

Huayan Robot Library

C++



修订记录

文件状态	[]草稿 [√] 正式发布 []正在修改		
当前版本	V1.0.15.0		
拟 制	华沿机器人系统软件组	日期	2025 年 12 月 11 日
审 核	华沿机器人系统测试组	日期	2025 年 12 月 12 日
批 准	华沿机器人	日期	2025 年 12 月 15 日
发布日期	2025 年 12 月 15 日		
生效日期	2025 年 12 月 15 日		

A - 增加 M - 修订 D - 删除

版本编号	版本日期	支持的控制器版本	更新说明
V1.0.0.0	2022.05.12	20220314.180	A 创建文件
V1.0.0.1	2022.07.07	20220624.203	A 新增接口
V1.0.0.2	2022.08.05	20220720.207	A 新增接口
V1.0.0.3	2022.08.28	20220825.214	A 新增 HRIF_SetToolMotion 接口
V1.0.0.4	2023.02.13	20220825.214	A 新增 HRIF_ReadOverride 接口
V1.0.0.5	2023.02.15	HR5.50.3	A 新增 HRIF_ReadPointByName 接口
V1.0.1.0	2023.03.20	HR5.49.7	A 新增 HRIF_GetErrorCodeStr 接口
V1.0.2.0	2023.03.21	HR5.50.5	A 新增 HRIF_SetPoseTrackingMaxMotionLimit, HRIF_SetPoseTrackingPIDParams, HRIF_SetPoseTrackingTargetPos, HRIF_SetPoseTrackingState, HRIF_SetUpdateTrackingPose
V1.0.2.1	2023.08.26	HR5.50.5	M ReadHoldingRegisters 和 WriteHoldingRegisters 的参数 nSlaveID 的取值范围改为 1~255
V1.0.2.2	2023.10.26	HR6.2.15	A 新增力控指令接口 HRIF_SetFreeDriveMotionFreedom、 HRIF_SetFTFreeFactor、 HRIF_SetTangentForceBounds
V1.0.2.3	2024.01.18	HR6.3.4	M HRIF_SetForceDataLimit 修改力限制范围接口
V1.0.3.0	2024-2-21	HR6.3.5	A 新增接口： HRIF_SetSimulation、HRIF_CalUcsPlane HRIF_CalUcsLine、HRIF_CalTcp3P、HRIF_CalTcp4P
V1.0.4.0	2024-03-01	HR6.3.6	A 新增接口 HRIF_InitPath、HRIF_PushPathPoints、HRIF_EndPushPathPoints、 HRIF_DelPath、HRIF_ReadPathList、HRIF_ReadPathInfo、 HRIF_UpdatePathName、HRIF_ReadPathState A 新增接口 HRIF_ConfigTCP、HRIF_ConfigUCS HRIF_MoveToSS、HRIF_SetCollideLeve HRIF_SetMoveParamsAO、HRIF_SetPoseTrackingStopTimeOut
V1.0.4.1	2024-03-13	HR6.3.6	M 修改轨迹运动类接口，轨迹名称由原来的 sTrackName 统一改为 sPathName
V1.0.5.0	2024-03-25	HR6.3.6	A 新增接口

			HRIF_OpenBrake、HRIF_CloseBrake、HRIF_ReadBreakStatus、 HRIF_ReadMaxPayload、HRIF_ReadPayload
V1.0.6.0	2024-04-09	HR6.3.7	A 新增接口 HRIF_SetFreeDriveCompensateForce、 HRIF_SetFTWrenchThresholds、HRIF_SetMaxFreeDriveVel、 HRIF_ReadFTMotionFreedom、HRIF_SetMaxSearchDistance、 HRIF_SetSteadyContactDeviationRange A 新增示例代码 机器人初始化、恒力控+MoveL
V1.0.6.1	2024-04-19	HR6.3.8	M HRIF_MoveS 修改参数顺序 M 修改通用运动类指令的速度限制范围
V1.0.7.0	2024-04-23	HR6.4.1	A 新增接口 HRIF_ReadUCSList、HRIF_ReadTCPList、 HRIF_EnableEndBTN、HRIF_ReadPointList
V1.0.8.0	2024-10-08	HR6.2.17	A 新增接口 HRIF_cdsSetIO
		HR6.3.1	A 新增接口 HRIF_AddSafePlane HRIF_DelSafePlane HRIF_ReadSafePlaneList HRIF_UpdateSafePlane HRIF_SetDepthThresholdForDampingArea
		HR6.4.1	A 新增接口 HRIF_MovePathJOL HRIF_GetMovePathJOLIndex HRIF_ReadFTFreeDriveSpeedMode HRIF_SetFTFreeDriveSpeedMode
		HR6.4SpeedL HR6.5.1	A 新增接口 HRIF_SpeedJ HRIF_SpeedL
		HR6.4.10	A 新增接口 HRIF_SetTriStageSwitch HRIF_ReadTriStageSwitch
V1.0.9.0	2024-10-15	HR5.51.9	A 新增接口 HRIF_EnterSafeGuard
		HR6.4.12	A 新增接口 HRIF_SwitchScript HRIF_ReadDefaultScript
		HR7.1.5	A 新增接口 HRIF_SetJointMaxVel_nJ HRIF_SetJointMaxAcc_nJ HRIF_SetMaxAcsRange_nJ HRIF_ReadCmdJointPos_nJ HRIF_ReadCmdJointCur_nJ HRIF_ReadActJointCur_nJ HRIF_ReadActJointPos_nJ HRIF_ReadCmdJointVel_nJ HRIF_ReadActJointVel_nJ HRIF_GetInverseKin_nJ HRIF_GetForwardKin_nJ HRIF_WayPointRel_nJ HRIF_MoveJ_nJ HRIF_MoveL_nJ HRIF_MoveC_nJ HRIF_ReadJointMaxVel_nJ HRIF_ReadJointMaxJerk_nJ HRIF_ReadJointMaxAcc_nJ HRIF_ReadPointByName_nJ HRIF_ReadActPos_nJ HRIF_WayPointEx_nJ HRIF_WayPoint2_nJ HRIF_WayPoint_nJ HRIF_ReadAxisErrorCode_nJ
V1.0.9.1	2024-10-16	HR5.50.3	M HRIF_ReadPointByName 修正接口的示例入参顺序
V1.0.9.2	2024-10-28	HR5.49.7	M HRIF_MoveE 修改参数说明
V1.0.10.0	2024-12-26	HR6.4.10	A 新增接口 HRIF_MoveAlignToZ
V1.0.10.1	2024-12-27	HR5.49.7	M HRIF_HRAppCmd 修改接口参数

V1.0.11.0	2025-3-14	HR7.2.3	A 新增接口 HRIF_WayPoint2ByLockAxis_nJ HRIF_MoveLByLockAxis_nJ HRIF_MoveCByLockAxis_nJ
V1.0.12.0	2025-3-14	HR6.5.10	A 新增接口 HRIF_SetBaseInstallingAngle HRIF_GetBaseInstallingAngle HRIF_GetLoadIdentifyResult HRIF_GetLastCalibParams HRIF_SetInitializeForceSensor HRIF_SetFTCalibration HRIF_ReadForceData HRIF_LoadIdentify HRIF_ShortJogJ HRIF_ShortJogL HRIF_LongJogJ HRIF_LongJogL HRIF_LongMoveEvent
V1.0.13.0	2025-4-15	6.5ZT.12 6.6.1	HRIF_CalTCPort HRIF_SetDftTCP HRIF_ReadActCoord HRIF_ReadActCoord_nJ
V1.0.14.0	2025-07-28	HR6.5.15	A 新增接口 HRIF_GetInverseDynamics HRIF_SetPathRefJoints HRIF_SetFTUCS HRIF_ReadFTUCS
		HR6.4.10	A 新增接口 HRIF_ReadMaxRange HRIF_ReadMaxRange_nJ
		HR5.49.7	A 新增接口 HRIF_HRAppData
		HR6.4.8	A 新增接口 HRIF_SetEndModbusInfo
V1.0.14.1	2025-6-19	HR6.5.14	M 修改接口 MoveC 和 MoveC_nJ 的 nFixedPosure 参数说明
V1.0.14.2	2025-11-10	HR6.4.10	M 修复 HRIF_MoveAlignToZ 使用说明
V1.0.14.3	2025-11-18	HR5.49.7	M HRIF_StartServo 增加版本变更参数说明 M HRIF_MoveE 增加支持版本说明
	2025-11-18	6.5ZT.12	M 修复 HRIF_SetDftTCP 使用示例
	2025-11-18	HR6.5.10	M 修复 HRIF_GetLoadIdentifyResult 参数说明
V1.0.15.0	2025-12-11	-	A 更新文档环境配置部分描述

第一章 概述	1
1.1. 编写目的	1
1.2. 名词解析	1
1.3. 数据类型	1
第二章 环境搭建	2
2.1. C++ SDK 出货清单	2
2.2. Windows 环境(MSVC 编译器)	2
2.2.1. 创建项目	2
2.2.2. 移动 SDK	4
2.2.3. 链接库文件	4
2.2.4. 运行程序	6
2.3. Windows 环境 (MinGW 编译器)	7
2.3.1. 创建项目	7
2.3.2. 移动 SDK	7
2.3.3. 链接库文件	8
2.3.4. 运行程序	9
2.4. Linux 环境	10
2.4.1. 创建项目	10
2.4.2. 移动 SDK	10
2.4.3. 链接库文件	11
2.4.4. 运行程序	12
第三章 可使用接口	13
3.1. 初始化指令	13
3.1.1. HRIF_Connect	13
3.1.2. HRIF_DisConnect	14
3.1.3. HRIF_IsConnected	15
3.1.4. HRIF_ShutdownRobot	16
3.1.5. HRIF_Connect2Box	17

3.1.6. HRIF_Electrify	18
3.1.7. HRIF_Blackout	19
3.1.8. HRIF_Connect2Controller	20
3.1.9. HRIF_IsControllerStarted	21
3.1.10. HRIF_SetSimulation	22
3.1.11. HRIF_IsSimulateRobot	23
3.1.12. HRIF_ReadVersion	24
3.1.13. HRIF_ReadRobotModel	26
3.1.14. HRIF_GetErrorCodeStr	27
3.2. 轴组控制指令	28
3.2.1. HRIF_GrpEnable	28
3.2.2. HRIF_GrpDisable	29
3.2.3. HRIF_GrpReset	30
3.2.4. HRIF_GrpStop	31
3.2.5. HRIF_GrpInterrupt	32
3.2.6. HRIF_GrpContinue	33
3.2.7. HRIF_GrpCloseFreeDriver	34
3.2.8. HRIF_GrpOpenFreeDriver	35
3.2.9. HRIF_OpenBrake	35
3.2.10. HRIF_CloseBrake	36
3.2.11. HRIF_ReadBrakeStatus	36
3.2.12. HRIF_MoveToSS	37
3.2.13. HRIF_EnterSafeGuard	38
3.2.14. HRIF_ReadTriStageSwitch	39
3.2.15. HRIF_SetTriStageSwitch	40
3.2.16. HRIF_MoveAlignToZ	40
3.2.17. HRIF_GetLoadIdentifyResult	42
3.2.18. HRIF_LoadIdentify	43
3.3. 脚本控制指令	44
3.3.1. HRIF_RunFunc	44
3.3.2. HRIF_StartScript	45
3.3.3. HRIF_StopScript	46
3.3.4. HRIF_PauseScript	47
3.3.5. HRIF_ContinueScript	48
3.3.6. HRIF_SwitchScript	49
3.3.7. HRIF_ReadDefaultScript	50
3.4. 电箱控制指令	51
3.4.1. HRIF_ReadBoxInfo	51
3.4.2. HRIF_ReadBoxCI	53
3.4.3. HRIF_ReadBoxDI	54

3.4.4. HRIF_ReadBoxCO	55
3.4.5. HRIF_ReadBoxDO	56
3.4.6. HRIF_ReadAllBoxCI	57
3.4.7. HRIF_ReadAllBoxDI	58
3.4.8. HRIF_ReadAllBoxCO	59
3.4.9. HRIF_ReadAllBoxDO	60
3.4.10. HRIF_ReadBoxAI	61
3.4.11. HRIF_ReadBoxAO	62
3.4.12. HRIF_SetBoxCO	63
3.4.13. HRIF_SetBoxDO	64
3.4.14. HRIF_SetBoxAOMode	65
3.4.15. HRIF_SetBoxAOVal	66
3.4.16. HRIF_SetEndDO	67
3.4.17. HRIF_SetMoveParamsAO	68
3.4.18. HRIF_ReadEndDI	69
3.4.19. HRIF_ReadEndDO	70
3.4.20. HRIF_ReadEndAI	71
3.4.21. HRIF_ReadEndBTN	72
3.4.22. HRIF_EnableEndBTN	73
3.4.23. HRIF_cdsSetIO	73
3.5. 状态读取与设置指令	76
3.5.1. HRIF_SetOverride	76
3.5.2. HRIF_SetToolMotion	77
3.5.3. HRIF_SetPayload	78
3.5.4. HRIF_SetJointMaxVel	79
3.5.5. HRIF_SetJointMaxVel_nJ	80
3.5.6. HRIF_SetJointMaxAcc	81
3.5.7. HRIF_SetJointMaxAcc_nJ	82
3.5.8. HRIF_SetLinearMaxVel	83
3.5.9. HRIF_SetLinearMaxAcc	84
3.5.10. HRIF_SetMaxAcsRange	85
3.5.11. HRIF_SetMaxAcsRange_nJ	86
3.5.12. HRIF_SetMaxPcsRange	88
3.5.13. HRIF_SetCollideLevel	89
3.5.14. HRIF_ReadMaxPayload	90
3.5.15. HRIF_ReadPayload	91
3.5.16. HRIF_ReadOverride	92
3.5.17. HRIF_ReadJointMaxVel	93
3.5.18. HRIF_ReadJointMaxVel_nJ	94
3.5.19. HRIF_ReadJointMaxAcc	95
3.5.20. HRIF_ReadJointMaxAcc_nJ	96

3.5.21. HRIF_ReadJointMaxJerk	97
3.5.22. HRIF_ReadJointMaxJerk_nJ	98
3.5.23. HRIF_ReadLinearMaxSpeed	99
3.5.24. HRIF_ReadEmergencyInfo	100
3.5.25. HRIF_ReadRobotState	101
3.5.26. HRIF_ReadRobotFlags	103
3.5.27. HRIF_ReadCurWaypointID	105
3.5.28. HRIF_ReadAxisErrorCode	106
3.5.29. HRIF_ReadAxisErrorCode_nJ	107
3.5.30. HRIF_ReadCurFSM	108
3.5.31. HRIF_ReadCurFSMFromCPS	109
3.5.32. HRIF_ReadPointByName	110
3.5.33. HRIF_ReadPointByName_nJ	112
3.5.34. HRIF_ReadPointList	114
3.5.35. HRIF_ReadMaxRange	114
3.5.36. HRIF_ReadMaxRange_nJ	116
3.6. 位置、速度、电流读取指令	118
3.6.1. HRIF_ReadActPos	118
3.6.2. HRIF_ReadActPos_nJ	120
3.6.3. HRIF_ReadCmdJointPos	122
3.6.4. HRIF_ReadCmdJointPos_nJ	123
3.6.5. HRIF_ReadActJointPos	124
3.6.6. HRIF_ReadActJointPos_nJ	125
3.6.7. HRIF_ReadCmdTcpPos	126
3.6.8. HRIF_ReadActTcpPos	127
3.6.9. HRIF_ReadCmdJointVel	128
3.6.10. HRIF_ReadCmdJointVel_nJ	129
3.6.11. HRIF_ReadActJointVel	130
3.6.12. HRIF_ReadActJointVel_nJ	131
3.6.13. HRIF_ReadCmdTcpVel	132
3.6.14. HRIF_ReadActTcpVel	133
3.6.15. HRIF_ReadCmdJointCur	134
3.6.16. HRIF_ReadCmdJointCur_nJ	135
3.6.17. HRIF_ReadActJointCur	136
3.6.18. HRIF_ReadActJointCur_nJ	137
3.6.19. HRIF_ReadTcpVelocity	138
3.7. 坐标转换计算指令	139
3.7.1. HRIF_Quaternion2RPY	139
3.7.2. HRIF_RPY2Quaternion	140
3.7.3. HRIF_GetInverseKin	141

3.7.4. HRIF_GetInverseKin_nJ	144
3.7.5. HRIF_GetForwardKin	147
3.7.6. HRIF_GetForwardKin_nJ	149
3.7.7. HRIF_Base2UcsTcp	151
3.7.8. HRIF_UcsTcp2Base	153
3.7.9. HRIF_PoseAdd	155
3.7.10. HRIF_PoseSub	157
3.7.11. HRIF_PoseTrans	159
3.7.12. HRIF_PoseInverse	161
3.7.13. HRIF_PoseDist	163
3.7.14. HRIF_Poseinterpolate	165
3.7.15. HRIF_PoseDefdFrame	167
3.7.16. HRIF_CalUcsPlane	170
3.7.17. HRIF_CalUcsLine	172
3.7.18. HRIF_CalTcp3P	174
3.7.19. HRIF_CalTcp4P	177
3.7.20. HRIF_CalTCPOrt	180
3.7.21. HRIF_GetInverseDynamics	182
3.8. 工具坐标与用户坐标读写指令	184
3.8.1. HRIF_SetTCP	184
3.8.2. HRIF_SetUCS	185
3.8.3. HRIF_ReadCurTCP	186
3.8.4. HRIF_ReadCurUCS	187
3.8.5. HRIF_SetTCPByName	188
3.8.6. HRIF_SetUCSByName	189
3.8.7. HRIF_ReadTCPByName	190
3.8.8. HRIF_ReadTCPList	191
3.8.9. HRIF_ReadUCSList	192
3.8.10. HRIF_ReadUCSByName	193
3.8.11. HRIF_ConfigTCP	194
3.8.12. HRIF_ConfigUCS	195
3.8.13. HRIF_SetBaseInstallingAngle	196
3.8.14. HRIF_GetBaseInstallingAngle	197
3.8.15. HRIF_SetDftTCP	198
3.8.16. HRIF_ReadActCoord	199
3.8.17. HRIF_ReadActCoord_nJ	201
3.9. 力控控制指令	203
3.9.1. HRIF_SetForceControlState	203
3.9.2. HRIF_ReadForceControlState	204
3.9.3. HRIF_SetForceToolCoordinateMotion	205

3.9.4. HRIF_ForceControlInterrupt	206
3.9.5. HRIF_ForceControlContinue	207
3.9.6. HRIF_SetForceZero	208
3.9.7. HRIF_SetMaxSearchVelocities	209
3.9.8. HRIF_SetControlFreedom	210
3.9.9. HRIF_SetForceControlStrategy	211
3.9.10. HRIF_SetFreeDrivePositionAndOrientation	212
3.9.11. HRIF_SetPIDControlParams	213
3.9.12. HRIF_SetMassParams	214
3.9.13. HRIF_SetDampParams	215
3.9.14. HRIF_SetStiffParams	216
3.9.15. HRIF_SetForceControlGoal	217
3.9.16. HRIF_SetControlGoal	218
3.9.17. HRIF_SetForceDataLimit	219
3.9.18. HRIF_SetForceDistanceLimit	221
3.9.19. HRIF_SetForceFreeDriveMode	222
3.9.20. HRIF_ReadFTCabData	223
3.9.21. HRIF_SetFTFreeDriveSpeedMode	224
3.9.22. HRIF_ReadFTFreeDriveSpeedMode	224
3.9.23. HRIF_SetFreeDriveMotionFreedom	225
3.9.24. HRIF_SetFTFreeFactor	227
3.9.25. HRIF_SetTangentForceBounds	228
3.9.26. HRIF_SetFreeDriveCompensateForce	229
3.9.27. HRIF_SetFTWrenchThresholds	230
3.9.28. HRIF_SetMaxFreeDriveVel	231
3.9.29. HRIF_ReadFTMotionFreedom	232
3.9.30. HRIF_SetMaxSearchDistance	233
3.9.31. HRIF_SetSteadyContactDeviationRange	234
3.9.32. HRIF_AddSafePlane	235
3.9.33. HRIF_UpdateSafePlane	237
3.9.34. HRIF_DelSafePlane	239
3.9.35. HRIF_ReadSafePlaneList	240
3.9.36. HRIF_ReadSafePlane	241
3.9.37. HRIF_SetDepthThresholdForDampingArea	242
3.9.38. HRIF_GetLastCalibParams	242
3.9.39. HRIF_SetInitializeForceSensor	243
3.9.40. HRIF_SetFTCalibration	245
3.9.41. HRIF_ReadForceData	246
3.9.42. HRIF_SetFTUCS	247
3.9.43. HRIF_ReadFTUCS	248
3.10. 通用运动类控制指令	249

3.10.1. HRIF_MoveRelJ	249
3.10.2. HRIF_MoveRelL	250
3.10.3. HRIF_WayPointRel	251
3.10.4. HRIF_WayPointRel_nJ	255
3.10.5. HRIF_IsMotionDone	260
3.10.6. HRIF_IsBlendingDone	261
3.10.7. HRIF_WayPointEx	262
3.10.8. HRIF_WayPointEx_nJ	266
3.10.9. HRIF_WayPoint	270
3.10.10. HRIF_WayPoint_nJ	274
3.10.11. HRIF_WayPoint2	278
3.10.12. HRIF_WayPoint2_nJ	282
3.10.13. HRIF_WayPoint2ByLockAxis_nJ	286
3.10.14. HRIF_MoveJ	291
3.10.15. HRIF_MoveJ_nJ	294
3.10.16. HRIF_MoveL	297
3.10.17. HRIF_MoveL_nJ	300
3.10.18. HRIF_MoveLByLockAxis_nJ	303
3.10.19. HRIF_MoveC	306
3.10.20. HRIF_MoveC_nJ	310
3.10.21. HRIF_MoveCByLockAxis_nJ	314
3.10.22. HRIF_MoveZ	318
3.10.23. HRIF_MoveE	322
3.10.24. HRIF_MoveS	326
3.10.25. HRIF_MoveLinearWeave	328
3.10.26. HRIF_MoveCircularWeave	332
3.10.27. HRIF_ShortJogJ	336
3.10.28. HRIF_ShortJogL	338
3.10.29. HRIF_LongJogJ	339
3.10.30. HRIF_LongJogL	340
3.10.31. HRIF_LongMoveEvent	341
3.11. 连续轨迹运动类控制指令	342
3.11.1. HRIF_StartPushMovePathJ	342
3.11.2. HRIF_PushMovePathJ	343
3.11.3. HRIF_EndPushMovePathJ	344
3.11.4. HRIF_MovePathJ	345
3.11.5. HRIF_ReadMovePathJState	346
3.11.6. HRIF_UpdateMovePathJName	348
3.11.7. HRIF_DelMovePathJ	349
3.11.8. HRIF_ReadTrackProcess	350
3.11.9. HRIF_InitMovePathL	351

3.11.10. HRIF_PushMovePathL	353
3.11.11. HRIF_PushMovePaths	355
3.11.12. HRIF_MovePathL	357
3.11.13. HRIF_SetMovePathOverride	358
3.11.14. HRIF_InitPath	358
3.11.15. HRIF_PushPathPoints	361
3.11.16. HRIF_EndPushPathPoints	362
3.11.17. HRIF_DelPath	363
3.11.18. HRIF_ReadPathList	363
3.11.19. HRIF_ReadPathInfo	364
3.11.20. HRIF_UpdatePathName	366
3.11.21. HRIF_ReadPathState	366
3.11.22. HRIF_MovePathJOL	369
3.11.23. HRIF_GetMovePathJOLIndex	373
3.11.24. HRIF_SetPathRefJoints	374
3.12. Servo 运动类控制指令	375
3.12.1. HRIF_StartServo	375
3.12.2. HRIF_PushServoJ	377
3.12.3. HRIF_PushServoP	378
3.12.4. HRIF_SpeedJ	379
3.12.5. HRIF_SpeedL	380
3.13. 相对跟踪运动类控制指令	382
3.13.1. HRIF_SetMoveTraceParams	382
3.13.2. HRIF_SetMoveTraceInitParams	383
3.13.3. HRIF_SetMoveTraceUcs	384
3.13.4. HRIF_SetTrackingState	385
3.14. 位置跟随运动类指令	386
3.14.1. HRIF_SetPoseTrackingMaxMotionLimit	386
3.14.2. HRIF_SetPoseTrackingStopTimeOut	387
3.14.3. HRIF_SetPoseTrackingPIDParams	388
3.14.4. HRIF_SetPoseTrackingTargetPos	389
3.14.5. HRIF_SetPoseTrackingState	390
3.14.6. HRIF_SetUpdateTrackingPose	391
3.15. 其他指令	392
3.15.1. HRIF_HRAppCmd	392
3.15.2. HRIF_HRAppData	392
3.15.3. HRIF_WriteEndHoldingRegisters	394
3.15.4. HRIF_ReadEndHoldingRegisters	396
3.15.5. HRIF_SetEndModbusInfo	398

3.16. 示例代码	400
------------------	-----

第一章 概述

1.1. 编写目的

1. 为使用 Huayan C++ 接口的人员提供接口使用说明;
2. 为研发人员进行接口编写和维护提供参考文件。

1.2. 名词解析

专用名称	解释
ACS	关节坐标, 单位为度(°)
PCS	空间坐标, 单位为毫米(mm)、度(°)
TCP	系统默认的工具坐标系, 初始值为 0,0,0,0,0,0
UCS	用户坐标系
Base	系统默认的用户坐标系, 初始值为 0,0,0,0,0,0
Tool	系统默认的工具坐标系

1.3. 数据类型

数据类型	长度	说明
char	1 byte	字符型
bool	1 byte	布尔型
unsigned int	4 bytes	无符号整型
int	4 bytes	整型
double	8 bytes	双精度浮点型
string	-	字符串
vector	-	容器

第二章 环境搭建

2.1. C++ SDK 出货清单

华沿机器人 C++ SDK 支持多平台、多架构的 C++ 编译器环境，自 1.0.15.0 版本起，标准出货件默认覆盖三大主流开发环境，分别为 Windows 平台的 MSVC 64 位编译器、MinGW 64 位编译器，以及 Linux 平台的 GCC 64 位编译器；针对 32 位系统及 Arm 架构的适配库，我们采取按需定制的供应策略，您可提供具体的开发环境信息，联系技术支持获取定制版本。

以下为该 C++ SDK 标准出货件的文件列表及对应说明。

平台	文件名	说明
include	HR_Pro.h	华沿机器人 SDK 仅包含一个头文件，供所有库共用
MSVC	HR_Pro.lib	Release 静态库
	HR_Pro.dll	Release 动态库
	HR_Prod.lib	Debug 静态库
	HR_Prod.dll	Debug 动态库
	pthreadVC2.dll	线程动态库
MinGW	libHR_Pro.dll.a	Release 导入库
	libHR_Pro.dll	Release 动态库
	libHR_Prod.dll.a	Debug 导入库
	libHR_Prod.dll	Debug 动态库
Linux	libHR_Pro.so	动态共享库

2.2. Windows 环境(MSVC 编译器)

2.2.1. 创建项目

使用 Visual Studio 创建一个 C++ 控制台程序，并添加一个源文件。

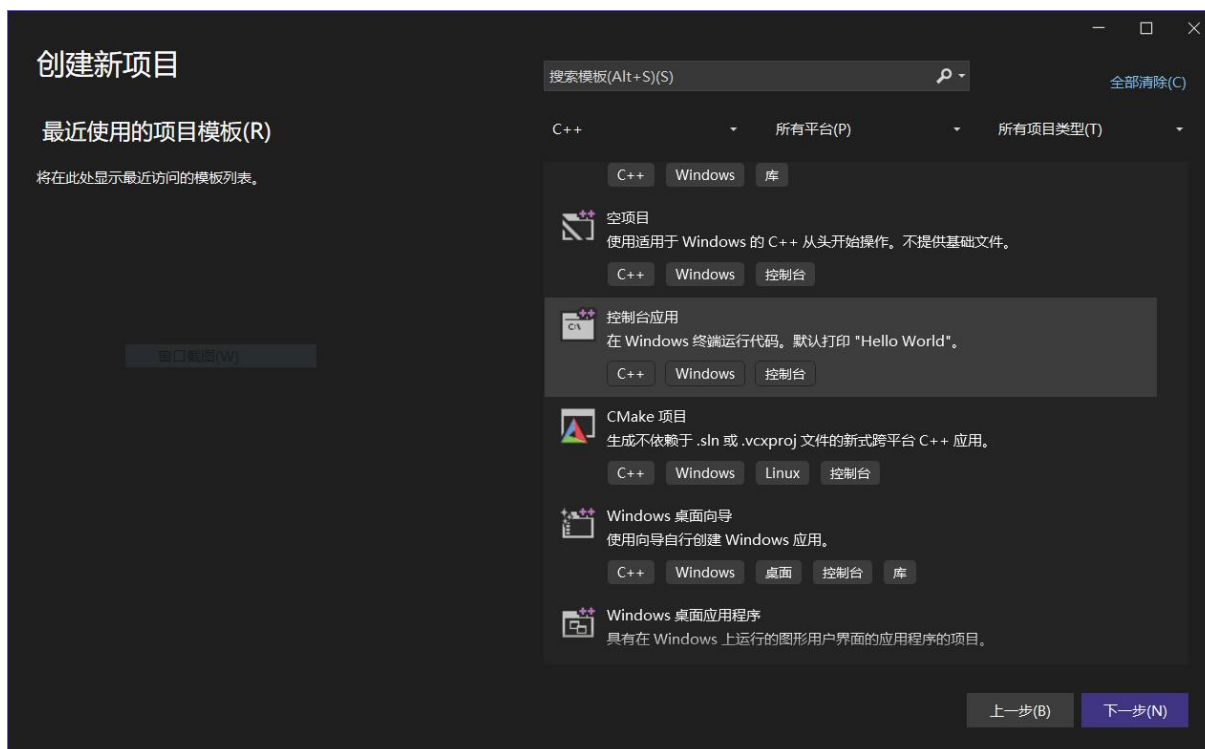


图 2.2.1 创建项目

分别切换 Debug 和 Release 模式进行编译，编译完成后，解决方案目录下会生成 x64/Debug 和 x64/Release 目录，可执行文件将存放在这两个目录中。

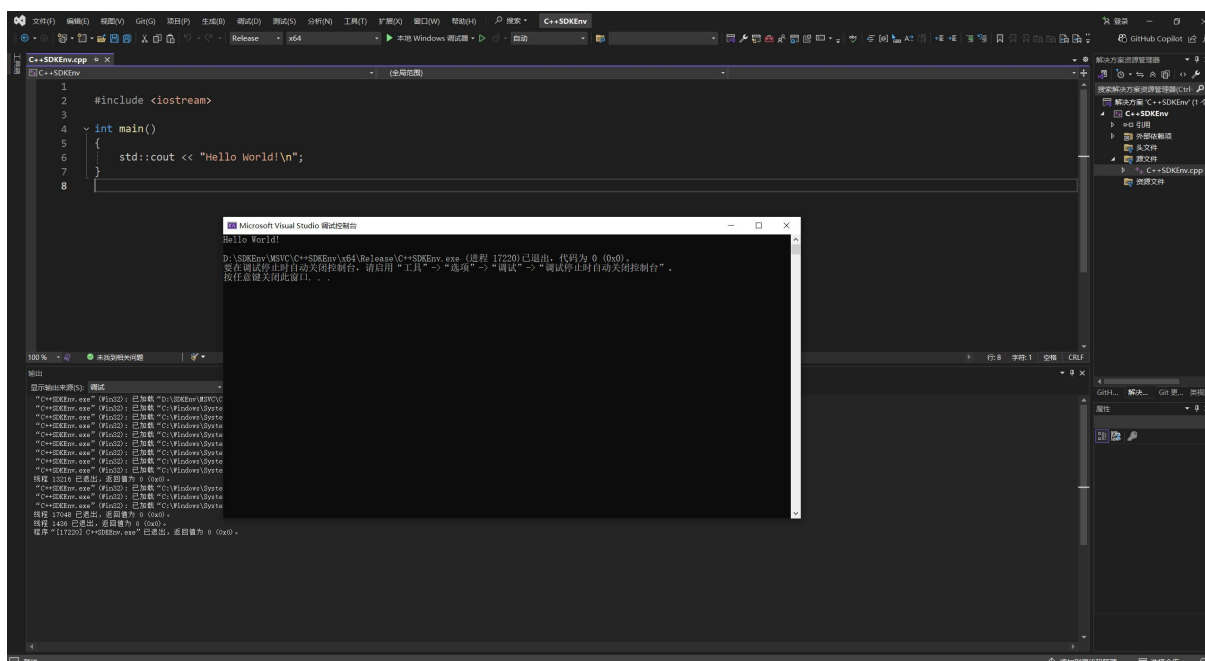


图 2.2.2 编译项目

2.2.2. 移动 SDK

进入项目文件夹，将 HR_Pro.h 文件复制到项目文件夹中。进入输出目录 x64/Debug，将 HR_Prod.lib、HR_Prod.dll 与 pthreadVC2.dll 文件放置到 Debug 模式下的输出目录；进入输出目录 x64/Release，将 HR_Prod.lib、HR_Prod.dll 与 pthreadVC2.dll 文件放置到 Release 模式下的输出目录。完成了将库文件发布中对应的动态库文件(dll)、静态库文件(lib)放在编译器生成的可执行程序同一个路径下。操作汇总图如下图所示。

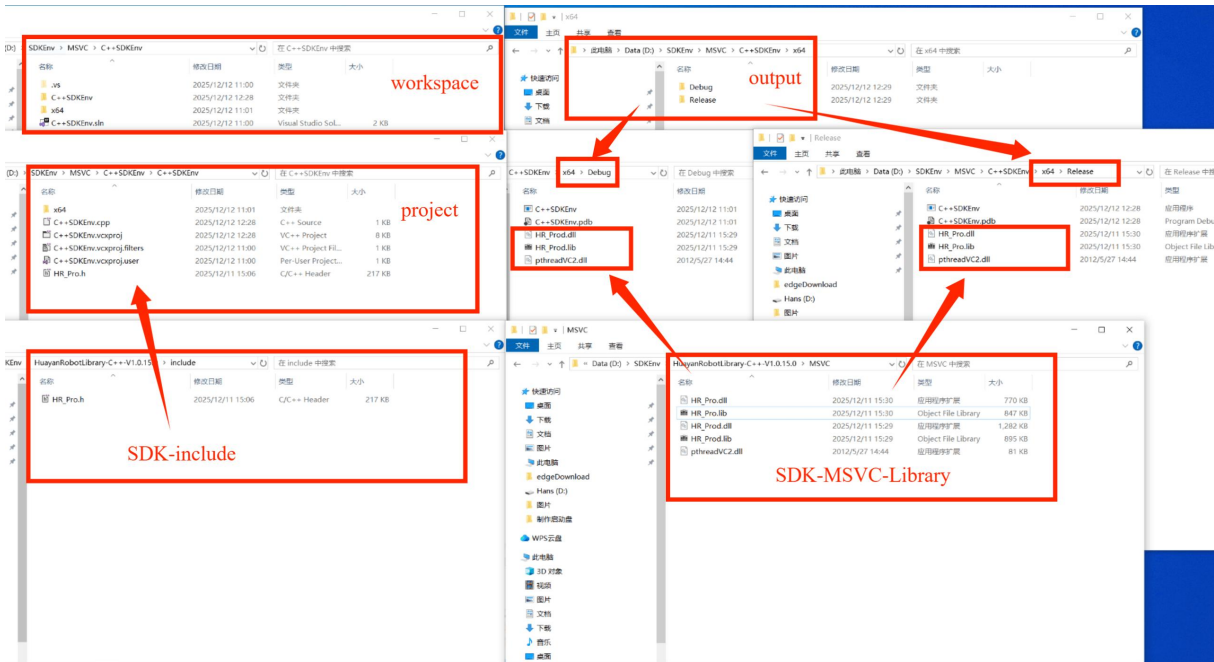


图 2.2.3 放置 SDK 文件

2.2.3. 链接库文件

在 IDE 中选中项目，在工具栏中选择项目->属性，打开项目属性页面并将配置设置为所有配置。如下图所示，在项目属性页的常规中获取输出目录并复制，复制并增加到 VC++ 目录的库目录位置。

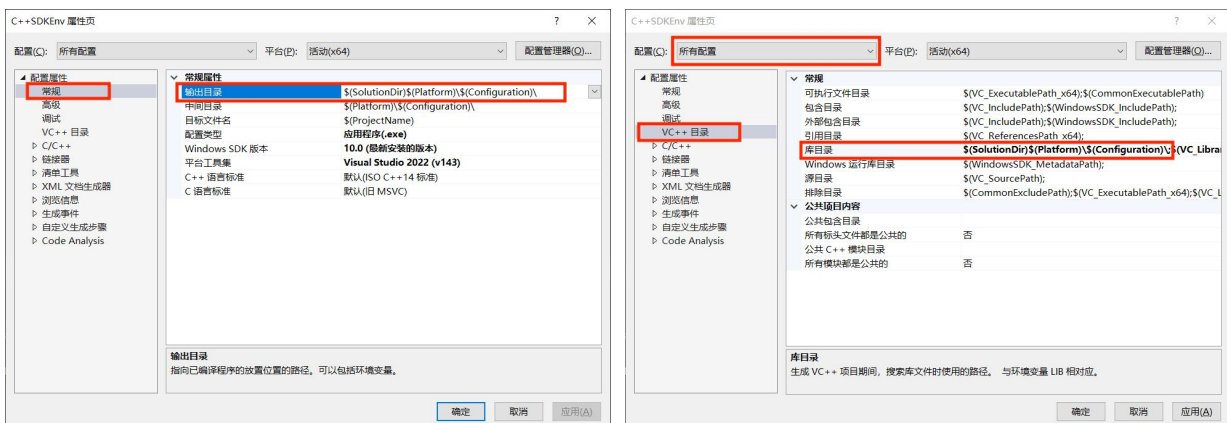


图 2.2.4 项目属性页配置

如下图所示为库目录的最终效果。

***注：**这里是以增加的方式在原来的系统库的基础上增加，如果删除了原来的内容可能带来负面影响。

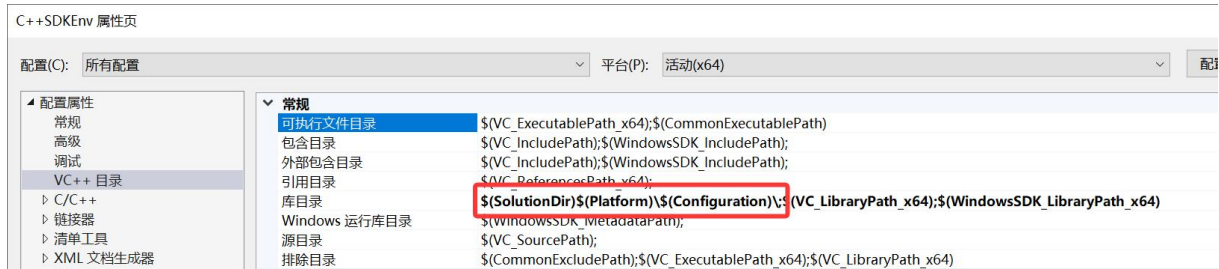


图 2.2.5 库目录最终效果

如下图所示，在项目属性页面将配置设置为 Debug。在链接器的输入菜单中的附加依赖项添加上 HR_Prod.lib 并应用。

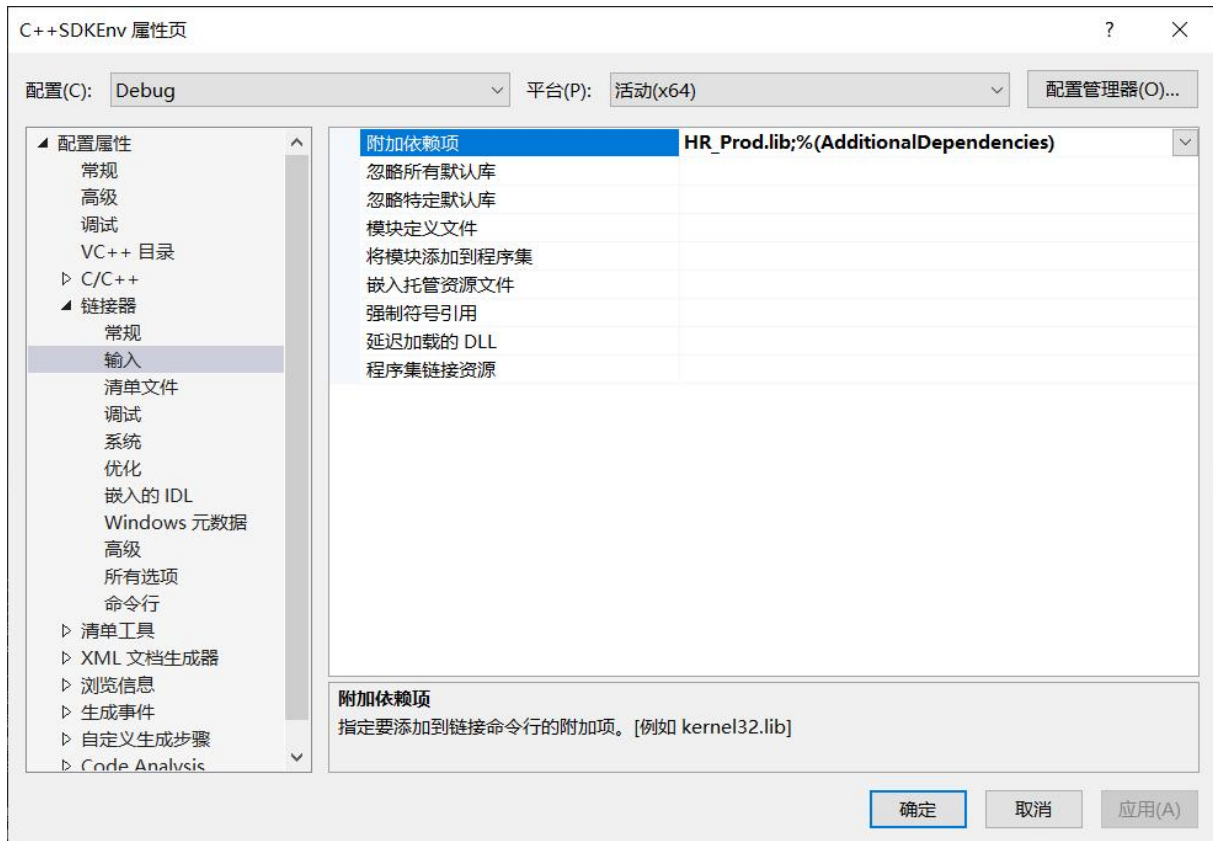


图 2.2.6 Debug 模式库加载

如下图所示，在项目属性页面将配置设置为 Release。在链接器的输入菜单中的附加依赖项添加上 HR_Prod.lib 并应用。

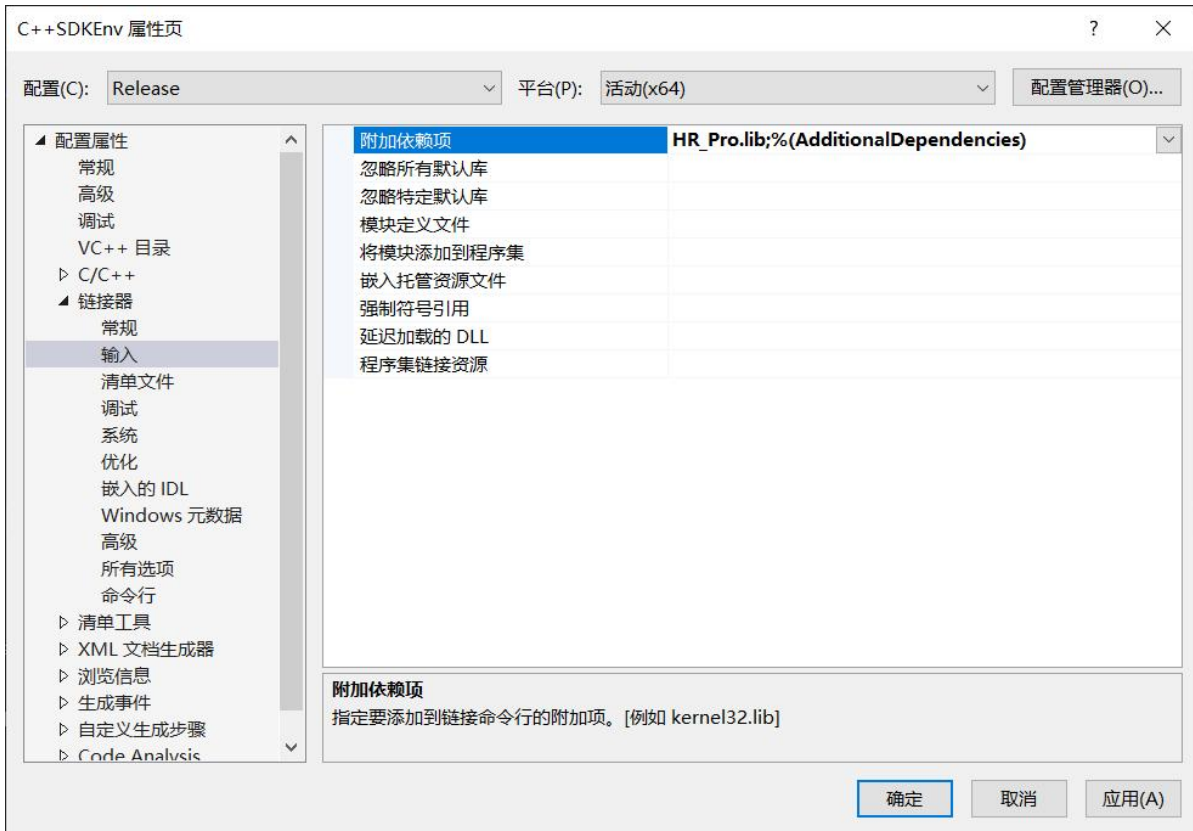


图 2.2.7 Release 模式库加载

2.2.4. 运行程序

在程序中使用 HR_Pro.h 文件，调用连接函数，连接机器人的 IP 的 10003 端口，进行编译运行，基本运行环境搭建完成如下图所示。

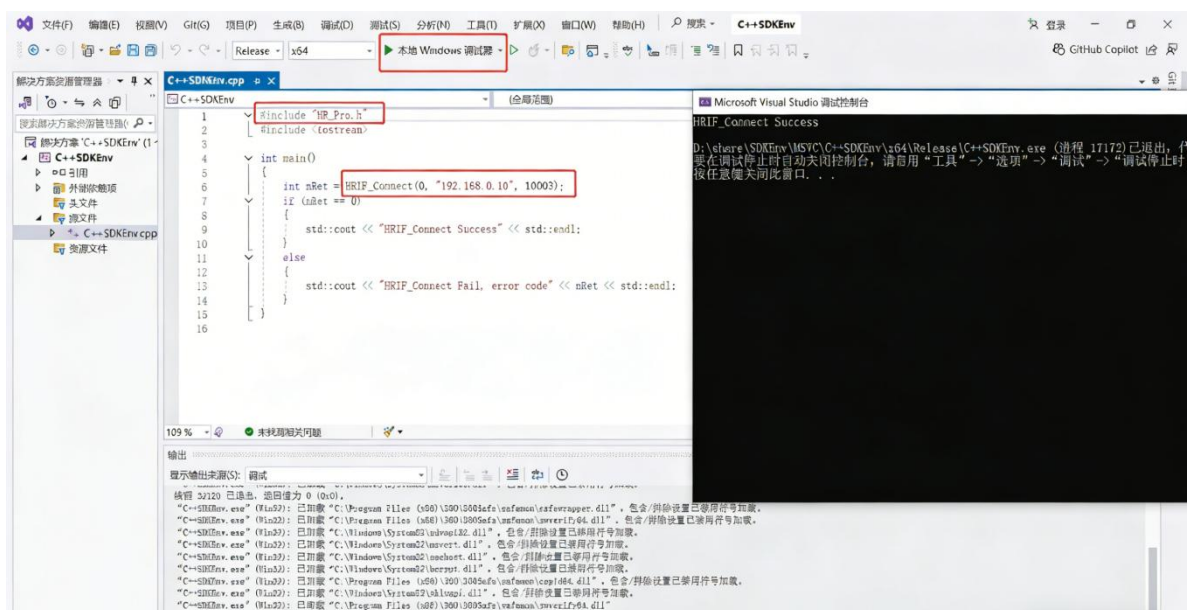
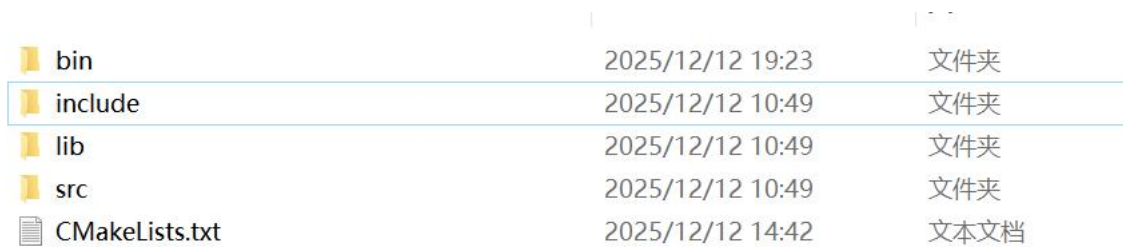


图 2.2.8 运行程序

2.3. Windows 环境 (MinGW 编译器)

2.3.1. 创建项目

使用 Visual Studio Code 创建一个 CMake 工程。在工作空间下新建 bin 文件夹存放可执行文件，新建 lib 文件夹放置静态库目录，新建 include 文件夹放置头文件，新建 src 文件夹放置源文件。工程下通过 CMakeLists.txt 文件来构建项目。



bin	2025/12/12 19:23	文件夹
include	2025/12/12 10:49	文件夹
lib	2025/12/12 10:49	文件夹
src	2025/12/12 10:49	文件夹
CMakeLists.txt	2025/12/12 14:42	文本文档

图 2.3.1 创建项目

2.3.2. 移动 SDK

进入项目文件夹，将 HR_Pro.h 文件复制到项目文件夹的 include 中。进入库目录 lib，将 libHR_Prod.dll.a 和 libHR_Pro.dll.a 文件放置到 lib 目录下；进入可执行文件目录 bin，将 libHR_Prod.dll 和 libHR_Pro.dll 文件放置到 bin 目录下。操作汇总图如下图所示。

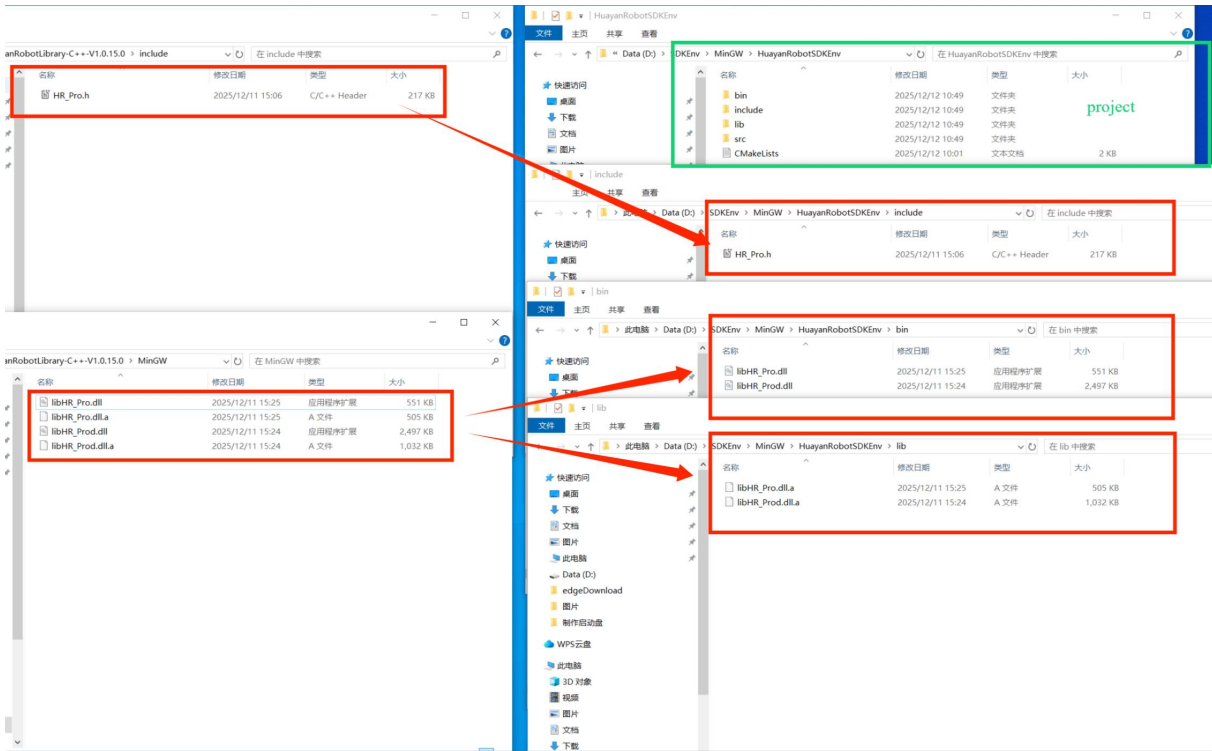


图 2.3.2 放置 SDK 文件

2.3.3. 链接库文件

使用 VSCode 打开工程，确保配置正确。

编写 CMakeLists.txt，定义编译构建规则：

1. 配置最小编译单元，将可执行文件输出目录设置为工程根目录下的 bin 目录；
2. 设置头文件目录为工程目录下的 include 目录，并通过 INCLUDE_DIRECTORIES 指令引入该目录；
3. 设置库文件目录为工程根目录下的 lib 目录，并根据编译模式选择对应的链接库；
4. 设置编译源文件为工程根目录下的 src/main.cpp 文件，并添加编译目标生成可执行文件；
5. 链接选择的库文件。

在 main.cpp 文件中编写简单连接到机器人，选择对应的编译器并执行构建操作，完成项目编译，项目构建流程如下图所示。

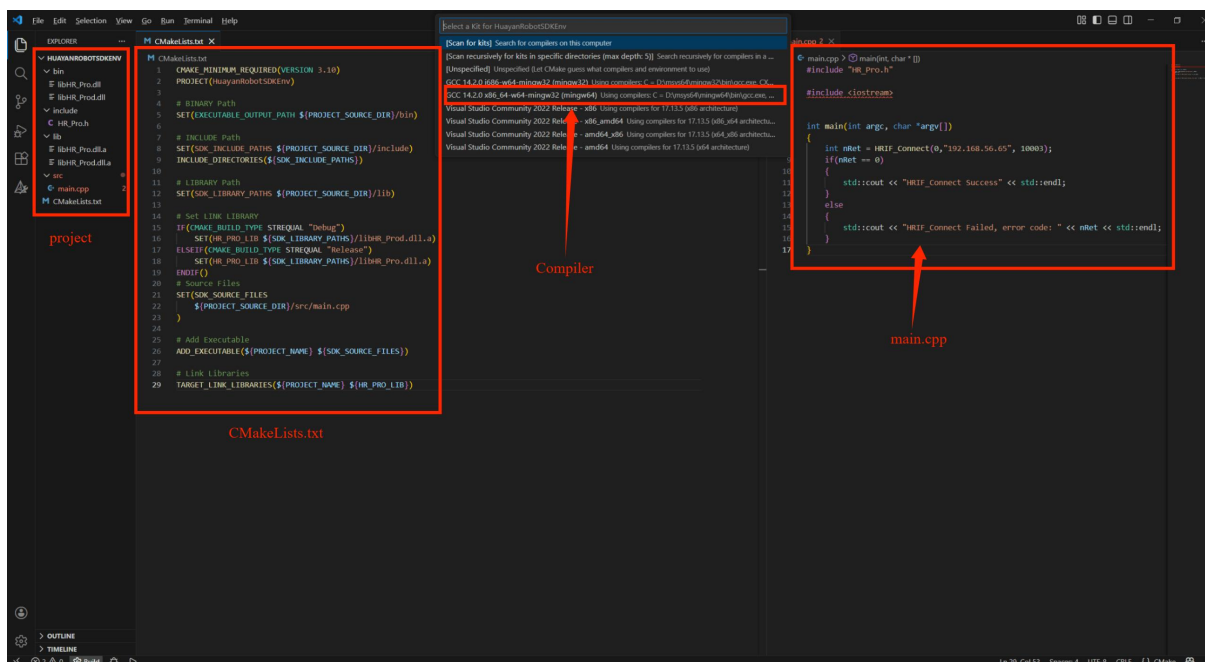


图 2.3.3 项目构建图

2.3.4. 运行程序

在程序中使用 HR_Pro.h 文件，调用连接函数，连接机器人的 IP 的 10003 端口，进行编译运行，基本运行环境搭建完成如下图所示。

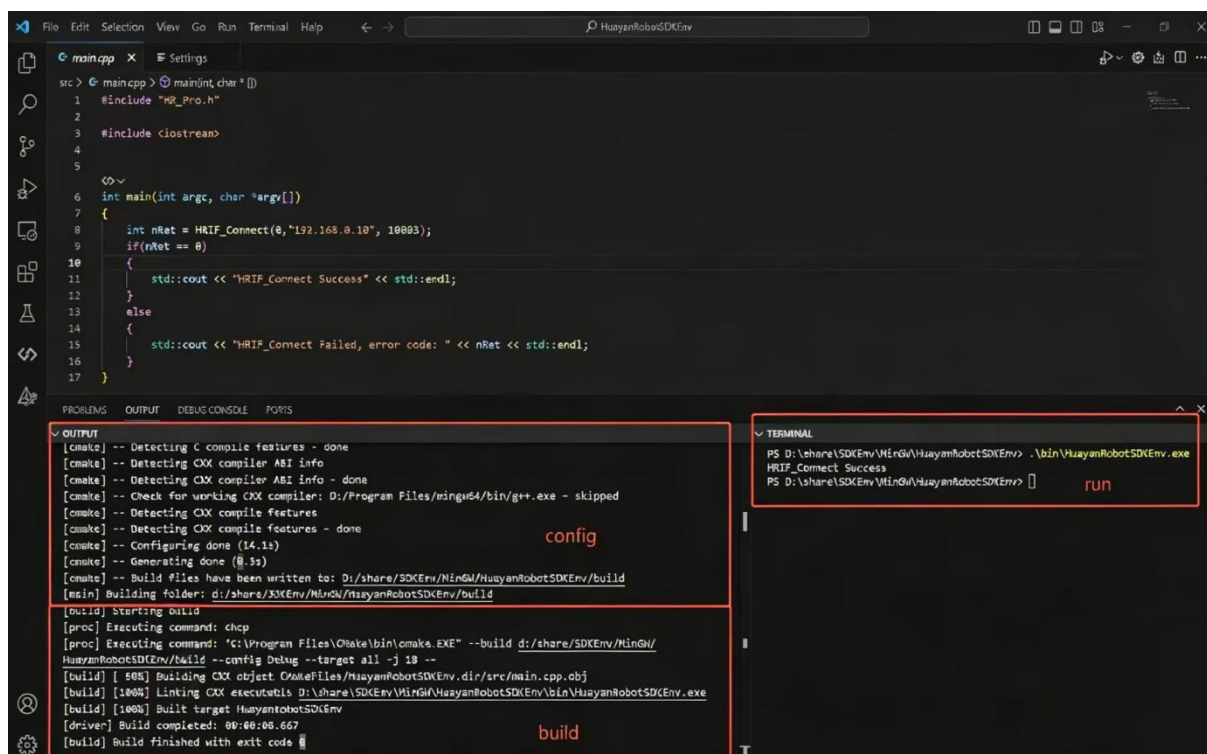


图 2.3.4 运行程序

2.4. Linux 环境

2.4.1. 创建项目

使用 Visual Studio Code 创建一个 CMake 工程。在工作空间下新建 bin 文件夹存放可执行文件，新建 lib 文件夹放置静态库目录，新建 include 文件夹放置头文件，新建 src 文件夹放置源文件。工程下通过 CMakeLists.txt 文件来构建项目。

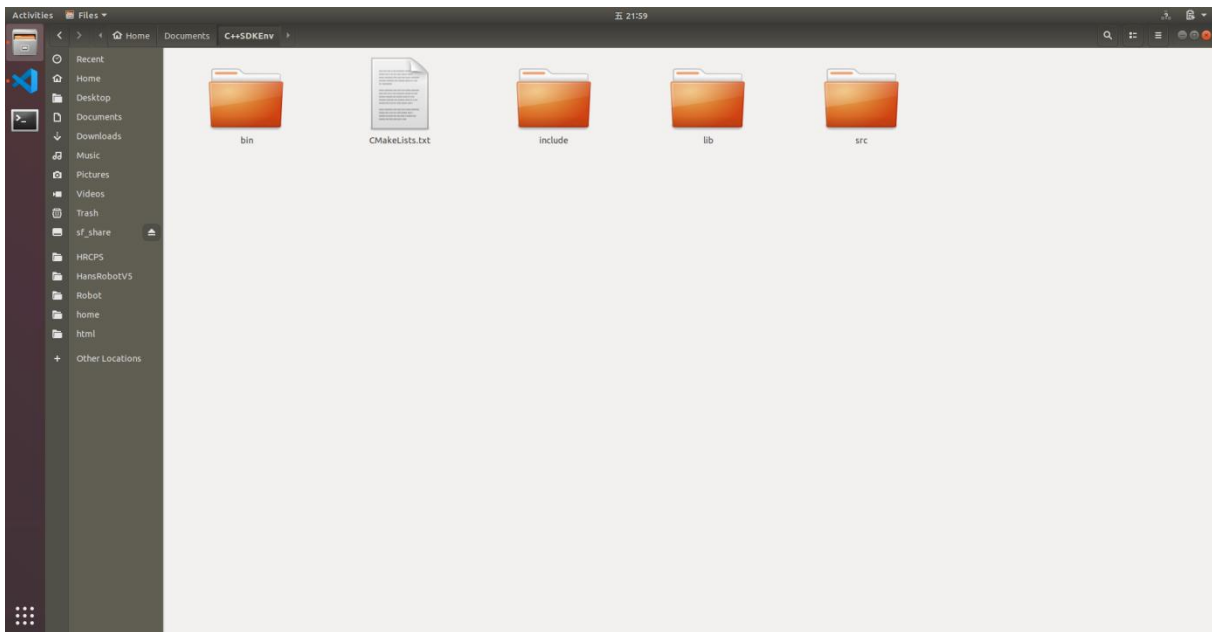


图 2.4.1 创建项目

2.4.2. 移动 SDK

进入项目文件夹，将 HR_Pro.h 文件复制到项目文件夹的 include 中。进入库目录 lib，将 libHR_Pro.so 文件放置到 lib 目录下。操作汇总图如下图所示。

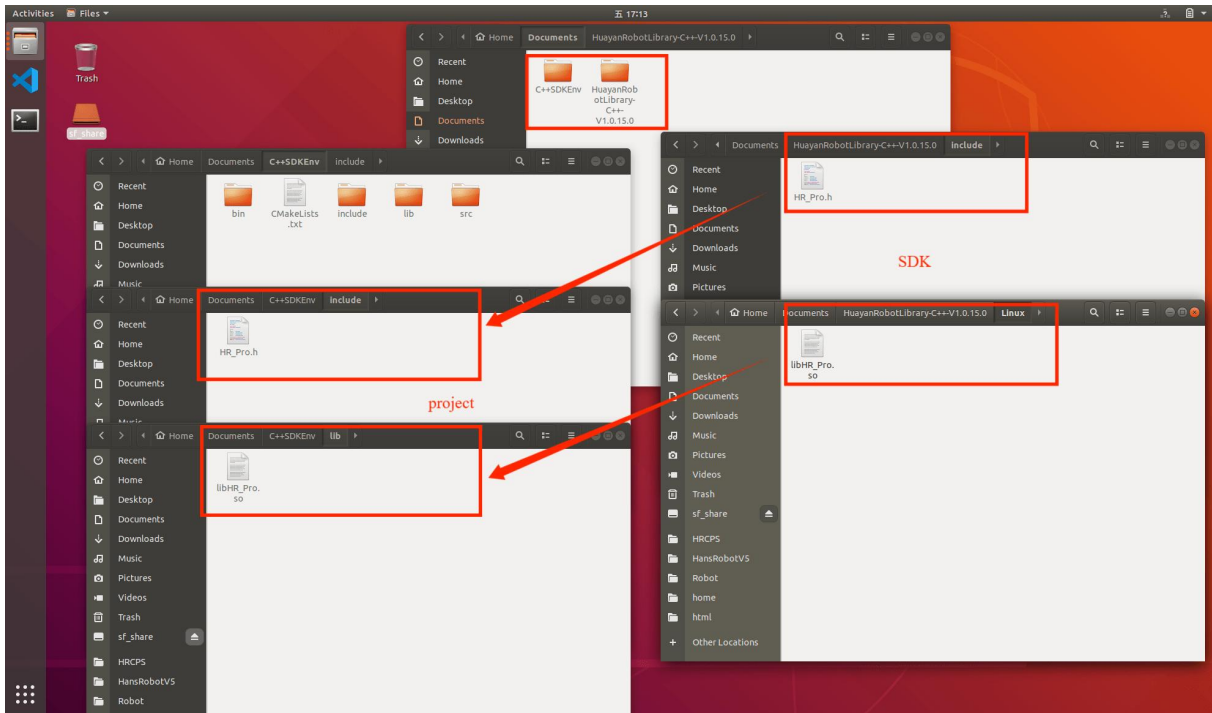


图 2.4.2 放置 SDK 文件

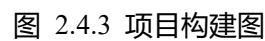
2.4.3. 链接库文件

使用 VSCode 打开工程，确保配置正确。

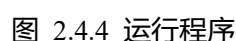
编写 CMakeLists.txt，定义编译构建规则：

1. 配置最小编译单元，将可执行文件输出目录设置为工程根目录下的 bin 目录；
2. 设置头文件目录为工程目录下的 include 目录，并通过 INCLUDE_DIRECTORIES 指令引入该目录；
3. 设置库文件目录为工程源目录下面的 lib 目录；
4. 设置编译源文件为工程源目录下面的 src/main.cpp 文件，并添加编译目标为可执行文件；
5. 链接库目录下的库文件 libHR_Pro.so。

在 main.cpp 文件中编写简单连接到机器人，选择对应的编译器并执行构建操作，完成项目编译，项目构建流程如下图所示。



在程序中使用 HR_Pro.h 文件，调用连接函数，连接机器人的 IP 的 10003 端口，进行编译运行，基本运行环境搭建完成如下图所示。



第三章 可使用接口

3.1. 初始化指令

3.1.1. HRIF_Connect

3.1.1.1. 描述：连接机器人服务器。

✓ 输入变量

输入变量	名称	数据类型	有效范围	内容
boxID	电箱 ID	unsigned int	0~5	电箱 ID 号, 默认值 = 0
hostName	控制器 IP 地址	char*	-	根据实际设置的 IP 地址定义
nPort	端口号	unsigned short	10003	端口号为 10003

✓ 返回值

返回值	名称	数据类型	有效范围	内容
nRet	返回值	int	≥ 0 的整型值	nRet=0: 返回函数调用成功 nRet>0: 返回调用失败的错误码

3.1.1.2. 示例

// 定义控制器 IP 地址

```
const char* hostname = "192.168.0.10";
```

// 定义控制器端口号

```
unsigned short nPort = 10003;
```

// 连接机器人服务器

```
int nRet = HRIF_Connect(0, hostname, nPort);
```

3.1.2. HRIF_DisConnect

3.1.2.1. 描述：断开连接机器人服务器。

✓ 输入变量

输入变量	名称	数据类型	有效范围	内容
boxID	电箱 ID	unsigned int	0~5	电箱 ID 号, 默认值= 0

✓ 返回值

返回值	名称	数据类型	有效范围	内容
nRet	返回值	int	>=0 的整型值	nRet=0: 返回函数调用成功 nRet>0: 返回调用失败的错误码

3.1.2.2. 示例

// 断开连接机器人服务器

```
int nRet = HRIF_DisConnect(0);
```

3.1.3. HRIF_IsConnected

3.1.3.1. 描述：判断控制器是否连接。

✓ 输入变量

输入变量	名称	数据类型	有效范围	内容
boxID	电箱 ID	unsigned int	0~5	电箱 ID 号, 默认值= 0

✓ 返回值

返回值	名称	数据类型	有效范围	内容
Ret	返回值	bool	false/true	false : 控制器未连接 true : 控制器已连接

3.1.3.2. 示例

// 控制器是否连接

```
bool Ret = HRIF_IsConnected(0);
```

3.1.4. HRIF_ShutdownRobot

3.1.4.1. 描述：控制器断电（断开机器人供电，系统关机）。

✓ 输入变量

输入变量	名称	数据类型	有效范围	内容
boxID	电箱 ID	unsigned int	0~5	电箱 ID 号，默认值= 0

✓ 返回值

返回值	名称	数据类型	有效范围	内容
nRet	返回值	int	≥ 0 的整型值	nRet=0: 返回函数调用成功 nRet>0: 返回调用失败的错误码

3.1.4.2. 示例

// 控制器断电

```
int nRet = HRIF_ShutdownRobot(0);
```

3.1.5. HRIF_Connect2Box

3.1.5.1. 描述：连接控制器电箱。

✓ 输入变量

输入变量	名称	数据类型	有效范围	内容
boxID	电箱 ID	unsigned int	0~5	电箱 ID 号，默认值= 0

注：1.在机器人系统开机过程中，会自动连接电箱控制板。

2.只有电箱控制板断开连接，也就是状态机状态为 2 时，才需要调用此接口。

✓ 返回值

返回值	名称	数据类型	有效范围	内容
nRet	返回值	int	≥ 0 的整型值	nRet=0: 返回函数调用成功 nRet>0: 返回调用失败的错误码

3.1.5.2. 示例

// 连接控制器电箱

```
int nRet = HRIF_Connect2Box(0);
```

3.1.6. HRIF_Electrify

3.1.6.1. 描述：机器人上电。

✓ 输入变量

输入变量	名称	数据类型	有效范围	内容
boxID	电箱 ID	unsigned int	0~5	电箱 ID 号, 默认值= 0

✓ 返回值

返回值	名称	数据类型	有效范围	内容
nRet	返回值	int	>=0 的整型值	nRet=0: 返回函数调用成功 nRet>0: 返回调用失败的错误码

3.1.6.2. 示例

// 机器人上电

```
int nRet = HRIF_Electrify(0);
```

3.1.7. HRIF_Blackout

3.1.7.1. 描述：机器人断电。

✓ 输入变量

输入变量	名称	数据类型	有效范围	内容
boxID	电箱 ID	unsigned int	0~5	电箱 ID 号, 默认值= 0

✓ 返回值

返回值	名称	数据类型	有效范围	内容
nRet	返回值	int	>=0 的整型值	nRet=0: 返回函数调用成功 nRet>0: 返回调用失败的错误码

3.1.7.2. 示例

// 机器人断电

```
int nRet = HRIF_Blackout(0);
```


3.1.8. HRIF_Connect2Controller

3.1.8.1. 描述：连接控制器。

✓ 输入变量

输入变量	名称	数据类型	有效范围	内容
boxID	电箱 ID	unsigned int	0~5	电箱 ID 号，默认值= 0

注：连接过程中会启动主站，初始化从站，配置参数，检查配置，完成后跳转到去使能状态。

✓ 返回值

返回值	名称	数据类型	有效范围	内容
nRet	返回值	int	≥ 0 的整型值	nRet=0: 返回函数调用成功 nRet>0: 返回调用失败的错误码

3.1.8.2. 示例

// 连接控制器

```
int nRet = HRIF_Connect2Controller(0);
```

3.1.9. HRIF_IsControllerStarted

3.1.9.1. 描述：判断控制器是否启动完成。

✓ 输入变量

输入变量	名称	数据类型	有效范围	内容
boxID	电箱 ID	unsigned int	0~5	电箱 ID 号, 默认值= 0

✓ 输出变量

输出变量	名称	数据类型	有效范围	内容
nStarted	是否启动完成	int	0/1	0: 未启动 1: 已启动

✓ 返回值

返回值	名称	数据类型	有效范围	内容
nRet	返回值	int	≥ 0 的整型值	nRet=0: 返回函数调用成功 nRet>0: 返回调用失败的错误码

3.1.9.2. 示例

```
int nStarted;
```

```
// 判断机器人是否启动完成
```

```
int nRet = HRIF_IsControllerStarted(0, nStarted);
```

3.1.10. HRIF_SetSimulation

3.1.10.1. 描述：设置机器人的模拟状态。

✓ 输入变量

输入变量	名称	数据类型	有效范围	内容
boxID	电箱 ID	unsigned int	0~5	电箱 ID 号，默认值= 0
rbtID	机器人 ID	unsigned int	0~5	机器人 ID 号，默认值= 0
nSimulation	是否为模拟 机器人	int	0/1	0: 真实机器人 1: 模拟机器人

注：该指令需要在机器人断电模式下才能正常调用，否则返回 20018 错误。

✓ 返回值

返回值	名称	数据类型	有效范围	内容
nRet	返回值	int	≥ 0 的整型值	nRet=0: 返回函数调用成功 nRet>0: 返回调用失败的错误码

3.1.10.2. 示例

// 设置当前机器人为模拟模式

```
int nRet = HRIF_SetSimulation(0,0, 1);
```

3.1.11. HRIF_IsSimulateRobot

3.1.11.1. 描述：判断是否为模拟机器人。

✓ 输入变量

输入变量	名称	数据类型	有效范围	内容
boxID	电箱 ID	unsigned int	0~5	电箱 ID 号, 默认值= 0

✓ 输出变量

输出变量	名称	数据类型	有效范围	内容
nSimulateRobot	是否为模拟 机器人	int	0/1	0: 真实机器人 1: 模拟机器人

✓ 返回值

返回值	名称	数据类型	有效范围	内容
nRet	返回值	int	≥ 0 的整型值	nRet=0: 返回函数调用成功 nRet>0: 返回调用失败的错误码

3.1.11.2. 示例

```
int nSimulateRobot;
```

```
// 判断是否为模拟机器人
```

```
int nRet = HRIF_IsSimulateRobot(0, nSimulateRobot);
```

3.1.12. HRIF_ReadVersion

3.1.12.1. 描述：读取控制器版本号。

✓ 输入变量

输入变量	名称	数据类型	有效范围	内容
boxID	电箱 ID	unsigned int	0~5	电箱 ID 号，默认值= 0
rbtID	机器人 ID	unsigned int	0~5	机器人 ID 号，默认值= 0

✓ 输出变量

输出变量	名称	数据类型	有效范围	内容
strVer	整体版本号	string	-	版本号的格式以 “.” 分隔，数字的意义如下： CPSVer.controlVer. BoxVerMajor. BoxVerM ID . BoxVerMin. AlgorithmVer. ElfinFirmwareVer
nCPSVersion	CPS 版本	int	-	CPS 版本号
nCodesysVersion	控制器版本	int	-	控制器版本号
nBoxVerMajor	电箱版本	int	0~4	电箱版本号： 0：模拟电箱 1~4：电箱版本号
nBoxVerMID	控制板固件 版本	int	-	控制板固件版本
nBoxVerMin	控制板固件 版本	int	-	控制板固件版本
nAlgorithmVer	算法版本	int	-	算法版本
nElfinFirmwareVer	固件版本	int	-	固件版本

✓ 返回值

返回值	名称	数据类型	有效范围	内容
nRet	返回值	int	>=0 的整型值	nRet=0: 返回函数调用成功 nRet>0: 返回调用失败的错误码

3.1.12.2. 示例

```
// 获取整体版本号
string strVer = "";
// 获取 CPS 版本号
int nCPSVersion = 0;
// 获取控制器版本号
int nCodesysVersion = 0;
// 获取电箱版本
int nBoxVerMajor = 0;
// 获取电箱固件版本
int nBoxVerMID = 0;
// 获取电箱固件版本
int nBoxVerMin = 0;
// 获取算法版本
int nAlgorithmVer = 0;
// 获取驱动固件版本
int nElfinFirmwareVer = 0;
// 获取版本号
int nRet = HRIF_ReadVersion(0, 0, strVer, nCPSVersion, nCodesysVersion, nBoxVerMajor, nBoxVerMID,
nBoxVerMin, nAlgorithmVer, nElfinFirmwareVer);
```

3.1.13. HRIF_ReadRobotModel

3.1.13.1. 描述：读取机器人机型。

✓ 输入变量

输入变量	名称	数据类型	有效范围	内容
boxID	电箱 ID	unsigned int	0~5	电箱 ID 号, 默认值= 0

✓ 输出变量

输出变量	名称	数据类型	有效范围	内容
strModel	类型	string	-	机器人机型

✓ 返回值

返回值	名称	数据类型	有效范围	内容
nRet	返回值	int	≥ 0 的整型值	nRet=0: 返回函数调用成功 nRet>0: 返回调用失败的错误码

3.1.13.2. 示例

```
string strModel = "";
```

```
// 读取机器人机型
```

```
int nRet = HRIF_ReadRobotModel(0, strModel);
```

3.1.14.HRIF_GetErrorCodeStr

3.1.14.1. 描述：读取错误码注解

✓ 输入变量

输入变量	名称	数据类型	有效范围	内容
boxID	电箱 ID	unsigned int	0~5	电箱 ID 号, 默认值= 0
nErrorCode	错误码	int	-	错误码

✓ 输出变量

输出变量	名称	数据类型	有效范围	内容
strErrorMsg	错误码解释	string	-	错误码解释的字符串

✓ 返回值

返回值	名称	数据类型	有效范围	内容
nRet	返回值	int	>=0 的整型值	nRet=0: 返回函数调用成功 nRet>0: 返回调用失败的错误码

3.1.14.2. 示例

```
int nErrorCode = 20018;
string strErrorMsg;
// 读取错误码注解
int nRet = HRIF_GetErrorCodeStr(0,nErrorCode,strErrorMsg);
```


3.2. 轴组控制指令

3.2.1. HRIF_GrpEnable

3.2.1.1. 描述：机器人使能命令。

✓ 输入变量

输入变量	名称	数据类型	有效范围	内容
boxID	电箱 ID	unsigned int	0~5	电箱 ID 号，默认值= 0
rbtID	机器人 ID	unsigned int	0~5	机器人 ID 号，默认值= 0

✓ 返回值

返回值	名称	数据类型	有效范围	内容
nRet	返回值	int	≥ 0 的整型值	nRet=0: 返回函数调用成功 nRet>0: 返回调用失败的错误码

3.2.1.2. 示例

// 机器人使能

```
int nRet= HRIF_GrpEnable(0,0);
```

3.2.2. HRIF_GrpDisable

3.2.2.1. 描述：机器人去使能命令。

✓ 输入变量

输入变量	名称	数据类型	有效范围	内容
boxID	电箱 ID	unsigned int	0~5	电箱 ID 号，默认值= 0
rbtID	机器人 ID	unsigned int	0~5	机器人 ID 号，默认值= 0

✓ 返回值

返回值	名称	数据类型	有效范围	内容
nRet	返回值	int	>=0 的整型值	nRet=0: 返回函数调用成功 nRet>0: 返回调用失败的错误码

3.2.2.2. 示例

// 机器人去使能

```
int nRet= HRIF_GrpDisable(0,0);
```

3.2.3. HRIF_GrpReset

3.2.3.1. 描述：机器人复位命令。

✓ 输入变量

输入变量	名称	数据类型	有效范围	内容
boxID	电箱 ID	unsigned int	0~5	电箱 ID 号，默认值= 0
rbtID	机器人 ID	unsigned int	0~5	机器人 ID 号，默认值= 0

✓ 返回值

返回值	名称	数据类型	有效范围	内容
nRet	返回值	int	>=0 的整型值	nRet=0: 返回函数调用成功 nRet>0: 返回调用失败的错误码

3.2.3.2. 示例

// 机器人复位

```
int nRet= HRIF_GrpReset(0,0);
```

3.2.4. HRIF_GrpStop

3.2.4.1. 描述：机器人停止运动命令。

✓ 输入变量

输入变量	名称	数据类型	有效范围	内容
boxID	电箱 ID	unsigned int	0~5	电箱 ID 号，默认值= 0
rbtID	机器人 ID	unsigned int	0~5	机器人 ID 号，默认值= 0

✓ 返回值

返回值	名称	数据类型	有效范围	内容
nRet	返回值	int	>=0 的整型值	nRet=0: 返回函数调用成功 nRet>0: 返回调用失败的错误码

3.2.4.2. 示例

// 机器人停止运动

```
int nRet= HRIF_GrpStop(0,0);
```

3.2.5. HRIF_GrpInterrupt

3.2.5.1. 描述：机器人暂停运动命令。

✓ 输入变量

输入变量	名称	数据类型	有效范围	内容
boxID	电箱 ID	unsigned int	0~5	电箱 ID 号，默认值= 0
rbtID	机器人 ID	unsigned int	0~5	机器人 ID 号，默认值= 0

✓ 返回值

返回值	名称	数据类型	有效范围	内容
nRet	返回值	int	≥ 0 的整型值	nRet=0: 返回函数调用成功 nRet>0: 返回调用失败的错误码

3.2.5.2. 示例

// 机器人暂停运动

```
int nRet= HRIF_GrpInterrupt(0,0);
```

3.2.6. HRIF_GrpContinue

3.2.6.1. 描述：机器人继续运动命令。

✓ 输入变量

输入变量	名称	数据类型	有效范围	内容
boxID	电箱 ID	unsigned int	0~5	电箱 ID 号，默认值= 0
rbtID	机器人 ID	unsigned int	0~5	机器人 ID 号，默认值= 0

✓ 返回值

返回值	名称	数据类型	有效范围	内容
nRet	返回值	int	>=0 的整型值	nRet=0: 返回函数调用成功 nRet>0: 返回调用失败的错误码

3.2.6.2. 示例

// 机器人继续运动

```
int nRet= HRIF_GrpContinue (0,0);
```

3.2.7. HRIF_GrpCloseFreeDriver

3.2.7.1. 描述：关闭零力示教。

✓ 输入变量

输入变量	名称	数据类型	有效范围	内容
boxID	电箱 ID	unsigned int	0~5	电箱 ID 号，默认值= 0
rbtID	机器人 ID	unsigned int	0~5	机器人 ID 号，默认值= 0

✓ 返回值

返回值	名称	数据类型	有效范围	内容
nRet	返回值	int	>=0 的整型值	nRet = 0: 返回函数调用成功 nRet >0: 返回调用失败的错误码

3.2.7.2. 示例

// 机器人关闭零力示教

```
int nRet= HRIF_GrpCloseFreeDriver(0,0);
```

3.2.8. HRIF_GrpOpenFreeDriver

3.2.8.1. 描述：打开零力示教。

✓ 输入变量

输入变量	名称	数据类型	有效范围	内容
boxID	电箱 ID	unsigned int	0~5	电箱 ID 号，默认值= 0
rbtID	机器人 ID	unsigned int	0~5	机器人 ID 号，默认值= 0

✓ 返回值

返回值	名称	数据类型	有效范围	内容
nRet	返回值	int	>=0 的整型值	nRet = 0: 返回函数调用成功 nRet > 0: 返回调用失败的错误码

3.2.8.2. 示例

// 机器人打开零力示教

```
int nRet = HRIF_GrpOpenFreeDriver(0,0);
```

3.2.9. HRIF_OpenBrake

3.2.9.1. 描述：松闸。

✓ 输入变量

输入变量	名称	数据类型	有效范围	内容
boxID	电箱 ID	unsigned int	0~5	电箱 ID 号，默认值= 0
rbtID	机器人 ID	unsigned int	0~5	机器人 ID 号，默认值= 0
nAxisID	轴 ID	int	0-5	需要松闸的目标轴 ID，对应关节 J1-J6

✓ 返回值

返回值	名称	数据类型	有效范围	内容
nRet	返回值	int	>=0 的整型值	nRet = 0: 返回函数调用成功 nRet > 0: 返回调用失败的错误码

3.2.9.2. 示例

// 松闸 (关节6)

```
int nRet = HRIF_OpenBrake(0,0,5);
```

3.2.10. HRIF_CloseBrake

3.2.10.1. 描述：抱闸。

✓ 输入变量

输入变量	名称	数据类型	有效范围	内容
boxID	电箱 ID	unsigned int	0~5	电箱 ID 号，默认值= 0
rbtID	机器人 ID	unsigned int	0~5	机器人 ID 号，默认值= 0
nAxisID	轴 ID	int	0-5	需要抱闸的目标轴 ID，对应关节 J1-J6

✓ 返回值

返回值	名称	数据类型	有效范围	内容
nRet	返回值	int	>=0 的整型值	nRet = 0: 返回函数调用成功 nRet >0: 返回调用失败的错误码

3.2.10.2. 示例

// 抱闸 (关节6)

```
int nRet = HRIF_CloseBrake(0,0,5);
```

3.2.11. HRIF_ReadBrakeStatus

3.2.11.1. 描述：读取各关节松/抱闸状态。

✓ 输入变量

输入变量	名称	数据类型	有效范围	内容
boxID	电箱 ID	unsigned int	0~5	电箱 ID 号，默认值= 0
rbtID	机器人 ID	unsigned int	0~5	机器人 ID 号，默认值= 0

✓ 输出变量

输出变量	名称	数据类型	有效范围	内容
stateJ1 ~ J6	松/抱闸状态	int	0/1	对应关节 1~6 的松/抱闸状态

✓ 返回值

返回值	名称	数据类型	有效范围	内容
nRet	返回值	int	≥ 0 的整型值	nRet = 0: 返回函数调用成功 nRet > 0: 返回调用失败的错误码

3.2.11.2. 示例

```
int stateJ1; int stateJ2; int stateJ3;
int stateJ4; int stateJ5; int stateJ6;
```

```
// 读取各关节松/抱闸状态
```

```
int nRet = HRIF_ReadBrakeStatus(0,0,stateJ1,stateJ2,stateJ3,stateJ4,stateJ5,stateJ6);
```

3.2.12. HRIF_MoveToSS

3.2.12.1. 描述：移动到安全位置。

✓ 输入变量

输入变量	名称	数据类型	有效范围	内容
boxID	电箱 ID	unsigned int	0~5	电箱 ID 号，默认值= 0
rbtID	机器人 ID	unsigned int	0~5	机器人 ID 号，默认值= 0

✓ 返回值

返回值	名称	数据类型	有效范围	内容
nRet	返回值	int	≥ 0 的整型值	nRet = 0: 返回函数调用成功 nRet > 0: 返回调用失败的错误码

3.2.12.2. 示例

```
// 移动到安全位置
```

```
int nRet = HRIF_MoveToSS(0,0);
```

3.2.13. HRIF_EnterSafeGuard

3.2.13.1.描述：强制进入安全光幕（软急停）。

✓ 输入变量

参数	名称	数据类型	有效范围	内容
boxID	电箱 ID	unsigned int	0~5	电箱 ID 号，默认值= 0
rbtID	机器人 ID	unsigned int	0~5	机器人 ID 号，默认值= 0
nFlag	状态标识	int	0/1	0：退出软急停 1：进入软急停

* 注：该指令用强制进入/退出安全光幕的方式进入/退出软急停。进入软急停后，示教器或者前端界面上看到的现象是进入了安全光幕。用本指令退出软急停后，需要在示教器或者前端界面点击“光幕恢复”。

✓ 返回值

返回值	名称	数据类型	有效范围	内容
nRet	错误码	int	>0 的整型值	返回调用失败的错误码

3.2.13.2.示例

// 指定状态为进入软急停

```
int nFlag=1;
```

// 机器人进入软急停

```
int nRet=HRIF_EnterSafeGuard(0,0,nFlag);
```

3.2.14. HRIF_ReadTriStageSwitch

3.2.14.1.描述：读取三段式按钮的开关以及模式。

✓ 输入变量

参数	名称	数据类型	有效范围	内容
boxID	电箱 ID	unsigned int	0~5	电箱 ID 号，默认值= 0
rbtID	机器人 ID	unsigned int	0~5	机器人 ID 号，默认值= 0

✓ 输出变量

参数	名称	数据类型	有效范围	内容
nThreeStageEnable	三段式开关	int	0/1	0: 关闭三段式按钮 1: 开启三段式
nThreeStageMode	三段式模式	int	0/1	0: 零力示教 1: 使能

✓ 返回值

返回值	名称	数据类型	有效范围	内容
nRet	错误码	int	>0 的整型值	返回调用失败的错误码

3.2.14.2. 示例

// 三段式按钮开关

```
int nThreeStageEnable=0;
```

// 三段式按钮模式

```
int nThreeStageMode=0;
```

// 读取三段式按钮开关和模式

```
int nRet=HRIF_ReadTriStageSwitch(0,0,nThreeStageEnable,nThreeStageMode);
```

3.2.15. HRIF_SetTriStageSwitch

3.2.15.1.描述：设置三段式按钮的开关以及模式。

✓ 输入变量

参数	名称	数据类型	有效范围	内容
boxID	电箱 ID	unsigned int	0~5	电箱 ID 号，默认值= 0
rbtID	机器人 ID	unsigned int	0~5	机器人 ID 号，默认值= 0
nThreeStageEnable	三段式开关	int	0/1	0: 关闭三段式按钮 1: 开启三段式
nThreeStageMode	三段式模式	int	0/1	0: 零力示教 1: 使能

✓ 返回值

返回值	名称	数据类型	有效范围	内容
nRet	错误码	int	>0 的整型值	返回调用失败的错误码

3.2.15.2.示例

// 打开三段式按钮开关

```
int nThreeStageEnable=1;
```

// 设置三段式按钮模式为使能模式

```
int nThreeStageMode=1;
```

// 设置三段式按钮开关和模式

```
int nRet=HRIF_SetTriStageSwitch(0,0,nThreeStageEnable,nThreeStageMode);
```

3.2.16. HRIF_MoveAlignToZ

3.2.16.1.描述：z 轴对齐。

✓ 输入变量

输入变量	名称	数据类型	有效范围	内容
boxID	电箱 ID	int	0~5	电箱 ID 号，默认值= 0
rbtID	机器人 ID	int	0~5	机器人 ID 号，默认值= 0
sTcpName	工具坐标名称	string	-	需要设置的工具坐标名称

sUcsName	用户坐标名称	string	-	需要设置的用户坐标名称
----------	--------	--------	---	-------------

* 注：须与 HRIF_LongMoveEvent 配合使用。

✓ 输出变量

输出变量	名称	数据类型	有效范围	内容
bIsReached	到位标志	bool	true/false	true: z 轴已经对齐 false: z 轴尚未对齐
dJ1 -dJ6	关节坐标	double	-	dJ1: 关节 1 坐标, 单位[°] dJ2: 关节 2 坐标, 单位[°] dJ3: 关节 3 坐标, 单位[°] dJ4: 关节 4 坐标, 单位[°] dJ5: 关节 5 坐标, 单位[°] dJ6: 关节 6 坐标, 单位[°]

✓ 返回值

返回值	名称	数据类型	有效范围	内容
nRet	返回值	int	>=0 的整型值	nRet = 0: 返回函数调用成功 nRet > 0: 返回调用失败的错误码

3.2.16.2. 示例

// 定义工具坐标变量

```
string sTcpName = "TCP";
```

// 定义用户坐标变量

```
string sUcsName = "Base";
```

```
bool bIsReached = false;
```

// 定义计算所得的 z 轴对齐所在的关节位置

```
double dJ1 = 0;double dJ2 = 0;double dJ3 = 0;double dJ4 = 0;double dJ5 = 0;double dJ6 = 0;
```

```
while (!bIsReached)
```

```
{
    int nRet = HRIF_MoveAlignToZ(0, 0, sTcpName, sUcsName, bIsReached, dJ1, dJ2, dJ3, dJ4, dJ5, dJ6);
    if (nRet != 0)
    {
        break;
    }
}
```

// HRIF_MoveAlignToZ 需要配合 HRIF_LongMoveEvent 一同使用

```

    HRIF_LongMoveEvent(0,0);
}

```

3.2.17.HRIF_GetLoadIdentifyResult

3.2.17.1.描述：查询机器人负载辨识的结果，包括当前进度、质量及质心坐标等信息。

✓ 输入变量

输入变量	名称	数据类型	有效范围	内容
boxID	电箱 ID	unsigned int	0~5	电箱 ID 号，默认值= 0
rbtID	机器人 ID	unsigned int	0~5	机器人 ID 号，默认值= 0

✓ 输出变量

输出变量	名称	数据类型	有效范围	内容
status	负载辨识状态	int	0~7	0, 成功 1, 第一条轨迹错误 2, 第二条轨迹错误 3, 第三条轨迹错误 4, 负载辨识运行中 5, 意外停止 6, 意外退出 7, 初始化, 负载辨识未开始
progress	进度状态	int	0~100	当前负载识别进度
mass	负载质量	double	-	识别出的负载质量, 单位[Kg]
cx	质心 X 坐标	double	-	负载质心 X 坐标, 单位[mm]
cy	质心 Y 坐标	double	-	负载质心 Y 坐标, 单位[mm]
cz	质心 Z 坐标	double	-	负载质心 Z 坐标, 单位[mm]

✓ 返回值

返回值	名称	数据类型	有效范围	内容
nRet	返回值	int	>=0 的整型值	nRet = 0: 返回函数调用成功 nRet >0: 返回调用失败的错误码

3.2.17.2.示例

//读取负载识别状态 status

```
int status=7;
```

```
//读取负载识别进度
```

```
int progress=0;
```

```
//读取负载质量
```

```
double mass=0;
```

```
//读取质点 x 坐标
```

```
double cx=0;
```

```
//读取质点 y 坐标
```

```
double cy=0;
```

```
//读取质点 z 坐标
```

```
double cz=0;
```

```
// 获取负载辨识结果
```

```
int nRet = HRIF_GetLoadIdentifyResult(0,0,status,progress,mass,cx,cy,cz);
```

3.2.18.HRIF_LoadIdentify

3.2.18.1.描述：开始负载识别

✓ 输入变量

输入变量	名称	数据类型	有效范围	内容
boxID	电箱 ID	unsigned int	0~5	电箱 ID 号，默认值= 0
rbtID	机器人 ID	unsigned int	0~5	机器人 ID 号，默认值= 0

✓ 返回值

返回值	名称	数据类型	有效范围	内容
nRet	返回值	int	>=0 的整型值	nRet = 0: 返回函数调用成功 nRet >0: 返回调用失败的错误码

3.2.18.2.示例

```
// 进行负载辨识
```

```
int nRet = HRIF_LoadIdentify(0,0);
```


3.3. 脚本控制指令

3.3.1. HRIF_RunFunc

3.3.1.1. 描述：运行指定脚本函数。

✓ 输入变量

输入变量	名称	数据类型	有效范围	内容
boxID	电箱 ID	unsigned int	0~5	电箱 ID 号，默认值= 0
strFuncName	函数名称	string	-	指定脚本函数名称，对应示教器界面的函数
Param	参数表	vector<string>	-	参数列表

✓ 返回值

返回值	名称	数据类型	有效范围	内容
nRet	返回值	int	>=0 的整型值	nRet = 0: 返回函数调用成功 nRet >0: 返回调用失败的错误码

3.3.1.2. 示例

// 指定运行函数 Func_1

```
string strFuncName = "Func_1";
```

// 定义输入参数

```
vector<string> param;
```

// 运行函数 Func_1

```
int nRet= HRIF_RunFunc(0, strFuncName, param);
```

3.3.2. HRIF_StartScript

3.3.2.1. 描述：执行脚本 Main 函数，调用后执行示教器页面编译好的脚本文件 Func_main 函数。

✓ 输入变量

输入变量	名称	数据类型	有效范围	内容
boxID	电箱 ID	unsigned int	0~5	电箱 ID 号，默认值=0

✓ 返回值

返回值	名称	数据类型	有效范围	内容
nRet	返回值	int	≥ 0 的整型值	nRet = 0: 返回函数调用成功 nRet > 0: 返回调用失败的错误码

3.3.2.2. 示例

// 运行脚本 Main 函数

```
int nRet= HRIF_StartScript(0);
```

3.3.3. HRIF_StopScript

3.3.3.1. 描述：停止脚本，调用后停止示教器页面正在执行的脚本文件。

✓ 输入变量

输入变量	名称	数据类型	有效范围	内容
boxID	电箱 ID	unsigned int	0~5	电箱 ID 号，默认值= 0

✓ 返回值

返回值	名称	数据类型	有效范围	内容
nRet	返回值	int	≥ 0 的整型值	nRet = 0: 返回函数调用成功 nRet > 0: 返回调用失败的错误码

3.3.3.2. 示例

// 停止当前正在运行的脚本

```
int nRet= HRIF_StopScript(0);
```

3.3.4. HRIF_PauseScript

3.3.4.1. 描述：暂停脚本，调用后暂停示教器页面正在执行的脚本文件。

✓ 输入变量

输入变量	名称	数据类型	有效范围	内容
boxID	电箱 ID	unsigned int	0~5	电箱 ID 号，默认值=0

✓ 返回值

返回值	名称	数据类型	有效范围	内容
nRet	返回值	int	≥ 0 的整型值	nRet = 0: 返回函数调用成功 nRet > 0: 返回调用失败的错误码

3.3.4.2. 示例

// 暂停当前运行的脚本

```
int nRet= HRIF_PauseScript(0);
```

3.3.5. HRIF_ContinueScript

3.3.5.1. 描述：继续脚本，调用后继续运行示教器页面已暂停的脚本文件。

✓ 输入变量

输入变量	名称	数据类型	有效范围	内容
boxID	电箱 ID	unsigned int	0~5	电箱 ID 号，默认值= 0

✓ 返回值

返回值	名称	数据类型	有效范围	内容
nRet	返回值	int	≥ 0 的整型值	nRet = 0: 返回函数调用成功 nRet > 0: 返回调用失败的错误码

3.3.5.2. 示例

// 继续运行当前暂停的脚本

```
int nRet= HRIF_ContinueScript(0);
```

3.3.6. HRIF_SwitchScript

3.3.6.1. 描述：应用指定的脚本文件。

✓ 输入变量

输入变量	名称	数据类型	有效范围	内容
boxID	电箱 ID	unsigned int	0~5	电箱 ID 号，默认值= 0
rbtID	机器人 ID	unsigned int	0~5	机器人 ID 号，默认值= 0
strScriptName	脚本名称	string	-	以.json 为扩展名的文件

✓ 返回值

返回值	名称	数据类型	有效范围	内容
nRet	返回值	int	≥ 0 的整型值	nRet = 0: 返回函数调用成功 nRet > 0: 返回调用失败的错误码

3.3.6.2. 示例

// 脚本文件名

```
string strScriptName = "SwitchScriptTest.json";
```

// 应用名为 *strScriptName* 的脚本文件

```
int nRet= HRIF_SwitchScript(0,0,strScriptName);
```

3.3.7. HRIF_ReadDefaultScript

3.3.7.1. 描述：读取系统当前应用的脚本程序的名字。

✓ 输入变量

输入变量	名称	数据类型	有效范围	内容
boxID	电箱 ID	unsigned int	0~5	电箱 ID 号，默认值=0
rbtID	机器人 ID	unsigned int	0~5	机器人 ID 号，默认值=0

✓ 输出变量

返回值	名称	数据类型	有效范围	内容
strScriptName	脚本名称	string	-	当前正在应用的脚本程序名字，以.json 为扩展名

✓ 返回值

返回值	名称	数据类型	有效范围	内容
nRet	返回值	int	≥ 0 的整型值	nRet = 0: 返回函数调用成功 nRet > 0: 返回调用失败的错误码

3.3.7.2. 示例

// 脚本文件名

string strScriptName;

// 读取当前应用的脚本文件

int nRet= HRIF_ReadDefaultScript(0,0,strScriptName);

3.4. 电箱控制指令

3.4.1. HRIF_ReadBoxInfo

3.4.1.1. 描述：读取电箱信息。

✓ 输入变量

输入变量	名称	数据类型	有效范围	内容
boxID	电箱 ID	unsigned int	0~5	电箱 ID 号，默认值= 0
rbtID	机器人 ID	unsigned int	0~5	机器人 ID 号，默认值= 0

✓ 输出变量

输出变量	名称	数据类型	有效范围	内容
nConnected	电箱连接状态	int	0/1	0: 未连接 1: 已连接
n48V_ON	48V 电压状态	int	0/1	0: 电压<36V 1: 电压>36V
db48OUT_Voltag	48V 输出电压值	double	0~53	48V 输出电压值
db48OUT_Current	48V 输出电流值	double	-	48V 输出电流值
nRemoteBTN	远程关机按钮状态	int	0/1	0: 无信号 1: 有信号
nThreeStageBTN	三段按钮状态	int	0/1	0: 无信号 1: 有信号

✓ 返回值

返回值	名称	数据类型	有效范围	内容
nRet	返回值	int	>=0 的整型值	nRet=0: 返回函数调用成功 nRet>0: 返回调用失败的错误码

3.4.1.2. 示例

// 定义电箱是否连接

int nConnected = 0;

// 48V 上电状态


```
int n48V_ON = 0;
// 48V 电压值

double db48OUT_Voltag = 0;
// 48V 电流值

double db48OUT_Current = 0;
// 远程关机按钮状态

int nRemoteBTN = 0;
// 三段按钮状态

int nThreeStageBTN = 0;
// 读取电箱信息

int nRet= HRIF_ReadBoxInfo(0,0, nConnected, n48V_ON, db48OUT_Voltag, db48OUT_Current,
nRemoteBTN, nThreeStageBTN);
```

3.4.2. HRIF_ReadBoxCI

3.4.2.1. 描述：读取电箱控制数字输入状态。

✓ 输入变量

输入变量	名称	数据类型	有效范围	内容
boxID	电箱 ID	unsigned int	0~5	电箱 ID 号, 默认值= 0
nBit	控制数字输入位	int	0~7	控制数字输入位索引

✓ 输出变量

输出变量	名称	数据类型	有效范围	内容
nVal	控制数字输入对应位状态	int	0/1	0: 低电平 1: 高电平

✓ 返回值

返回值	名称	数据类型	有效范围	内容
nRet	返回值	int	>=0 的整型值	nRet=0: 返回函数调用成功 nRet>0: 返回调用失败的错误码

3.4.2.2. 示例

// 读取结果

```
int nVal = 0;
```

// 定义需要读取的位

```
int nBit = 0;
```

// 读取电箱 nBit 位 CI 状态

```
int nRet= HRIF_ReadBoxCI (0, nBit, nVal);
```

3.4.3. HRIF_ReadBoxDI

3.4.3.1. 描述：读取电箱通用数字输入状态。

✓ 输入变量

输入变量	名称	数据类型	有效范围	内容
boxID	电箱 ID	unsigned int	0~5	电箱 ID 号，默认值= 0
nBit	通用数字输入位	int	0~7	通用数字输入位索引

✓ 输出变量

输出变量	名称	数据类型	有效范围	内容
nVal	通用数字输入对应位状态	int	0/1	0: 低电平 1: 高电平

✓ 返回值

返回值	名称	数据类型	有效范围	内容
nRet	返回值	int	>=0 的整型值	nRet=0: 返回函数调用成功 nRet>0: 返回调用失败的错误码

3.4.3.2. 示例

// 读取结果

```
int nVal = 0;
```

// 定义需要读取的位

```
int nBit = 0;
```

// 读取电箱 nBit 位 DI 状态

```
int nRet= HRIF_ReadBoxDI(0, nBit, nVal);
```

3.4.4. HRIF_ReadBoxCO

3.4.4.1. 描述：读取电箱控制数字输出状态。

✓ 输入变量

输入变量	名称	数据类型	有效范围	内容
boxID	电箱 ID	unsigned int	0~5	电箱 ID 号, 默认值= 0
nBit	控制数字输出位	int	0~7	控制数字输出位索引

✓ 输出变量

输出变量	名称	数据类型	有效范围	内容
nVal	控制数字输出 对应位状态	int	0/1	0: 低电平 1: 高电平

✓ 返回值

返回值	名称	数据类型	有效范围	内容
nRet	返回值	int	>=0 的整型值	nRet=0: 返回函数调用成功 nRet>0: 返回调用失败的错误码

3.4.4.2. 示例

// 读取结果

```
int nVal = 0;
```

// 定义需要读取的位

```
int nBit = 0;
```

// 读取电箱 nBit 位 CO 状态

```
int nRet= HRIF_ReadBoxCO(0, nBit, nVal);
```

3.4.5. HRIF_ReadBoxDO

3.4.5.1. 描述：读取电箱通用数字输出状态。

✓ 输入变量

输入变量	名称	数据类型	有效范围	内容
boxID	电箱 ID	unsigned int	0~5	电箱 ID 号, 默认值= 0
nBit	通用数字输出位	int	0~7	通用数字输出位索引

✓ 输出变量

输出变量	名称	数据类型	有效范围	内容
nVal	通用数字输出 对应位状态	int	0/1	0: 低电平 1: 高电平

✓ 返回值

返回值	名称	数据类型	有效范围	内容
nRet	返回值	int	>=0 的整型值	nRet=0: 返回函数调用成功 nRet>0: 返回调用失败的错误码

3.4.5.2. 示例

// 读取结果

int nVal = 0;

// 定义需要读取的位

int nBit = 0;

// 读取电箱 nBit 位 DO 状态

int nRet= HRIF_ReadBoxDO(0, nBit, nVal);

3.4.6. HRIF_ReadAllBoxCI

3.4.6.1. 描述：读取电箱所有控制数字输入状态。

✓ 输入变量

输入变量	名称	数据类型	有效范围	内容
boxID	电箱 ID	int	0~5	电箱 ID 号, 默认值 =0

✓ 输出变量

输出变量	名称	数据类型	有效范围	内容
BoxCI	控制数字输入 对应位状态	vector<int>	-	BoxCI[0]~BoxCI[7] 8 个 CI 的状态值,取 0/1

✓ 返回值

返回值	名称	数据类型	有效范围	内容
nRet	返回值	int	≥0 的整型值	nRet=0: 返回函数调用成功 nRet>0: 返回调用失败的错误码

3.4.6.2. 示例

// 创建 CI 状态结构体

```
vector<int> BoxCI;
```

// 一次性读取电箱所有 CI 位状态

```
int nRet = HRIF_ReadAllBoxCI(0, BoxCI);
```

3.4.7. HRIF_ReadAllBoxDI

3.4.7.1. 描述：读取电箱所有通用数字输入状态。

✓ 输入变量

输入变量	名称	数据类型	有效范围	内容
boxID	电箱 ID	int	0~5	电箱 ID 号, 默认值 =0

✓ 输出变量

输出变量	名称	数据类型	有效范围	内容
BoxDI	控制数字输入 对应位状态	vector<int>	-	BoxDI[0]~BoxDI[7]: 8 个 DI 的状态值,取 0/1

✓ 返回值

返回值	名称	数据类型	有效范围	内容
nRet	返回值	int	≥0 的整型值	nRet=0: 返回函数调用成功 nRet>0: 返回调用失败的错误码

3.4.7.2. 示例

// 创建 DI 状态数组

```
vector<int> BoxDI;
```

// 一次性读取电箱所有 DI 位状态

```
int nRet = HRIF_ReadAllBoxDI(0, BoxDI);
```

3.4.8. HRIF_ReadAllBoxCO

3.4.8.1. 描述：读取电箱所有控制数字输出状态。

✓ 输入变量

输入变量	名称	数据类型	有效范围	内容
boxID	电箱 ID	int	0~5	电箱 ID 号, 默认值 =0

✓ 输出变量

输出变量	名称	数据类型	有效范围	内容
BoxCO	控制数字输入 对应位状态	vector<int>	-	BoxCO[0]~BoxCO[7]: 8 个 CO 的状态值,取 0/1

✓ 返回值

返回值	名称	数据类型	有效范围	内容
nRet	返回值	int	≥0 的整型值	nRet=0: 返回函数调用成功 nRet>0: 返回调用失败的错误码

3.4.8.2. 示例

// 创建 CO 状态数组

```
vector<int> BoxCO;
```

// 一次性读取电箱所有 CO 位状态

```
int nRet = HRIF_ReadAllBoxCO(0, BoxCO);
```


3.4.9. HRIF_ReadAllBoxDO

3.4.9.1. 描述：读取电箱所有通用数字输出状态。

✓ 输入变量

输入变量	名称	数据类型	有效范围	内容
boxID	电箱 ID	int	0~5	电箱 ID 号, 默认值 =0

✓ 输出变量

输出变量	名称	数据类型	有效范围	内容
BoxDO	控制数字输入 对应位状态	vector<int>	-	BoxDO[0]~BoxDO[7]: 8 个 DO 的状态值,取 0/1

✓ 返回值

返回值	名称	数据类型	有效范围	内容
nRet	返回值	int	>=0 的整型值	nRet=0: 返回函数调用成功 nRet>0: 返回调用失败的错误码

3.4.9.2. 示例

// 创建 DO 状态数组

```
vector<int> BoxDO;
```

// 一次性读取电箱所有 DO 位状态

```
int nRet = HRIF_ReadAllBoxDI(0, BoxDO);
```

3.4.10. HRIF_ReadBoxAI

3.4.10.1. 描述：读取电箱模拟量输入值。

✓ 输入变量

输入变量	名称	数据类型	有效范围	内容
boxID	电箱 ID	unsigned int	0~5	电箱 ID 号, 默认值= 0
nBit	模拟量输入通道	int	0/1	0: 模拟量通道 0 1: 模拟量通道 1

✓ 输出变量

输出变量	名称	数据类型	有效范围	内容
dVal	对应的模拟量通道值	double	0~20	电流模式:4-20mA 电压模式:0-10V

✓ 返回值

返回值	名称	数据类型	有效范围	内容
nRet	返回值	int	≥ 0 的整型值	nRet=0: 返回函数调用成功 nRet>0: 返回调用失败的错误码

3.4.10.2. 示例

// 读取结果

```
double dVal = 0;
```

// 定义需要读取的位

```
int nBit = 0;
```

// 读取电箱 nBit 通道 AI 值

```
int nRet= HRIF_ReadBoxAI(0, nBit, dVal);
```

3.4.11. HRIF_ReadBoxAO

3.4.11.1. 描述：读取电箱模拟量输出值。

✓ 输入变量

输入变量	名称	数据类型	有效范围	内容
boxID	电箱 ID	unsigned int	0~5	电箱 ID 号, 默认值= 0
nBit	模拟量输出通道	int	0/1	0: 模拟量通道 0 1: 模拟量通道 1

✓ 输出变量

输出变量	名称	数据类型	有效范围	内容
nMode	对应的模拟量通道模式	int	1/2	1: 电压模式 2: 电流模式:
dVal	对应的模拟量通道值	double	0~20	电流模式:4-20mA 电压模式:0-10V

✓ 返回值

返回值	名称	数据类型	有效范围	内容
nRet	返回值	int	>=0 的整型值	nRet=0: 返回函数调用成功 nRet>0: 返回调用失败的错误码

3.4.11.2. 示例

// 读取结果

```
double dVal = 0;
```

// 读取的当前模式

```
int nMode = 0;
```

// 定义需要读取的位

```
int nBit = 0;
```

// 读取电箱 nBit 通道 AO 值

```
int nRet= HRIF_ReadBoxAO(0, nBit, nMode, dVal);
```

3.4.12. HRIF_SetBoxCO

3.4.12.1. 描述：设置电箱控制数字输出状态。

✓ 输入变量

输入变量	名称	数据类型	有效范围	内容
boxID	电箱 ID	unsigned int	0~5	电箱 ID 号，默认值= 0
nBit	控制数字输出位	int	0~7	控制数字输出位索引
nVal	控制数字输出状态	int	0/1	0: 低电平 1: 高电平

✓ 返回值

返回值	名称	数据类型	有效范围	内容
nRet	返回值	int	>=0 的整型值	nRet=0: 返回函数调用成功 nRet>0: 返回调用失败的错误码

3.4.12.2. 示例

// 定义设置状态

```
int nVal = 0;
```

// 定义需要设置的位

```
int nBit = 0;
```

// 设置电箱第 nBit 位 CO 状态为 nVal

```
int nRet= HRIF_SetBoxCO(0, nBit, nVal);
```

3.4.13. HRIF_SetBoxDO

3.4.13.1. 描述：设置电箱通用数字输出状态。

✓ 输入变量

输入变量	名称	数据类型	有效范围	内容
boxID	电箱 ID	unsigned int	0~5	电箱 ID 号，默认值= 0
nBit	通用数字输出位	int	0~7	通用数字输出位索引
nVal	通用数字输出状态	int	0/1	0: 低电平 1: 高电平

✓ 返回值

返回值	名称	数据类型	有效范围	内容
nRet	返回值	int	>=0 的整型值	nRet=0: 返回函数调用成功 nRet>0: 返回调用失败的错误码

3.4.13.2. 示例

// 定义设置状态

```
int nVal = 0;
```

// 定义需要设置的位

```
int nBit = 0;
```

// 设置电箱第 nBit 位 DO 状态为 nVal

```
int nRet= HRIF_SetBoxDO(0, nBit,nVal);
```

3.4.14. HRIF_SetBoxAOMode

3.4.14.1. 描述：设置电箱模拟量输出模式。

✓ 输入变量

输入变量	名称	数据类型	有效范围	内容
boxID	电箱 ID	unsigned int	0~5	电箱 ID 号, 默认值= 0
nBit	模拟量通道	int	0/1	模拟量通道
nVal	模式	int	1/2	1: 电压模式 2: 电流模式

✓ 返回值：

返回值	名称	数据类型	有效范围	内容
nRet	返回值	int	>=0 的整型值	nRet=0: 返回函数调用成功 nRet>0: 返回调用失败的错误码

3.4.14.2. 示例

// 定义需要设置的通道

```
int nBit = 0;
```

// 定义设置模式

```
int nVal = 0;
```

// 设置电箱第 nBit 通道 AO 模式为 nVal

```
int nRet= HRIF_SetBoxAOMode(0, nBit , nVal);
```

3.4.15. HRIF_SetBoxAOVal

3.4.15.1. 描述：设置电箱模拟量输出值。

✓ 输入变量

输入变量	名称	数据类型	有效范围	内容
boxID	电箱 ID	unsigned int	0~5	电箱 ID 号, 默认值= 0
nBit	模拟量通道	int	0/1	模拟量通道
dVal	对应模拟量值	double	0~20	电流模式:4-20mA 电压模式:0-10V
nMode	模式	int	1/2	1: 电压模式 2: 电流模式

✓ 返回值

返回值	名称	数据类型	有效范围	内容
nRet	返回值	int	>=0 的整型值	nRet=0: 返回函数调用成功 nRet>0: 返回调用失败的错误码

3.4.15.2. 示例

// 定义设置值

```
double dVal = 0;
```

// 定义需要设置的通道

```
int nBit = 0;
```

// 定义需要设置的模式为电压模式

```
int nMode = 1;
```

// 设置电箱第 nBit 通道 AO 模式为电压, 值为 dVal

```
int nRet= HRIF_SetBoxAOVal(0, nBit, dVal, nMode);
```

3.4.16. HRIF_SetEndDO

3.4.16.1. 描述：设置末端数字输出状态。

✓ 输入变量

输入变量	名称	数据类型	有效范围	内容
boxID	电箱 ID	unsigned int	0~5	电箱 ID 号，默认值= 0
rbtID	机器人 ID	unsigned int	0~5	机器人 ID 号，默认值= 0
nBit	末端数字输出位	int	0~2	通用数字输出位索引，末端只有 3 个 DO
nVal	末端数字输出状态	int	0/1	0: 低电平 1: 高电平

✓ 返回值

返回值	名称	数据类型	有效范围	内容
nRet	返回值	int	≥ 0 的整型值	nRet=0: 返回函数调用成功 nRet>0: 返回调用失败的错误码

3.4.16.2. 示例

// 定义设置值

```
int nVal = 0;
```

// 定义需要设置的位

```
int nBit = 0;
```

// 设置末端 nBit 位 DO 状态

```
int nRet= HRIF_SetEndDO(0,0, nBit, nVal);
```


3.4.17. HRIF_SetMoveParamsAO

3.4.17.1. 描述：设置焊接模拟量输出电压。

✓ 输入变量

输入变量	名称	数据类型	有效范围	内容
boxID	电箱 ID	unsigned int	0~5	电箱 ID 号，默认值=0
rbtID	机器人 ID	unsigned int	0~5	机器人 ID 号，默认值=0
nState	通道开关状态	int	0/1	0: 关闭 1: 开启
nIndex	电压模拟量通道	int	0/1	0: 通道 AO0 1: 通道 AO1
dInitAO	初始化电压	double	0-10V	初始化电压，单位：V
dWeldingAO	焊接电压	double	0-10V	焊接电压，单位：V

✓ 返回值

返回值	名称	数据类型	有效范围	内容
nRet	返回值	int	≥0 的整型值	nRet=0: 返回函数调用成功 nRet>0: 返回调用失败的错误码

3.4.17.2. 示例

// 定义通道开关状态

```
int nState = 0;
```

// 定义电压模拟量通道

```
int nIndex = 0;
```

// 定义初始化电压

```
int dInitAO = 0;
```

// 定义实际焊接电压

```
int dWeldingAO = 0;
```

// 设置焊接模拟量输出电压

```
int nRet = HRIF_SetMoveParamsAO(0,0, nState, nIndex,dInitAO,dWeldingAO);
```

3.4.18. HRIF_ReadEndDI

3.4.18.1. 描述：读取末端数字输入状态。

✓ 输入变量

输入变量	名称	数据类型	有效范围	内容
boxID	电箱 ID	unsigned int	0~5	电箱 ID 号，默认值= 0
rbtID	机器人 ID	unsigned int	0~5	机器人 ID 号，默认值= 0
nBit	末端数字输入位	int	0~2	末端数字输入对应位状态

✓ 输出变量

输出变量	名称	数据类型	有效范围	内容
nVal	末端数字输入对应位状态	int	0/1	0: 低电平 1: 高电平

✓ 返回值

返回值	名称	数据类型	有效范围	内容
nRet	返回值	int	>=0 的整型值	nRet=0: 返回函数调用成功 nRet>0: 返回调用失败的错误码

3.4.18.2. 示例

// 读取结果

```
int nVal = 0;
```

// 定义需要读取的位

```
int nBit = 0;
```

// 读取末端 nBit 位 DI 状态

```
int nRet= HRIF_ReadEndDI(0,0, nBit, nVal);
```

3.4.19. HRIF_ReadEndDO

3.4.19.1. 描述：读取末端数字输出状态。

✓ 输入变量

输入变量	名称	数据类型	有效范围	内容
boxID	电箱 ID	unsigned int	0~5	电箱 ID 号，默认值= 0
rbtID	机器人 ID	unsigned int	0~5	机器人 ID 号，默认值= 0
nBit	末端数字输出位	int	0~2	末端数字输出对应位索引

✓ 输出变量

输出变量	名称	数据类型	有效范围	内容
nVal	末端数字输出 对应位状态	int	0/1	0: 低电平 1: 高电平

✓ 返回值

返回值	名称	数据类型	有效范围	内容
nRet	返回值	int	>=0 的整型值	nRet=0: 返回函数调用成功 nRet>0: 返回调用失败的错误码

3.4.19.2. 示例

// 读取结果

```
int nVal = 0;
```

// 定义需要读取的位

```
int nBit = 0;
```

// 读取末端 nBit 位 DO 状态

```
int nRet= HRIF_ReadEndDO(0,0, nBit, nVal);
```

3.4.20. HRIF_ReadEndAI

3.4.20.1. 描述：读取末端模拟量输入值。

✓ 输入变量

输入变量	名称	数据类型	有效范围	内容
boxID	电箱 ID	unsigned int	0~5	电箱 ID 号，默认值= 0
rbtID	机器人 ID	unsigned int	0~5	机器人 ID 号，默认值= 0
nBit	模拟量输入通道	int	0/1	0: 模拟量通道 0 1: 模拟量通道 1

✓ 输出变量

输出变量	名称	数据类型	有效范围	内容
dVal	对应的模拟量通道值	double	0~20	电流模式:4~20mA 电压模式:0-10V

✓ 返回值

返回值	名称	数据类型	有效范围	内容
nRet	返回值	int	≥ 0 的整型值	nRet=0: 返回函数调用成功 nRet>0: 返回调用失败的错误码

3.4.20.2. 示例

// 读取结果

```
double dVal = 0;
```

// 定义需要读取的位

```
int nBit = 0;
```

// 读取末端 nBit 通道 AI 值

```
int nRet= HRIF_ReadEndAI(0,0, nBit, dVal);
```

3.4.21. HRIF_ReadEndBTN

3.4.21.1. 描述：读取末端按键状态，根据搭载的末端类型，各状态表示含义会有区别。

✓ 输入变量

输入变量	名称	数据类型	有效范围	内容
boxID	电箱 ID	unsigned int	0~5	电箱 ID 号，默认值= 0
rbtID	机器人 ID	unsigned int	0~5	机器人 ID 号，默认值= 0

✓ 输出变量

输出变量	名称	数据类型	有效范围	内容
nBit1	按键 1 状态	int	0/1	0: 松开 1: 按下
nBit2	按键 2 状态	int	0/1	0: 松开 1: 按下
nBit3	按键 3 状态	int	0/1	0: 松开 1: 按下
nBit4	按键 4 状态	int	0/1	0: 松开 1: 按下

✓ 返回值

返回值	名称	数据类型	有效范围	内容
nRet	返回值	int	>=0 的整型值	nRet=0: 返回函数调用成功 nRet>0: 返回调用失败的错误码

3.4.21.2. 示例

// 定义需要读取的位

```
int nBit1 = 0; int nBit2 = 0;
```

```
int nBit3 = 0; int nBit4 = 0;
```

// 读取末端 nBit 通道 AI 值

```
int nRet= HRIF_ReadEndBTN(0,0, nBit1,nBit2,nBit3,nBit4);
```

3.4.22. HRIF_EnableEndBTN

3.4.22.1. 描述：启用/禁用末端按键。

✓ 输入变量

输入变量	名称	数据类型	有效范围	内容
boxID	电箱 ID	unsigned int	0~5	电箱 ID 号，默认值= 0
rbtID	机器人 ID	unsigned int	0~5	机器人 ID 号，默认值= 0
nStatus	模块按钮状态	int	0/1	0: 禁用 1: 启用

✓ 返回值

返回值	名称	数据类型	有效范围	内容
nRet	返回值	int	≥ 0 的整型值	nRet=0: 返回函数调用成功 nRet>0: 返回调用失败的错误码

3.4.22.2. 示例

// 定义需要设置的状态

```
int nStatus = 0;
```

// 启用末端按钮

```
int nRet = HRIF_EnableEndBTN(0,0,nStatus);
```

3.4.23. HRIF_cdsSetIO

3.4.23.1.描述：在运动指令到达目标点位前设置 IO。

✓ 输入变量

参数	名称	数据类型	有效范围	内容
boxID	电箱 ID	unsigned int	0~5	电箱 ID 号，默认值= 0
rbtID	机器人 ID	unsigned int	0~5	机器人 ID 号，默认值= 0
nEndDOMask	需要更改的 EndDO 按 bit	int	-	EndDO0~EndDO7 对应到 nEndDOMask 的 bit0~bit7, 如果某个 bit 是 1, 则其对应的

	标识			EndDO 的值需要更改，否则无需更改。
nEndDOVal	各个需要更改的 EndDO 的目标状态	int	-	EndDO0~EndDO7 对应到 nEndDOVal 的 bit0~bit7。 0: 低电平 1: 高电平
nBoxDOMask	需要更改的 BoxDO 按 bit 标识	int	-	BoxDO0~BoxDO7 对应到 nBoxDOMask 的 bit0~bit7，如果某个 bit 是 1，则其对应的 BoxDO 的值需要更改，否则无需更改。
nBoxDOVal	各个需要更改的 BoxDO 的目标状态	int	-	BoxDO0~BoxDO7 对应到 nBoxDOVal 的 bit0~bit7。 0: 低电平 1: 高电平
nBoxCOMask	需要更改的 BoxCO 按 bit 标识	int	-	BoxCO~BoxCO 对应到 nBoxCOMask 的 bit0~bit7，如果某个 bit 是 1，则其对应的 BoxCO 的值需要更改，否则无需更改。
nBoxCOVal	各个需要更改的 BoxCO 的目标状态	int	-	BoxCO~BoxCO 对应到 nBoxCOVal 的 bit0~bit7。 0: 低电平 1: 高电平
nBoxAOCH0_Mask	BoxAOCH0 是否需要更改的标识	int	0/1	0: 无需更改 1: 需要更改
nBoxAOCH0_Mode	模式	int	1/2	1: 电压模式 2: 电流模式
nBoxAOCH1_Mask	BoxAOCH1 是否需要更改的标识	int	0/1	0: 无需更改 1: 需要更改
nBoxAOCH1_Mode	模式	int	1/2	1: 电压模式 2: 电流模式
dbBoxAOCH0_Val	对应模拟量值	double	0~20	电流模式: 4-20mA 电压模式: 0-10V
dbBoxAOCH1_Val	对应模拟量值	double	0~20	电流模式: 4-20mA 电压模式: 0-10V

✓ 返回值

返回值	名称	数据类型	有效范围	内容
nRet	错误码	int	>=0 的整型值	返回调用失败的错误码

3.4.23.2. 示例

//需要更改的 EndDO 按 bit 标识

int nEndDOMask=1;

//各个需要更改的 EndDO 的目标状态

int nEndDOVal=0;

//需要更改的 BoxDO 按 bit 标识

int nBoxDOMask=0;

//各个需要更改的 BoxDO 的目标状态

int nBoxDOVal=0;

//需要更改的 BoxCO 按 bit 标识

int nBoxCOMask=0;

//各个需要更改的 BoxCO 的目标状态

int nBoxCOVal=0;

//BoxAOCH0 是否需要更改的标识

int nBoxAOCH0_Mask=0;

//模式

int nBoxAOCH0_Mode=0;

//BoxAOCH1 是否需要更改的标识

int nBoxAOCH1_Mask=0;

//模式

int nBoxAOCH1_Mode=0;

//对应模拟量值

double dbBoxAOCH0_Val=0;

//对应模拟量值

double dbBoxAOCH1_Val=0;

//在运动指令到达目标点位前设置 IO

```
int  nRet=HRIF_cdsSetIO(0,0,nEndDOMask,      nEndDOVal,  nBoxDOMask,  nBoxDOVal,  nBoxCOMask,
nBoxCOVal,  nBoxAOCH0_Mask,  nBoxAOCH0_Mode,  nBoxAOCH1_Mask,  nBoxAOCH1_Mode,
dbBoxAOCH0_Val, dbBoxAOCH1_Val);
```


3.5. 状态读取与设置指令

3.5.1. HRIF_SetOverride

3.5.1.1. 描述：设置速度比。

✓ 输入变量

输入变量	名称	数据类型	有效范围	内容
boxID	电箱 ID	unsigned int	0~5	电箱 ID 号，默认值= 0
rbtID	机器人 ID	unsigned int	0~5	机器人 ID 号，默认值= 0
dOverride	速度比	double	0.01~1	需要设置的速度比(0.01~1)

✓ 返回值

返回值	名称	数据类型	有效范围	内容
nRet	返回值	int	≥ 0 的整型值	nRet=0: 返回函数调用成功 nRet>0: 返回调用失败的错误码

3.5.1.2. 示例

// 需要设置的速度比

```
double dOverride = 0.5;
```

// 设置当前速度比

```
int nRet= HRIF_SetOverride(0,0, dOverride);
```

3.5.2. HRIF_SetToolMotion

3.5.2.1. 描述：开启或关闭 Tool 坐标系运动模式。

✓ 输入变量

输入变量	名称	数据类型	有效范围	内容
boxID	电箱 ID	unsigned int	0~5	电箱 ID 号，默认值= 0
rbtID	机器人 ID	unsigned int	0~5	机器人 ID 号，默认值= 0
nState	状态	int	0/1	0: 关闭 1: 开启

✓ 返回值

返回值	名称	数据类型	有效范围	内容
nRet	返回值	int	≥ 0 的整型值	nRet=0: 返回函数调用成功 nRet>0: 返回调用失败的错误码

3.5.2.2. 示例

// 需要设置的 Tool 运动状态

int nState = 1;

// 设置 Tool 运动状态

int nRet= HRIF_SetToolMotion(0,0, nState);

3.5.3. HRIF_SetPayload

3.5.3.1. 描述：设置当前负载参数。

✓ 输入变量

输入变量	名称	数据类型	有效范围	内容
boxID	电箱 ID	unsigned int	0~5	电箱 ID 号，默认值= 0
rbtID	机器人 ID	unsigned int	0~5	机器人 ID 号，默认值= 0
dMass	质量	double	0~允许最大负载	质量，范围为 0 到当前允许最大负载，单位[kg]
dX	质心 X 方向偏移	double	>0	质心 X 方向偏移，单位[mm]
dY	质心 Y 方向偏移	double	>0	质心 Y 方向偏移，单位[mm]
dZ	质心 Z 方向偏移	double	>0	质心 Z 方向偏移，单位[mm]
bUpdateDB	可选参数	unsigned char	1/3	可选参数，不写默认为 1 1：修改负载，修改重心，写入数据库 3：修改负载，不修改重心，写入数据库

✓ 返回值

返回值	名称	数据类型	有效范围	内容
nRet	返回值	int	>=0 的整型值	nRet=0: 返回函数调用成功 nRet>0: 返回调用失败的错误码

3.5.3.2. 示例

```
double dMass = 1.5;
```

```
double dX = 10;
```

```
double dY = 20;
```

```
double dZ = 30;
```

```
// 设置当前负载参数
```

```
int nRet= HRIF_SetPayload(0,0,dMass,dX,dY,dZ);
```

3.5.4. HRIF_SetJointMaxVel

3.5.4.1. 描述：设置关节最大运动速度。

✓ 输入变量

输入变量	名称	数据类型	有效范围	内容
boxID	电箱 ID	unsigned int	0~5	电箱 ID 号，默认值=0
rbtID	机器人 ID	unsigned int	0~5	机器人 ID 号，默认值=0
dJ1	J1 轴最大速度	double	0~200	J1 轴最大速度，单位[°/s]
dJ2	J2 轴最大速度	double	0~200	J2 轴最大速度，单位[°/s]
dJ3	J3 轴最大速度	double	0~200	J3 轴最大速度，单位[°/s]
dJ4	J4 轴最大速度	double	0~200	J4 轴最大速度，单位[°/s]
dJ5	J5 轴最大速度	double	0~200	J5 轴最大速度，单位[°/s]
dJ6	J6 轴最大速度	double	0~200	J6 轴最大速度，单位[°/s]

* 注：关节速度有效范围需要参考具体机型。

✓ 返回值

返回值	名称	数据类型	有效范围	内容
nRet	返回值	int	>=0 的整型值	nRet=0: 返回函数调用成功 nRet>0: 返回调用失败的错误码

3.5.4.2. 示例

// 定义需要设置关节最大速度

```
double dJ1 = 180; double dJ2 = 180; double dJ3 = 180;
```

```
double dJ4 = 180; double dJ5 = 180; double dJ6 = 180;
```

// 设置最大关节速度

```
int nRet= HRIF_SetJointMaxVel(0,0, dJ1 , dJ2 , dJ3 , dJ4 , dJ5 , dJ6 );
```

3.5.5. HRIF_SetJointMaxVel_nJ

3.5.5.1. 描述：设置关节最大运动速度。**注意：这是扩展轴版本所用接口**

✓ 输入变量

输入变量	名称	数据类型	有效范围	内容
boxID	电箱 ID	unsigned int	0~5	电箱 ID 号，默认值= 0
rbtID	机器人 ID	unsigned int	0~5	机器人 ID 号，默认值= 0
Joints	第 i 轴轴最大速度	JointsData	0~200	第 i 轴轴最大速度，单位[°/s]

* 注：关节速度有效范围需要参考具体机型。

✓ 返回值

返回值	名称	数据类型	有效范围	内容
nRet	返回值	int	>=0 的整型值	nRet=0: 返回函数调用成功 nRet>0: 返回调用失败的错误码

3.5.5.2. 示例

// 定义需要设置关节最大速度

```
JointsData Joints;
Joints.dJoint[0] = 180;
Joints.dJoint[1] = 180;
Joints.dJoint[2] = 180;
Joints.dJoint[3] = 180;
Joints.dJoint[4] = 180;
Joints.dJoint[5] = 180;
Joints.dJoint[6] = 180;
```

// 设置最大关节速度

```
int nRet = HRIF_SetJointMaxVel_nJ(0, 0, Joints);
```

3.5.6. HRIF_SetJointMaxAcc

3.5.6.1. 描述：设置关节最大运动加速度。

✓ 输入变量

输入变量	名称	数据类型	有效范围	内容
boxID	电箱 ID	unsigned int	0~5	电箱 ID 号，默认值= 0
rbtID	机器人 ID	unsigned int	0~5	机器人 ID 号，默认值= 0
dJ1	J1 轴最大加速度	double	0~720	J1 轴最大加速度，单位[°/s ²]
dJ2	J2 轴最大加速度	double	0~720	J2 轴最大加速度，单位[°/s ²]
dJ3	J3 轴最大加速度	double	0~720	J3 轴最大加速度，单位[°/s ²]
dJ4	J4 轴最大加速度	double	0~720	J4 轴最大加速度，单位[°/s ²]
dJ5	J5 轴最大加速度	double	0~720	J5 轴最大加速度，单位[°/s ²]
dJ6	J6 轴最大加速度	double	0~720	J6 轴最大加速度，单位[°/s ²]

* 注：关节加速度有效范围需要参考具体机型。

✓ 返回值

返回值	名称	数据类型	有效范围	内容
nRet	返回值	int	>=0 的整型值	nRet=0: 返回函数调用成功 nRet>0: 返回调用失败的错误码

3.5.6.2. 示例

// 定义需要设置关节最大加速度

```
double dJ1 = 100; double dJ2 = 100; double dJ3 = 100;
```

```
double dJ4 = 100; double dJ5 = 100; double dJ6 = 100;
```

// 设置最大关节加速度

```
int nRet= HRIF_SetJointMaxAcc(0,0, dJ1 , dJ2 , dJ3 , dJ4 , dJ5 , dJ6 );
```

3.5.7. HRIF_SetJointMaxAcc_nJ

3.5.7.1. 描述：设置关节最大运动加速度。**注意：这是扩展轴版本所用接口**

✓ 输入变量

输入变量	名称	数据类型	有效范围	内容
boxID	电箱 ID	unsigned int	0~5	电箱 ID 号，默认值= 0
rbtID	机器人 ID	unsigned int	0~5	机器人 ID 号，默认值= 0
Joints	第 i 轴轴最大加 速度	JointsData	0~720	第 i 轴轴最加大速度，单位[°/s²]

* 注：关节加速度有效范围需要参考具体机型。

✓ 返回值

返回值	名称	数据类型	有效范围	内容
nRet	返回值	int	>=0 的整型值	nRet=0: 返回函数调用成功 nRet>0: 返回调用失败的错误码

3.5.7.2. 示例

// 定义需要设置关节最大加速度

```
JointsData Joints;
Joints.dJoint[0] = 180;
Joints.dJoint[1] = 180;
Joints.dJoint[2] = 180;
Joints.dJoint[3] = 180;
Joints.dJoint[4] = 180;
Joints.dJoint[5] = 180;
Joints.dJoint[6] = 180;
```

// 设置最大关节加速度

```
int nRet = HRIF_SetJointMaxAcc_nJ(0, 0, Joints);
```

3.5.8. HRIF_SetLinearMaxVel

3.5.8.1. 描述：设置直线运动最大速度。

✓ 输入变量

输入变量	名称	数据类型	有效范围	内容
boxID	电箱 ID	unsigned int	0~5	电箱 ID 号，默认值= 0
rbtID	机器人 ID	unsigned int	0~5	机器人 ID 号，默认值= 0
dMaxVel	最大直线速度	double	0~3000	最大直线速度，默认[2000]，单位[mm/s]

* 注：直线速度有效范围需要参考具体机型。

✓ 返回值

返回值	名称	数据类型	有效范围	内容
nRet	返回值	int	≥ 0 的整型值	nRet=0: 返回函数调用成功 nRet>0: 返回调用失败的错误码

3.5.8.2. 示例

// 定义需要设置直线最大速度

```
double dMaxVel = 100;
```

// 设置最大直线速度

```
int nRet= HRIF_SetLinearMaxVel(0,0, dMaxVel);
```


3.5.9. HRIF_SetLinearMaxAcc

3.5.9.1. 描述：设置直线运动最大加速度。

✓ 输入变量

输入变量	名称	数据类型	有效范围	内容
boxID	电箱 ID	unsigned int	0~5	电箱 ID 号，默认值= 0
rbtID	机器人 ID	unsigned int	0~5	机器人 ID 号，默认值= 0
dMaxAcc	最大直线加速度	double	0~2500	最大直线加速度，默认[2500]，单位[mm/s ²]

* 注：直线加速度有效范围需要参考具体机型。

✓ 返回值

返回值	名称	数据类型	有效范围	内容
nRet	返回值	int	>=0 的整型值	nRet=0: 返回函数调用成功 nRet>0: 返回调用失败的错误码

3.5.9.2. 示例

// 定义需要设置直线最大速度

```
double dMaxAcc = 2500;
```

// 设置最大直线速度

```
int nRet= HRIF_SetLinearMaxAcc(0,0, dMaxAcc);
```

3.5.10. HRIF_SetMaxAcsRange

3.5.10.1. 描述：设置关节最大运动范围。

✓ 输入变量

输入变量	名称	数据类型	有效范围	内容
boxID	电箱 ID	unsigned int	0~5	电箱 ID 号，默认值= 0
rbtID	机器人 ID	unsigned int	0~5	机器人 ID 号，默认值= 0
dMaxJ1- dMaxJ6	关节最大范围	double	-	dMaxJ1:关节 1 最大运动范围，单位[°] dMaxJ2:关节 2 最大运动范围，单位[°] dMaxJ3:关节 3 最大运动范围，单位[°] dMaxJ4:关节 4 最大运动范围，单位[°] dMaxJ5:关节 5 最大运动范围，单位[°] dMaxJ6:关节 6 最大运动范围，单位[°]
dMinJ1- dMinJ6	关节最小范围	double	-	dMinJ1:关节 1 最小运动范围，单位[°] dMinJ2:关节 2 最小运动范围，单位[°] dMinJ3:关节 3 最小运动范围，单位[°] dMinJ4:关节 4 最小运动范围，单位[°] dMinJ5:关节 5 最小运动范围，单位[°] dMinJ6:关节 6 最小运动范围，单位[°]

✓ 返回值

返回值	名称	数据类型	有效范围	内容
nRet	返回值	int	>=0 的整型值	nRet=0: 返回函数调用成功 nRet>0: 返回调用失败的错误码

3.5.10.2. 示例

// 定义需要设置的关节最大运动范围

```
double dMaxJ1 = 100; double dMaxJ2 = 100; double dMaxJ3 = 100;
double dMaxJ4 = 100; double dMaxJ5 = 100; double dMaxJ6 = 100;
```

// 定义需要设置的关节最小运动范围

```
double dMinJ1 = -100; double dMinJ2 = -100; double dMinJ3 = -100;
double dMinJ4 = -100; double dMinJ5 = -100; double dMinJ6 = -100;
```

// 设置关节运动范围

```
int nRet= HRIF_SetMaxAcsRange (0,0, dMaxJ1, dMaxJ2, dMaxJ3, dMaxJ4, dMaxJ5, dMaxJ6,
dMinJ1, dMinJ2, dMinJ3, dMinJ4, dMinJ5, dMinJ6);
```

3.5.11. HRIF_SetMaxAcsRange_nJ

3.5.11.1. 描述：设置关节最大运动范围。 **注意：这是扩展轴版本所用接口**

✓ 输入变量

输入变量	名称	数据类型	有效范围	内容
boxID	电箱 ID	unsigned int	0~5	电箱 ID 号，默认值= 0
rbtID	机器人 ID	unsigned int	0~5	机器人 ID 号，默认值= 0
jointsMaxAcs	关节最大范围	JointsData	-	jointsMaxAcs.joint[i]: 关节 i 最大运动范围，单位[°]
jointsMinAcs	关节最小范围	JointsData	-	jointsMinAcs.joint[i]: 关节 i 最小运动范围，单位[°]

✓ 返回值

返回值	名称	数据类型	有效范围	内容
nRet	返回值	int	>=0 的整型值	nRet=0: 返回函数调用成功 nRet>0: 返回调用失败的错误码

3.5.11.2. 示例

// 定义需要设置的关节最大运动范围

```
JointsData jointsMaxAcs;
jointsMaxAcs.dJoint[0] = 100;
jointsMaxAcs.dJoint[1] = 100;
jointsMaxAcs.dJoint[2] = 100;
jointsMaxAcs.dJoint[3] = 100;
jointsMaxAcs.dJoint[4] = 100;
jointsMaxAcs.dJoint[5] = 100;
jointsMaxAcs.dJoint[6] = 100;
```

// 定义需要设置的关节最小运动范围

```
JointsData jointsMinAcs;  
jointsMinAcs.dJoint[0] = -100;  
jointsMinAcs.dJoint[1] = -100;  
jointsMinAcs.dJoint[2] = -100;  
jointsMinAcs.dJoint[3] = -100;  
jointsMinAcs.dJoint[4] = -100;  
jointsMinAcs.dJoint[5] = -100;  
jointsMinAcs.dJoint[6] = -100;  
  
// 设置关节运动范围  
  
int nRet = HRIF_SetMaxAcsRange_nJ(0, 0, jointsMaxAcs, jointsMinAcs);
```

3.5.12. HRIF_SetMaxPcsRange

3.5.12.1. 描述：设置空间最大运动范围。

✓ 输入变量

输入变量	名称	数据类型	有效范围	内容
boxID	电箱 ID	unsigned int	0~5	电箱 ID 号，默认值= 0
rbtID	机器人 ID	unsigned int	0~5	机器人 ID 号，默认值= 0
dMaxX- dMaxZ	XYZ 最大范围	double	-	XYZ 最大范围，单位[mm]
dMinX- dMinZ	XYZ 最小范围	double	-	XYZ 最小范围，单位[mm]
dUcs_X-dUcs_Rz	基于用户坐标	double	-	dX: X 坐标，单位[mm] dY: Y 坐标，单位[mm] dZ: Z 坐标，单位[mm] dRx: Rx 坐标，单位[°] dRy: Ry 坐标，单位[°] dRz: Rz 坐标，单位[°]

✓ 返回值

返回值	名称	数据类型	有效范围	内容
nRet	返回值	int	>=0 的整型值	nRet=0: 返回函数调用成功 nRet>0: 返回调用失败的错误码

3.5.12.2. 示例

// 定义需要设置的空间最大运动范围

```
double dMaxX = 100; double dMaxY = 100; double dMaxZ = 100;
```

// 定义需要设置的空间最小运动范围

```
double dMinX = -100; double dMinY = -100; double dMinZ = -100;
```

// 定义用户坐标变量

```
double dUcs_X = 0; double dUcs_Y = 0; double dUcs_Z = 0;
```

```
double dUcs_Rx = 0; double dUcs_Ry = 0; double dUcs_Rz = 0;
```

// 设置关节运动范围

```
int nRet= HRIF_SetMaxPcsRange (0,0, dMaxX, dMaxY, dMaxZ, dMinX, dMinY, dMinZ,  
dUcs_X, dUcs_Y, dUcs_Z, dUcs_Rx, dUcs_Ry, dUcs_Rx);
```

3.5.13. HRIF_SetCollideLevel

3.5.13.1. 描述：设置安全风险等级。

✓ 输入变量

输入变量	名称	数据类型	有效范围	内容
boxID	电箱 ID	unsigned int	0~5	电箱 ID 号，默认值= 0
rbtID	机器人 ID	unsigned int	0~5	机器人 ID 号，默认值= 0
nSafeLevel	安全风险等级	int	-	安全风险等级 (0-5) 0: 安全风险低 5: 安全风险高

✓ 返回值

返回值	名称	数据类型	有效范围	内容
nRet	返回值	int	>=0 的整型值	nRet=0: 返回函数调用成功 nRet>0: 返回调用失败的错误码

3.5.13.2. 示例

// 定义安全风险等级

int nSafeLevel = 3;

// 设置安全风险等级

int nRet= HRIF_SetCollideLevel(0,0, nSafeLevel);

3.5.14. HRIF_ReadMaxPayload

3.5.14.1. 描述：读取末端最大负载。

✓ 输入变量

输入变量	名称	数据类型	有效范围	内容
boxID	电箱 ID	unsigned int	0~5	电箱 ID 号，默认值= 0
rbtID	机器人 ID	unsigned int	0~5	机器人 ID 号，默认值= 0

✓ 输出变量

输出变量	名称	数据类型	有效范围	内容
dbMaxPayload	最大负载	double	-	末端最大负载，单位：kg

✓ 返回值

返回值	名称	数据类型	有效范围	内容
nRet	返回值	int	≥ 0 的整型值	nRet=0: 返回函数调用成功 nRet>0: 返回调用失败的错误码

3.5.14.2. 示例

// 定义最大负载值

```
double dbMaxPayload = 0;
```

// 读取末端最大负载

```
int nRet = HRIF_ReadMaxPayload(0,0,dbMaxPayload);
```

3.5.15. HRIF_ReadPayload

3.5.15.1. 描述：读取当前负载参数。

✓ 输入变量

输入变量	名称	数据类型	有效范围	内容
boxID	电箱 ID	unsigned int	0~5	电箱 ID 号，默认值= 0
rbtID	机器人 ID	unsigned int	0~5	机器人 ID 号，默认值= 0

✓ 输出变量

输出变量	名称	数据类型	有效范围	内容
dMass	质量	double	0~允许最大 负载	质量，范围为 0 到当前允许最大负载，单位 [kg]
dX	质心 X 方向偏移	double	>0	质心 X 方向偏移，单位[mm]
dY	质心 Y 方向偏移	double	>0	质心 Y 方向偏移，单位[mm]
dZ	质心 Z 方向偏移	double	>0	质心 Z 方向偏移，单位[mm]

✓ 返回值

返回值	名称	数据类型	有效范围	内容
nRet	返回值	int	>=0 的整型值	nRet=0: 返回函数调用成功 nRet>0: 返回调用失败的错误码

3.5.15.2. 示例

// 定义负载值

```
double dMass = 0;
```

// 定义最大负载值

```
double dX = 0;
```

// 定义质心 Y 方向偏移

```
double dY = 0;
```

// 定义质心 Z 方向偏移

```
double dZ = 0;
```

// 读取当前负载参数

```
int nRet = HRIF_ReadPayload(0,0,dMass,dX,dY,dZ);
```


3.5.16. HRIF_ReadOverride

3.5.16.1. 描述：读取当前速度比。

✓ 输入变量

输入变量	名称	数据类型	有效范围	内容
boxID	电箱 ID	unsigned int	0~5	电箱 ID 号，默认值= 0
rbtID	机器人 ID	unsigned int	0~5	机器人 ID 号，默认值= 0

✓ 输出变量

输出变量	名称	数据类型	有效范围	内容
dOverride	速度比	double	0~1	保存获取到的当前速度比

✓ 返回值

返回值	名称	数据类型	有效范围	内容
nRet	返回值	int	≥ 0 的整型值	nRet=0: 返回函数调用成功 nRet>0: 返回调用失败的错误码

3.5.16.2. 示例

// 定义速度比变量

```
double dOverride = 0;
```

// 读取当前速度比

```
int nRet= HRIF_ReadOverride (0, 0, dOverride);
```

3.5.17. HRIF_ReadJointMaxVel

3.5.17.1. 描述：读取关节最大运动速度。

✓ 输入变量

输入变量	名称	数据类型	有效范围	内容
boxID	电箱 ID	unsigned int	0~5	电箱 ID 号，默认值= 0
rbtID	机器人 ID	unsigned int	0~5	机器人 ID 号，默认值= 0

✓ 输出变量

输出变量	名称	数据类型	有效范围	内容
dJ1	J1 轴最大速度	double	0~200	J1 轴最大速度，单位[°/s]
dJ2	J2 轴最大速度	double	0~200	J2 轴最大速度，单位[°/s]
dJ3	J3 轴最大速度	double	0~200	J3 轴最大速度，单位[°/s]
dJ4	J4 轴最大速度	double	0~200	J4 轴最大速度，单位[°/s]
dJ5	J5 轴最大速度	double	0~200	J5 轴最大速度，单位[°/s]
dJ6	J6 轴最大速度	double	0~200	J6 轴最大速度，单位[°/s]

✓ 返回值

返回值	名称	数据类型	有效范围	内容
nRet	返回值	int	>=0 的整型值	nRet=0: 返回函数调用成功 nRet>0: 返回调用失败的错误码

3.5.17.2. 示例

// 定义关节最大运动速度变量

```
double dJ1 = 0; double dJ2 = 0; double dJ3 = 0;
```

```
double dJ4 = 0; double dJ5 = 0; double dJ6 = 0;
```

// 读取最大关节速度

```
int nRet= HRIF_ReadJointMaxVel(0,0, dJ1 , dJ2 , dJ3 , dJ4 , dJ5 , dJ6 );
```

3.5.18. HRIF_ReadJointMaxVel_nJ

3.5.18.1. 描述：读取关节最大运动速度。**注意：这是扩展轴版本所用接口**

✓ 输入变量

输入变量	名称	数据类型	有效范围	内容
boxID	电箱 ID	unsigned int	0~5	电箱 ID 号，默认值= 0
rbtID	机器人 ID	unsigned int	0~5	机器人 ID 号，默认值= 0

✓ 输出变量

输出变量	名称	数据类型	有效范围	内容
Joints	各轴最大速度	JointsData	0~200	Joints.dJoint[i]:第 i 轴最大速度，单位[°/s]

✓ 返回值

返回值	名称	数据类型	有效范围	内容
nRet	返回值	int	>=0 的整型值	nRet=0: 返回函数调用成功 nRet>0: 返回调用失败的错误码

3.5.18.2. 示例

// 定义关节最大运动速度变量

JointsData Joints;

// 读取最大关节速度

```
int nRet = HRIF_ReadJointMaxVel_nJ(0, 0, Joints);
```

3.5.19. HRIF_ReadJointMaxAcc

3.5.19.1. 描述：读取关节最大运动加速度。

✓ 输入变量

输入变量	名称	数据类型	有效范围	内容
boxID	电箱 ID	unsigned int	0~5	电箱 ID 号，默认值= 0
rbtID	机器人 ID	unsigned int	0~5	机器人 ID 号，默认值= 0

✓ 输出变量

输出变量	名称	数据类型	有效范围	内容
dJ1	J1 轴最大加速度	double	0~720	J1 轴最大加速度，单位[°/s ²]
dJ2	J2 轴最大加速度	double	0~720	J2 轴最大加速度，单位[°/s ²]
dJ3	J3 轴最大加速度	double	0~720	J3 轴最大加速度，单位[°/s ²]
dJ4	J4 轴最大加速度	double	0~720	J4 轴最大加速度，单位[°/s ²]
dJ5	J5 轴最大加速度	double	0~720	J5 轴最大加速度，单位[°/s ²]
dJ6	J6 轴最大加速度	double	0~720	J6 轴最大加速度，单位[°/s ²]

✓ 返回值

返回值	名称	数据类型	有效范围	内容
nRet	返回值	int	>=0 的整型值	nRet=0: 返回函数调用成功 nRet>0: 返回调用失败的错误码

3.5.19.2. 示例

// 定义关节最大运动加速度变量

```
double dJ1 = 0; double dJ2 = 0; double dJ3 = 0;
```

```
double dJ4 = 0; double dJ5 = 0; double dJ6 = 0;
```

// 读取最大关节加速度

```
int nRet= HRIF_ReadJointMaxAcc(0,0, dJ1 , dJ2 , dJ3 , dJ4 , dJ5 , dJ6 );
```

3.5.20. HRIF_ReadJointMaxAcc_nJ

3.5.20.1. 描述：读取关节最大运动加速度。**注意：这是扩展轴版本所用接口**

✓ 输入变量

输入变量	名称	数据类型	有效范围	内容
boxID	电箱 ID	unsigned int	0~5	电箱 ID 号，默认值= 0
rbtID	机器人 ID	unsigned int	0~5	机器人 ID 号，默认值= 0

✓ 输出变量

输出变量	名称	数据类型	有效范围	内容
Joints	各轴最大速度	JointsData	0~200	Joints.dJoint[i]:第 i 轴最大加速度，单位[°/s²]

✓ 返回值

返回值	名称	数据类型	有效范围	内容
nRet	返回值	int	≥0 的整型值	nRet=0: 返回函数调用成功 nRet>0: 返回调用失败的错误码

3.5.20.2. 示例

// 定义关节最大运动加速度变量

JointsData Joints;

// 读取最大关节加速度

```
int nRet = HRIF_ReadJointMaxAcc_nJ(0, 0, Joints);
```

3.5.21. HRIF_ReadJointMaxJerk

3.5.21.1. 描述：读取关节最大运动加加速度。

✓ 输入变量

输入变量	名称	数据类型	有效范围	内容
boxID	电箱 ID	unsigned int	0~5	电箱 ID 号，默认值= 0
rbtID	机器人 ID	unsigned int	0~5	机器人 ID 号，默认值= 0

✓ 输出变量

输出变量	名称	数据类型	有效范围	内容
dJ1	J1 轴最大加加速度	double	0~3600	J1 轴最大加加速度，单位[°/s³]
dJ2	J2 轴最大加加速度	double	0~3600	J2 轴最大加加速度，单位[°/s³]
dJ3	J3 轴最大加加速度	double	0~3600	J3 轴最大加加速度，单位[°/s³]
dJ4	J4 轴最大加加速度	double	0~3600	J4 轴最大加加速度，单位[°/s³]
dJ5	J5 轴最大加加速度	double	0~3600	J5 轴最大加加速度，单位[°/s³]
dJ6	J6 轴最大加加速度	double	0~3600	J6 轴最大加加速度，单位[°/s³]

✓ 返回值

返回值	名称	数据类型	有效范围	内容
nRet	返回值	int	>=0 的整型值	nRet=0: 返回函数调用成功 nRet>0: 返回调用失败的错误码

3.5.21.2. 示例

// 定义关节最大运动加加速度变量

```
double dJ1 = 0; double dJ2 = 0; double dJ3 = 0;
```

```
double dJ4 = 0; double dJ5 = 0; double dJ6 = 0;
```

// 读取最大关节加加速度

```
int nRet= HRIF_ReadJointMaxJerk(0,0, dJ1 , dJ2 , dJ3 , dJ4 , dJ5 , dJ6 );
```

3.5.22. HRIF_ReadJointMaxJerk_nJ

3.5.22.1. 描述：读取关节最大运动加加速度。**注意：这是扩展轴版本所用接口**

✓ 输入变量

输入变量	名称	数据类型	有效范围	内容
boxID	电箱 ID	unsigned int	0~5	电箱 ID 号，默认值= 0
rbtID	机器人 ID	unsigned int	0~5	机器人 ID 号，默认值= 0

✓ 输出变量

输出变量	名称	数据类型	有效范围	内容
Joints	各轴最大速度	JointsData	0~200	Joints.dJoint[i]; 第 i 轴最大加加速度，单位[°/s³]

✓ 返回值

返回值	名称	数据类型	有效范围	内容
nRet	返回值	int	>=0 的整型值	nRet=0: 返回函数调用成功 nRet>0: 返回调用失败的错误码

3.5.22.2. 示例

// 定义关节最大运动加加速度变量

JointsData Joints;

// 读取最大关节加加速度

int nRet = HRIF_ReadJointMaxJerk_nJ(0, 0, Joints);

3.5.23. HRIF_ReadLinearMaxSpeed

3.5.23.1. 描述：读取直线运动最大速度参数。

✓ 输入变量

输入变量	名称	数据类型	有效范围	内容
boxID	电箱 ID	unsigned int	0~5	电箱 ID 号，默认值= 0
rbtID	机器人 ID	unsigned int	0~5	机器人 ID 号，默认值= 0

✓ 输出变量

输出变量	名称	数据类型	有效范围	内容
dMaxVel	最大直线速度	double	0~3000	最大直线速度，默认[2000]，单位[mm/ s]
dMaxAcc	最大直线加速度	double	0~2500	最大直线加速度，默认[2500]，单位[mm/ s ²]
dMaxJerk	最大直线加加速度	double	0~100000	最大直线加加速度，默认[100000]，单位[mm/s ³]

✓ 返回值

返回值	名称	数据类型	有效范围	内容
nRet	返回值	int	>=0 的整型值	nRet=0: 返回函数调用成功 nRet>0: 返回调用失败的错误码

3.5.23.2. 示例

// 定义直线最大运动速度变量

```
double dMaxVel = 0; double dMaxAcc = 0; double dMaxJerk = 0;
```

// 读取最大直线速度

```
int nRet= HRIF_ReadLinearMaxSpeed (0,0, dMaxVel , dMaxAcc, dMaxJerk);
```


3.5.24. HRIF_ReadEmergencyInfo

3.5.24.1. 描述：读取急停信息。

✓ 输入变量

输入变量	名称	数据类型	有效范围	内容
boxID	电箱 ID	unsigned int	0~5	电箱 ID 号，默认值= 0
rbtID	机器人 ID	unsigned int	0~5	机器人 ID 号，默认值= 0

✓ 输出变量

输出变量	名称	数据类型	有效范围	内容
nESTO_Error	急停错误	int	0/1	急停回路有两路，当两路信号不相同，则认为急停回路有错误，值为 1
nESTO	急停信号	int	0/1	急停信号，发生急停时，会断 48V 输出到本体的供电，但是不会断 220V 到 48V 的供电
nSafeGuard_Error	安全光幕错误	int	0/1	安全光幕回路有两路，当两路信号不相同，则认为安全光幕回路有错误，值为 1
nSafeGuard	安全光幕信号	int	0/1	安全光幕信号，发生安全光幕时，会停止机器人运动，并且不再接受运动指令，不会断本体供电

✓ 返回值

返回值	名称	数据类型	有效范围	内容
nRet	返回值	int	≥ 0 的整型值	nRet=0: 返回函数调用成功 nRet>0: 返回调用失败的错误码

3.5.24.2. 示例

// 定义直线最大运动速度变量

```
int nESTO_Error = 0; int nESTO = 0; int nSafeGuard_Error = 0; int nSafeGuard = 0;
```

// 读取最大直线速度

```
int nRet= HRIF_ReadEmergencyInfo (0,0, nESTO_Error, nESTO, nSafeGuard_Error, nSafeGuard);
```

3.5.25. HRIF_ReadRobotState

3.5.25.1. 描述：读取当前机器人状态标志。

✓ 输入变量

输入变量	名称	数据类型	有效范围	内容
boxID	电箱 ID	unsigned int	0~5	电箱 ID 号，默认值= 0
rbtID	机器人 ID	unsigned int	0~5	机器人 ID 号，默认值= 0

✓ 输出变量

输出变量	名称	数据类型	有效范围	内容
nMovingState	运动状态	int	0/1	0：机器人不处于运动状态 1：机器人运动中
nEnableState	使能状态	int	0/1	0：机器人未使能 1：机器人已使能
nErrorState	错误状态	int	0/1	0：未发生错误 1：有错误发生
nErrorCode	错误码	int	>= 0	报错错误码
nErrorAxis	错误轴 ID	int	0~5	错误轴 ID
nBreaking	抱闸状态	int	0/1	抱闸状态（松闸后受重力作用可能导致轴掉落） 0：抱闸 1：松闸
nPause	暂停状态	int	0/1	0：未处于暂停状态 1：已处于暂停状态
nEmergencyStop	急停状态	int	0/1	0：未处于急停状态 1：已处于急停状态
nSaftyGuard	安全光幕状态	int	0/1	0：未处于安全光幕状态 1：已处于安全光幕状态
nElectrify	上电状态	int	0/1	0：未上电状态 1：已上电状态
nIsConnectToBox	电箱连接状态	int	0/1	0：电箱未连接

				1: 电箱已连接
nBlendingDone	路点运动状态	int	0/1	0: 路点运动中 1: 路点运动完成
nInPos	到位状态	int	0/1	0: 机器人实际位置还没有运动到命令位置 1: 运动到位(命令与实际位置基本没有误差)

✓ 返回值

返回值	名称	数据类型	有效范围	内容
nRet	返回值	int	>=0 的整型值	nRet=0: 返回函数调用成功 nRet>0: 返回调用失败的错误码

3.5.25.2. 示例

// 定义需要读取的机器人状态变量

```
int nMovingState = 0; int nEnableState = 0; int nErrorState = 0; int nErrorCode = 0;
int nErrorAxis = 0; int nBreaking = 0; int nPause = 0; int nEmergencyStop = 0;
int nSafetyGuard = 0; int nElectrify = 0; int nIsConnectToBox = 0; int nBlendingDone = 0; int nInPos = 0;
```

// 读取状态

```
int nRet= HRIF_ReadRobotState(0,0, nMovingState, nEnableState,nErrorState, nErrorCode, nErrorAxis, nBreaking,
nPause, nEmergencyStop,nSafetyGuard, nElectrify, nIsConnectToBox, nBlendingDone, nInPos);
```

3.5.26. HRIF_ReadRobotFlags

3.5.26.1. 描述：读取当前机器人状态标志。

✓ 输入变量

输入变量	名称	数据类型	有效范围	内容
boxID	电箱 ID	unsigned int	0~5	电箱 ID 号，默认值=0
rbtID	机器人 ID	unsigned int	0~5	机器人 ID 号，默认值=0

✓ 输出变量

输出变量	名称	数据类型	有效范围	内容
nMovingState	运动状态	int	0/1	0: 机器人不处于运动状态 1: 机器人运动中
nEnableState	使能状态	int	0/1	0: 机器人未使能 1: 机器人已使能
nErrorState	错误状态	int	0/1	0: 未发生错误 1: 有错误发生
nErrorCode	错误码	int	>= 0	报错错误码
nErrorAxis	错误轴 ID	int	0~5	错误轴 ID
nBreaking	抱闸状态	int	0/1	抱闸状态 (松闸后受重力作用可能导致轴掉落) 0: 抱闸 1: 松闸
nPause	暂停状态	int	0/1	0: 未处于暂停状态 1: 已处于暂停状态
nBlendingDone	路点运动状态	int	0/1	0: 路点运动中 1: 路点运动完成

✓ 返回值

返回值	名称	数据类型	有效范围	内容
nRet	返回值	int	>=0 的整型值	nRet=0: 返回函数调用成功 nRet>0: 返回调用失败的错误码

3.5.26.2. 示例

// 定义需要读取的机器人状态变量

```
int nMovingState = 0; int nEnableState = 0; int nErrorState = 0; int nErrorCode = 0;
```

```
int nErrorAxis = 0; int nBreaking = 0; int nPause = 0; int nEmergencyStop = 0;
```

```
int nSaftyGuard = 0; int nElectrify = 0; int nIsConnectToBox = 0; int nBlendingDone = 0; int nInPos = 0;
```

// 读取状态

```
int nRet= HRIF_ReadRobotFlags(0,0, nMovingState, nEnableState,  
nErrorState, nErrorCode, nErrorAxis, nBreaking, nPause, nBlendingDone);
```

3.5.27. HRIF_ReadCurWaypointID

3.5.27.1. 描述：读取 WayPoint 当前运动 ID 号。

✓ 输入变量

输入变量	名称	数据类型	有效范围	内容
boxID	电箱 ID	unsigned int	0~5	电箱 ID 号，默认值= 0
rbtID	机器人 ID	unsigned int	0~5	机器人 ID 号，默认值= 0

✓ 输出变量

输出变量	名称	数据类型	有效范围	内容
strCurWaypointID	当前 ID	string	-	路点当前运动 ID，与 WayPoint,MoveJ, MoveL,MoveC 里设置的路点 ID 一致

✓ 返回值

返回值	名称	数据类型	有效范围	内容
nRet	返回值	int	>=0 的整型值	nRet=0: 返回函数调用成功 nRet>0: 返回调用失败的错误码

3.5.27.2. 示例

// 定义路点运动 ID 变量

```
string strCurWaypointID = "";
```

// 读取路点 ID

```
int nRet= HRIF_ReadCurWaypointID(0,0, strCurWaypointID );
```

3.5.28. HRIF_ReadAxisErrorCode

3.5.28.1. 描述：读取错误码。

✓ 输入变量

输入变量	名称	数据类型	有效范围	内容
boxID	电箱 ID	unsigned int	0~5	电箱 ID 号，默认值= 0
rbtID	机器人 ID	unsigned int	0~5	机器人 ID 号，默认值= 0

✓ 输出变量

输出变量	名称	数据类型	有效范围	内容
nErrorCode	当前错误码	int	≥ 0	当前错误码，没有错误码时= 0
nJ1	J1 轴错误码	int	≥ 0	J1 轴错误码，没有错误码时= 0
nJ2	J2 轴错误码	int	≥ 0	J2 轴错误码，没有错误码时= 0
nJ3	J3 轴错误码	int	≥ 0	J3 轴错误码，没有错误码时= 0
nJ4	J4 轴错误码	int	≥ 0	J4 轴错误码，没有错误码时= 0
nJ5	J5 轴错误码	int	≥ 0	J5 轴错误码，没有错误码时= 0
nJ6	J6 轴错误码	int	≥ 0	J6 轴错误码，没有错误码时= 0

✓ 返回值

返回值	名称	数据类型	有效范围	内容
nRet	返回值	int	≥ 0 的整型值	nRet=0: 返回函数调用成功 nRet>0: 返回调用失败的错误码

3.5.28.2. 示例

// 定义错误码变量

```
int nErrorCode = 0;
```

// 定义各轴错误码变量

```
int nJ1= 0; int nJ2= 0; int nJ3= 0; int nJ4= 0; int nJ5= 0; int nJ6= 0;
```

// 读取错误码

```
int nRet= HRIF_ReadAxisErrorCode(0,0, nErrorCode, nJ1, nJ2, nJ3, nJ4, nJ5, nJ6);
```

3.5.29. HRIF_ReadAxisErrorCode_nJ

3.5.29.1. 描述：读取错误码。**注意：这是扩展轴版本所用接口**

✓ 输入变量

输入变量	名称	数据类型	有效范围	内容
boxID	电箱 ID	unsigned int	0~5	电箱 ID 号，默认值= 0
rbtID	机器人 ID	unsigned int	0~5	机器人 ID 号，默认值= 0

✓ 输出变量

输出变量	名称	数据类型	有效范围	内容
nErrorCode	当前错误码	int	≥ 0	当前错误码，没有错误码时= 0
nAxisErrorCode	各轴错误码	JointsData	≥ 0	nAxisErrorCode.dJoint[i]: 第 i 轴错误码，没有错误码时= 0

✓ 返回值

返回值	名称	数据类型	有效范围	内容
nRet	返回值	int	≥ 0 的整型值	nRet=0: 返回函数调用成功 nRet>0: 返回调用失败的错误码

3.5.29.2. 示例

// 定义错误码变量

```
int nErrorCode = 0;
```

// 定义各轴错误码变量

```
JointsData nAxisErrorCode;
```

// 读取错误码

```
int nRet= HRIF_ReadAxisErrorCode_nJ(0,0, nErrorCode, nAxisErrorCode);
```


3.5.30. HRIF_ReadCurFSM

3.5.30.1. 描述：读取当前状态机及描述。

✓ 输入变量

输入变量	名称	数据类型	有效范围	内容
boxID	电箱 ID	unsigned int	0~5	电箱 ID 号，默认值= 0
rbtID	机器人 ID	unsigned int	0~5	机器人 ID 号，默认值= 0

✓ 输出变量

输出变量	名称	数据类型	有效范围	内容
nCurFSM	当前状态机	int	≥ 0	状态机，可以参考状态机列表
strCurFSM	当前状态机描述	string	-	当前状态机描述

✓ 返回值

返回值	名称	数据类型	有效范围	内容
nRet	返回值	int	≥ 0 的整型值	nRet=0: 返回函数调用成功 nRet>0: 返回调用失败的错误码

3.5.30.2. 示例

```
int nCurFSM = 0;
string strCurFSM = "";
// 读取当前状态机状态及描述
int nRet = HRIF_ReadCurFSM(0,0, nCurFSM, strCurFSM);
```

3.5.31. HRIF_ReadCurFSMFromCPS

3.5.31.1. 描述：读取当前状态机。

✓ 输入变量

输入变量	名称	数据类型	有效范围	内容
boxID	电箱 ID	unsigned int	0~5	电箱 ID 号，默认值= 0
rbtID	机器人 ID	unsigned int	0~5	机器人 ID 号，默认值= 0

✓ 输出变量

输出变量	名称	数据类型	有效范围	内容
nCurFSM	当前状态机	int	≥ 0	状态机，可以参考状态机列表

✓ 返回值

返回值	名称	数据类型	有效范围	内容
nRet	返回值	int	≥ 0 的整型值	nRet=0: 返回函数调用成功 nRet>0: 返回调用失败的错误码

3.5.31.2. 示例

```
int nCurFSM = 0;
```

```
// 读取当前状态机状态
```

```
int nRet = HRIF_ReadCurFSMFromCPS(0,0, nCurFSM);
```

3.5.32. HRIF_ReadPointByName

3.5.32.1. 描述：根据点位名称读取其位置信息。

✓ 输入变量

输入变量	名称	数据类型	有效范围	内容
boxID	电箱 ID	unsigned int	0~5	电箱 ID 号，默认值= 0
rbtID	机器人 ID	unsigned int	0~5	机器人 ID 号，默认值= 0
strPointName	点位名称	string		需要读取的点位名称

✓ 输出变量

输出变量	名称	数据类型	有效范围	内容
dJ1 -dJ6	关节坐标	double	-	dJ1: 关节 1 坐标, 单位[°] dJ2: 关节 2 坐标, 单位[°] dJ3: 关节 3 坐标, 单位[°] dJ4: 关节 4 坐标, 单位[°] dJ5: 关节 5 坐标, 单位[°] dJ6: 关节 6 坐标, 单位[°]
dX -dRz	迪卡尔坐标	double	-	dX: X 坐标, 单位[mm] dY: Y 坐标, 单位[mm] dZ: Z 坐标, 单位[mm] dRx: Rx 坐标, 单位[°] dRy: Ry 坐标, 单位[°] dRz: Rz 坐标, 单位[°]
dTcp_X-dTcp_Rz	当前工具坐标	double	-	dX: X 坐标, 单位[mm] dY: Y 坐标, 单位[mm] dZ: Z 坐标, 单位[mm] dRx: Rx 坐标, 单位[°] dRy: Ry 坐标, 单位[°] dRz: Rz 坐标, 单位[°]
dUcs_X-dUcs_Rz	当前用户坐标	double	-	dX: X 坐标, 单位[mm]

				dY: Y 坐标, 单位[mm] dZ: Z 坐标, 单位[mm] dRx: Rx 坐标, 单位[°] dRy: Ry 坐标, 单位[°] dRz: Rz 坐标, 单位[°]
--	--	--	--	---

✓ 返回值

返回值	名称	数据类型	有效范围	内容
nRet	返回值	Int	>=0 的整型值	nRet=0: 返回函数调用成功 nRet>0: 返回调用失败的错误码

3.5.32.2. 示例

// 定义关节位置变量

```
double dJ1 = 0; double dJ2 = 0; double dJ3 = 0;
double dJ4 = 0; double dJ5 = 0; double dJ6 = 0;
```

// 定义空间位置变量

```
double dX = 0; double dY = 0; double dZ = 0;
double dRx = 0; double dRy = 0; double dRz = 0;
```

// 定义工具坐标变量

```
double dTcp_X = 0; double dTcp_Y = 0; double dTcp_Z = 0;
double dTcp_Rx = 0; double dTcp_Ry = 0; double dTcp_Rz = 0;
```

// 定义用户坐标变量

```
double dUcs_X = 0; double dUcs_Y = 0; double dUcs_Z = 0;
double dUcs_Rx = 0; double dUcs_Ry = 0; double dUcs_Rz = 0;
```

// 读取点位 "Point_2" 的位置信息

```
int nRet= HRIF_ReadPointByName(0,0,"Point_2",dJ1 , dJ2 , dJ3 , dJ4 , dJ5 , dJ6 ,dX , dY , dZ , dRx , dRy , dRz ,
dTcp_X, dTcp_Y, dTcp_Z, dTcp_Rx, dTcp_Ry, dTcp_Rz,dUcs_X, dUcs_Y, dUcs_Z, dUcs_Rx, dUcs_Ry, dUcs_Rz);
```

3.5.33. HRIF_ReadPointByName_nJ

3.5.33.1. 描述：根据点位名称读取其位置信息。**注意：这是扩展轴版本所用接口**

✓ 输入变量

输入变量	名称	数据类型	有效范围	内容
boxID	电箱 ID	unsigned int	0~5	电箱 ID 号，默认值= 0
rbtID	机器人 ID	unsigned int	0~5	机器人 ID 号，默认值= 0
strPointName	点位名称	string		需要读取的点位名称

✓ 输出变量

输出变量	名称	数据类型	有效范围	内容
Joints	关节坐标	JointsData	-	Joints.dJoint[i]: 关节 i 坐标, 单位[°]
dX -dRz	迪卡尔坐标	double	-	dX: X 坐标, 单位[mm] dY: Y 坐标, 单位[mm] dZ: Z 坐标, 单位[mm] dRx: Rx 坐标, 单位[°] dRy: Ry 坐标, 单位[°] dRz: Rz 坐标, 单位[°]
dTcp_X-dTcp_Rz	当前工具坐标	double	-	dX: X 坐标, 单位[mm] dY: Y 坐标, 单位[mm] dZ: Z 坐标, 单位[mm] dRx: Rx 坐标, 单位[°] dRy: Ry 坐标, 单位[°] dRz: Rz 坐标, 单位[°]
dUcs_X-dUcs_Rz	当前用户坐标	double	-	dX: X 坐标, 单位[mm] dY: Y 坐标, 单位[mm] dZ: Z 坐标, 单位[mm] dRx: Rx 坐标, 单位[°] dRy: Ry 坐标, 单位[°] dRz: Rz 坐标, 单位[°]

✓ 返回值

返回值	名称	数据类型	有效范围	内容
nRet	返回值	Int	≥ 0 的整型值	nRet=0: 返回函数调用成功 nRet>0: 返回调用失败的错误码

3.5.33.2. 示例

// 定义关节位置变量

JointsData Joints;

// 定义空间位置变量

double dX = 0; double dY = 0; double dZ = 0;

double dRx = 0; double dRy = 0; double dRz = 0;

// 定义工具坐标变量

double dTcp_X = 0; double dTcp_Y = 0; double dTcp_Z = 0;

double dTcp_Rx = 0; double dTcp_Ry = 0; double dTcp_Rz = 0;

// 定义用户坐标变量

double dUcs_X = 0; double dUcs_Y = 0; double dUcs_Z = 0;

double dUcs_Rx = 0; double dUcs_Ry = 0; double dUcs_Rz = 0;

// 读取点位 "Point_2" 的位置信息

```
int nRet= HRIF_ReadPointByName_nJ(0,0, "Point_2", Joints,
    dX , dY , dZ , dRx , dRy , dRz ,
    dTcp_X, dTcp_Y, dTcp_Z, dTcp_Rx, dTcp_Ry, dTcp_Rz,
    dUcs_X, dUcs_Y, dUcs_Z, dUcs_Rx, dUcs_Ry, dUcs_Rz);
```

3.5.34. HRIF_ReadPointList

3.5.34.1. 描述：读取系统中保存的点名称列表。

✓ 输入变量

输入变量	名称	数据类型	有效范围	内容
boxID	电箱 ID	unsigned int	0~5	电箱 ID 号，默认值= 0
rbtID	机器人 ID	unsigned int	0~5	机器人 ID 号，默认值= 0

✓ 输出变量

输出变量	名称	数据类型	有效范围	内容
PointList	点名称列表	vector<string>	-	保存读取到的所有点名称

✓ 返回值

返回值	名称	数据类型	有效范围	内容
nRet	返回值	int	>=0 的整型值	nRet = 0: 返回函数调用成功 nRet > 0: 返回调用失败的错误码

3.5.34.2. 示例

// 定义保存点名称列表的变量

```
vector<string> PointList;
```

// 读取点名称列表

```
int nRet = HRIF_ReadPointList(0,0, PointList);
```

3.5.35. HRIF_ReadMaxRange

3.5.35.1. 描述：读取机器人工作范围。

✓ 输入变量

输入变量	名称	数据类型	有效范围	内容
boxID	电箱 ID	unsigned int	0~5	电箱 ID 号，默认值= 0
rbtID	机器人 ID	unsigned int	0~5	机器人 ID 号，默认值= 0

✓ 输出变量

输出变量	名称	数据类型	有效范围	内容
MaxJointRange	关节最大范围	vector<double>	-	MaxJointRange[0]:关节 1 最大运动范围, 单位[°] MaxJointRange[1]:关节 2 最大运动范围, 单位[°] MaxJointRange[2]:关节 3 最大运动范围, 单位[°] MaxJointRange[3]:关节 4 最大运动范围, 单位[°] MaxJointRange[4]:关节 5 最大运动范围, 单位[°] MaxJointRange[5]:关节 6 最大运动范围, 单位[°]
MinJointRange	关节最小范围	vector<double>	-	MinJointRange[0]:关节 1 最小运动范围, 单位[°] MinJointRange[1]:关节 2 最小运动范围, 单位[°] MinJointRange[2]:关节 3 最小运动范围, 单位[°] MinJointRange[3]:关节 4 最小运动范围, 单位[°] MinJointRange[4]:关节 5 最小运动范围, 单位[°] MinJointRange[5]:关节 6 最小运动范围, 单位[°]
MaxLinerRange	X-Rz 最大范围	vector<double>	-	XYZ 最小范围, 单位[mm] RxRyRz 最小范围, 单位[°]
MinLinerRange	X-Rz 最小范围	vector<double>	-	XYZ 最大范围, 单位[mm] RxRyRz 最大范围, 单位[°]

✓ 返回值

返回值	名称	数据类型	有效范围	内容
nRet	返回值	int	>=0 的整型值	nRet = 0: 返回函数调用成功 nRet > 0: 返回调用失败的错误码

3.5.35.2. 示例

// 定义保存工作范围的变量

```
vector<double> MaxJointRange;
vector<double> MinJointRange;
vector<double> MaxLinerRange;
vector<double> MinLinerRange;
```

// 读取机器人工作范围

```
int nRet = HRIF_ReadMaxRange(0,0, MaxJointRange,MinJointRange,MaxLinerRange,MinLinerRange);
```


3.5.36. HRIF_ReadMaxRange_nJ

3.5.36.1. 描述：读取机器人工作范围。**注意：这是扩展轴版本所用接口，7 系列版本不支持**

✓ 输入变量

输入变量	名称	数据类型	有效范围	内容
boxID	电箱 ID	unsigned int	0~5	电箱 ID 号，默认值=0
rbtID	机器人 ID	unsigned int	0~5	机器人 ID 号，默认值=0

✓ 输出变量

输出变量	名称	数据类型	有效范围	内容
MaxJointRange	关节最大范围	vector<double>	-	MaxJointRange[0]:关节 1 最大运动范围, 单位[°] MaxJointRange[1]:关节 2 最大运动范围, 单位[°] MaxJointRange[2]:关节 3 最大运动范围, 单位[°] MaxJointRange[3]:关节 4 最大运动范围, 单位[°] MaxJointRange[4]:关节 5 最大运动范围, 单位[°] MaxJointRange[5]:关节 6 最大运动范围, 单位[°]
MinJointRange	关节最小范围	vector<double>	-	MinJointRange[0]:关节 1 最小运动范围, 单位[°] MinJointRange[1]:关节 2 最小运动范围, 单位[°] MinJointRange[2]:关节 3 最小运动范围, 单位[°] MinJointRange[3]:关节 4 最小运动范围, 单位[°] MinJointRange[4]:关节 5 最小运动范围, 单位[°] MinJointRange[5]:关节 6 最小运动范围, 单位[°]
MaxLinerRange	X-Rz 最大范围	vector<double>	-	XYZ 最小范围, 单位[mm] RxRyRz 最小范围, 单位[°]
MinLinerRange	X-Rz 最小范围	vector<double>	-	XYZ 最大范围, 单位[mm] RxRyRz 最大范围, 单位[°]

✓ 返回值

返回值	名称	数据类型	有效范围	内容
nRet	返回值	int	>=0 的整型值	nRet = 0: 返回函数调用成功 nRet >0: 返回调用失败的错误码

3.5.36.2. 示例

// 定义保存工作范围的变量

```
vector<double> MaxJointRange;
```

```
vector<double> MinJointRange;
```

```
vector<double> MaxLinerRange;
```

```
vector<double> MinLinerRange;
```

// 读取机器人工作范围

```
int nRet = HRIF_ReadMaxRange_nJ(0,0, MaxJointRange,MinJointRange,MaxLinerRange,MinLinerRange);
```

3.6. 位置、速度、电流读取指令

3.6.1. HRIF_ReadActPos

3.6.1.1. 描述：读取当前实际位置信息。

✓ 输入变量

输入变量	名称	数据类型	有效范围	内容
boxID	电箱 ID	unsigned int	0~5	电箱 ID 号，默认值= 0
rbtID	机器人 ID	unsigned int	0~5	机器人 ID 号，默认值= 0

✓ 输出变量

输出变量	名称	数据类型	有效范围	内容
dX -dRz	迪卡尔坐标	double	-	dX: X 坐标, 单位[mm] dY: Y 坐标, 单位[mm] dZ: Z 坐标, 单位[mm] dRx: Rx 坐标, 单位[°] dRy: Ry 坐标, 单位[°] dRz: Rz 坐标, 单位[°]
dJ1 -dJ6	关节坐标	double	-	dJ1: 关节 1 坐标, 单位[°] dJ2: 关节 2 坐标, 单位[°] dJ3: 关节 3 坐标, 单位[°] dJ4: 关节 4 坐标, 单位[°] dJ5: 关节 5 坐标, 单位[°] dJ6: 关节 6 坐标, 单位[°]
dTcp_X-dTcp_Rz	当前工具坐标	double	-	dX: X 坐标, 单位[mm] dY: Y 坐标, 单位[mm] dZ: Z 坐标, 单位[mm] dRx: Rx 坐标, 单位[°] dRy: Ry 坐标, 单位[°] dRz: Rz 坐标, 单位[°]

dUcs_X-dUcs_Rz	当前用户坐标	double	-	dX: X 坐标, 单位[mm] dY: Y 坐标, 单位[mm] dZ: Z 坐标, 单位[mm] dRx: Rx 坐标, 单位[°] dRy: Ry 坐标, 单位[°] dRz: Rz 坐标, 单位[°]
----------------	--------	--------	---	---

✓ 返回值

返回值	名称	数据类型	有效范围	内容
nRet	返回值	int	≥ 0 的整型值	nRet=0: 返回函数调用成功 nRet>0: 返回调用失败的错误码

3.6.1.2. 示例

// 定义空间位置变量

```
double dX = 0; double dY = 0; double dZ = 0;
double dRx = 0; double dRy = 0; double dRz = 0;
```

// 定义关节位置变量

```
double dJ1 = 0; double dJ2 = 0; double dJ3 = 0;
double dJ4 = 0; double dJ5 = 0; double dJ6 = 0;
```

// 定义工具坐标变量

```
double dTcp_X = 0; double dTcp_Y = 0; double dTcp_Z = 0;
double dTcp_Rx = 0; double dTcp_Ry = 0; double dTcp_Rz = 0;
```

// 定义用户坐标变量

```
double dUcs_X = 0; double dUcs_Y = 0; double dUcs_Z = 0;
double dUcs_Rx = 0; double dUcs_Ry = 0; double dUcs_Rz = 0;
```

// 读取当前实际位置信息

```
int nRet= HRIF_ReadActPos(0,0, dX , dY , dZ , dRx , dRy , dRz , dJ1 , dJ2 , dJ3 , dJ4 , dJ5 , dJ6 ,dTcp_X, dTcp_Y,
dTcp_Z, dTcp_Rx, dTcp_Ry, dTcp_Rz,dUcs_X, dUcs_Y, dUcs_Z, dUcs_Rx, dUcs_Ry, dUcs_Rz);
```

3.6.2. HRIF_ReadActPos_nJ

3.6.2.1. 描述：读取当前实际位置信息。**注意：这是扩展轴版本所用接口**

✓ 输入变量

输入变量	名称	数据类型	有效范围	内容
boxID	电箱 ID	unsigned int	0~5	电箱 ID 号，默认值= 0
rbtID	机器人 ID	unsigned int	0~5	机器人 ID 号，默认值= 0

✓ 输出变量

输出变量	名称	数据类型	有效范围	内容
dX -dRz	迪卡尔坐标	double	-	dX: X 坐标, 单位[mm] dY: Y 坐标, 单位[mm] dZ: Z 坐标, 单位[mm] dRx: Rx 坐标, 单位[°] dRy: Ry 坐标, 单位[°] dRz: Rz 坐标, 单位[°]
Joints	关节坐标	JointsData	-	Joints.dJoint[i]: 关节 i 坐标, 单位[°]
dTcp_X- dTcp_Rz	当前工具坐标	double	-	dX: X 坐标, 单位[mm] dY: Y 坐标, 单位[mm] dZ: Z 坐标, 单位[mm] dRx: Rx 坐标, 单位[°] dRy: Ry 坐标, 单位[°] dRz: Rz 坐标, 单位[°]
dUcs_X- dUcs_Rz	当前用户坐标	double	-	dX: X 坐标, 单位[mm] dY: Y 坐标, 单位[mm] dZ: Z 坐标, 单位[mm] dRx: Rx 坐标, 单位[°] dRy: Ry 坐标, 单位[°] dRz: Rz 坐标, 单位[°]

✓ 返回值

返回值	名称	数据类型	有效范围	内容
nRet	返回值	int	≥ 0 的整型值	nRet=0: 返回函数调用成功 nRet>0: 返回调用失败的错误码

3.6.2.2. 示例

// 定义空间位置变量

```
double dX = 0; double dY = 0; double dZ = 0;  
double dRx = 0; double dRy = 0; double dRz = 0;
```

// 定义关节位置变量

```
JointsData Joints;
```

// 定义工具坐标变量

```
double dTcp_X = 0; double dTcp_Y = 0; double dTcp_Z = 0;  
double dTcp_Rx = 0; double dTcp_Ry = 0; double dTcp_Rz = 0;
```

// 定义用户坐标变量

```
double dUcs_X = 0; double dUcs_Y = 0; double dUcs_Z = 0;  
double dUcs_Rx = 0; double dUcs_Ry = 0; double dUcs_Rz = 0;
```

// 读取当前实际位置信息

```
int nRet = HRIF_ReadActPos_nJ(0,0, dX , dY , dZ , dRx , dRy , dRz , Joints ,dTcp_X, dTcp_Y, dTcp_Z, dTcp_Rx,  
dTcp_Ry, dTcp_Rz,dUcs_X, dUcs_Y, dUcs_Z, dUcs_Rx, dUcs_Ry, dUcs_Rz);
```

3.6.3. HRIF_ReadCmdJointPos

3.6.3.1. 描述：读取关节命令位置。

✓ 输入变量

输入变量	名称	数据类型	有效范围	内容
boxID	电箱 ID	unsigned int	0~5	电箱 ID 号，默认值= 0
rbtID	机器人 ID	unsigned int	0~5	机器人 ID 号，默认值= 0

✓ 输出变量

输出变量	名称	数据类型	有效范围	内容
dJ1 -dJ6	关节坐标	double	-	dJ1: 关节 1 坐标, 单位[°] dJ2: 关节 2 坐标, 单位[°] dJ3: 关节 3 坐标, 单位[°] dJ4: 关节 4 坐标, 单位[°] dJ5: 关节 5 坐标, 单位[°] dJ6: 关节 6 坐标, 单位[°]

✓ 返回值

返回值	名称	数据类型	有效范围	内容
nRet	返回值	int	>=0 的整型值	nRet=0: 返回函数调用成功 nRet>0: 返回调用失败的错误码

3.6.3.2. 示例

// 定义关节命令位置

```
double dJ1 = 0; double dJ2 = 0; double dJ3 = 0;
```

```
double dJ4 = 0; double dJ5 = 0; double dJ6 = 0;
```

// 读取关节命令位置

```
int nRet= HRIF_ReadCmdJointPos(0,0, dJ1 , dJ2 , dJ3 , dJ4 , dJ5 , dJ6 );
```

3.6.4. HRIF_ReadCmdJointPos_nJ

3.6.4.1. 描述：读取关节命令位置。**注意：这是扩展轴版本所用接口**

✓ 输入变量

输入变量	名称	数据类型	有效范围	内容
boxID	电箱 ID	unsigned int	0~5	电箱 ID 号，默认值= 0
rbtID	机器人 ID	unsigned int	0~5	机器人 ID 号，默认值= 0

✓ 输出变量

输出变量	名称	数据类型	有效范围	内容
Joints	关节坐标	JointsData	-	Joints.joint[i]: 关节 i 命令位置，单位[°]

✓ 返回值

返回值	名称	数据类型	有效范围	内容
nRet	返回值	int	>=0 的整型值	nRet=0: 返回函数调用成功 nRet>0: 返回调用失败的错误码

3.6.4.2. 示例

// 定义关节命令位置

JointsData Joints;

// 读取关节命令位置

```
int nRet = HRIF_ReadCmdJointPos_nJ(0, 0, Joints);
```


3.6.5. HRIF_ReadActJointPos

3.6.5.1. 描述：读取关节实际位置。

✓ 输入变量

输入变量	名称	数据类型	有效范围	内容
boxID	电箱 ID	unsigned int	0~5	电箱 ID 号，默认值= 0
rbtID	机器人 ID	unsigned int	0~5	机器人 ID 号，默认值= 0

✓ 输出变量

输出变量	名称	数据类型	有效范围	内容
dJ1 -dJ6	关节坐标	double	-	dJ1: 关节 1 实际位置, 单位[°] dJ2: 关节 2 实际位置, 单位[°] dJ3: 关节 3 实际位置, 单位[°] dJ4: 关节 4 实际位置, 单位[°] dJ5: 关节 5 实际位置, 单位[°] dJ6: 关节 6 实际位置, 单位[°]

✓ 返回值

返回值	名称	数据类型	有效范围	内容
nRet	返回值	int	>=0 的整型值	nRet=0: 返回函数调用成功 nRet>0: 返回调用失败的错误码

3.6.5.2. 示例

// 定义关节实际位置

```
double dJ1 = 0; double dJ2 = 0; double dJ3 = 0;
```

```
double dJ4 = 0; double dJ5 = 0; double dJ6 = 0;
```

// 读取关节实际位置

```
int nRet= HRIF_ReadActJointPos(0,0, dJ1 , dJ2 , dJ3 , dJ4 , dJ5 , dJ6 );
```

3.6.6. HRIF_ReadActJointPos_nJ

3.6.6.1. 描述：读取关节实际位置。**注意：这是扩展轴版本所用接口**

✓ 输入变量

输入变量	名称	数据类型	有效范围	内容
boxID	电箱 ID	unsigned int	0~5	电箱 ID 号，默认值= 0
rbtID	机器人 ID	unsigned int	0~5	机器人 ID 号，默认值= 0

✓ 输出变量

输出变量	名称	数据类型	有效范围	内容
Joints	关节坐标	JointsData	-	Joints.joint[i]: 关节 i 命令位置，单位[°]

✓ 返回值

返回值	名称	数据类型	有效范围	内容
nRet	返回值	int	>=0 的整型值	nRet=0: 返回函数调用成功 nRet>0: 返回调用失败的错误码

3.6.6.2. 示例

// 定义关节实际位置

JointsData Joints;

// 读取关节实际位置

int nRet= HRIF_ReadActJointPos_nJ(0,0, Joints);

3.6.7. HRIF_ReadCmdTcpPos

3.6.7.1. 描述：读取命令 TCP 位置。

✓ 输入变量

输入变量	名称	数据类型	有效范围	内容
boxID	电箱 ID	unsigned int	0~5	电箱 ID 号，默认值= 0
rbtID	机器人 ID	unsigned int	0~5	机器人 ID 号，默认值= 0

✓ 输出变量

输出变量	名称	数据类型	有效范围	内容
dX -dRz	TCP 坐标	double	-	dX: X 坐标, 单位[mm] dY: Y 坐标, 单位[mm] dZ: Z 坐标, 单位[mm] dRx: Rx 坐标, 单位[°] dRy: Ry 坐标, 单位[°] dRz: Rz 坐标, 单位[°]

✓ 返回值

返回值	名称	数据类型	有效范围	内容
nRet	返回值	int	>=0 的整型值	nRet=0: 返回函数调用成功 nRet>0: 返回调用失败的错误码

3.6.7.2. 示例

// TCP 命令位置

```
double dX = 0; double dY = 0; double dZ = 0;
double dRx = 0; double dRy = 0; double dRz = 0;
```

// 读取 TCP 命令位置

```
int nRet= HRIF_ReadCmdTcpPos(0,0, dX , dY , dZ , dRx , dRy , dRz );
```

3.6.8. HRIF_ReadActTcpPos

3.6.8.1. 描述：读取实际 TCP 位置。

✓ 输入变量

输入变量	名称	数据类型	有效范围	内容
boxID	电箱 ID	unsigned int	0~5	电箱 ID 号，默认值= 0
rbtID	机器人 ID	unsigned int	0~5	机器人 ID 号，默认值= 0

✓ 输出变量

输出变量	名称	数据类型	有效范围	内容
dX -dRz	TCP 坐标	double	-	dX: X 坐标, 单位[mm] dY: Y 坐标, 单位[mm] dZ: Z 坐标, 单位[mm] dRx: Rx 坐标, 单位[°] dRy: Ry 坐标, 单位[°] dRz: Rz 坐标, 单位[°]

✓ 返回值

返回值	名称	数据类型	有效范围	内容
nRet	返回值	int	>=0 的整型值	nRet=0: 返回函数调用成功 nRet>0: 返回调用失败的错误码

3.6.8.2. 示例

// TCP 实际位置

```
double dX = 0; double dY = 0; double dZ = 0;
double dRx = 0; double dRy = 0; double dRz = 0;
```

// 读取 TCP 实际位置

```
int nRet= HRIF_ReadActTcpPos(0,0, dX , dY , dZ , dRx , dRy , dRz );
```

3.6.9. HRIF_ReadCmdJointVel

3.6.9.1. 描述：读取关节命令速度。

✓ 输入变量

输入变量	名称	数据类型	有效范围	内容
boxID	电箱 ID	unsigned int	0~5	电箱 ID 号，默认值= 0
rbtID	机器人 ID	unsigned int	0~5	机器人 ID 号，默认值= 0

✓ 输出变量

输出变量	名称	数据类型	有效范围	内容
dJ1 -dJ6	关节速度	double	-	dJ1: 关节 1 命令速度, 单位[°/s] dJ2: 关节 2 命令速度, 单位[°/s] dJ3: 关节 3 命令速度, 单位[°/s] dJ4: 关节 4 命令速度, 单位[°/s] dJ5: 关节 5 命令速度, 单位[°/s] dJ6: 关节 6 命令速度, 单位[°/s]

✓ 返回值

返回值	名称	数据类型	有效范围	内容
nRet	返回值	int	>=0 的整型值	nRet=0: 返回函数调用成功 nRet>0: 返回调用失败的错误码

3.6.9.2. 示例

// 定义关节实际位置

```
double dJ1 = 0; double dJ2 = 0; double dJ3 = 0;
```

```
double dJ4 = 0; double dJ5 = 0; double dJ6 = 0;
```

// 读取关节实际位置

```
int nRet= HRIF_ReadCmdJointVel(0,0, dJ1 , dJ2 , dJ3 , dJ4 , dJ5 , dJ6 );
```

3.6.10. HRIF_ReadCmdJointVel_nJ

3.6.10.1. 描述：读取关节命令速度。**注意：这是扩展轴版本所用接口**

✓ 输入变量

输入变量	名称	数据类型	有效范围	内容
boxID	电箱 ID	unsigned int	0~5	电箱 ID 号，默认值= 0
rbtID	机器人 ID	unsigned int	0~5	机器人 ID 号，默认值= 0

✓ 输出变量

输出变量	名称	数据类型	有效范围	内容
Joints	关节速度	JointsData	-	Joints.dJoint[i]：关节 i 命令速度，单位[°/s]

✓ 返回值

返回值	名称	数据类型	有效范围	内容
nRet	返回值	int	≥ 0 的整型值	nRet=0: 返回函数调用成功 nRet>0: 返回调用失败的错误码

3.6.10.2. 示例

// 定义关节实际位置

JointsData Joints;

// 读取关节实际位置

```
int nRet = HRIF_ReadCmdJointVel_nJ(0, 0, Joints);
```

3.6.11. HRIF_ReadActJointVel

3.6.11.1. 描述：读取关节实际速度。

✓ 输入变量

输入变量	名称	数据类型	有效范围	内容
boxID	电箱 ID	unsigned int	0~5	电箱 ID 号，默认值= 0
rbtID	机器人 ID	unsigned int	0~5	机器人 ID 号，默认值= 0

✓ 输出变量

输出变量	名称	数据类型	有效范围	内容
dJ1 -dJ6	关节速度	double	-	dJ1: 关节 1 实际速度, 单位[°/s] dJ2: 关节 2 实际速度, 单位[°/s] dJ3: 关节 3 实际速度, 单位[°/s] dJ4: 关节 4 实际速度, 单位[°/s] dJ5: 关节 5 实际速度, 单位[°/s] dJ6: 关节 6 实际速度, 单位[°/s]

✓ 返回值

返回值	名称	数据类型	有效范围	内容
nRet	返回值	int	>=0 的整型值	nRet=0: 返回函数调用成功 nRet>0: 返回调用失败的错误码

3.6.11.2. 示例

// 定义关节实际速度

```
double dJ1 = 0; double dJ2 = 0; double dJ3 = 0;
```

```
double dJ4 = 0; double dJ5 = 0; double dJ6 = 0;
```

// 读取关节实际速度

```
int nRet= HRIF_ReadActJointVel(0,0, dJ1 , dJ2 , dJ3 , dJ4 , dJ5 , dJ6 );
```

3.6.12. HRIF_ReadActJointVel_nJ

3.6.12.1. 描述：读取关节实际速度。**注意：这是扩展轴版本所用接口**

✓ 输入变量

输入变量	名称	数据类型	有效范围	内容
boxID	电箱 ID	unsigned int	0~5	电箱 ID 号，默认值= 0
rbtID	机器人 ID	unsigned int	0~5	机器人 ID 号，默认值= 0

✓ 输出变量

输出变量	名称	数据类型	有效范围	内容
Joints	关节速度	JointsData	-	Joints.dJoint[i]：关节 i 实际速度，单位[°/s]

✓ 返回值

返回值	名称	数据类型	有效范围	内容
nRet	返回值	int	>=0 的整型值	nRet=0: 返回函数调用成功 nRet>0: 返回调用失败的错误码

3.6.12.2. 示例

// 定义关节实际速度

JointsData Joints;

// 读取关节实际速度

```
int nRet = HRIF_ReadActJointVel_nJ(0, 0, Joints);
```


3.6.13. HRIF_ReadCmdTcpVel

3.6.13.1. 描述：读取命令 TCP 速度。

✓ 输入变量

输入变量	名称	数据类型	有效范围	内容
boxID	电箱 ID	unsigned int	0~5	电箱 ID 号, 默认值= 0
rbtID	机器人 ID	unsigned int	0~5	机器人 ID 号, 默认值= 0

✓ 输出变量

输出变量	名称	数据类型	有效范围	内容
dX -dRz	TCP 速度	double	-	dX: X 速度, 单位[mm/s] dY: Y 速度, 单位[mm/s] dZ: Z 速度, 单位[mm/s] dRx: Rx 速度, 单位[°/s] dRy: Ry 速度, 单位[°/s] dRz: Rz 速度, 单位[°/s]

✓ 返回值

返回值	名称	数据类型	有效范围	内容
nRet	返回值	int	>=0 的整型值	nRet=0: 返回函数调用成功 nRet>0: 返回调用失败的错误码

3.6.13.2. 示例

// TCP 命令速度

```
double dX = 0; double dY = 0; double dZ = 0;
double dRx = 0; double dRy = 0; double dRz = 0;
```

// 读取 TCP 命令速度

```
int nRet= HRIF_ReadCmdTcpVel(0,0, dX , dY , dZ , dRx , dRy , dRz );
```

3.6.14. HRIF_ReadActTcpVel

3.6.14.1. 描述：读取实际 TCP 速度。

✓ 输入变量

输入变量	名称	数据类型	有效范围	内容
boxID	电箱 ID	unsigned int	0~5	电箱 ID 号，默认值=0
rbtID	机器人 ID	unsigned int	0~5	机器人 ID 号，默认值=0

✓ 输出变量

输出变量	名称	数据类型	有效范围	内容
dX - dRz	TCP 速度	double	-	dX: X 速度, 单位[mm/s] dY: Y 速度, 单位[mm/s] dZ: Z 速度, 单位[mm/s] dRx: Rx 速度, 单位[°/s] dRy: Ry 速度, 单位[°/s] dRz: Rz 速度, 单位[°/s]

✓ 返回值

返回值	名称	数据类型	有效范围	内容
nRet	返回值	int	>=0 的整型值	nRet=0: 返回函数调用成功 nRet>0: 返回调用失败的错误码

3.6.14.2. 示例

// TCP 实际速度

```
double dX = 0; double dY = 0; double dZ = 0;
double dRx = 0; double dRy = 0; double dRz = 0;
```

// 读取 TCP 实际速度

```
int nRet= HRIF_ReadActTcpVel(0,0, dX , dY , dZ , dRx , dRy , dRz );
```

3.6.15. HRIF_ReadCmdJointCur

3.6.15.1. 描述：读取关节命令电流。

✓ 输入变量

输入变量	名称	数据类型	有效范围	内容
boxID	电箱 ID	unsigned int	0~5	电箱 ID 号，默认值= 0
rbtID	机器人 ID	unsigned int	0~5	机器人 ID 号，默认值= 0

✓ 输出变量

输出变量	名称	数据类型	有效范围	内容
dJ1 -dJ6	关节电流	double	-	dJ1: 关节 1 命令电流, 单位[A] dJ2: 关节 2 命令电流, 单位[A] dJ3: 关节 3 命令电流, 单位[A] dJ4: 关节 4 命令电流, 单位[A] dJ5: 关节 5 命令电流, 单位[A] dJ6: 关节 6 命令电流, 单位[A]

✓ 返回值

返回值	名称	数据类型	有效范围	内容
nRet	返回值	int	>=0 的整型值	nRet=0: 返回函数调用成功 nRet>0: 返回调用失败的错误码

3.6.15.2. 示例

// 定义关节命令电流

```
double dJ1 = 0; double dJ2 = 0; double dJ3 = 0;
```

```
double dJ4 = 0; double dJ5 = 0; double dJ6 = 0;
```

// 读取关节命令电流

```
int nRet= HRIF_ReadCmdJointCur(0,0, dJ1 , dJ2 , dJ3 , dJ4 , dJ5 , dJ6 );
```

3.6.16. HRIF_ReadCmdJointCur_nJ

3.6.16.1. 描述：读取关节命令电流。**注意：这是扩展轴版本所用接口**

✓ 输入变量

输入变量	名称	数据类型	有效范围	内容
boxID	电箱 ID	unsigned int	0~5	电箱 ID 号，默认值= 0
rbtID	机器人 ID	unsigned int	0~5	机器人 ID 号，默认值= 0

✓ 输出变量

输出变量	名称	数据类型	有效范围	内容
Joints	关节电流	JointsData	-	Joints.dJoint[i]: 关节 i 命令电流, 单位[A]

✓ 返回值

返回值	名称	数据类型	有效范围	内容
nRet	返回值	int	>=0 的整型值	nRet=0: 返回函数调用成功 nRet>0: 返回调用失败的错误码

3.6.16.2. 示例

// 定义关节命令电流

JointsData Joints;

// 读取关节命令电流

```
int nRet = HRIF_ReadCmdJointCur_nJ(0, 0, Joints);
```

3.6.17. HRIF_ReadActJointCur

3.6.17.1. 描述：读取关节实际电流。

✓ 输入变量

输入变量	名称	数据类型	有效范围	内容
boxID	电箱 ID	unsigned int	0~5	电箱 ID 号，默认值= 0
rbtID	机器人 ID	unsigned int	0~5	机器人 ID 号，默认值= 0

✓ 输出变量

输出变量	名称	数据类型	有效范围	内容
dJ1 -dJ6	关节电流	double	-	dJ1: 关节 1 实际电流, 单位[A] dJ2: 关节 2 实际电流, 单位[A] dJ3: 关节 3 实际电流, 单位[A] dJ4: 关节 4 实际电流, 单位[A] dJ5: 关节 5 实际电流, 单位[A] dJ6: 关节 6 实际电流, 单位[A]

✓ 返回值

返回值	名称	数据类型	有效范围	内容
nRet	返回值	int	>=0 的整型值	nRet=0: 返回函数调用成功 nRet>0: 返回调用失败的错误码

3.6.17.2. 示例

// 定义关节实际电流

```
double dJ1 = 0; double dJ2 = 0; double dJ3 = 0;
```

```
double dJ4 = 0; double dJ5 = 0; double dJ6 = 0;
```

// 读取关节实际电流

```
int nRet= HRIF_ReadActJointCur(0,0, dJ1 , dJ2 , dJ3 , dJ4 , dJ5 , dJ6 );
```

3.6.18. HRIF_ReadActJointCur_nJ

3.6.18.1. 描述：读取关节实际电流。**注意：这是扩展轴版本所用接口**

✓ 输入变量

输入变量	名称	数据类型	有效范围	内容
boxID	电箱 ID	unsigned int	0~5	电箱 ID 号，默认值= 0
rbtID	机器人 ID	unsigned int	0~5	机器人 ID 号，默认值= 0

✓ 输出变量

输出变量	名称	数据类型	有效范围	内容
Joints	关节电流	JointsData	-	Joints.dJoint[i]: 关节 i 实际电流, 单位[A]

✓ 返回值

返回值	名称	数据类型	有效范围	内容
nRet	返回值	int	>=0 的整型值	nRet=0: 返回函数调用成功 nRet>0: 返回调用失败的错误码

3.6.18.2. 示例

// 定义关节实际电流

JointsData Joints;

// 读取关节实际电流

int nRet= HRIF_ReadActJointCur_nJ(0, 0, Joints);

3.6.19. HRIF_ReadTcpVelocity

3.6.19.1. 描述：读取 TCP 末端速度。

✓ 输入变量

输入变量	名称	数据类型	有效范围	内容
boxID	电箱 ID	unsigned int	0~5	电箱 ID 号，默认值= 0
rbtID	机器人 ID	unsigned int	0~5	机器人 ID 号，默认值= 0

✓ 输出变量

输出变量	名称	数据类型	有效范围	内容
dCmdVel	命令速度	double	-	命令速度，单位[mm/s]
dActVel	实际速度	double	-	实际速度，单位[mm/s]

✓ 返回值

返回值	名称	数据类型	有效范围	内容
nRet	返回值	int	≥ 0 的整型值	nRet=0: 返回函数调用成功 nRet>0: 返回调用失败的错误码

3.6.19.2. 示例

// 定义 TCP 速度变量

```
double dCmdVel = 0;
```

```
double dActVel = 0;
```

// 读取当前末端 TCP 速度

```
int nRet= HRIF_ReadTcpVelocity(0,0, dCmdVel, dActVel);
```

3.7. 坐标转换计算指令

3.7.1. HRIF_Quaternion2RPY

3.7.1.1. 描述：四元素转欧拉角。

✓ 输入变量

输入变量	名称	数据类型	有效范围	内容
boxID	电箱 ID	unsigned int	0~5	电箱 ID 号, 默认值= 0
rbtID	机器人 ID	unsigned int	0~5	机器人 ID 号, 默认值= 0
dQuaW	W	double	-	W
dQuaX	Xi	double	-	Xi
dQuaY	Yj	double	-	Yj
dQuaZ	Zk	double	-	Zk

✓ 输出变量

输出变量	名称	数据类型	有效范围	内容
dRx	欧拉角 Rx	double	$\geq -180, < 180$	欧拉角 Rx
dRy	欧拉角 Ry	double	$\geq -180, < 180$	欧拉角 Ry
dRz	欧拉角 Rz	double	$\geq -180, < 180$	欧拉角 Rz

✓ 返回值

返回值	名称	数据类型	有效范围	内容
nRet	返回值	int	≥ 0 的整型值	nRet=0: 返回函数调用成功 nRet>0: 返回调用失败的错误码

3.7.1.2. 示例

// 需要转换的四元素变量

```
double dQuaW = 0; double dQuaX = 0; double dQuaY = 0; double dQuaZ = 0;
```

// 转换后的欧拉角结果

```
double dRx = 0; double dRy = 0; double dRz = 0;
```

// 转换

```
int nRet= HRIF_Quaternion2RPY(0,0, dQuaW, dQuaX, dQuaY, dQuaZ, dRx , dRy , dRz );
```


3.7.2. HRIF_RPY2Quaternion

3.7.2.1. 描述：欧拉角转四元素。

✓ 输入变量

输入变量	名称	数据类型	有效范围	内容
boxID	电箱 ID	unsigned int	0~5	电箱 ID 号, 默认值= 0
rbtID	机器人 ID	unsigned int	0~5	机器人 ID 号, 默认值= 0
Rx	欧拉角 Rx	double	$\geq -180, \leq 180$	欧拉角 Rx
Ry	欧拉角 Ry	double	$\geq -180, \leq 180$	欧拉角 Ry
Rz	欧拉角 Rz	double	$\geq -180, \leq 180$	欧拉角 Rz

✓ 输出变量

输出变量	名称	数据类型	有效范围	内容
dQuaW	W	double	-	W
dQuaX	Xi	double	-	Xi
dQuaY	Yj	double	-	Yj
dQuaZ	Zk	double	-	Zk

✓ 返回值

返回值	名称	数据类型	有效范围	内容
nRet	返回值	int	≥ 0 的整型值	nRet=0: 返回函数调用成功 nRet>0: 返回调用失败的错误码

3.7.2.2. 示例

// 需要转换的转换后的欧拉角变量

```
double dRx = 0; double dRy = 0; double dRz = 0;
```

// 转换后的四元素变量

```
double dQuaW = 0; double dQuaX = 0; double dQuaY = 0; double dQuaZ = 0;
```

// 转换

```
int nRet= HRIF_RPY2Quaternion (0,0, dRx , dRy , dRz , dQuaW, dQuaX, dQuaY, dQuaZ);
```

3.7.3. HRIF_GetInverseKin

3.7.3.1. 描述：运动学逆解，由指定用户坐标系位置和工具坐标系下的迪卡尔坐标计算对应的关节坐标位置。

✓ 输入变量

输入变量	名称	数据类型	有效范围	内容
boxID	电箱 ID	unsigned int	0~5	电箱 ID 号，默认值= 0
rbtID	机器人 ID	unsigned int	0~5	机器人 ID 号，默认值= 0
dCoord_X- dCoord_Rz	需要计算逆解的目标迪卡尔位置	double	-	需要计算逆解的目标迪卡尔位置： dCoord_X: X 坐标，单位[mm] dCoord_Y: Y 坐标，单位[mm] dCoord_Z: Z 坐标，单位[mm] dCoord_Rx: Rx 坐标，单位[°] dCoord_Ry: Ry 坐标，单位[°] dCoord_Rz: Rz 坐标，单位[°]
dTcp_X-dTcp_Rz	工具坐标	double	-	目标位置是否包含工具坐标(不包含工具坐标则所有值= 0): dTcp_X: X 坐标，单位[mm] dTcp_Y: Y 坐标，单位[mm] dTcp_Z: Z 坐标，单位[mm] dTcp_Rx: Rx 坐标，单位[°] dTcp_Ry: Ry 坐标，单位[°] dTcp_Rz: Rz 坐标，单位[°]
dUcs_X-dUcs_Rz	用户坐标	double	-	目标位置是否包含用户坐标(不包含用户坐标则所有值= 0): dUcs_X: X 坐标，单位[mm] dUcs_Y: Y 坐标，单位[mm] dUcs_Z: Z 坐标，单位[mm] dUcs_Rx: Rx 坐标，单位[°] dUcs_Ry: Ry 坐标，单位[°] dUcs_Rz: Rz 坐标，单位[°]

dJ1 -dJ6	参考关节坐标, 逆解出现多个解时需要根据参考关节坐标选取最终解	double	-	dJ1: 关节 1 坐标, 单位[°] dJ2: 关节 2 坐标, 单位[°] dJ3: 关节 3 坐标, 单位[°] dJ4: 关节 4 坐标, 单位[°] dJ5: 关节 5 坐标, 单位[°] dJ6: 关节 6 坐标, 单位[°]
----------	---------------------------------	--------	---	--

✓ 输出变量

输出变量	名称	数据类型	有效范围	内容
dTargetJ1-dTargetJ6	关节坐标	double	-	dTargetJ1: 关节 1 坐标, 单位[°] dTargetJ2: 关节 2 坐标, 单位[°] dTargetJ3: 关节 3 坐标, 单位[°] dTargetJ4: 关节 4 坐标, 单位[°] dTargetJ5: 关节 5 坐标, 单位[°] dTargetJ6: 关节 6 坐标, 单位[°]

✓ 返回值

返回值	名称	数据类型	有效范围	内容
nRet	返回值	int	>=0 的整型值	nRet=0: 返回函数调用成功 nRet>0: 返回调用失败的错误码

3.7.3.2. 示例

// 定义需要转换的空间位置变量

```
double dCoord_X = 0; double dCoord_Y = 0; double dCoord_Z = 0;
double dCoord_Rx = 0; double dCoord_Ry = 0; double dCoord_Rz = 0;
```

// 定义工具坐标变量

```
double dTcp_X = 0; double dTcp_Y = 0; double dTcp_Z = 0;
double dTcp_Rx = 0; double dTcp_Ry = 0; double dTcp_Rz = 0;
```

// 定义工具坐标变量

```
double dUcs_X = 0; double dUcs_Y = 0; double dUcs_Z = 0;
double dUcs_Rx = 0; double dUcs_Ry = 0; double dUcs_Rz = 0;
```

// 定义参考关节位置变量

```
double dJ1 = 0; double dJ2 = 0; double dJ3 = 0;
```

```
double dJ4 = 0; double dJ5 = 0; double dJ6 = 0;
```

```
// 定义转换结果
```

```
double dTargetJ1 = 0; double dTargetJ2 = 0; double dTargetJ3 = 90;
```

```
double dTargetJ4 = 0; double dTargetJ5 = 90; double dTargetJ6 = 0;
```

```
// 求逆解
```

```
int nRet= HRIF_GetInverseKin(0, 0,dCoord_X, dCoord_Y, dCoord_Z, dCoord_Rx, dCoord_Ry, dCoord_Rz,
```

```
dTcp_X, dTcp_Y, dTcp_Z, dTcp_Rx, dTcp_Ry, dTcp_Rz,
```

```
dUcs_X, dUcs_Y, dUcs_Z, dUcs_Rx, dUcs_Ry, dUcs_Rz,
```

```
dJ1 , dJ2 , dJ3 , dJ4 , dJ5 , dJ6 ,
```

```
dTargetJ1, dTargetJ2, dTargetJ3, dTargetJ4, dTargetJ5, dTargetJ6);
```

3.7.4. HRIF_GetInverseKin_nJ

3.7.4.1. 描述：运动学逆解，由指定用户坐标系位置和工具坐标系下的迪卡尔坐标计算对应的关节坐标位置。

注意：这是扩展轴版本所用接口

✓ 输入变量

输入变量	名称	数据类型	有效范围	内容
boxID	电箱 ID	unsigned int	0~5	电箱 ID 号，默认值= 0
rbtID	机器人 ID	unsigned int	0~5	机器人 ID 号，默认值= 0
dCoord_X- dCoord_Rz	需要计算逆解的目标迪卡尔位置	double	-	需要计算逆解的目标迪卡尔位置： dCoord_X: X 坐标，单位[mm] dCoord_Y: Y 坐标，单位[mm] dCoord_Z: Z 坐标，单位[mm] dCoord_Rx: Rx 坐标，单位[°] dCoord_Ry: Ry 坐标，单位[°] dCoord_Rz: Rz 坐标，单位[°]
dTcp_X-dTcp_Rz	工具坐标	double	-	目标位置是否包含工具坐标(不包含工具坐标则所有值= 0): dTcp_X: X 坐标，单位[mm] dTcp_Y: Y 坐标，单位[mm] dTcp_Z: Z 坐标，单位[mm] dTcp_Rx: Rx 坐标，单位[°] dTcp_Ry: Ry 坐标，单位[°] dTcp_Rz: Rz 坐标，单位[°]
dUcs_X-dUcs_Rz	用户坐标	double	-	目标位置是否包含用户坐标(不包含用户坐标则所有值= 0): dUcs_X: X 坐标，单位[mm] dUcs_Y: Y 坐标，单位[mm] dUcs_Z: Z 坐标，单位[mm] dUcs_Rx: Rx 坐标，单位[°] dUcs_Ry: Ry 坐标，单位[°]

				dUcs_Rz: Rz 坐标, 单位[°]
Joints	参考关节坐标, 逆解出现多个解时需要根据参考关节坐标选取最终解	JointsData	-	Joints.dJoint[i]: 关节 i 坐标, 单位[°]

✓ 输出变量

输出变量	名称	数据类型	有效范围	内容
dTargetJoints	关节坐标	JointsData	-	dTargetJoints.dJoint[i]: 关节 i 坐标, 单位[°]

✓ 返回值

返回值	名称	数据类型	有效范围	内容
nRet	返回值	int	≥ 0 的整型值	nRet=0: 返回函数调用成功 nRet>0: 返回调用失败的错误码

3.7.4.2. 示例

// 定义需要转换的空间位置变量

```
double dCoord_X = 0; double dCoord_Y = 0; double dCoord_Z = 0;
double dCoord_Rx = 0; double dCoord_Ry = 0; double dCoord_Rz = 0;
```

// 定义工具坐标变量

```
double dTcp_X = 0; double dTcp_Y = 0; double dTcp_Z = 0;
double dTcp_Rx = 0; double dTcp_Ry = 0; double dTcp_Rz = 0;
```

// 定义工具坐标变量

```
double dUcs_X = 0; double dUcs_Y = 0; double dUcs_Z = 0;
double dUcs_Rx = 0; double dUcs_Ry = 0; double dUcs_Rz = 0;
```

// 定义参考关节位置变量

```
JointsData Joints;
```

// 定义转换结果

```
JointsData dTargetJoints;
```

// 求逆解

```
int nRet= HRIF_GetInverseKin_nJ(0, 0,dCoord_X, dCoord_Y, dCoord_Z, dCoord_Rx, dCoord_Ry, dCoord_Rz,
```

```
dTcp_X, dTcp_Y, dTcp_Z, dTcp_Rx, dTcp_Ry, dTcp_Rz,  
dUcs_X, dUcs_Y, dUcs_Z, dUcs_Rx, dUcs_Ry, dUcs_Rz, Joints, dTargetJoints);
```

3.7.5. HRIF_GetForwardKin

3.7.5.1. 描述：运动学正解，由关节坐标位置计算指定用户坐标系和工具坐标系下的迪卡尔坐标位置。

✓ 输入变量

输入变量	名称	数据类型	有效范围	内容
boxID	电箱 ID	unsigned int	0~5	电箱 ID 号，默认值= 0
rbtID	机器人 ID	unsigned int	0~5	机器人 ID 号，默认值= 0
dJ1 -dJ6	需要计算正解的关节坐标	double	-	dJ1: 关节 1 坐标, 单位[°] dJ2: 关节 2 坐标, 单位[°] dJ3: 关节 3 坐标, 单位[°] dJ4: 关节 4 坐标, 单位[°] dJ5: 关节 5 坐标, 单位[°] dJ6: 关节 6 坐标, 单位[°]
dTcp_X-dTcp_Rz	工具坐标	double	-	目标位置是否包含工具坐标(不包含工具坐标则所有值= 0): dTcp_X: X 坐标, 单位[mm] dTcp_Y: Y 坐标, 单位[mm] dTcp_Z: Z 坐标, 单位[mm] dTcp_Rx: Rx 坐标, 单位[°] dTcp_Ry: Ry 坐标, 单位[°] dTcp_Rz: Rz 坐标, 单位[°]
dUcs_X-dUcs_Rz	用户坐标	double	-	目标位置是否包含用户坐标(不包含用户坐标则所有值= 0): dUcs_X: X 坐标, 单位[mm] dUcs_Y: Y 坐标, 单位[mm] dUcs_Z: Z 坐标, 单位[mm] dUcs_Rx: Rx 坐标, 单位[°] dUcs_Ry: Ry 坐标, 单位[°] dUcs_Rz: Rz 坐标, 单位[°]

✓ 输出变量

输出变量	名称	数据类型	有效范围	内容
dTarget_X- dTarget_Rz	目标迪卡尔 坐标	double	-	计算正解的目标迪卡尔位置： dTargetX: X 坐标, 单位[mm] dTargetY: Y 坐标, 单位[mm] dTargetZ: Z 坐标, 单位[mm] dTargetRx: Rx 坐标, 单位[°] dTargetRy: Ry 坐标, 单位[°] dTargetRz: Rz 坐标, 单位[°]

✓ 返回值

返回值	名称	数据类型	有效范围	内容
nRet	返回值	int	>=0 的整型值	nRet=0: 返回函数调用成功 nRet>0: 返回调用失败的错误码

3.7.5.2. 示例

// 定义需要转换的关节位置变量

```
double dJ1 = 0; double dJ2 = 0; double dJ3 = 0;
double dJ4 = 0; double dJ5 = 0; double dJ6 = 0;
```

// 定义工具坐标变量

```
double dTcp_X = 0; double dTcp_Y = 0; double dTcp_Z = 0;
double dTcp_Rx = 0; double dTcp_Ry = 0; double dTcp_Rz = 0;
```

// 定义用户坐标变量

```
double dUcs_X = 0; double dUcs_Y = 0; double dUcs_Z = 0;
double dUcs_Rx = 0; double dUcs_Ry = 0; double dUcs_Rz = 0;
```

// 定义转换后的空间位置结果

```
double dTarget_X = 0; double dTarget_Y = 0; double dTarget_Z = 0;
double dTarget_Rx = 0; double dTarget_Ry = 0; double dTarget_Rz = 0;
```

// 求正解

```
int nRet= HRIF_GetForwardKin(0,0, dJ1 , dJ2 , dJ3 , dJ4 , dJ5 , dJ6 ,
dTcp_X, dTcp_Y, dTcp_Z, dTcp_Rx, dTcp_Ry, dTcp_Rz, dUcs_X, dUcs_Y, dUcs_Z, dUcs_Rx, dUcs_Ry, dUcs_Rz,
dTarget_X, dTarget_Y, dTarget_Z, dTarget_Rx, dTarget_Ry, dTarget_Rz);
```

3.7.6. HRIF_GetForwardKin_nJ

3.7.6.1. 描述：运动学正解，由关节坐标位置计算指定用户坐标系和工具坐标系下的迪卡尔坐标位置。**注意：**

这是扩展轴版本所用接口

✓ 输入变量

输入变量	名称	数据类型	有效范围	内容
boxID	电箱 ID	unsigned int	0~5	电箱 ID 号，默认值= 0
rbtID	机器人 ID	unsigned int	0~5	机器人 ID 号，默认值= 0
Joints	需要计算正解的关节坐标	JointsData	-	Joints.dJoint[i]：关节坐标，单位[°]
dTcp_X-dTcp_Rz	工具坐标	double	-	目标位置是否包含工具坐标(不包含工具坐标则所有值= 0)： dTcp_X: X 坐标，单位[mm] dTcp_Y: Y 坐标，单位[mm] dTcp_Z: Z 坐标，单位[mm] dTcp_Rx: Rx 坐标，单位[°] dTcp_Ry: Ry 坐标，单位[°] dTcp_Rz: Rz 坐标，单位[°]
dUcs_X-dUcs_Rz	用户坐标	double	-	目标位置是否包含用户坐标(不包含用户坐标则所有值= 0)： dUcs_X: X 坐标，单位[mm] dUcs_Y: Y 坐标，单位[mm] dUcs_Z: Z 坐标，单位[mm] dUcs_Rx: Rx 坐标，单位[°] dUcs_Ry: Ry 坐标，单位[°] dUcs_Rz: Rz 坐标，单位[°]

✓ 输出变量

输出变量	名称	数据类型	有效范围	内容
dTarget_X-dTarget_Rz	目标迪卡尔	double	-	计算正解的目标迪卡尔位置：

	坐标			dTargetX: X 坐标, 单位[mm] dTargetY: Y 坐标, 单位[mm] dTargetZ: Z 坐标, 单位[mm] dTargetRx: Rx 坐标, 单位[°] dTargetRy: Ry 坐标, 单位[°] dTargetRz: Rz 坐标, 单位[°]
--	----	--	--	---

✓ 返回值

返回值	名称	数据类型	有效范围	内容
nRet	返回值	int	>=0 的整型值	nRet=0: 返回函数调用成功 nRet>0: 返回调用失败的错误码

3.7.6.2. 示例

// 定义需要转换的关节位置变量

```
JointsData Joints;
```

```
Joints.dJoint[0] = 30; Joints.dJoint[1] = 0; Joints.dJoint[2] = 90;
```

```
Joints.dJoint[3] = 0; Joints.dJoint[4] = 90; Joints.dJoint[5] = 0; Joints.dJoint[6] = 10;
```

// 定义工具坐标变量

```
double dTcp_X = 0; double dTcp_Y = 0; double dTcp_Z = 0;
```

```
double dTcp_Rx = 0; double dTcp_Ry = 0; double dTcp_Rz = 0;
```

// 定义用户坐标变量

```
double dUcs_X = 0; double dUcs_Y = 0; double dUcs_Z = 0;
```

```
double dUcs_Rx = 0; double dUcs_Ry = 0; double dUcs_Rz = 0;
```

// 定义转换后的空间位置结果

```
double dTarget_X = 0; double dTarget_Y = 0; double dTarget_Z = 0;
```

```
double dTarget_Rx = 0; double dTarget_Ry = 0; double dTarget_Rz = 0;
```

// 求正解

```
int nRet= HRIF_GetForwardKin_nJ(0,0,Joints ,
```

```
dTcp_X, dTcp_Y, dTcp_Z, dTcp_Rx, dTcp_Ry, dTcp_Rz, dUcs_X, dUcs_Y, dUcs_Z, dUcs_Rx, dUcs_Ry, dUcs_Rz,
dTarget_X, dTarget_Y, dTarget_Z, dTarget_Rx, dTarget_Ry, dTarget_Rz);
```

3.7.7. HRIF_Base2UcsTcp

3.7.7.1. 描述：由基坐标系下的坐标位置计算指定用户坐标系和工具坐标系下的迪卡尔坐标位置。

✓ 输入变量

输入变量	名称	数据类型	有效范围	内容
boxID	电箱 ID	unsigned int	0~5	电箱 ID 号, 默认值= 0
rbtID	机器人 ID	unsigned int	0~5	机器人 ID 号, 默认值= 0
dCoord_X- dCoord_Rz	基于基座坐 标系的迪卡 尔位置	double	-	需要转换的迪卡尔位置: dCoord_X: X 坐标, 单位[mm] dCoord_Y: Y 坐标, 单位[mm] dCoord_Z: Z 坐标, 单位[mm] dCoord_Rx: Rx 坐标, 单位[°] dCoord_Ry: Ry 坐标, 单位[°] dCoord_Rz: Rz 坐标, 单位[°]
dTcp_X-dTcp_Rz	工具坐标	double	-	目标位置是否包含工具坐标(不包含工具坐 标则所有值= 0): dTcp_X: X 坐标, 单位[mm] dTcp_Y: Y 坐标, 单位[mm] dTcp_Z: Z 坐标, 单位[mm] dTcp_Rx: Rx 坐标, 单位[°] dTcp_Ry: Ry 坐标, 单位[°] dTcp_Rz: Rz 坐标, 单位[°]
dUcs_X-dUcs_Rz	用户坐标	double	-	目标位置是否包含用户坐标(不包含用户坐 标则所有值= 0): dUcs_X: X 坐标, 单位[mm] dUcs_Y: Y 坐标, 单位[mm] dUcs_Z: Z 坐标, 单位[mm] dUcs_Rx: Rx 坐标, 单位[°] dUcs_Ry: Ry 坐标, 单位[°] dUcs_Rz: Rz 坐标, 单位[°]

✓ 输出变量

输出变量	名称	数据类型	有效范围	内容
dTarget_X- dTarget_Rz	目标迪卡尔坐标	double	-	指定用户坐标系和工具坐标系下的迪卡尔坐标位置： dTargetX: X 坐标, 单位[mm] dTargetY: Y 坐标, 单位[mm] dTargetZ: Z 坐标, 单位[mm] dTargetRx: Rx 坐标, 单位[°] dTargetRy: Ry 坐标, 单位[°] dTargetRz: Rz 坐标, 单位[°]

✓ 返回值

返回值	名称	数据类型	有效范围	内容
nRet	返回值	int	>=0 的整型值	nRet=0: 返回函数调用成功 nRet>0: 返回调用失败的错误码

3.7.7.2. 示例
// 定义需要转换的空间位置变量

```
double dCoord_X = 0; double dCoord_Y = 0; double dCoord_Z = 0;
double dCoord_Rx = 0; double dCoord_Ry = 0; double dCoord_Rz = 0;
```

// 定义工具坐标变量

```
double dTcp_X = 0; double dTcp_Y = 0; double dTcp_Z = 0;
double dTcp_Rx = 0; double dTcp_Ry = 0; double dTcp_Rz = 0;
```

// 定义用户坐标变量

```
double dUcs_X = 0; double dUcs_Y = 0; double dUcs_Z = 0;
double dUcs_Rx = 0; double dUcs_Ry = 0; double dUcs_Rz = 0;
```

// 定义转换后的空间位置结果

```
double dTarget_X = 0; double dTarget_Y = 0; double dTarget_Z = 90;
double dTarget_Rx = 0; double dTarget_Ry = 90; double dTarget_Rz = 0;
```

// 基座坐标转换为用户坐标

```
int nRet= HRIF_Base2UcsTcp(0,0,dCoord_X,dCoord_Y,dCoord_Z,dCoord_Rx,dCoord_Ry,dCoord_Rz,
dTcp_X, dTcp_Y, dTcp_Z, dTcp_Rx, dTcp_Ry, dTcp_Rz,
dUcs_X, dUcs_Y, dUcs_Z, dUcs_Rx, dUcs_Ry, dUcs_Rz,
dTarget_X, dTarget_Y, dTarget_Z, dTarget_Rx, dTarget_Ry, dTarget_Rz);
```

3.7.8. HRIF_UcsTcp2Base

3.7.8.1. 描述：由指定用户坐标系和工具坐标系下的迪卡尔坐标位置计算基坐标系下的坐标位置。

✓ 输入变量

输入变量	名称	数据类型	有效范围	内容
boxID	电箱 ID	unsigned int	0~5	电箱 ID 号，默认值= 0
rbtID	机器人 ID	unsigned int	0~5	机器人 ID 号，默认值= 0
dCoord_X- dCoord_Rz	迪卡尔位置	double	-	指定用户坐标系和工具坐标系下的迪卡尔坐标位置： dCoord_X: X 坐标，单位[mm] dCoord_Y: Y 坐标，单位[mm] dCoord_Z: Z 坐标，单位[mm] dCoord_Rx: Rx 坐标，单位[°] dCoord_Ry: Ry 坐标，单位[°] dCoord_Rz: Rz 坐标，单位[°]
dTcp_X-dTcp_Rz	工具坐标	double	-	目标位置是否包含工具坐标(不包含工具坐标则所有值=0): dTcp_X: X 坐标，单位[mm] dTcp_Y: Y 坐标，单位[mm] dTcp_Z: Z 坐标，单位[mm] dTcp_Rx: Rx 坐标，单位[°] dTcp_Ry: Ry 坐标，单位[°] dTcp_Rz: Rz 坐标，单位[°]
dUcs_X-dUcs_Rz	用户坐标	double	-	目标位置是否包含用户坐标(不包含用户坐标则所有值=0): dUcs_X: X 坐标，单位[mm] dUcs_Y: Y 坐标，单位[mm] dUcs_Z: Z 坐标，单位[mm] dUcs_Rx: Rx 坐标，单位[°] dUcs_Ry: Ry 坐标，单位[°] dUcs_Rz: Rz 坐标，单位[°]

✓ 输出变量

输出变量	名称	数据类型	有效范围	内容
dTarget_X- dTarget_Rz	目标迪卡尔 坐标	double	-	基座坐标系下的迪卡尔坐标位置： dTargetX: X 坐标, 单位[mm] dTargetY: Y 坐标, 单位[mm] dTargetZ: Z 坐标, 单位[mm] dTargetRx: Rx 坐标, 单位[°] dTargetRy: Ry 坐标, 单位[°] dTargetRz: Rz 坐标, 单位[°]

✓ 返回值

返回值	名称	数据类型	有效范围	内容
nRet	返回值	int	>=0 的整型值	nRet=0: 返回函数调用成功 nRet>0: 返回调用失败的错误码

3.7.8.2. 示例
// 定义需要转换的空间位置变量

```
double dCoord_X = 0; double dCoord_Y = 0; double dCoord_Z = 0;
double dCoord_Rx = 0; double dCoord_Ry = 0; double dCoord_Rz = 0;
```

// 定义工具坐标变量

```
double dTcp_X = 0; double dTcp_Y = 0; double dTcp_Z = 0;
double dTcp_Rx = 0; double dTcp_Ry = 0; double dTcp_Rz = 0;
```

// 定义用户坐标变量

```
double dUcs_X = 0; double dUcs_Y = 0; double dUcs_Z = 0;
double dUcs_Rx = 0; double dUcs_Ry = 0; double dUcs_Rz = 0;
```

// 定义转换后的空间位置结果

```
double dTarget_X = 0; double dTarget_Y = 0; double dTarget_Z = 90;
double dTarget_Rx = 0; double dTarget_Ry = 90; double dTarget_Rz = 0;
```

// 用户坐标转换为基座坐标

```
int nRet= HRIF_UcsTcp2Base(0,0, dCoord_X,dCoord_Y,dCoord_Z,dCoord_Rx,dCoord_Ry,dCoord_Rz,
dTcp_X, dTcp_Y, dTcp_Z, dTcp_Rx, dTcp_Ry, dTcp_Rz,
dUcs_X, dUcs_Y, dUcs_Z, dUcs_Rx, dUcs_Ry, dUcs_Rz,
dTarget_X, dTarget_Y, dTarget_Z, dTarget_Rx, dTarget_Ry, dTarget_Rz);
```

3.7.9. HRIF_PoseAdd

3.7.9.1. 描述：点位加法计算，使用矩阵左乘运算(第二个点左乘第一个点)。

✓ 输入变量

输入变量	名称	数据类型	有效范围	内容
boxID	电箱 ID	unsigned int	0~5	电箱 ID 号，默认值= 0
rbtID	机器人 ID	unsigned int	0~5	机器人 ID 号，默认值= 0
dPose1_X- dPose1_Rz	空间坐标 1	double	-	需要计算的空间坐标 1： dPose1_X: X 坐标，单位[mm] dPose1_Y: Y 坐标，单位[mm] dPose1_Z: Z 坐标，单位[mm] dPose1_Rx: Rx 坐标，单位[°] dPose1_Ry: Ry 坐标，单位[°] dPose1_Rz: Rz 坐标，单位[°]
dPose2_X- dPose2_Rz	空间坐标 2	double	-	需要计算的空间坐标 2： dPose2_X: X 坐标，单位[mm] dPose2_Y: Y 坐标，单位[mm] dPose2_Z: Z 坐标，单位[mm] dPose2_Rx: Rx 坐标，单位[°] dPose2_Ry: Ry 坐标，单位[°] dPose2_Rz: Rz 坐标，单位[°]

✓ 输出变量

输出变量	名称	数据类型	有效范围	内容
dPose3_X- dPose3_Rz	计算结果	double	-	计算结果： dPose3_X: X 坐标，单位[mm] dPose3_Y: Y 坐标，单位[mm] dPose3_Z: Z 坐标，单位[mm] dPose3_Rx: Rx 坐标，单位[°] dPose3_Ry: Ry 坐标，单位[°] dPose3_Rz: Rz 坐标，单位[°]

✓ 返回值

返回值	名称	数据类型	有效范围	内容
nRet	返回值	int	≥ 0 的整型值	nRet=0: 返回函数调用成功 nRet>0: 返回调用失败的错误码

3.7.9.2. 示例

// 定义需要计算的空间坐标 1

```
double dPose1_X = 420; double dPose1_Y = 0; double dPose1_Z = 445;
double dPose1_Rx = 180; double dPose1_Ry = 0; double dPose1_Rz = 180;
```

// 定义需要计算的空间坐标 2

```
double dPose2_X = 420; double dPose2_Y = 50; double dPose2_Z = 445;
double dPose2_Rx = 180; double dPose2_Ry = 0; double dPose2_Rz = 180;
```

// 计算结果

```
double dPose3_X = 0; double dPose3_Y = 0; double dPose3_Z = 0;
double dPose3_Rx = 0; double dPose3_Ry = 0; double dPose3_Rz = 0;
```

// 计算结果

```
int nRet= HRIF_PoseAdd(0,0, dPose1_X, dPose1_Y, dPose1_Z, dPose1_Rx, dPose1_Ry, dPose1_Rz,
dPose2_X, dPose2_Y, dPose2_Z, dPose2_Rx, dPose2_Ry, dPose2_Rz,
dPose3_X, dPose3_Y, dPose3_Z, dPose3_Rx, dPose3_Ry, dPose3_Rz);
```

3.7.10. HRIF_PoseSub

3.7.10.1. 描述：点位减法计算，以第二个点为参考点。

✓ 输入变量

输入变量	名称	数据类型	有效范围	内容
boxID	电箱 ID	unsigned int	0~5	电箱 ID 号，默认值= 0
rbtID	机器人 ID	unsigned int	0~5	机器人 ID 号，默认值= 0
dPose1_X- dPose1_Rz	空间坐标 1	double	-	需要计算的空间坐标 1： dPose1_X: X 坐标，单位[mm] dPose1_Y: Y 坐标，单位[mm] dPose1_Z: Z 坐标，单位[mm] dPose1_Rx: Rx 坐标，单位[°] dPose1_Ry: Ry 坐标，单位[°] dPose1_Rz: Rz 坐标，单位[°]
dPose2_X- dPose2_Rz	空间坐标 2	double	-	需要计算的空间坐标 2： dPose2_X: X 坐标，单位[mm] dPose2_Y: Y 坐标，单位[mm] dPose2_Z: Z 坐标，单位[mm] dPose2_Rx: Rx 坐标，单位[°] dPose2_Ry: Ry 坐标，单位[°] dPose2_Rz: Rz 坐标，单位[°]

✓ 输出变量

输出变量	名称	数据类型	有效范围	内容
dPose3_X- dPose3_Rz	计算坐标	double	-	计算结果： dPose3_X: X 坐标，单位[mm] dPose3_Y: Y 坐标，单位[mm] dPose3_Z: Z 坐标，单位[mm] dPose3_Rx: Rx 坐标，单位[°] dPose3_Ry: Ry 坐标，单位[°] dPose3_Rz: Rz 坐标，单位[°]

✓ 返回值

返回值	名称	数据类型	有效范围	内容
nRet	返回值	int	≥ 0 的整型值	nRet=0: 返回函数调用成功 nRet>0: 返回调用失败的错误码

3.7.10.2. 示例

// 定义需要计算的空间坐标 1

```
double dPose1_X = 420; double dPose1_Y = 0; double dPose1_Z = 445;
double dPose1_Rx = 180; double dPose1_Ry = 0; double dPose1_Rz = 180;
```

// 定义需要计算的空间坐标 2

```
double dPose2_X = 420; double dPose2_Y = 50; double dPose2_Z = 445;
double dPose2_Rx = 180; double dPose2_Ry = 0; double dPose2_Rz = 180;
```

// 计算结果

```
double dPose3_X = 0; double dPose3_Y = 0; double dPose3_Z = 0;
double dPose3_Rx = 0; double dPose3_Ry = 0; double dPose3_Rz = 0;
```

// 计算结果

```
int nRet= HRIF_PoseSub(0,0, dPose1_X, dPose1_Y, dPose1_Z, dPose1_Rx, dPose1_Ry, dPose1_Rz,
dPose2_X, dPose2_Y, dPose2_Z, dPose2_Rx, dPose2_Ry, dPose2_Rz,
dPose3_X, dPose3_Y, dPose3_Z, dPose3_Rx, dPose3_Ry, dPose3_Rz);
```

3.7.11. HRIF_PoseTrans

3.7.11.1. 描述：坐标变换。

✓ 输入变量

输入变量	名称	数据类型	有效范围	内容
boxID	电箱 ID	unsigned int	0~5	电箱 ID 号，默认值= 0
rbtID	机器人 ID	unsigned int	0~5	机器人 ID 号，默认值= 0
dPose1_X- dPose1_Rz	坐标位置 1	double	-	坐标位置 1: dPose1_X: X 坐标, 单位[mm] dPose1_Y: Y 坐标, 单位[mm] dPose1_Z: Z 坐标, 单位[mm] dPose1_Rx: Rx 坐标, 单位[°] dPose1_Ry: Ry 坐标, 单位[°] dPose1_Rz: Rz 坐标, 单位[°]
dPose2_X- dPose2_Rz	坐标位置 2	double	-	坐标位置 2: dPose2_X: X 坐标, 单位[mm] dPose2_Y: Y 坐标, 单位[mm] dPose2_Z: Z 坐标, 单位[mm] dPose2_Rx: Rx 坐标, 单位[°] dPose2_Ry: Ry 坐标, 单位[°] dPose2_Rz: Rz 坐标, 单位[°]

✓ 输出变量

输出变量	名称	数据类型	有效范围	内容
dPose3_X- dPose3_Rz	计算坐标	double	-	计算结果: dPose3_X: X 坐标, 单位[mm] dPose3_Y: Y 坐标, 单位[mm] dPose3_Z: Z 坐标, 单位[mm] dPose3_Rx: Rx 坐标, 单位[°] dPose3_Ry: Ry 坐标, 单位[°] dPose3_Rz: Rz 坐标, 单位[°]

✓ 返回值

返回值	名称	数据类型	有效范围	内容
nRet	返回值	int	≥ 0 的整型值	nRet=0: 返回函数调用成功 nRet>0: 返回调用失败的错误码

3.7.11.2. 示例

// 定义需要计算的空间坐标 1

```
double dPose1_X = 420; double dPose1_Y = 0; double dPose1_Z = 445;
double dPose1_Rx = 180; double dPose1_Ry = 0; double dPose1_Rz = 180;
```

// 定义需要计算的空间坐标 2

```
double dPose2_X = 420; double dPose2_Y = 50; double dPose2_Z = 445;
double dPose2_Rx = 180; double dPose2_Ry = 0; double dPose2_Rz = 180;
```

// 计算结果

```
double dPose3_X = 0; double dPose3_Y = 0; double dPose3_Z = 0;
double dPose3_Rx = 0; double dPose3_Ry = 0; double dPose3_Rz = 0;
```

// 计算结果

```
int nRet= HRIF_PoseTrans(0,0, dPose1_X, dPose1_Y, dPose1_Z, dPose1_Rx, dPose1_Ry, dPose1_Rz,
dPose2_X, dPose2_Y, dPose2_Z, dPose2_Rx, dPose2_Ry, dPose2_Rz,
dPose3_X, dPose3_Y, dPose3_Z, dPose3_Rx, dPose3_Ry, dPose3_Rz);
```

3.7.12. HRIF_PoseInverse

3.7.12.1. 描述：坐标逆变换。

✓ 输入变量

输入变量	名称	数据类型	有效范围	内容
boxID	电箱 ID	unsigned int	0~5	电箱 ID 号，默认值= 0
rbtID	机器人 ID	unsigned int	0~5	机器人 ID 号，默认值= 0
dPose1_X- dPose1_Rz	空间坐标 1	double	-	需要计算的空间坐标 1： dPose1_X: X 坐标，单位[mm] dPose1_Y: Y 坐标，单位[mm] dPose1_Z: Z 坐标，单位[mm] dPose1_Rx: Rx 坐标，单位[°] dPose1_Ry: Ry 坐标，单位[°] dPose1_Rz: Rz 坐标，单位[°]

✓ 输出变量

输出变量	名称	数据类型	有效范围	内容
dPose3_X- dPose3_Rz	计算坐标	double	-	计算结果： dPose3_X: X 坐标，单位[mm] dPose3_Y: Y 坐标，单位[mm] dPose3_Z: Z 坐标，单位[mm] dPose3_Rx: Rx 坐标，单位[°] dPose3_Ry: Ry 坐标，单位[°] dPose3_Rz: Rz 坐标，单位[°]

✓ 返回值

返回值	名称	数据类型	有效范围	内容
nRet	返回值	int	>=0 的整型值	nRet=0: 返回函数调用成功 nRet>0: 返回调用失败的错误码

3.7.12.2. 示例

// 定义需要计算的空间坐标 1

```
double dPose1_X = 420; double dPose1_Y = 0; double dPose1_Z = 445;  
double dPose1_Rx = 180; double dPose1_Ry = 0; double dPose1_Rz = 180;
```

```
// 计算结果
```

```
double dPose3_X = 0; double dPose3_Y = 0; double dPose3_Z = 0;  
double dPose3_Rx = 0; double dPose3_Ry = 0; double dPose3_Rz = 0;
```

```
// 计算结果
```

```
int nRet= HRIF_PoseInverse(0,0, dPose1_X, dPose1_Y, dPose1_Z, dPose1_Rx, dPose1_Ry, dPose1_Rz,  
dPose3_X, dPose3_Y, dPose3_Z, dPose3_Rx, dPose3_Ry, dPose3_Rz);
```

3.7.13. HRIF_PoseDist

3.7.13.1. 描述：计算点位距离。

✓ 输入变量

输入变量	名称	数据类型	有效范围	内容
boxID	电箱 ID	unsigned int	0~5	电箱 ID 号，默认值= 0
rbtID	机器人 ID	unsigned int	0~5	机器人 ID 号，默认值= 0
dPose1_X- dPose1_Rz	空间坐标 1	double	-	需要计算的空间坐标 1： dPose1_X: X 坐标，单位[mm] dPose1_Y: Y 坐标，单位[mm] dPose1_Z: Z 坐标，单位[mm] dPose1_Rx: Rx 坐标，单位[°] dPose1_Ry: Ry 坐标，单位[°] dPose1_Rz: Rz 坐标，单位[°]
dPose2_X- dPose2_Rz	空间坐标 2	double	-	需要计算的空间坐标 2： dPose2_X: X 坐标，单位[mm] dPose2_Y: Y 坐标，单位[mm] dPose2_Z: Z 坐标，单位[mm] dPose2_Rx: Rx 坐标，单位[°] dPose2_Ry: Ry 坐标，单位[°] dPose2_Rz: Rz 坐标，单位[°]

✓ 输出变量

输出变量	名称	数据类型	有效范围	内容
dDistance	点位距离	double	-	点位距离，单位[mm]
dAngle	姿态距离	double	-	姿态距离，单位[°]

✓ 返回值

返回值	名称	数据类型	有效范围	内容
nRet	返回值	int	>=0 的整型值	nRet=0: 返回函数调用成功 nRet>0: 返回调用失败的错误码

3.7.13.2. 示例

// 定义需要计算的空间坐标 1

```
double dPose1_X = 420; double dPose1_Y = 0; double dPose1_Z = 445;  
double dPose1_Rx = 180; double dPose1_Ry = 0; double dPose1_Rz = 180;
```

// 定义需要计算的空间坐标 2

```
double dPose2_X = 420; double dPose2_Y = 50; double dPose2_Z = 445;  
double dPose2_Rx = 180; double dPose2_Ry = 0; double dPose2_Rz = 180;
```

// 计算结果

```
double dDistance = 0;  
double dAngle = 0;
```

// 计算结果

```
int nRet= HRIF_PoseDist(0,0, dPose1_X, dPose1_Y, dPose1_Z, dPose1_Rx, dPose1_Ry, dPose1_Rz,  
dPose2_X, dPose2_Y, dPose2_Z, dPose2_Rx, dPose2_Ry, dPose2_Rz,  
dDistance, dAngle);
```

3.7.14. HRIF_Poseinterpolate

3.7.14.1. 描述：空间位置直线插补计算。

✓ 输入变量

输入变量	名称	数据类型	有效范围	内容
boxID	电箱 ID	unsigned int	0~5	电箱 ID 号，默认值= 0
rbtID	机器人 ID	unsigned int	0~5	机器人 ID 号，默认值= 0
dPose1_X- dPose1_Rz	空间坐标 1	double	-	需要计算的空间坐标 1： dPose1_X: X 坐标，单位[mm] dPose1_Y: Y 坐标，单位[mm] dPose1_Z: Z 坐标，单位[mm] dPose1_Rx: Rx 坐标，单位[°] dPose1_Ry: Ry 坐标，单位[°] dPose1_Rz: Rz 坐标，单位[°]
dPose2_X- dPose2_Rz	空间坐标 2	double	-	需要计算的空间坐标 2： dPose2_X: X 坐标，单位[mm] dPose2_Y: Y 坐标，单位[mm] dPose2_Z: Z 坐标，单位[mm] dPose2_Rx: Rx 坐标，单位[°] dPose2_Ry: Ry 坐标，单位[°] dPose2_Rz: Rz 坐标，单位[°]
dAlpha	插补比例	double	0-1	dAlpha= 0: dPose3= dPose1 dAlpha= 1: dPose3= dPose2 0-1: 按照 dPose1 到 dPose2 的位置取比例为 dAlpha 的位置返回 dPose3

✓ 输出变量

输出变量	名称	数据类型	有效范围	内容
dPose3_X- dPose3_Rz	计算坐标	double	-	计算结果： dPose3_X: X 坐标，单位[mm]

				dPose3_Y: Y 坐标, 单位[mm] dPose3_Z: Z 坐标, 单位[mm] dPose3_Rx: Rx 坐标, 单位[°] dPose3_Ry: Ry 坐标, 单位[°] dPose3_Rz: Rz 坐标, 单位[°]
--	--	--	--	---

✓ 返回值

返回值	名称	数据类型	有效范围	内容
nRet	返回值	int	≥ 0 的整型值	nRet=0: 返回函数调用成功 nRet>0: 返回调用失败的错误码

3.7.14.2. 示例

// 定义需要计算的空间坐标 1

```
double dPose1_X = 420; double dPose1_Y = 0; double dPose1_Z = 445;
double dPose1_Rx = 180; double dPose1_Ry = 0; double dPose1_Rz = 180;
```

// 定义需要计算的空间坐标 2

```
double dPose2_X = 420; double dPose2_Y = 50; double dPose2_Z = 445;
double dPose2_Rx = 180; double dPose2_Ry = 0; double dPose2_Rz = 180;
```

// 插补比例

```
double dAlpha = 0.5;
```

// 计算结果

```
double dPose3_X = 0; double dPose3_Y = 0; double dPose3_Z = 0;
double dPose3_Rx = 0; double dPose3_Ry = 0; double dPose3_Rz = 0;
```

// 计算结果

```
int nRet= HRIF_Poseinterpolate(0,0, dPose1_X, dPose1_Y, dPose1_Z, dPose1_Rx, dPose1_Ry, dPose1_Rz,
dPose2_X, dPose2_Y, dPose2_Z, dPose2_Rx, dPose2_Ry, dPose2_Rz, dAlpha,
dPose3_X, dPose3_Y, dPose3_Z, dPose3_Rx, dPose3_Ry, dPose3_Rz);
```

3.7.15. HRIF_PoseDefdFrame

3.7.15.1. 描述：以轨迹中心旋转计算，p1,p2,p3 为旋转前选取的轨迹的特征点，p4,p5,p6 为旋转后选取的轨迹的特征点，计算结果表示为旋转特征的用户坐标系。

✓ 输入变量

输入变量	名称	数据类型	有效范围	内容
boxID	电箱 ID	unsigned int	0~5	电箱 ID 号，默认值= 0
rbtID	机器人 ID	unsigned int	0~5	机器人 ID 号，默认值= 0
dPose1_X- dPose1_Z	空间坐标 1	double	-	需要计算的空间坐标 1： dPose1_X: X 坐标，单位[mm] dPose1_Y: Y 坐标，单位[mm] dPose1_Z: Z 坐标，单位[mm]
dPose2_X- dPose2_Z	空间坐标 2	double	-	需要计算的空间坐标 2： dPose2_X: X 坐标，单位[mm] dPose2_Y: Y 坐标，单位[mm] dPose2_Z: Z 坐标，单位[mm]
dPose3_X- dPose3_Z	空间坐标 3	double	-	需要计算的空间坐标 3： dPose3_X: X 坐标，单位[mm] dPose3_Y: Y 坐标，单位[mm] dPose3_Z: Z 坐标，单位[mm]
dPose4_X- dPose4_Z	空间坐标 4	double	-	需要计算的空间坐标 4： dPose4_X: X 坐标，单位[mm] dPose4_Y: Y 坐标，单位[mm] dPose4_Z: Z 坐标，单位[mm]
dPose5_X- dPose5_Z	空间坐标 5	double	-	需要计算的空间坐标 5： dPose5_X: X 坐标，单位[mm] dPose5_Y: Y 坐标，单位[mm] dPose5_Z: Z 坐标，单位[mm]
dPose6_X- dPose6_Z	空间坐标 6	double	-	需要计算的空间坐标 6：

				dPose6_X: X 坐标, 单位[mm] dPose6_Y: Y 坐标, 单位[mm] dPose6_Z: Z 坐标, 单位[mm]
--	--	--	--	--

✓ 输出变量

输出变量	名称	数据类型	有效范围	内容
dUcs_X- dUcs_Rz	计算结果	double	-	计算结果: dUcs_X: X 坐标, 单位[mm] dUcs_Y: Y 坐标, 单位[mm] dUcs_Z: Z 坐标, 单位[mm] dUcs_Rx: Rx 坐标, 单位[°] dUcs_Ry: Ry 坐标, 单位[°] dUcs_Rz: Rz 坐标, 单位[°]

✓ 返回值

返回值	名称	数据类型	有效范围	内容
nRet	返回值	int	>=0 的整型值	nRet=0: 返回函数调用成功 nRet>0: 返回调用失败的错误码

3.7.15.2. 示例

// 定义需要计算的空间坐标 1

```
double dPose1_X = 0; double dPose1_Y = 0; double dPose1_Z = 0;
```

// 定义需要计算的空间坐标 2

```
double dPose2_X = 0; double dPose2_Y = 0; double dPose2_Z = 0;
```

// 定义需要计算的空间坐标 3

```
double dPose3_X = 0; double dPose3_Y = 0; double dPose3_Z = 0;
```

// 定义需要计算的空间坐标 4

```
double dPose4_X = 0; double dPose4_Y = 0; double dPose4_Z = 0;
```

// 定义需要计算的空间坐标 5

```
double dPose5_X = 0; double dPose5_Y = 0; double dPose5_Z = 0;
```

// 定义需要计算的空间坐标 6

```
double dPose6_X = 0; double dPose6_Y = 0; double dPose6_Z = 0;
```

// 计算结果

```
double dUcs_X = 0; double dUcs_Y = 0; double dUcs_Z = 0;  
double dUcs_Rx = 0; double dUcs_Ry = 0; double dUcs_Rz = 0;
```

// 计算结果

```
int nRet= HRIF_PoseDefdFrame(0,0, dPose1_X, dPose1_Y, dPose1_Z, dPose2_X, dPose2_Y, dPose2_Z, dPose3_X,  
dPose3_Y, dPose3_Z, dPose4_X, dPose4_Y, dPose4_Z, dPose5_X, dPose5_Y, dPose5_Z, dPose6_X, dPose6_Y,  
dPose6_Z,dUcs_X, dUcs_Y, dUcs_Z, dUcs_Rx, dUcs_Ry, dUcs_Rz);
```

3.7.16. HRIF_CalUcsPlane

3.7.16.1. 描述：通过三点平面法计算 UCS。

✓ 输入变量

输入变量	名称	数据类型	有效范围	内容
boxID	电箱 ID	unsigned int	0~5	电箱 ID 号，默认值=0
rbtID	机器人 ID	unsigned int	0~5	机器人 ID 号，默认值=0
dPose1_X- dPose1_Z	空间坐标 1	double	-	点 1 在 Base 坐标系下系统默认 TCP 的位置： dPose1_X: X 坐标，单位[mm] dPose1_Y: Y 坐标，单位[mm] dPose1_Z: Z 坐标，单位[mm]
dPose2_X- dPose2_Z	空间坐标 2	double	-	点 2 在 Base 坐标系下系统默认 TCP 的位置： dPose2_X: X 坐标，单位[mm] dPose2_Y: Y 坐标，单位[mm] dPose2_Z: Z 坐标，单位[mm]
dPose3_X- dPose3_Z	空间坐标 3	double	-	点 3 在 Base 坐标系下系统默认 TCP 的位置： dPose3_X: X 坐标，单位[mm] dPose3_Y: Y 坐标，单位[mm] dPose3_Z: Z 坐标，单位[mm]

✓ 输出变量

输出变量	名称	数据类型	有效范围	内容
dRetPose_X- dRetPose_Rz	计算结果	double	-	计算得出的 UCS 位姿： dRetPose_X: X 坐标，单位[mm] dRetPose_Y: Y 坐标，单位[mm] dRetPose_Z: Z 坐标，单位[mm] dRetPose_Rx: Rx 坐标，单位[°] dRetPose_Ry: Ry 坐标，单位[°] dRetPose_Rz: Rz 坐标，单位[°]

✓ 返回值

返回值	名称	数据类型	有效范围	内容
nRet	返回值	int	≥ 0 的整型值	nRet=0: 返回函数调用成功 nRet>0: 返回调用失败的错误码

3.7.16.2. 示例

// 定义需要计算的空间坐标 1

```
double dPose1_X = 10; double dPose1_Y = 0; double dPose1_Z = 0;
```

// 定义需要计算的空间坐标 2

```
double dPose2_X = 0; double dPose2_Y = 10; double dPose2_Z = 0;
```

// 定义需要计算的空间坐标 3

```
double dPose3_X = 0; double dPose3_Y = 0; double dPose3_Z = 10;
```

// 计算结果变量定义

```
double dRetPose_X = 0; double dRetPose_Y = 0; double dRetPose_Z = 0;
```

```
double dRetPose_Rx = 0; double dRetPose_Ry = 0; double dRetPose_Rz = 0;
```

// 获取计算结果, 函数调用后, nRet 值为 0, dRetPose_X~dRetPose_Rz 的值分别为: 10,0,0,180,-54.736,-135

```
int nRet= HRIF_CalUcsPlane(0,0, dPose1_X, dPose1_Y, dPose1_Z, dPose2_X, dPose2_Y, dPose2_Z, dPose3_X,
dPose3_Y, dPose3_Z, dRetPose_X, dRetPose_Y, dRetPose_Z, dRetPose_Rx, dRetPose_Ry, dRetPose_Rz);
```


3.7.17. HRIF_CalUcsLine

3.7.17.1. 描述：通过两点直线法计算 UCS。

✓ 输入变量

输入变量	名称	数据类型	有效范围	内容
boxID	电箱 ID	unsigned int	0~5	电箱 ID 号，默认值= 0
rbtID	机器人 ID	unsigned int	0~5	机器人 ID 号，默认值= 0
dPose1_X- dPose1_Rz	空间坐标 1	double	-	点 1 在 Base 坐标系下系统默认 TCP 的位姿： dPose1_X: X 坐标，单位[mm] dPose1_Y: Y 坐标，单位[mm] dPose1_Z: Z 坐标，单位[mm] dPose1_Rx: Rx 坐标，单位[°] dPose1_Ry: Ry 坐标，单位[°] dPose1_Rz: Rz 坐标，单位[°]
dPose2_X- dPose2_Rz	空间坐标 2	double	-	点 2 在 Base 坐标系下系统默认 TCP 的位姿： dPose2_X: X 坐标，单位[mm] dPose2_Y: Y 坐标，单位[mm] dPose2_Z: Z 坐标，单位[mm] dPose2_Rx: Rx 坐标，单位[°] dPose2_Ry: Ry 坐标，单位[°] dPose2_Rz: Rz 坐标，单位[°]

✓ 输出变量

输出变量	名称	数据类型	有效范围	内容
dRetPose_X- dRetPose_Rz	计算结果	double	-	计算得出的 UCS 位姿： dRetPose_X: X 坐标，单位[mm] dRetPose_Y: Y 坐标，单位[mm] dRetPose_Z: Z 坐标，单位[mm] dRetPose_Rx: Rx 坐标，单位[°] dRetPose_Ry: Ry 坐标，单位[°] dRetPose_Rz: Rz 坐标，单位[°]

✓ 返回值

返回值	名称	数据类型	有效范围	内容
nRet	返回值	int	≥ 0 的整型值	nRet=0: 返回函数调用成功 nRet>0: 返回调用失败的错误码

3.7.17.2. 示例

// 定义需要计算的空间坐标 1

```
double dPose1_X = 10; double dPose1_Y = 0; double dPose1_Z = 0; double dPose1_Rx = 10; double dPose1_Ry = 0;
double dPose1_Rz = 0;
```

// 定义需要计算的空间坐标 2

```
double dPose2_X = 0; double dPose2_Y = 10; double dPose2_Z = 0; double dPose2_Rx = 0; double dPose2_Ry = 10;
double dPose2_Rz = 0;
```

// 计算结果变量定义

```
double dRetPose_X = 0; double dRetPose_Y = 0; double dRetPose_Z = 0;
double dRetPose_Rx = 0; double dRetPose_Ry = 0; double dRetPose_Rz = 0;
```

// 获取计算结果, 函数调用后, nRet 值为 0, dRetPose_X~dRetPose_Rz 的值分别为: 10,0,0,0,-7.107,-45

```
int nRet= HRIF_CalUcsLine(0,0, dPose1_X, dPose1_Y, dPose1_Z, dPose1_Rx, dPose1_Ry, dPose1_Rz, dPose2_X,
dPose2_Y, dPose2_Z, dPose2_Rx, dPose2_Ry, dPose2_Rz, dRetPose_X, dRetPose_Y, dRetPose_Z, dRetPose_Rx,
dRetPose_Ry, dRetPose_Rz);
```

3.7.18. HRIF_CalTcp3P

3.7.18.1. 描述：通过三点平面法计算 TCP。

✓ 输入变量

输入变量	名称	数据类型	有效范围	内容
boxID	电箱 ID	unsigned int	0~5	电箱 ID 号，默认值= 0
rbtID	机器人 ID	unsigned int	0~5	机器人 ID 号，默认值= 0
dPose1_X- dPose1_Rz	空间坐标 1	double	-	点 1 在 Base 坐标系下系统默认 TCP 的位姿： dPose1_X: X 坐标，单位[mm] dPose1_Y: Y 坐标，单位[mm] dPose1_Z: Z 坐标，单位[mm] dPose1_Rx: Rx 坐标，单位[°] dPose1_Ry: Ry 坐标，单位[°] dPose1_Rz: Rz 坐标，单位[°]
dPose2_X- dPose2_Rz	空间坐标 2	double	-	点 2 在 Base 坐标系下系统默认 TCP 的位姿： dPose2_X: X 坐标，单位[mm] dPose2_Y: Y 坐标，单位[mm] dPose2_Z: Z 坐标，单位[mm] dPose2_Rx: Rx 坐标，单位[°] dPose2_Ry: Ry 坐标，单位[°] dPose2_Rz: Rz 坐标，单位[°]
dPose3_X- dPose3_Rz	空间坐标 3	double	-	点 3 在 Base 坐标系下系统默认 TCP 的位置： dPose3_X: X 坐标，单位[mm] dPose3_Y: Y 坐标，单位[mm] dPose3_Z: Z 坐标，单位[mm] dPose3_Rx: Rx 坐标，单位[°] dPose3_Ry: Ry 坐标，单位[°] dPose3_Rz: Rz 坐标，单位[°]

✓ 输出变量

输出变量	名称	数据类型	有效范围	内容
------	----	------	------	----

dRetPose_X- dRetPose_Rz	计算结果	double	-	计算得出的 UCS 位姿： dRetPose_X: X 坐标, 单位[mm] dRetPose_Y: Y 坐标, 单位[mm] dRetPose_Z: Z 坐标, 单位[mm] dRetPose_Rx: Rx 坐标, 单位[°], 一般是 0 dRetPose_Ry: Ry 坐标, 单位[°], 一般是 0 dRetPose_Rz: Rz 坐标, 单位[°], 一般是 0
quality	结果质量	int	0/1/2	0: 良好 1: 差 (计算结果最好不用) 2: 异常

✓ 返回值

返回值	名称	数据类型	有效范围	内容
nRet	返回值	int	>=0 的整型值	nRet=0: 返回函数调用成功 nRet>0: 返回调用失败的错误码

3.7.18.2. 示例

// 定义需要计算的空间坐标 1

```
double dPose1_X = 10; double dPose1_Y = 0; double dPose1_Z = 0; double dPose1_Rx = 10; double dPose1_Ry = 0;
double dPose1_Rz = 0;
```

// 定义需要计算的空间坐标 2

```
double dPose2_X = 0; double dPose2_Y = 10; double dPose2_Z = 0; double dPose2_Rx = 10; double dPose2_Ry = 0;
double dPose2_Rz = 0;
```

// 定义需要计算的空间坐标 3

```
double dPose3_X = 0; double dPose3_Y = 10; double dPose3_Z = 0; double dPose3_Rx = 10; double dPose3_Ry = 0;
double dPose3_Rz = 0;
```

// 计算结果变量定义

```
double dRetPose_X = 0; double dRetPose_Y = 0; double dRetPose_Z = 0;
double dRetPose_Rx = 0; double dRetPose_Ry = 0; double dRetPose_Rz = 0;
```

// 计算结果质量变量定义

```
int quality = 0;
```

// 获取计算结果, 函数调用后, nRet 值为 20031 (点位计算失败)。

```
int nRet = HRIF_CalTcp3P(0, 0, dPose1_X, dPose1_Y, dPose1_Z, dPose1_Rx, dPose1_Ry, dPose1_Rz, dPose2_X,
```

dPose2_Y, dPose2_Z, dPose2_Rx, dPose2_Ry, dPose2_Rz, dPose3_X, dPose3_Y, dPose3_Z, dPose3_Rx, dPose3_Ry, dPose3_Rz, dRetPose_X, dRetPose_Y, dRetPose_Z, dRetPose_Rx, dRetPose_Ry, dRetPose_Rz, quality);

3.7.19. HRIF_CalTcp4P

3.7.19.1. 描述：通过四点平面法计算 TCP。

✓ 输入变量

输入变量	名称	数据类型	有效范围	内容
boxID	电箱 ID	unsigned int	0~5	电箱 ID 号，默认值= 0
rbtID	机器人 ID	unsigned int	0~5	机器人 ID 号，默认值= 0
dPose1_X- dPose1_Rz	空间坐标 1	double	-	点 1 在 Base 坐标系下系统默认 TCP 的位姿： dPose1_X: X 坐标，单位[mm] dPose1_Y: Y 坐标，单位[mm] dPose1_Z: Z 坐标，单位[mm] dPose1_Rx: Rx 坐标，单位[°] dPose1_Ry: Ry 坐标，单位[°] dPose1_Rz: Rz 坐标，单位[°]
dPose2_X- dPose2_Rz	空间坐标 2	double	-	点 2 在 Base 坐标系下系统默认 TCP 的位姿： dPose2_X: X 坐标，单位[mm] dPose2_Y: Y 坐标，单位[mm] dPose2_Z: Z 坐标，单位[mm] dPose2_Rx: Rx 坐标，单位[°] dPose2_Ry: Ry 坐标，单位[°] dPose2_Rz: Rz 坐标，单位[°]
dPose3_X- dPose3_Rz	空间坐标 3	double	-	点 3 在 Base 坐标系下系统默认 TCP 的位置： dPose3_X: X 坐标，单位[mm] dPose3_Y: Y 坐标，单位[mm] dPose3_Z: Z 坐标，单位[mm] dPose3_Rx: Rx 坐标，单位[°] dPose3_Ry: Ry 坐标，单位[°] dPose3_Rz: Rz 坐标，单位[°]
dPose4_X- dPose4_Rz	空间坐标 4	double	-	点 4 在 Base 坐标系下系统默认 TCP 的位置： dPose4_X: X 坐标，单位[mm]

				dPose4_Y: Y 坐标, 单位[mm] dPose4_Z: Z 坐标, 单位[mm] dPose4_Rx: Rx 坐标, 单位[°] dPose4_Ry: Ry 坐标, 单位[°] dPose4_Rz: Rz 坐标, 单位[°]
--	--	--	--	---

✓ 输出变量

输出变量	名称	数据类型	有效范围	内容
dRetPose_X- dRetPose_Rz	计算结果	double	-	计算得出的 UCS 位姿: dRetPose_X: X 坐标, 单位[mm] dRetPose_Y: Y 坐标, 单位[mm] dRetPose_Z: Z 坐标, 单位[mm] dRetPose_Rx: Rx 坐标, 单位[°], 一般是 0 dRetPose_Ry: Ry 坐标, 单位[°], 一般是 0 dRetPose_Rz: Rz 坐标, 单位[°], 一般是 0
quality	结果质量	int	0/1/2	0: 良好 1: 差 (计算结果最好不用) 2: 异常
errorIndex_P1~4	源点的错误 指示	int	0/1	0: 正常 1: 异常

✓ 返回值

返回值	名称	数据类型	有效范围	内容
nRet	返回值	int	>=0 的整型值	nRet=0: 返回函数调用成功 nRet>0: 返回调用失败的错误码

3.7.19.2. 示例
// 定义需要计算的空间坐标 1

```
double dPose1_X = 10; double dPose1_Y = 0; double dPose1_Z = 0; double dPose1_Rx = 10; double dPose1_Ry = 0;
double dPose1_Rz = 0;
```

// 定义需要计算的空间坐标 2

```
double dPose2_X = 0; double dPose2_Y = 10; double dPose2_Z = 0; double dPose2_Rx = 10; double dPose2_Ry = 0;
double dPose2_Rz = 0;
```

// 定义需要计算的空间坐标 3

```
double dPose3_X = 0; double dPose3_Y = 10; double dPose3_Z = 0; double dPose3_Rx = 10; double dPose3_Ry = 0;  
double dPose3_Rz = 0;
```

// 定义需要计算的空间坐标 4

```
double dPose4_X = 0; double dPose4_Y = 10; double dPose4_Z = 0; double dPose4_Rx = 10; double dPose4_Ry = 0;  
double dPose4_Rz = 0;
```

// 计算结果变量定义

```
double dRetPose_X = 0; double dRetPose_Y = 0; double dRetPose_Z = 0;  
double dRetPose_Rx = 0; double dRetPose_Ry = 0; double dRetPose_Rz = 0;
```

// 计算结果质量变量定义

```
int quality = 0; int errorIndex_P1=0; int errorIndex_P2=0; int errorIndex_P3=0; int errorIndex_P4=0;
```

// 获取计算结果，函数调用后，nRet 值为 0，quality 的值为 2，errorIndex_P1~P3 的值为 1，errorIndex_P4 的值为 0。

```
int nRet= HRIF_CalTcp4P(0,0, dPose1_X, dPose1_Y, dPose1_Z, dPose1_Rx, dPose1_Ry, dPose1_Rz, dPose2_X,  
dPose2_Y, dPose2_Z, dPose2_Rx, dPose2_Ry, dPose2_Rz, dPose3_X, dPose3_Y, dPose3_Z, dPose3_Rx, dPose3_Ry,  
dPose3_Rz, dPose4_X, dPose4_Y, dPose4_Z, dPose4_Rx, dPose4_Ry, dPose4_Rz, dRetPose_X, dRetPose_Y,  
dRetPose_Z, dRetPose_Rx, dRetPose_Ry, dRetPose_Rz,  
quality,errorIndex_P1,errorIndex_P2,errorIndex_P3,errorIndex_P4);
```


3.7.20.HRIF_CalTCPOrt

3.7.20.1. 描述：标定 tcp 方向

✓ 输入变量

输入变量	名称	数据类型	有效范围	内容
boxID	电箱 ID	unsigned int	0~5	电箱 ID 号, 默认值=0
rbtID	机器人 ID	unsigned int	0~5	机器人 ID 号, 默认值=0
UCS_X- UCS_Rz	UCS 方向	double	-	当前设置的 ucs 方向: UCS_X: X 坐标, 单位[mm] UCS_Y: Y 坐标, 单位[mm] UCS_Z: Z 坐标, 单位[mm] UCS_Rx: Rx 坐标, 单位[°] UCS_Ry: Ry 坐标, 单位[°] UCS_Rz: Rz 坐标, 单位[°]
TCP_X- TCP_Rz	默认 TCP 的 位姿	double	-	点 2 在 Base 坐标系下系统默认 TCP 的位姿: TCP_X: X 坐标, 单位[mm] TCP_Y: Y 坐标, 单位[mm] TCP_Z: Z 坐标, 单位[mm] TCP_Rx: Rx 坐标, 单位[°] TCP_Ry: Ry 坐标, 单位[°] TCP_Rz: Rz 坐标, 单位[°]

✓ 输出变量

输出变量	名称	数据类型	有效范围	内容
RX-RY- RZ	计算结果	double	-	计算得出的 UCS 方向: Rx: Rx 坐标, 单位[°], 一般是 0 Ry: Ry 坐标, 单位[°], 一般是 0 Rz: Rz 坐标, 单位[°], 一般是 0

✓ 返回值

返回值	名称	数据类型	有效范围	内容
nRet	返回值	int	≥ 0 的整型值	nRet=0: 返回函数调用成功 nRet>0: 返回调用失败的错误码

3.7.20.2. 示例

// 定义默认 TCP 的位姿

```
double TCP_x=0;double TCP_y=0;double TCP_z=0;
double TCP_Rx=0;double TCP_Ry=0;double TCP_Rz=0;
```

// 定义 UCS 方向

```
double UCS_x=84.303;double UCS_y=341.461;double UCS_z=457.954;
double UCS_Rx=179.863;double UCS_Ry=1.916;double UCS_Rz=-107.973;
```

// 进行标定

```
int nRet=HRIF_CalTCPOrt(0, 0,UCS_x,UCS_y,UCS_z,UCS_Rx,UCS_Ry,UCS_Rz,
                        TCP_x,TCP_y,TCP_z,TCP_Rx,TCP_Ry,TCP_Rz,RX,RY,RZ);
```

3.7.21. HRIF_GetInverseDynamics

3.7.21.1.描述：动力学逆解，由指定关节位置、关节速度、关节加速度计算对应的关节扭矩。

✓ 输入变量

参数	名称	数据类型	有效范围	内容
nRbtID	机器人 id	unsigned int	0~5	机器 ID 号，默认值=0
dPose_J1-dPose_J6	需要计算逆解的关节位置	double	-	需要计算逆解的关节坐标： dPose_J1: 关节 1 坐标，单位[°] dPose_J2: 关节 2 坐标，单位[°] dPose_J3: 关节 3 坐标，单位[°] dPose_J4: 关节 4 坐标，单位[°] dPose_J5: 关节 5 坐标，单位[°] dPose_J6: 关节 6 坐标，单位[°]
dJ1-dJ6	需要计算逆解的关节速度	double	-	dVel_J1: 关节 1 速度，单位[°/s] dVel_J2: 关节 2 速度，单位[°/s] dVel_J3: 关节 3 速度，单位[°/s] dVel_J4: 关节 4 速度，单位[°/s] dVel_J5: 关节 5 速度，单位[°/s] dVel_J6: 关节 6 速度，单位[°/s]
dTcp_X-dTcp_Rz	需要计算逆解的关节加速度	double	-	dAcc_J1: 关节 1 加速度，单位[°/s²] dAcc_J2: 关节 2 加速度，单位[°/s²] dAcc_J3: 关节 3 加速度，单位[°/s²] dAcc_J4: 关节 4 加速度，单位[°/s²] dAcc_J5: 关节 5 加速度，单位[°/s²] dAcc_J6: 关节 6 加速度，单位[°/s²]

✓ 输出变量

返回值	名称	数据类型	有效范围	内容
dTorq_J1-dTorq_J6	计算所得的关节扭矩	double	-	dTorq_J1: 关节 1 扭矩，单位[Nm] dTorq_J2: 关节 2 扭矩，单位[Nm] dTorq_J3: 关节 3 扭矩，单位[Nm]

				dTorq_J4: 关节 4 扭矩, 单位[Nm]
				dTorq_J5: 关节 5 扭矩, 单位[Nm]
				dTorq_J6: 关节 6 扭矩, 单位[Nm]

✓ 返回值

返回值	名称	数据类型	有效范围	内容
nRet	返回值	int	≥ 0 的整型值	nRet=0: 返回函数调用成功 nRet>0: 返回调用失败的错误码

3.7.21.2. 示例

// 定义关节参数

```
double dPose_J1 = 0, dPose_J2 = 0, dPose_J3 = 90, dPose_J4 = 0, dPose_J5 = 90, dPose_J6 = 0;
double dVel_J1 = 0, dVel_J2 = 0, dVel_J3 = 0, dVel_J4 = 0, dVel_J5 = 0, dVel_J6 = 0;
double dAcc_J1 = 0, dAcc_J2 = 0, dAcc_J3 = 0, dAcc_J4 = 0, dAcc_J5 = 0, dAcc_J6 = 0;
double dTorq_J1 = 0, dTorq_J2 = 0, dTorq_J3 = 0, dTorq_J4 = 0, dTorq_J5 = 0, dTorq_J6 = 0;
```

// 获取逆解计算结果

```
int nRet = HRIF_GetInverseDynamics(0, dPose_J1, dPose_J2, dPose_J3, dPose_J4, dPose_J5, dPose_J6,
dVel_J1, dVel_J2, dVel_J3, dVel_J4, dVel_J5, dVel_J6, dAcc_J1, dAcc_J2, dAcc_J3, dAcc_J4, dAcc_J5, dAcc_J6,
dTorq_J1, dTorq_J2, dTorq_J3, dTorq_J4, dTorq_J5, dTorq_J6);
```

3.8. 工具坐标与用户坐标读写指令

3.8.1. HRIF_SetTCP

3.8.1.1. 描述：设置当前工具坐标，不写入配置文件，重启后失效。

✓ 输入变量

输入变量	名称	数据类型	有效范围	内容
boxID	电箱 ID	unsigned int	0~5	电箱 ID 号，默认值= 0
rbtID	机器人 ID	unsigned int	0~5	机器人 ID 号，默认值= 0
dTcp_X-dTcp_Rz	工具坐标	double	-	需要设置的工具坐标： dTcp_X: X 坐标，单位[mm] dTcp_Y: Y 坐标，单位[mm] dTcp_Z: Z 坐标，单位[mm] dTcp_Rx: Rx 坐标，单位[°] dTcp_Ry: Ry 坐标，单位[°] dTcp_Rz: Rz 坐标，单位[°]

✓ 返回值

返回值	名称	数据类型	有效范围	内容
nRet	返回值	int	>=0 的整型值	nRet=0: 返回函数调用成功 nRet>0: 返回调用失败的错误码

3.8.1.2. 示例

// 定义工具坐标变量

```
double dTcp_X = 0; double dTcp_Y = 0; double dTcp_Z = 0;
double dTcp_Rx = 0; double dTcp_Ry = 0; double dTcp_Rz = 0;
```

// 设置工具坐标

```
int nRet= HRIF_SetTCP(0,0, dTcp_X, dTcp_Y, dTcp_Z, dTcp_Rx, dTcp_Ry, dTcp_Rz);
```

3.8.2. HRIF_SetUCS

3.8.2.1. 描述：设置当前用户坐标，不写入配置文件，重启后失效。

✓ 输入变量

输入变量	名称	数据类型	有效范围	内容
boxID	电箱 ID	unsigned int	0~5	电箱 ID 号，默认值= 0
rbtID	机器人 ID	unsigned int	0~5	机器人 ID 号，默认值= 0
dUcs_X-dUcs_Rz	用户坐标	double	-	需要设置的用户坐标： dUcs_X: X 坐标，单位[mm] dUcs_Y: Y 坐标，单位[mm] dUcs_Z: Z 坐标，单位[mm] dUcs_Rx: Rx 坐标，单位[°] dUcs_Ry: Ry 坐标，单位[°] dUcs_Rz: Rz 坐标，单位[°]

✓ 返回值

返回值	名称	数据类型	有效范围	内容
nRet	返回值	int	>=0 的整型值	nRet=0: 返回函数调用成功 nRet>0: 返回调用失败的错误码

3.8.2.2. 示例

// 定义用户坐标变量

```
double dUcs_X = 0; double dUcs_Y = 0; double dUcs_Z = 0;
double dUcs_Rx = 0; double dUcs_Ry = 0; double dUcs_Rz = 0;
```

// 设置用户坐标

```
int nRet= HRIF_SetUCS(0,0, dUcs_X, dUcs_Y, dUcs_Z, dUcs_Rx, dUcs_Ry, dUcs_Rz);
```

3.8.3. HRIF_ReadCurTCP

3.8.3.1. 描述：读取当前设置的工具坐标值。

✓ 输入变量

输入变量	名称	数据类型	有效范围	内容
boxID	电箱 ID	unsigned int	0~5	电箱 ID 号，默认值= 0
rbtID	机器人 ID	unsigned int	0~5	机器人 ID 号，默认值= 0

✓ 输出变量

输出变量	名称	数据类型	有效范围	内容
dTcp_X-dTcp_Rz	工具坐标	double	-	读取到的工具坐标： dTcp_X: X 坐标，单位[mm] dTcp_Y: Y 坐标，单位[mm] dTcp_Z: Z 坐标，单位[mm] dTcp_Rx: Rx 坐标，单位[°] dTcp_Ry: Ry 坐标，单位[°] dTcp_Rz: Rz 坐标，单位[°]

✓ 返回值

返回值	名称	数据类型	有效范围	内容
nRet	返回值	int	≥0 的整型值	nRet=0: 返回函数调用成功 nRet>0: 返回调用失败的错误码

3.8.3.2. 示例

// 定义工具坐标变量

```
double dTcp_X = 0; double dTcp_Y = 0; double dTcp_Z = 0;
double dTcp_Rx = 0; double dTcp_Ry = 0; double dTcp_Rz = 0;
```

// 读取工具坐标

```
int nRet= HRIF_ReadCurTCP(0,0, dTcp_X, dTcp_Y, dTcp_Z, dTcp_Rx, dTcp_Ry, dTcp_Rz);
```

3.8.4. HRIF_ReadCurUCS

3.8.4.1. 描述：读取当前设置的用户坐标值。

✓ 输入变量

输入变量	名称	数据类型	有效范围	内容
boxID	电箱 ID	unsigned int	0~5	电箱 ID 号，默认值= 0
rbtID	机器人 ID	unsigned int	0~5	机器人 ID 号，默认值= 0

✓ 输出变量

输出变量	名称	数据类型	有效范围	内容
dUcs_X-dUcs_Rz	用户坐标	double	-	读取到的用户坐标： dUcs_X : X 坐标，单位[mm] dUcs_Y : Y 坐标，单位[mm] dUcs_Z : Z 坐标，单位[mm] dUcs_Rx : Rx 坐标，单位[°] dUcs_Ry : Ry 坐标，单位[°] dUcs_Rz : Rz 坐标，单位[°]

✓ 返回值

返回值	名称	数据类型	有效范围	内容
nRet	返回值	int	≥0 的整型值	nRet=0: 返回函数调用成功 nRet>0: 返回调用失败的错误码

3.8.4.2. 示例

// 定义用户坐标变量

```
double dUcs_X = 0; double dUcs_Y = 0; double dUcs_Z = 0;
```

```
double dUcs_Rx = 0; double dUcs_Ry = 0; double dUcs_Rz = 0;
```

// 读取用户坐标

```
int nRet= HRIF_ReadCurUCS(0,0, dUcs_X, dUcs_Y, dUcs_Z, dUcs_Rx, dUcs_Ry, dUcs_Rz);
```


3.8.5. HRIF_SetTCPByName

3.8.5.1. 描述：通过名称设置工具坐标列表中的值为当前工具坐标，对应名称为示教器配置页面 TCP 示教的工具名称。

✓ 输入变量

输入变量	名称	数据类型	有效范围	内容
boxID	电箱 ID	unsigned int	0~5	电箱 ID 号，默认值= 0
rbtID	机器人 ID	unsigned int	0~5	机器人 ID 号，默认值= 0
sTcpName	工具坐标名称	string	-	需要设置的工具坐标名称

✓ 返回值

返回值	名称	数据类型	有效范围	内容
nRet	返回值	int	≥ 0 的整型值	nRet=0: 返回函数调用成功 nRet>0: 返回调用失败的错误码

3.8.5.2. 示例

// 需要下发的工具坐标名称

```
string sTcpName = "TCP";
```

// 设置工具坐标

```
int nRet= HRIF_SetTCPByName(0,0, sTcpName);
```

3.8.6. HRIF_SetUCSByName

3.8.6.1. 描述：通过名称设置用户坐标列表中的值为当前用户坐标，对应名称为示教器配置页面用户坐标示教的名称。

✓ 输入变量

输入变量	名称	数据类型	有效范围	内容
boxID	电箱 ID	unsigned int	0~5	电箱 ID 号，默认值= 0
rbtID	机器人 ID	unsigned int	0~5	机器人 ID 号，默认值= 0
sUcsName	用户坐标名称	string	-	需要设置的用户坐标名称

✓ 返回值

返回值	名称	数据类型	有效范围	内容
nRet	返回值	int	≥ 0 的整型值	nRet = 0: 返回函数调用成功 nRet > 0: 返回调用失败的错误码

3.8.6.2. 示例

// 需要下发的用户坐标名称

```
string sUcsName = "UCS";
```

// 设置用户坐标

```
int nRet= HRIF_SetUCSByName(0,0, sUcsName);
```

3.8.7. HRIF_ReadTCPByName

3.8.7.1. 描述：通过名称读取指定 TCP 坐标，对应名称为示教器配置页面 TCP 示教的工具名称。

✓ 输入变量

输入变量	名称	数据类型	有效范围	内容
boxID	电箱 ID	unsigned int	0~5	电箱 ID 号，默认值= 0
rbtID	机器人 ID	unsigned int	0~5	机器人 ID 号，默认值= 0
sTcpName	工具坐标名称	string	-	需要读取的工具坐标名称

✓ 输出变量

输出变量	名称	数据类型	有效范围	内容
dTcp_X-dTcp_Rz	工具坐标	double	-	读取到的工具坐标： dTcp_X : X 坐标，单位[mm] dTcp_Y : Y 坐标，单位[mm] dTcp_Z : Z 坐标，单位[mm] dTcp_Rx : Rx 坐标，单位[°] dTcp_Ry : Ry 坐标，单位[°] dTcp_Rz : Rz 坐标，单位[°]

✓ 返回值

返回值	名称	数据类型	有效范围	内容
nRet	返回值	int	≥0 的整型值	nRet = 0: 返回函数调用成功 nRet > 0: 返回调用失败的错误码

3.8.7.2. 示例

// 需要读取的工具坐标名称

```
string sTcpName = "TCP";
```

// 定义读取到的工具坐标结果变量

```
double dTcp_X = 0; double dTcp_Y = 0; double dTcp_Z = 0;
```

```
double dTcp_Rx = 0; double dTcp_Ry = 0; double dTcp_Rz = 0;
```

// 读取工具坐标

```
int nRet= HRIF_ReadTCPByName(0,0,sTcpName,dTcp_X,dTcp_Y,dTcp_Z, dTcp_Rx, dTcp_Ry, dTcp_Rz);
```

3.8.8. HRIF_ReadTCPList

3.8.8.1. 描述：读取系统中保存的 TCP 名称列表。

✓ 输入变量

输入变量	名称	数据类型	有效范围	内容
boxID	电箱 ID	unsigned int	0~5	电箱 ID 号，默认值=0
rbtID	机器人 ID	unsigned int	0~5	机器人 ID 号，默认值=0

✓ 输出变量

输出变量	名称	数据类型	有效范围	内容
TCPList	TCP 名称列表	vector<string>	-	保存读取到的所有 TCP 名称

✓ 返回值

返回值	名称	数据类型	有效范围	内容
nRet	返回值	int	≥ 0 的整型值	nRet = 0: 返回函数调用成功 nRet > 0: 返回调用失败的错误码

3.8.8.2. 示例

// 定义保存 TCP 列表的变量

```
vector<string> TCPList;
```

// 读取 TCP 名称列表

```
int nRet = HRIF_ReadTCPList(0,0, TCPList);
```

3.8.9. HRIF_ReadUCSList

3.8.9.1. 描述：读取系统中保存的 UCS 名称列表。

✓ 输入变量

输入变量	名称	数据类型	有效范围	内容
boxID	电箱 ID	unsigned int	0~5	电箱 ID 号，默认值= 0
rbtID	机器人 ID	unsigned int	0~5	机器人 ID 号，默认值= 0

✓ 输出变量

输出变量	名称	数据类型	有效范围	内容
UCSList	UCS 名称列表	vector<string>	-	保存读取到的所有 UCS 名称

✓ 返回值

返回值	名称	数据类型	有效范围	内容
nRet	返回值	int	≥ 0 的整型值	nRet = 0: 返回函数调用成功 nRet > 0: 返回调用失败的错误码

3.8.9.2. 示例

// 定义保存 UCS 列表的变量

```
vector<string> UCSList;
```

// 读取用户坐标系名称列表

```
int nRet = HRIF_ReadUCSList(0,0, UCSList);
```

3.8.10. HRIF_ReadUCSByName

3.8.10.1. 描述：通过名称读取指定 UCS 坐标，对应名称为示教器配置页面用户坐标示教的用户坐标名称。

✓ 输入变量

输入变量	名称	数据类型	有效范围	内容
boxID	电箱 ID	unsigned int	0~5	电箱 ID 号，默认值=0
rbtID	机器人 ID	unsigned int	0~5	机器人 ID 号，默认值=0
sUcsName	用户坐标名称	string	-	需要读取的用户坐标名称

✓ 输出变量

输出变量	名称	数据类型	有效范围	内容
dUcs_X-dUcs_Rz	用户坐标	double	-	读取到的用户坐标： dUcs_X : X 坐标，单位[mm] dUcs_Y : Y 坐标，单位[mm] dUcs_Z : Z 坐标，单位[mm] dUcs_Rx : Rx 坐标，单位[°] dUcs_Ry : Ry 坐标，单位[°] dUcs_Rz : Rz 坐标，单位[°]

✓ 返回值

返回值	名称	数据类型	有效范围	内容
nRet	返回值	int	≥0 的整型值	nRet = 0: 返回函数调用成功 nRet > 0: 返回调用失败的错误码

3.8.10.2. 示例

// 需要读取的用户坐标名称

```
string sUcsName = "UCS";
```

// 定义读取到的用户坐标变量

```
double dUcs_X = 0; double dUcs_Y = 0; double dUcs_Z = 0;
```

```
double dUcs_Rx = 0; double dUcs_Ry = 0; double dUcs_Rz = 0;
```

// 读取用户坐标

```
int nRet= HRIF_ReadUCSByName(0,0, sUcsName , dUcs_X, dUcs_Y, dUcs_Z, dUcs_Rx, dUcs_Ry, dUcs_Rz);
```

3.8.11. HRIF_ConfigTCP

3.8.11.1. 描述：新建指定名称的 TCP 和值。

✓ 输入变量

输入变量	名称	数据类型	有效范围	内容
boxID	电箱 ID	unsigned int	0~5	电箱 ID 号，默认值= 0
rbtID	机器人 ID	unsigned int	0~5	机器人 ID 号，默认值= 0
sTcpName	工具坐标名称	string	-	需要新建的工具坐标名称
dTcp_X-dTcp_Rz	工具坐标	double	-	需要设置的工具坐标： dTcp_X: X 坐标，单位[mm] dTcp_Y: Y 坐标，单位[mm] dTcp_Z: Z 坐标，单位[mm] dTcp_Rx: Rx 坐标，单位[°] dTcp_Ry: Ry 坐标，单位[°] dTcp_Rz: Rz 坐标，单位[°]

✓ 返回值

返回值	名称	数据类型	有效范围	内容
nRet	返回值	int	>=0 的整型值	nRet=0: 返回函数调用成功 nRet>0: 返回调用失败的错误码

3.8.11.2. 示例

// 定义工具坐标名称

```
string sTcpName = "TCP_A";
```

// 定义工具坐标变量

```
double dTcp_X = 0; double dTcp_Y = 0; double dTcp_Z = 10;
```

```
double dTcp_Rx = 0; double dTcp_Ry = 0; double dTcp_Rz = 0;
```

// 新建指定名称的 TCP 和值

```
int nRet = HRIF_ConfigTCP(0,0, sTcpName , dTcp_X, dTcp_Y, dTcp_Z, dTcp_Rx, dTcp_Ry, dTcp_Rz);
```

3.8.12. HRIF_ConfigUCS

3.8.12.1. 描述：新建指定名称的 UCS 和值。

✓ 输入变量

输入变量	名称	数据类型	有效范围	内容
boxID	电箱 ID	unsigned int	0~5	电箱 ID 号, 默认值= 0
rbtID	机器人 ID	unsigned int	0~5	机器人 ID 号, 默认值= 0
sUcsName	用户坐标名称	string	-	需要新建的用户坐标名称
dUcs_X-dUcs_Rz	用户坐标	double	-	需要设置的用户坐标: dUcs_X: X 坐标, 单位[mm] dUcs_Y: Y 坐标, 单位[mm] dUcs_Z: Z 坐标, 单位[mm] dUcs_Rx: Rx 坐标, 单位[°] dUcs_Ry: Ry 坐标, 单位[°] dUcs_Rz: Rz 坐标, 单位[°]

✓ 返回值

返回值	名称	数据类型	有效范围	内容
nRet	返回值	int	>=0 的整型值	nRet=0: 返回函数调用成功 nRet>0: 返回调用失败的错误码

3.8.12.2. 示例

// 定义用户坐标名称

```
string sUcsName = " Point_A";
```

// 定义用户坐标变量

```
double dUcs_X = 0; double dUcs_Y = 0; double dUcs_Z = 10;
```

```
double dUcs_Rx = 0; double dUcs_Ry = 0; double dUcs_Rz = 0;
```

// 新建指定名称的 UCS 和值

```
int nRet = HRIF_ConfigUCS(0,0, sUcsName , dUcs_X, dUcs_Y, dUcs_Z, dUcs_Rx, dUcs_Ry, dUcs_Rz);
```


3.8.13.HRIF_SetBaseInstallingAngle

3.8.13.1. 描述：设定机座安装角度

✓ 输入变量

输入变量	名称	数据类型	有效范围	内容
boxID	电箱 ID	int	0~5	电箱 ID 号, 默认值= 0
rbtID	机器人 ID	int	0~5	机器人 ID 号, 默认值= 0
nRotation	机座旋转角度	int	-360~360	设定的机座旋转角度, 单位[°]
nTilt	机座倾斜角度	int	-360~360	设定的机座倾斜角度, 单位[°]

✓ 返回值

返回值	名称	数据类型	有效范围	内容
nRet	返回值	int	>=0 的整型值	nRet=0: 返回函数调用成功 nRet>0: 返回调用失败的错误码

3.8.13.2. 示例

// 定义机座旋转角度

int nRotation= 0;

// 定义机座倾斜角度

int nTilt= 0;

// 设定机座安装角度

int nRet = HRIF_SetBaseInstallingAngle(0,0, nRotation,nTilt);

3.8.14.HRIF_GetBaseInstallingAngle

3.8.14.1. 描述：读取机座安装角度

✓ 输入变量

输入变量	名称	数据类型	有效范围	内容
boxID	电箱 ID	int	0~5	电箱 ID 号, 默认值= 0
rbtID	机器人 ID	int	0~5	机器人 ID 号, 默认值= 0

✓ 输出变量

输出变量	名称	数据类型	有效范围	内容
nRotation	机座旋转角度	int	-360~360	读取的机座旋转角度, 单位[°]
nTilt	机座倾斜角度	int	-360~360	读取的机座倾斜角度, 单位[°]

✓ 返回值

返回值	名称	数据类型	有效范围	内容
nRet	返回值	int	>=0 的整型值	nRet=0: 返回函数调用成功 nRet>0: 返回调用失败的错误码

3.8.14.2. 示例

// 获取机座旋转角度

```
int nRotation= 0;
```

// 获取机座倾斜角度

```
int nTilt= 0;
```

// 设定机座安装角度

```
int nRet = HRIF_GetBaseInstallingAngle(0,0,nRotation,nTilt);
```

3.8.15.HRIF_SetDftTCP

3.8.15.1. 描述：设置当前默认的 TCP

✓ 输入变量

输入变量	名称	数据类型	有效范围	内容
boxID	电箱 ID	int	0~5	电箱 ID 号, 默认值= 0
rbtID	机器人 ID	int	0~5	机器人 ID 号, 默认值= 0
Name	TCP 名称	string	-	需要设置为默认的 TCP 的名字

✓ 返回值

返回值	名称	数据类型	有效范围	内容
nRet	返回值	int	≥ 0 的整型值	nRet=0: 返回函数调用成功 nRet>0: 返回调用失败的错误码

3.8.15.2. 示例

// 定义 TCP 名称

```
string TcpName="TCP";
```

// 设置当前默认的 TCP

```
int nRet=HRIF_SetDftTCP(0,0,TcpName);
```

3.8.16.HRIF_ReadActCoord

3.8.16.1. 描述:通过输入 TCP 和 UCS 名称, 获取机械臂当前状态下关节坐标值 and 用户坐标

✓ 输入变量

输入变量	名称	数据类型	有效范围	内容
boxID	电箱 ID	int	0~5	电箱 ID 号, 默认值=0
rbtID	机器人 ID	int	0~5	机器人 ID 号, 默认值=0
TcpName	TCP 名称	String		tcp 名称
UcsName	Ucs 名称	String		Ucs 名称

✓ 输出变量

输出变量	名称	数据类型	有效范围	内容
J1~J6	关节坐标	double	~	当前关节坐标
dX-dRz	坐标	double	~	指定 Ucs 和 Tcp 下的坐标: dTcp_X: X 坐标, 单位[mm] dTcp_Y: Y 坐标, 单位[mm] dTcp_Z: Z 坐标, 单位[mm] dTcp_Rx: Rx 坐标, 单位[°] dTcp_Ry: Ry 坐标, 单位[°] dTcp_Rz: Rz 坐标, 单位[°]

✓ 返回值

返回值	名称	数据类型	有效范围	内容
nRet	返回值	int	>=0 的整型值	nRet=0: 返回函数调用成功 nRet>0: 返回调用失败的错误码

3.8.16.2. 示例

// 定义 TCP 名称

```
string TcpName="TCP";
```

// 定义 UCS 名称

```
string UcsName="Base";
```

// 定义关节变量

```
double dJ1 = 0; double dJ2 = 0; double dJ3 = 0;
```

```
double dJ4 = 0; double dJ5 = 0; double dJ6 = 0;
```

// 定义用户坐标变量

```
double dX = 0; double dY = 0; double dZ = 0;
```

```
double dRx = 0; double dRy = 0; double dRz = 0;
```

// 获取当前的关节坐标值和用户坐标系下的值

```
int nRet=HRIF_ReadActCoord(0,0,0,TcpName,UcsName,dJ1,dJ2,dJ3,dJ4,dJ5,dJ6,dX,dY,dZ,dRx,dRy,dRz);
```

3.8.17.HRIF_ReadActCoord_nJ

3.8.17.1. 描述:通过输入 TCP 和 UCS 名称, 获取机械臂当前状态下关节坐标值和用户坐标

✓ 输入变量

输入变量	名称	数据类型	有效范围	内容
boxID	电箱 ID	int	0~5	电箱 ID 号, 默认值=0
rbtID	机器人 ID	int	0~5	机器人 ID 号, 默认值=0
TcpName	TCP 名称	String		tcp 名称
UcsName	Ucs 名称	String		Ucs 名称

✓ 输出变量

输出变量	名称	数据类型	有效范围	内容
Joints	当前关节坐标	JointsData	-	Joints.dJoint[i]: 关节坐标, 单位[°]
dX-dRz	坐标	double	~	指定 Ucs 和 Tcp 下的坐标: dTcp_X: X 坐标, 单位[mm] dTcp_Y: Y 坐标, 单位[mm] dTcp_Z: Z 坐标, 单位[mm] dTcp_Rx: Rx 坐标, 单位[°] dTcp_Ry: Ry 坐标, 单位[°] dTcp_Rz: Rz 坐标, 单位[°]

✓ 返回值

返回值	名称	数据类型	有效范围	内容
nRet	返回值	int	>=0 的整型值	nRet=0: 返回函数调用成功 nRet>0: 返回调用失败的错误码

3.8.17.2. 示例

// 定义 TCP 名称

```
string TcpName="TCP";
```

// 定义 UCS 名称

```
string UcsName="Base";
```

// 定义关节变量

JointsData Joints;

// 定义用户坐标变量

double dX = 0; double dY = 0; double dZ = 0;

double dRx = 0; double dRy = 0; double dRz = 0;

// 获取当前的关节坐标值和用户坐标系下的值

int nRet=HRIF_ReadActCoord_nJ(0,0,0,TcpName,UcsName,Joints,dX,dY,dZ,dRx,dRy,dRz);

3.9. 力控控制指令

3.9.1. HRIF_SetForceControlState

3.9.1.1. 描述：开启/关闭力控。

✓ 输入变量

输入变量	名称	数据类型	有效范围	内容
boxID	电箱 ID	unsigned int	0~5	电箱 ID 号，默认值= 0
rbtID	机器人 ID	unsigned int	0~5	机器人 ID 号，默认值= 0
nState	力控状态	int	0/1	力控状态 0: 关闭力控 1: 开启力控

注：力控开启后，机器人将进入运动状态（力控探寻阶段）。

✓ 返回值

返回值	名称	数据类型	有效范围	内容
nRet	返回值	int	>=0 的整型值	nRet = 0: 返回函数调用成功 nRet >0: 返回调用失败的错误码

3.9.1.2. 示例

// 开启力控

```
int nState = 1;
```

```
int nRet = HRIF_SetForceControlState(0,0, nState);
```

// 关闭力控

```
int nState = 0;
```

```
int nRet = HRIF_SetForceControlState(0,0, nState);
```


3.9.2. HRIF_ReadForceControlState

3.9.2.1. 描述：读取当前力控状态。

✓ 输入变量

输入变量	名称	数据类型	有效范围	内容
boxID	电箱 ID	unsigned int	0~5	电箱 ID 号，默认值= 0
rbtID	机器人 ID	unsigned int	0~5	机器人 ID 号，默认值= 0

✓ 输出变量

输出变量	名称	数据类型	有效范围	内容
nState	力控状态	int	0/1/2/3	力控状态： 0：关闭状态 1：开力控探寻状态 2：力控探寻完成状态 3：力控自由驱动状态

✓ 返回值

返回值	名称	数据类型	有效范围	内容
nRet	返回值	int	>=0 的整型值	nRet = 0: 返回函数调用成功 nRet > 0: 返回调用失败的错误码

3.9.2.2. 示例

```
int nState = 0;
// 读取当前力控状态
int nRet = HRIF_ReadForceControlState(0,0, nState);
```

3.9.3. HRIF_SetForceToolCoordinateMotion

3.9.3.1. 描述：设置力控坐标系控制方向为 Tool 坐标系方向。

✓ 输入变量

输入变量	名称	数据类型	有效范围	内容
boxID	电箱 ID	unsigned int	0~5	电箱 ID 号，默认值= 0
rbtID	机器人 ID	unsigned int	0~5	机器人 ID 号，默认值= 0
nMode	模式	int	0/1	当前力控坐标系是否为 Tool 坐标系方向 0：当前力控坐标系为当前 UCS 方向 1：当前力控坐标系为 Tool 方向

✓ 返回值

返回值	名称	数据类型	有效范围	内容
nRet	返回值	int	>=0 的整型值	nRet = 0: 返回函数调用成功 nRet >0: 返回调用失败的错误码

3.9.3.2. 示例

```
int nMode = 1;
// 设置力控坐标系控制方向为 Tool 坐标系方向
int nRet = HRIF_SetForceToolCoordinateMotion(0,0, nMode);
```

3.9.4. HRIF_ForceControlinterrupt

3.9.4.1. 描述：暂停力控运动。（此接口功能已屏蔽）

✓ 输入变量

输入变量	名称	数据类型	有效范围	内容
boxID	电箱 ID	unsigned int	0~5	电箱 ID 号，默认值=0
rbtID	机器人 ID	unsigned int	0~5	机器人 ID 号，默认值=0

✓ 返回值

返回值	名称	数据类型	有效范围	内容
nRet	返回值	int	>=0 的整型值	nRet = 0: 返回函数调用成功 nRet >0: 返回调用失败的错误码

3.9.4.2. 示例

// 暂停力控运动

```
int nRet = HRIF_ForceControlinterrupt(0,0);
```

3.9.5. HRIF_ForceControlContinue

3.9.5.1. 描述：继续力控运动。（此接口功能已屏蔽）

✓ 输入变量

输入变量	名称	数据类型	有效范围	内容
boxID	电箱 ID	unsigned int	0~5	电箱 ID 号，默认值= 0
rbtID	机器人 ID	unsigned int	0~5	机器人 ID 号，默认值= 0

✓ 返回值

返回值	名称	数据类型	有效范围	内容
nRet	返回值	int	>=0 的整型值	nRet = 0: 返回函数调用成功 nRet >0: 返回调用失败的错误码

3.9.5.2. 示例

// 继续力控运动

```
int nRet = HRIF_ForceControlContinue(0,0);
```

3.9.6. HRIF_SetForceZero

3.9.6.1. 描述：力控清零，在原有数据的基础上重新标定力传感器。

✓ 输入变量

输入变量	名称	数据类型	有效范围	内容
boxID	电箱 ID	unsigned int	0~5	电箱 ID 号，默认值= 0
rbtID	机器人 ID	unsigned int	0~5	机器人 ID 号，默认值= 0

✓ 返回值

返回值	名称	数据类型	有效范围	内容
nRet	返回值	int	>=0 的整型值	nRet = 0: 返回函数调用成功 nRet >0: 返回调用失败的错误码

3.9.6.2. 示例

// 力控清零

```
int nRet = HRIF_SetForceZero(0,0);
```

3.9.7. HRIF_SetMaxSearchVelocities

3.9.7.1. 描述：设置力控探寻的最大直线速度及角速度。

✓ 输入变量

输入变量	名称	数据类型	有效范围	内容
boxID	电箱 ID	unsigned int	0~5	电箱 ID 号, 默认值= 0
rbtID	机器人 ID	unsigned int	0~5	机器人 ID 号, 默认值= 0
dMaxLinearVelocity	直线速度	double	1~50	探寻最大直线速度, 单位[mm/s]
dMaxAngularVelocity	角速度	double	1~10	探寻最大角速度, 单位[°/s]

✓ 返回值

返回值	名称	数据类型	有效范围	内容
nRet	返回值	int	>=0 的整型值	nRet = 0: 返回函数调用成功 nRet >0: 返回调用失败的错误码

3.9.7.2. 示例

// 力控探寻最大直线速度

```
double dMaxLinearVelocity = 15;
```

// 力控探寻最大姿态角速度

```
double dMaxAngularVelocity = 5;
```

// 设置力控探寻最大直线速度及角速度

```
int nRet = HRIF_SetMaxSearchVelocities(0,0, dMaxLinearVelocity, dMaxAngularVelocity);
```

3.9.8. HRIF_SetControlFreedom

3.9.8.1. 描述：设置力控探寻自由度。

✓ 输入变量

输入变量	名称	数据类型	有效范围	内容
boxID	电箱 ID	unsigned int	0~5	电箱 ID 号，默认值= 0
rbtID	机器人 ID	unsigned int	0~5	机器人 ID 号，默认值= 0
nX-nRz	各方向自由度	int	0/1	各方向探寻自由度开关： 0：关闭 1：开启

✓ 返回值

返回值	名称	数据类型	有效范围	内容
nRet	返回值	int	>=0 的整型值	nRet = 0: 返回函数调用成功 nRet >0: 返回调用失败的错误码

3.9.8.2. 示例

// 定义力控探寻自由度状态

int nX = 0; int nY = 0; int nZ = 1;

int nRx = 0; int nRy = 0; int nRz = 0;

// 设置力控探寻自由度状态

int nRet = HRIF_SetControlFreedom(0,0, nX, nY, nZ, nRx, nRy, nRz);

3.9.9. HRIF_SetForceControlStrategy

3.9.9.1. 描述：设置力控控制策略。

✓ 输入变量

输入变量	名称	数据类型	有效范围	内容
boxID	电箱 ID	unsigned int	0~5	电箱 ID 号，默认值= 0
rbtID	机器人 ID	unsigned int	0~5	机器人 ID 号，默认值= 0
nStrategy	控制策略	int	0/2	力控控制策略： 0：恒力模式 2：越障模式

✓ 返回值

返回值	名称	数据类型	有效范围	内容
nRet	返回值	int	≥ 0 的整型值	nRet = 0: 返回函数调用成功 nRet > 0: 返回调用失败的错误码

3.9.9.2. 示例

// 定义力控策略

```
int nStrategy = 0;
```

// 设置力控策略为恒力模式

```
int nRet = HRIF_SetForceControlStrategy(0,0, nStrategy);
```


3.9.10. HRIF_SetFreeDrivePositionAndOrientation

3.9.10.1. 描述：设置力传感器中心相对于法兰盘的安装位置和方向。

✓ 输入变量

输入变量	名称	数据类型	有效范围	内容
boxID	电箱 ID	unsigned int	0~5	电箱 ID 号，默认值= 0
rbtID	机器人 ID	unsigned int	0~5	机器人 ID 号，默认值= 0
dX -dRz	笛卡尔坐标	double	-	力传感器相对于法兰盘安装位置和姿态： dX: X 坐标，单位[mm] dY: Y 坐标，单位[mm] dZ: Z 坐标，单位[mm] dRx: Rx 坐标，单位[°] dRy: Ry 坐标，单位[°] dRz: Rz 坐标，单位[°]

✓ 返回值

返回值	名称	数据类型	有效范围	内容
nRet	返回值	int	>=0 的整型值	nRet = 0: 返回函数调用成功 nRet >0: 返回调用失败的错误码

3.9.10.2. 示例

// 定义力传感器安装位置和方向

```
double dX = 0; double dY = 0; double dZ = 0;
```

```
double dRx = 0; double dRy = 0; double dRz = 0;
```

// 设置力传感器的安装位置和方向

```
int nRet= HRIF_SetFreeDrivePositionAndOrientation(0,0, dX , dY , dZ , dRx , dRy , dRz );
```

3.9.11. HRIF_SetPIDControlParams

3.9.11.1. 描述：设置恒力控 PID 参数。（此接口已屏蔽）

✓ 输入变量

输入变量	名称	数据类型	有效范围	内容
boxID	电箱 ID	unsigned int	0~5	电箱 ID 号，默认值=0
rbtID	机器人 ID	unsigned int	0~5	机器人 ID 号，默认值=0
dFp	PID 参数	double	-	PID 参数 Fp
dFi	PID 参数	double	-	PID 参数 Fi
dFd	PID 参数	double	-	PID 参数 Fd
dTp	PID 参数	double	-	PID 参数 Tp
dTi	PID 参数	double	-	PID 参数 Ti
dTd	PID 参数	double	-	PID 参数 Td

✓ 返回值

返回值	名称	数据类型	有效范围	内容
nRet	返回值	int	>=0 的整型值	nRet = 0: 返回函数调用成功 nRet >0: 返回调用失败的错误码

3.9.11.2. 示例

// 定义 PID 参数

```
double dFp = 1.0; double dFi= 0.1; double dFd = 0;
```

```
double dTp = 1.0; double dTi = 0.1; double dTd = 0;
```

// 设置 PID 参数

```
int nRet = HRIF_SetPIDControlParams(0,0, dFp, dFi, dFd, dTp, dTi, dTd);
```

3.9.12. HRIF_SetMassParams

3.9.12.1. 描述：设置恒力控质量/惯量控制参数。

✓ 输入变量

输入变量	名称	数据类型	有效范围	内容
boxID	电箱 ID	unsigned int	0~5	电箱 ID 号，默认值= 0
rbtID	机器人 ID	unsigned int	0~5	机器人 ID 号，默认值= 0
dXMass -dRzMass	质量/惯量控制 参数	double	-	质量/惯量控制参数： dXMass: X 方向质量参数 dYMass: Y 方向质量参数 dZMass: Z 方向质量参数 dRxMass: Rx 方向惯量参数 dRyMass: Ry 方向惯量参数 dRzMass: Rz 方向惯量参数

✓ 返回值

返回值	名称	数据类型	有效范围	内容
nRet	返回值	int	>=0 的整型值	nRet = 0: 返回函数调用成功 nRet > 0: 返回调用失败的错误码

3.9.12.2. 示例

// 定义质量/惯量参数

```
double dXMass = 40; double dYMass = 40; double dZMass = 40;
```

```
double dRxMass = 10; double dRyMass = 10; double dRzMass = 10;
```

// 设置质量/惯量参数

```
int nRet = HRIF_SetMassParams(0,0,dXMass,dYMass,dZMass,dRxMass,dRyMass,dRzMass);
```

3.9.13. HRIF_SetDampParams

3.9.13.1. 描述：设置恒力控制阻尼(b)控制参数。

✓ 输入变量

输入变量	名称	数据类型	有效范围	内容
boxID	电箱 ID	unsigned int	0~5	电箱 ID 号，默认值= 0
rbtID	机器人 ID	unsigned int	0~5	机器人 ID 号，默认值= 0
dXDamp -dRzDamp	阻尼控制参数	double	-	阻尼控制参数： dXDamp: X 方向阻尼参数 dYDamp: Y 方向阻尼参数 dZDamp: Z 方向阻尼参数 dRxDamp: Rx 方向阻尼参数 dRyDamp: Ry 方向阻尼参数 dRzDamp: Rz 方向阻尼参数

✓ 返回值

返回值	名称	数据类型	有效范围	内容
nRet	返回值	int	>=0 的整型值	nRet = 0: 返回函数调用成功 nRet > 0: 返回调用失败的错误码

3.9.13.2. 示例

// 定义阻尼参数

```
double dXDamp = 800; double dYDamp = 800; double dZDamp = 800;
```

```
double dRxDamp = 40; double dRyDamp = 40; double dRzDamp = 40;
```

// 设置阻尼参数

```
int nRet = HRIF_SetDampParams(0,0,dXDamp,dYDamp,dZDamp,dRxDamp,dRyDamp,dRzDamp);
```

3.9.14. HRIF_SetStiffParams

3.9.14.1. 描述：设置恒力控制刚度(k)控制参数。

✓ 输入变量

输入变量	名称	数据类型	有效范围	内容
boxID	电箱 ID	unsigned int	0~5	电箱 ID 号，默认值= 0
rbtID	机器人 ID	unsigned int	0~5	机器人 ID 号，默认值= 0
dXStiff -dRzStiff	刚度控制参数	double	-	刚度控制参数： dXStiff: X 方向刚度参数 dYStiff: Y 方向刚度参数 dZStiff: Z 方向刚度参数 dRxStiff: Rx 方向刚度参数 dRyStiff: Ry 方向刚度参数 dRzStiff: Rz 方向刚度参数

✓ 返回值

返回值	名称	数据类型	有效范围	内容
nRet	返回值	int	>=0 的整型值	nRet = 0: 返回函数调用成功 nRet >0: 返回调用失败的错误码

3.9.14.2. 示例

// 定义刚度参数

```
double dXStiff = 1000; double dYStiff = 1000; double dZStiff = 1000;
```

```
double dRxStiff = 100; double dRyStiff = 100; double dRzStiff = 100;
```

// 设置刚度参数

```
int nRet = HRIF_SetStiffParams(0,0,dXStiff,dYStiff,dZStiff,dRxStiff,dRyStiff,dRzStiff);
```

3.9.15. HRIF_SetForceControlGoal

3.9.15.1. 描述：设置恒力控目标力大小。

✓ 输入变量

输入变量	名称	数据类型	有效范围	内容
boxID	电箱 ID	unsigned int	0~5	电箱 ID 号，默认值= 0
rbtID	机器人 ID	unsigned int	0~5	机器人 ID 号，默认值= 0
dXForce -dRzForce	力控目标力	double	-	恒力控目标力大小： dXForce: X 方向，单位[N] dYForce: Y 方向，单位[N] dZForce: Z 方向，单位[N] dRxForce: Rx 方向，单位[N·m] dRyForce: Ry 方向，单位[N·m] dRzForce: Rz 方向，单位[N·m]

✓ 返回值

返回值	名称	数据类型	有效范围	内容
nRet	返回值	int	>=0 的整型值	nRet = 0: 返回函数调用成功 nRet >0: 返回调用失败的错误码

3.9.15.2. 示例

// 定义力控目标力

```
double dXForce = 0; double dYForce = 0; double dZForce = 10;
```

```
double dRxForce = 0; double dRyForce = 0; double dRzForce = 0;
```

// 设置力控目标力 Z 方向 10N

```
int nRet = HRIF_SetForceControlGoal(0,0, dXForce, dYForce, dZForce, dRxForce, dRyForce, dRzForce);
```

3.9.16.HRIF_SetControlGoal

3.9.16.1. 描述：设置力控目标力和目标距离(0,0,力控目标距离暂未启用)。

✓ 输入变量

输入变量	名称	数据类型	有效范围	内容
boxID	电箱 ID	int	0~5	电箱 ID 号，默认值=0
rbtID	机器人 ID	int	0~5	机器人 ID 号，默认值=0
dWrench_X-dWrench_Rz	力控目标力	float	-	力控目标力： dWrenchX: X 方向，单位[N] dWrenchY: Y 方向，单位[N] dWrenchZ: Z 方向，单位[N] dWrenchRx: Rx 方向，单位[N • m] dWrenchRy: Ry 方向，单位[N • m] dWrenchRz: Rz 方向，单位 N • m
dDistance_X-dDistance_Rz	力控目标距离	float	-	力控目标距离： dDistanceX: X 方向，单位[mm] dDistanceY: Y 方向，单位[mm] dDistanceZ: Z 方向，单位[mm] dDistanceRx: Rx 方向，单位[°] dDistanceRy: Ry 方向，单位[°] dDistanceRz: Rz 方向，单位[°]

✓ 返回值

返回值	名称	数据类型	有效范围	内容
nRet	返回值	int	>=0 的整型值	nRet=0：返回函数调用成功 nRet>0：返回调用失败的错误码

3.9.16.2. 示例

// 定义力控目标力

```
double dWrench_X=0; double dWrench_Y=0; double dWrench_Z=0;
double dWrench_Rx=0; double dWrench_Ry=0; double dWrench_Rz=0;
```

// 定义力控目标距离

```
double dDistance_X=0; double dDistance_Y=0; double dDistance_Z=0;
double dDistance_Rx=0; double dDistance_Ry=0; double dDistance_Rz=0;
```

```
Int nRet = HRIF_SetControlGoal ( 0, 0, dWrench_X, dWrench_Y, dWrench_Z, dWrench_Rx, dWrench_Ry,
dWrench_Rz ,dDistance_X ,dDistance_Y ,dDistance_Z ,dDistance_Rx ,dDistance_Ry ,dDistance_Rz );
```

3.9.17. HRIF_SetForceDataLimit

3.9.17.1. 描述：设置力保护限制范围，力传感器超过此范围后控制器断电。

✓ 输入变量

输入变量	名称	数据类型	有效范围	内容
boxID	电箱 ID	unsigned int	0~5	电箱 ID 号，默认值= 0
rbtID	机器人 ID	unsigned int	0~5	机器人 ID 号，默认值= 0
dMax_X-dMax_Rz	正方向阈值	double	0~1000	正方向力最大阈值： dMax_X: X 方向，单位[N] dMax_Y: Y 方向，单位[N] dMax_Z: Z 方向，单位[N] dMax_Rx: Rx 方向，单位[N·m] dMax_Ry: Ry 方向，单位[N·m] dMax_Rz: Rz 方向，单位[N·m]
dMin_X-dMin_Rz	负方向阈值	double	0~1000	负方向力最大阈值： dMin_X: X 方向，单位[N] dMin_Y: Y 方向，单位[N] dMin_Z: Z 方向，单位[N] dMin_Rx: Rx 方向，单位[N·m] dMin_Ry: Ry 方向，单位[N·m] dMin_Rz: Rz 方向，单位[N·m]

✓ 返回值

返回值	名称	数据类型	有效范围	内容
nRet	返回值	int	>=0 的整型值	nRet = 0: 返回函数调用成功 nRet >0: 返回调用失败的错误码

3.9.17.2. 示例

// 设置正方向力最大阈值

```
double dMax_X = 500; double dMax_Y = 500; double dMax_Z = 500;
```

```
double dMax_Rx = 50; double dMax_Ry = 50; double dMax_Rz = 50;
```

// 设置负方向力最大阈值


```
double dMin_X = 300; double dMin_Y = 300; double dMin_Z = 300;  
double dMin_Rx = 30; double dMin_Ry = 30; double dMin_Rz = 30;  
// 设置力保护限制范围  
int nRet = HRIF_SetForceDataLimit(0,0, dMax_X, dMax_Y, dMax_Z, dMax_Rx, dMax_Ry, dMax_Rz,  
dMin_X, dMin_Y, dMin_Z, dMin_Rx, dMin_Ry, dMin_Rz);
```

3.9.18. HRIF_SetForceDistanceLimit

3.9.18.1. 描述：设置力控形变范围。（此接口功能已弃用）

✓ 输入变量

输入变量	名称	数据类型	有效范围	内容
boxID	电箱 ID	unsigned int	0~5	电箱 ID 号，默认值= 0
rbtID	机器人 ID	unsigned int	0~5	机器人 ID 号，默认值= 0
dAllowDistance	允许最大距离	double		允许最大形变距离，单位[mm]
dStrengthLevel	位置与边界设置 偏离距离的 幂次项	double	2/3	位置与边界设置偏离距离的幂次项： 2：阻力与偏离边界的平方项成比例； 3：阻力与偏离边界的立方项成比例；

✓ 返回值

返回值	名称	数据类型	有效范围	内容
nRet	返回值	int	>=0 的整型值	nRet = 0: 返回函数调用成功 nRet > 0: 返回调用失败的错误码

3.9.18.2. 示例

// 定义允许最大距离

```
double dAllowDistance = 10;
```

// 定义位置与边界设置偏离距离的幂次项

```
double dStrengthLevel = 2;
```

// 设置力控形变范围

```
int nRet = HRIF_SetForceDistanceLimit(0,0, dAllowDistance, dStrengthLevel);
```

3.9.19. HRIF_SetForceFreeDriveMode

3.9.19.1. 描述：开启/关闭力控自由驱动。

✓ 输入变量

输入变量	名称	数据类型	有效范围	内容
boxID	电箱 ID	unsigned int	0~5	电箱 ID 号，默认值= 0
rbtID	机器人 ID	unsigned int	0~5	机器人 ID 号，默认值= 0
bEnable	是否开启	bool	false/true	false: 关闭 true: 开启

✓ 返回值

返回值	名称	数据类型	有效范围	内容
nRet	返回值	int	≥ 0 的整型值	nRet = 0: 返回函数调用成功 nRet > 0: 返回调用失败的错误码

3.9.19.2. 示例

// 开启力控自由驱动

```
bool bEnable = true;
```

```
int nRet = HRIF_SetForceFreeDriveMode(0,0, bEnable);
```

// 关闭力控自由驱动

```
bool bEnable = false;
```

```
int nRet = HRIF_SetForceFreeDriveMode(0,0, bEnable);
```

3.9.20. HRIF_ReadFTCabData

3.9.20.1. 描述：读取标定后的力传感器数据。

✓ 输入变量

输入变量	名称	数据类型	有效范围	内容
boxID	电箱 ID	unsigned int	0~5	电箱 ID 号，默认值=0
rbtID	机器人 ID	unsigned int	0~5	机器人 ID 号，默认值=0

✓ 输出变量

输出变量	名称	数据类型	有效范围	内容
dX -dRz	标定后的力传感器数据	double	-	dX: X 坐标, 单位[N] dY: Y 坐标, 单位[N] dZ: Z 坐标, 单位[N] dRx: Rx 坐标, 单位[N·m] dRy: Ry 坐标, 单位[N·m] dRz: Rz 坐标, 单位[N·m]

✓ 返回值

返回值	名称	数据类型	有效范围	内容
nRet	返回值	int	>=0 的整型值	nRet = 0: 返回函数调用成功 nRet >0: 返回调用失败的错误码

3.9.20.2. 示例

```
double dX = 0; double dY = 0; double dZ = 0;
double dRx = 0; double dRy = 0; double dRz = 0;
// 读取标定后力传感器数据
int nRet = HRIF_ReadFTCabData(0,0, dX, dY, dZ, dRx, dRy, dRz);
```

3.9.21. HRIF_SetFTFreeDriveSpeedMode

3.9.21.1. 描述：设置自由驱动的速度模式。

✓ 输入变量

输入变量	名称	数据类型	有效范围	内容
boxID	电箱 ID	unsigned int	0~5	电箱 ID 号，默认值= 0
rbtID	机器人 ID	unsigned int	0~5	机器人 ID 号，默认值= 0
nMode	速度模式	unsigned int	0~3	速度模式，默认为 0 0：正常速度模式 1：慢速模式 2：快速模式 3：焊接模式

✓ 返回值

返回值	名称	数据类型	有效范围	内容
nRet	返回值	int	>=0 的整型值	nRet = 0: 返回函数调用成功 nRet > 0: 返回调用失败的错误码

3.9.21.2. 示例

```
int nMode = 3;
// 设置速度模式为焊接模式
int nRet = HRIF_SetFTFreeDriveSpeedMode(0,0,nMode);
```

3.9.22. HRIF_ReadFTFreeDriveSpeedMode

3.9.22.1. 描述：读取设定后的自由驱动速度模式。

✓ 输入变量

输入变量	名称	数据类型	有效范围	内容
boxID	电箱 ID	unsigned int	0~5	电箱 ID 号，默认值= 0
rbtID	机器人 ID	unsigned int	0~5	机器人 ID 号，默认值= 0

✓ 输出变量

输出变量	名称	数据类型	有效范围	内容
nMode	速度模式	unsigned int	0~3	速度模式，默认为 0 0：正常速度模式 1：慢速模式 2：快速模式 3：焊接模式

✓ 返回值

返回值	名称	数据类型	有效范围	内容
nRet	返回值	int	≥ 0 的整型值	nRet = 0: 返回函数调用成功 nRet > 0: 返回调用失败的错误码

3.9.22.2. 示例

```
int nMode = 0;
```

```
// 读取设定的自由驱动速度模式
```

```
int nRet = HRIF_ReadFTFreeDriveSpeedMode(0,0, nMode);
```

3.9.23. HRIF_SetFreeDriveMotionFreedom

3.9.23.1. 描述：设置力控自由驱动末端自由度。

✓ 输入变量

输入变量	名称	数据类型	有效范围	内容
boxID	电箱 ID	unsigned int	0~5	电箱 ID 号，默认值= 0
rbtID	机器人 ID	unsigned int	0~5	机器人 ID 号，默认值= 0
nX-nRz	各方向自由度	int	0/1	各方向探寻自由度开关： 0：关闭 1：开启

✓ 返回值

返回值	名称	数据类型	有效范围	内容
nRet	返回值	int	≥ 0 的整型值	nRet = 0: 返回函数调用成功 nRet > 0: 返回调用失败的错误码

3.9.23.2. 示例

// 定义力控自由驱动自由度状态

```
int nX = 0; int nY = 0; int nZ = 1;
```

```
int nRx = 0; int nRy = 0; int nRz = 0;
```

// 设置力控自由驱动自由度状态

```
int nRet = HRIF_SetFreeDriveMotionFreedom(0,0, nX, nY, nZ, nRx, nRy, nRz);
```

3.9.24. HRIF_SetFTFreeFactor

3.9.24.1. 描述：设置平移柔顺度和旋转柔顺度。

✓ 输入变量

输入变量	名称	数据类型	有效范围	内容
boxID	电箱 ID	unsigned int	0~5	电箱 ID 号，默认值= 0
rbtID	机器人 ID	unsigned int	0~5	机器人 ID 号，默认值= 0
dLinear	平移柔顺度	double	0~100	力控自由驱动平移柔顺度
dAngular	旋转柔顺度	double	0~100	力控自由驱动旋转柔顺度

✓ 返回值

返回值	名称	数据类型	有效范围	内容
nRet	返回值	int	≥ 0 的整型值	nRet = 0: 返回函数调用成功 nRet > 0: 返回调用失败的错误码

3.9.24.2. 示例

```
// 定义平移柔顺度
double dLinear = 50;
// 定义旋转柔顺度
double dAngular = 50;
// 设置力控自由驱动平移柔顺度和旋转柔顺度
int nRet = HRIF_SetFTFreeFactor(0,0, dLinear, dAngular);
```


3.9.25. HRIF_SetTangentForceBounds

3.9.25.1. 描述：设置 X/Y 方向切向力最大值、最小值和最大上抬速度。

✓ 输入变量

输入变量	名称	数据类型	有效范围	内容
boxID	电箱 ID	unsigned int	0~5	电箱 ID 号，默认值= 0
rbtID	机器人 ID	unsigned int	0~5	机器人 ID 号，默认值= 0
dMax	最大值	double	0~500	X/Y 方向切向力最大值，单位：N
dMin	最小值	double	0~500	X/Y 方向切向力最小值，单位：N
dVel	抬升速度	double	0~100	最大抬升速度，单位：mm/s

✓ 返回值

返回值	名称	数据类型	有效范围	内容
nRet	返回值	int	>=0 的整型值	nRet = 0: 返回函数调用成功 nRet >0: 返回调用失败的错误码

3.9.25.2. 示例

```
// 定义 X/Y 方向切向力最大值
double dMax = 25;
// 定义 X/Y 方向切向力最小值
double dMin = 15;
// 定义越障上抬最大速度
double dVel = 30;
// 设置 X/Y 方向切向力最大值、最小值和上抬最大速度
int nRet = HRIF_SetTangentForceBounds(0,0, dMax, dMin, dVel);
```

3.9.26. HRIF_SetFreeDriveCompensateForce

3.9.26.1. 描述：设置 FreeDrive 模式下的定向补偿力大小及矢量方向。

✓ 输入变量

输入变量	名称	数据类型	有效范围	内容
boxID	电箱 ID	unsigned int	0~5	电箱 ID 号, 默认值= 0
rbtID	机器人 ID	unsigned int	0~5	机器人 ID 号, 默认值= 0
dForce	补偿力	double	0~500	补偿力大小, 单位: N
dVector	力方向向量	double	-	补偿力的矢量方向[x,y,z]

✓ 返回值

返回值	名称	数据类型	有效范围	内容
nRet	返回值	int	≥ 0 的整型值	nRet = 0: 返回函数调用成功 nRet > 0: 返回调用失败的错误码

3.9.26.2. 示例

```
// 定义补偿力大小
double dForce = 10;
// 定义补偿力在基坐标系下的矢量方向
double dX = 0;
double dY = 0;
double dZ = 0;
// 设置 FreeDrive 模式下的定向补偿力大小和矢量方向[x, y, z]
int nRet = HRIF_SetFreeDriveCompensateForce(0,0,dForce,dX,dY,dZ);
```

3.9.27. HRIF_SetFTWrenchThresholds

3.9.27.1. 描述：设置力控自由驱动启动阈值（力与力矩）。

✓ 输入变量

输入变量	名称	数据类型	有效范围	内容
boxID	电箱 ID	unsigned int	0~5	电箱 ID 号，默认值= 0
rbtID	机器人 ID	unsigned int	0~5	机器人 ID 号，默认值= 0
dForceThreshold	力	double	0~100	自由驱动力启动阈值，单位[N]
dTorqueThreshold	力矩	double	0~10	自由驱动力矩启动阈值，单位[N·m]

✓ 返回值

返回值	名称	数据类型	有效范围	内容
nRet	返回值	int	>=0 的整型值	nRet = 0: 返回函数调用成功 nRet >0: 返回调用失败的错误码

3.9.27.2. 示例

// 定义力阈值

double dForceThreshold;

// 定义力矩阈值

double dTorqueThreshold;

// 设置力控自由驱动启动阈值（力与力矩）

int nRet = HRIF_SetFTWrenchThresholds(0,0,dForceThreshold,dTorqueThreshold);

3.9.28. HRIF_SetMaxFreeDriveVel

3.9.28.1. 描述：设置力控自由驱动最大直线速度及姿态角速度。

✓ 输入变量

输入变量	名称	数据类型	有效范围	内容
boxID	电箱 ID	unsigned int	0~5	电箱 ID 号, 默认值= 0
rbtID	机器人 ID	unsigned int	0~5	机器人 ID 号, 默认值= 0
dMaxLinearVelocity	直线速度	double	1~1000	自由驱动的最大直线速度, 单位[mm/s]
dMaxAngularVelocity	姿态角速度	double	1~50	自由驱动的最大姿态角速度, 单位[°/s]

✓ 返回值

返回值	名称	数据类型	有效范围	内容
nRet	返回值	int	≥ 0 的整型值	nRet = 0: 返回函数调用成功 nRet > 0: 返回调用失败的错误码

3.9.28.2. 示例

// 定义自由驱动最大直线速度

```
double dMaxLinearVelocity;
```

// 定义自由驱动最大角速度

```
double dMaxAngularVelocity;
```

// 设置力控自由驱动最大直线速度及姿态角速度

```
int nRet = HRIF_SetMaxFreeDriveVel(0,0,dMaxLinearVelocity,dMaxAngularVelocity);
```

3.9.29. HRIF_ReadFTMotionFreedom

3.9.29.1. 描述：读取力控自由驱动的末端自由度。

✓ 输入变量

输入变量	名称	数据类型	有效范围	内容
boxID	电箱 ID	unsigned int	0~5	电箱 ID 号，默认值= 0
rbtID	机器人 ID	unsigned int	0~5	机器人 ID 号，默认值= 0

✓ 输出变量

输出变量	名称	数据类型	有效范围	内容
nX-nRz	各方向自由度	int	0/1	各方向探寻自由度开关： 0：关闭 1：开启

✓ 返回值

返回值	名称	数据类型	有效范围	内容
nRet	返回值	int	>=0 的整型值	nRet=0: 返回函数调用成功 nRet>0: 返回调用失败的错误码

3.9.29.2. 示例

```
int nX; int nY; int nZ;
```

```
int nRx; int nRy; int nRz;
```

```
// 读取力控自由驱动的末端自由度
```

```
int nRet = HRIF_ReadFTMotionFreedom(0,0,nX,nY,nZ,nRx,nRy,nRz);
```

3.9.30. HRIF_SetMaxSearchDistance

3.9.30.1. 描述：设置各自由度力控探寻最大距离。

✓ 输入变量

输入变量	名称	数据类型	有效范围	内容
boxID	电箱 ID	unsigned int	0~5	电箱 ID 号，默认值= 0
rbtID	机器人 ID	unsigned int	0~5	机器人 ID 号，默认值= 0
Dis_X	X 方向探寻边界	double	-	X 方向探寻边界，单位：mm
Dis_Y	Y 方向探寻边界	double	-	Y 方向探寻边界，单位：mm
Dis_Z	Z 方向探寻边界	double	-	Z 方向探寻边界，单位：mm
Dis_RX	RX 方向探寻边界	double	-	RX 方向探寻边界，单位：°
Dis_RY	RY 方向探寻边界	double	-	RY 方向探寻边界，单位：°
Dis_RZ	RZ 方向探寻边界	double	-	RZ 方向探寻边界，单位：°

✓ 返回值

返回值	名称	数据类型	有效范围	内容
nRet	返回值	int	>=0 的整型值	nRet = 0: 返回函数调用成功 nRet > 0: 返回调用失败的错误码

3.9.30.2. 示例

// 定义各自由度力控探寻最大距离

```
double Dis_X = 300; double Dis_Y = 300; double Dis_Z = 300;
```

```
double Dis_RX = 20; double Dis_RY = 20; double Dis_RZ = 20;
```

// 设置各自由度力控探寻最大距离

```
int nRet = HRIF_SetMaxSearchDistance(0,0,Dis_X,Dis_Y,Dis_Z,Dis_RX,Dis_RY,Dis_RZ);
```

3.9.31. HRIF_SetSteadyContactDeviationRange

3.9.31.1. 描述：设置恒力控稳定阶段边界。

✓ 输入变量

输入变量	名称	数据类型	有效范围	内容
boxID	电箱 ID	unsigned int	0~5	电箱 ID 号，默认值= 0
rbtID	机器人 ID	unsigned int	0~5	机器人 ID 号，默认值= 0
Pos_X~Dis_RZ	正方向边界	double	>0	正方向边界（与探寻方向一致），单位： mm
Neg_X ~ Neg_RZ	负方向边界	double	<0	负方向边界（与探寻方向相反），单位： mm

✓ 返回值

返回值	名称	数据类型	有效范围	内容
nRet	返回值	int	>=0 的整型值	nRet = 0: 返回函数调用成功 nRet >0: 返回调用失败的错误码

3.9.31.2. 示例

```
double Pos_X = 100; double Pos_Y = 100; double Pos_Z = 100;
double Pos_RX = 20; double Pos_RY = 20; double Pos_RZ = 20;
```

```
double Neg_X = -100; double Neg_Y = -100; double Neg_Z = -100;
double Neg_RX = -20; double Neg_RY = -20; double Neg_RZ = -20;
```

// 设置恒力控稳定阶段边界

```
int nRet = HRIF_SetSteadyContactDeviationRange(0,0,Pos_X,Pos_Y,Pos_Z,Pos_RX,Pos_RY,Pos_RZ,
Neg_X,Neg_Y,Neg_Z,Neg_RX,Neg_RY,Neg_RZ);
```

3.9.32. HRIF_AddSafePlane

3.9.32.1. 描述：添加虚拟墙平面；虚拟墙平面即安全平面，只是相应的功能不同，如果添加的安全平面名称已经存在，会报 20006 错误；最后一个参数为是否激活，如果不激活，不会检查 UCS 是否存在；

✓ 输入变量

输入变量	名称	数据类型	有效范围	内容
boxID	电箱 ID	unsigned int	0~5	电箱 ID 号，默认值= 0
rbtID	机器人 ID	unsigned int	0~5	机器人 ID 号，默认值= 0
Name	平面名称	string	——	要添加的平面名称
UcsName	用户坐标名称	string	——	所使用的用户坐标系
Mode	安全模式	int	0/1	正常/缩减模式
Display	显示	int	0/1	是否显示目标平面
Switch	启用	int	0/1	是否启用目标平面

✓ 返回值

返回值	名称	数据类型	有效范围	内容
nRet	返回值	int	>=0 的整型值	nRet = 0: 返回函数调用成功 nRet >0: 返回调用失败的错误码

3.9.32.2. 示例

// 定义平面名称

```
string Name = "border_1";
```

//定义使用的用户坐标名称

```
string UcsName = "Plane_1";
```

//定义安全模式

```
int Mode = 0;
```

//定义是否显示

```
int Display = 0;
```

//定义是否启用

```
int Switch = 1;
```

// 添加虚拟墙平面

```
int nRet = HRIF_AddSafePlane(0, 0, Name, UcsName , Mode, Display, Switch);
```

3.9.33. HRIF_UpdateSafePlane

3.9.33.1. 描述：修改更新虚拟墙平面属性；虚拟墙平面即安全平面，只是相应的功能不同；最后一个参数为是否激活，如果不激活，不会检查 UCS 是否存在；

✓ 输入变量

输入变量	名称	数据类型	有效范围	内容
boxID	电箱 ID	unsigned int	0~5	电箱 ID 号，默认值= 0
rbtID	机器人 ID	unsigned int	0~5	机器人 ID 号，默认值= 0
Name	平面名称	string	——	要添加的平面名称
UcsName	用户坐标名称	string	——	所使用的用户坐标系
Mode	安全模式	int	0/1	正常/缩减模式
Display	显示	int	0/1	是否显示目标平面
Switch	启用	int	0/1	是否启用目标平面

✓ 返回值

返回值	名称	数据类型	有效范围	内容
nRet	返回值	int	>=0 的整型值	nRet = 0: 返回函数调用成功 nRet >0: 返回调用失败的错误码

3.9.33.2. 示例

// 定义平面名称

```
string Name = "border_1";
```

//定义使用的用户坐标名称

```
string UcsName = "Plane_1";
```

//定义安全模式

```
int Mode = 0;
```

//定义是否显示

```
int Display = 0;
```

//定义是否启用

```
int Switch = 1;
```

// 添加虚拟墙平面

```
int nRet = HRIF_UpdateSafePlane(0, 0, Name, UcsName , Mode, Display, Switch);
```

3.9.34. HRIF_DelSafePlane

3.9.34.1. 描述：删除虚拟墙平面；虚拟墙平面即安全平面，只是相应的功能不同，如果指定的安全平面名称不存在，报 20006 错误；

✓ 输入变量

输入变量	名称	数据类型	有效范围	内容
boxID	电箱 ID	unsigned int	0~5	电箱 ID 号，默认值= 0
rbtID	机器人 ID	unsigned int	0~5	机器人 ID 号，默认值= 0
Name	平面名称	string	——	要删除的平面名称

✓ 返回值

返回值	名称	数据类型	有效范围	内容
nRet	返回值	int	≥ 0 的整型值	nRet = 0: 返回函数调用成功 nRet > 0: 返回调用失败的错误码

3.9.34.2. 示例

// 定义要删除的平面名称

string Name = "border_1";

//删除虚拟墙平面

int nRet = HRIF_DelSafePlane(0, 0, Name);

3.9.35. HRIF_ReadSafePlaneList

3.9.35.1. 描述：返回结果为所有安全平面的名字清单，虚拟墙平面即安全平面，只是相应的功能不同；

✓ 输入变量

输入变量	名称	数据类型	有效范围	内容
boxID	电箱 ID	unsigned int	0~5	电箱 ID 号，默认值= 0
rbtID	机器人 ID	unsigned int	0~5	机器人 ID 号，默认值= 0

✓ 输出变量

输出变量	名称	数据类型	有效范围	内容
BordersName	所有虚拟墙平面名称	vector<string>	——	虚拟墙平面名称

✓ 返回值

返回值	名称	数据类型	有效范围	内容
nRet	返回值	int	>=0 的整型值	nRet = 0: 返回函数调用成功 nRet >0: 返回调用失败的错误码

3.9.35.2. 示例

// 定义接收所有虚拟墙平面名称的列表

```
vector<string> BordersName;
```

//读取当前所有的虚拟墙平面名称

```
int nRet = HRIF_ReadSafePlaneList(0, 0, BordersName );
```

3.9.36. HRIF_ReadSafePlane

3.9.36.1. 描述：返回结果为指定安全平面的详细参数；虚拟墙平面即安全平面，只是相应的功能不同，如果指定的安全平面不存在，返回结果为空；

✓ 输入变量

输入变量	名称	数据类型	有效范围	内容
boxID	电箱 ID	unsigned int	0~5	电箱 ID 号，默认值= 0
rbtID	机器人 ID	unsigned int	0~5	机器人 ID 号，默认值= 0
Name	平面名称	string	——	要读取信息的平面名称

✓ 输出变量

输出变量	名称	数据类型	有效范围	内容
UcsName	用户坐标名称	string	——	所使用的用户坐标系
Mode	安全模式	int	0/1	正常/缩减模式
Display	显示	int	0/1	是否显示目标平面
Switch	启用	int	0/1	是否启用目标平面

✓ 返回值

返回值	名称	数据类型	有效范围	内容
nRet	返回值	int	≥ 0 的整型值	nRet = 0: 返回函数调用成功 nRet > 0: 返回调用失败的错误码

3.9.36.2. 示例

// 定义要读取的平面名称

```
string BorderName = "border_1";
```

// 定义接收目标平面信息的相关变量

```
string UcsName = "";
```

```
int Mode = 0;
```

```
int Display = 0;
```

```
int Switch = 0;
```

//读取目标虚拟墙平面的详细信息

```
int nRet = HRIF_ReadSafePlane(0, 0, BorderName, UcsName, Mode, Display, Switch);
```

3.9.37. HRIF_SetDepthThresholdForDampingArea

3.9.37.1. 描述：设置虚拟墙开始产生阻尼时的距离阈值。

✓ 输入变量

输入变量	名称	数据类型	有效范围	内容
boxID	电箱 ID	unsigned int	0~5	电箱 ID 号，默认值= 0
rbtID	机器人 ID	unsigned int	0~5	机器人 ID 号，默认值= 0
dDepth	阈值大小	double	-	触发虚拟墙的阈值大小，单位：mm

✓ 返回值

返回值	名称	数据类型	有效范围	内容
nRet	返回值	int	>=0 的整型值	nRet = 0: 返回函数调用成功 nRet >0: 返回调用失败的错误码

3.9.37.2. 示例

// 定义阈值大小

```
double dDepth = 100;
```

// 设置虚拟墙开始产生阻尼时的距离阈值

```
int nRet = HRIF_SetDepthThresholdForDampingArea(0, 0, dDepth);
```

3.9.38. HRIF_GetLastCalibParams

3.9.38.1. 描述：获取力控标定结果

✓ 输入变量

输入变量	名称	数据类型	有效范围	内容
boxID	电箱 ID	unsigned int	0~5	电箱 ID 号，默认值= 0
rbtID	机器人 ID	unsigned int	0~5	机器人 ID 号，默认值= 0

✓ 输出变量

输出变量	名称	数据类型	有效范围	内容
Fx0-Fz0	x-z 方向标定力	double	-	x-z 方向标定力
Mx0-Mz0	x-z 方向力矩	double	-	x-z 方向力矩

X-Z	重心偏移	double	-	x-z 方向重心偏移
G	重力	double	-	重力
InstallRotationAngel	安装角度	double	-	机器安装角度

✓ 返回值

返回值	名称	数据类型	有效范围	内容
nRet	返回值	int	>=0 的整型值	nRet = 0: 返回函数调用成功 nRet >0: 返回调用失败的错误码

3.9.38.2. 示例

//获取 x,y,z,方向上标定力

```
double Fx0 = 1;
```

```
double Fy0 = 2;
```

```
double Fz0 = 3;
```

//获取下 x,y,z 方向力矩

```
double Mx0 = 4;
```

```
double My0 = 5;
```

```
double Mz0 = 6;
```

//获取重心

```
double G = 7;
```

//获取下 x,y,z 方向上的重心偏移

```
double X = 8;
```

```
double Y = 9;
```

```
double Z = 10;
```

//获取安装角度

```
double InstallRotationAngel = 11;
```

```
int Ret = HRIF_GetLastCalibParams(0,0,Fx0,Fy0,Fz0,Mx0,My0,Mz0,G,X,Y,Z,InstallRotationAngel);
```

3.9.39. HRIF_SetInitializeForceSensor

3.9.39.1. 描述：设定力控标定结果

✓ 输入变量

输入变量	名称	数据类型	有效范围	内容
------	----	------	------	----

boxID	电箱 ID	unsigned int	0~5	电箱 ID 号, 默认值= 0
rbtID	机器人 ID	unsigned int	0~5	机器人 ID 号, 默认值= 0
Fx0-Fz0	x-z 方向标定力	double	-	x-z 方向标定力
Mx0-Mz0	x-z 方向力矩	double	-	x-z 方向力矩
X-Z	重心偏移	double	-	x-z 方向重心偏移
G	重力	double	-	重力
InstallRotationAngel	安装角度	double	-	机器安装角度

✓ 返回值

返回值	名称	数据类型	有效范围	内容
nRet	返回值	int	>=0 的整型值	nRet = 0: 返回函数调用成功 nRet >0: 返回调用失败的错误码

3.9.39.2. 示例

//设置 x,y,z,方向上标定力

```
double Fx0 = 1;
double Fy0 = 2;
double Fz0 = 3;
```

//设置 x,y,z 方向力矩

```
double Mx0 = 4;
double My0 = 5;
double Mz0 = 6;
```

//设置重心

```
double G = 7;
```

//设置 x,y,z 方向上的重心偏移

```
double X = 8;
double Y = 9;
double Z = 10;
```

//设置安装角度

```
double InstallRotationAngel = 11;
int Ret = HRIF_SetInitializeForceSensor(0,0,Fx0,Fy0,Fz0,Mx0,My0,Mz0,G,X,Y,Z,InstallRotationAngel);
```

3.9.40.HRIF_SetFTCalibration

3.9.40.1. 描述：计算力控标定结果

✓ 输入变量

输入变量	名称	数据类型	有效范围	内容
boxID	电箱 ID	unsigned int	0~5	电箱 ID 号，默认值= 0
rbtID	机器人 ID	unsigned int	0~5	机器人 ID 号，默认值= 0
pointnum	标定点位数据	int	8~16	标定点位个数，8~16 个
param	标定后数据	Double[[12]	参数总数必须为 12 的倍数，且在 96~192 之间。	依次存放 X, Y, Z, Rx, Ry, Rz, Fx, Fy, Fz, Tx, Ty, Tz, 每 12 个参数为一组数据，支持 8~16 组数据（即参数个数 96~192）。前六个为位置坐标，后六个为力控数据，单位：mm、N

✓ 返回值

返回值	名称	数据类型	有效范围	内容
nRet	返回值	int	>=0 的整型值	nRet = 0: 返回函数调用成功 nRet >0: 返回调用失败的错误码

3.9.40.2. 示例

//设置点位数

```
int pointnum=8;
vector<vector<int>> param(pointnum,vector<int>(12, 0));
```

//设置标定后数据

```
for(int i=0;i<pointnum;i++)
{
    for(int j=0;j<12;j++)
    {
        param[i][j]=0;
    }
}
int nRet=HRIF_SetFTCalibration(0,0,pointnum,param);
```

3.9.41.HRIF_ReadForceData

3.9.41.1. 描述：读取力传感器原始数据

✓ 输入变量

输入变量	名称	数据类型	有效范围	内容
boxID	电箱 ID	unsigned int	0~5	电箱 ID 号，默认值= 0
rbtID	机器人 ID	unsigned int	0~5	机器人 ID 号，默认值= 0

✓ 输出变量

输出变量	名称	数据类型	有效范围	内容
forcedata	原始力数据	double	——	力传感器提供的原始力数据： dX：x 方向的原始力数据； dY：y 方向的原始力数据； dZ：z 方向的原始力数据； dRX：RX 方向的原始力数据； dRY：RY 方向的原始力数据； dRZ：RZ 方向的原始力数据；

✓ 返回值

返回值	名称	数据类型	有效范围	内容
nRet	返回值	int	>=0 的整型值	nRet = 0: 返回函数调用成功 nRet >0: 返回调用失败的错误码

3.9.41.2. 示例

```
double dX=0;
double dY=0;
double dZ=0;
double dRX=0;
double dRY=0;
double dRZ=0;
```

//读取当前力传感器原始数据

```
int nRet=hr_HRIF_ReadForceData(0, 0, dX,dY,dZ,dRX,dRY,dRZ);
```

3.9.42. HRIF_SetFTUCS

3.9.42.1. 描述：设置力控用户坐标，不写入配置文件，重启后为默认 base。

✓ 输入变量

输入变量	名称	数据类型	有效范围	内容
boxID	电箱 ID	unsigned int	0~5	电箱 ID 号，默认值= 0
rbtID	机器人 ID	unsigned int	0~5	机器人 ID 号，默认值= 0
dUcs_X-dUcs_Rz	用户坐标	double	-	需要设置的用户坐标： dUcs_X: X 坐标，单位[mm] dUcs_Y: Y 坐标，单位[mm] dUcs_Z: Z 坐标，单位[mm] dUcs_Rx: Rx 坐标，单位[°] dUcs_Ry: Ry 坐标，单位[°] dUcs_Rz: Rz 坐标，单位[°]

✓ 返回值

返回值	名称	数据类型	有效范围	内容
nRet	返回值	int	>=0 的整型值	nRet=0: 返回函数调用成功 nRet>0: 返回调用失败的错误码

3.9.42.2. 示例

// 定义力控用户坐标变量

```
double dUcs_X = 0; double dUcs_Y = 0; double dUcs_Z = 0;
```

```
double dUcs_Rx = 0; double dUcs_Ry = 0; double dUcs_Rz = 0;
```

// 设置力控用户坐标

```
int nRet= HRIF_SetFTUCS(0,0, dUcs_X, dUcs_Y, dUcs_Z, dUcs_Rx, dUcs_Ry, dUcs_Rz);
```

3.9.43. HRIF_ReadFTUCS

3.9.43.1. 描述：读取力控用户坐标值。

✓ 输入变量

输入变量	名称	数据类型	有效范围	内容
boxID	电箱 ID	unsigned int	0~5	电箱 ID 号，默认值= 0
rbtID	机器人 ID	unsigned int	0~5	机器人 ID 号，默认值= 0

✓ 输出变量

输出变量	名称	数据类型	有效范围	内容
dUcs_X-dUcs_Rz	用户坐标	double	-	读取到的用户坐标： dUcs_X : X 坐标，单位[mm] dUcs_Y : Y 坐标，单位[mm] dUcs_Z : Z 坐标，单位[mm] dUcs_Rx : Rx 坐标，单位[°] dUcs_Ry : Ry 坐标，单位[°] dUcs_Rz : Rz 坐标，单位[°]

✓ 返回值

返回值	名称	数据类型	有效范围	内容
nRet	返回值	int	≥0 的整型值	nRet=0: 返回函数调用成功 nRet>0: 返回调用失败的错误码

3.9.43.2. 示例

// 定义力控用户坐标变量

```
double dUcs_X = 0; double dUcs_Y = 0; double dUcs_Z = 0;
```

```
double dUcs_Rx = 0; double dUcs_Ry = 0; double dUcs_Rz = 0;
```

// 读取力控用户坐标

```
int nRet= HRIF_ReadFTUCS(0,0, dUcs_X, dUcs_Y, dUcs_Z, dUcs_Rx, dUcs_Ry, dUcs_Rz);
```

3.10. 通用运动类控制指令

3.10.1. HRIF_MoveRelJ

3.10.1.1. 描述：关节相对运动。

✓ 输入变量

参数	名称	数据类型	有效范围	内容
boxID	电箱 ID	unsigned int	0~5	电箱 ID 号, 默认值= 0
rbtID	机器人 ID	unsigned int	0~5	机器人 ID 号, 默认值= 0
nAxisID	轴 ID	int	0~5	运动的目标轴 ID , 对应关节 J1-J6
nDirection	方向	int	0/1	0: 负方向 1: 正方向
dDistance	运动距离	double	-	相对运动距离

✓ 返回值

返回值	名称	数据类型	有效范围	内容
nRet	返回值	int	≥ 0 的整型值	nRet = 0: 返回函数调用成功 nRet > 0: 返回调用失败的错误码

3.10.1.2. 示例

//定义轴 ID

```
int nAxisID = 0;
```

//定义运动方向

```
int nDirection= 0;
```

//定义运动距离

```
double dDistance= 5;
```

//关节相对运动

```
int nRet= HRIF_MoveRelJ(0,0,nAxisID ,nDirection,dDistance);
```

3.10.2. HRIF_MoveRelL

3.10.2.1. 空间相对运动。

✓ 输入变量

参数	名称	数据类型	有效范围	内容
boxID	电箱 ID	unsigned int	0~5	电箱 ID 号, 默认值= 0
rbtID	机器人 ID	unsigned int	0~5	机器人 ID 号, 默认值= 0
nAxisID	轴 ID	int	0~5	运动的目标轴 ID , 对应空间坐标 X-Rz
nDirection	方向	int	0/1	0: 负方向 1: 正方向
dDistance	运动距离	double	-	相对运动距离
nToolMotion	运动坐标类型	int	0/1	0: 按当前选择的用户坐标运动 1: 按 Tool 坐标运动

✓ 返回值

返回值	名称	数据类型	有效范围	内容
nRet	返回值	int	>=0 的整型值	nRet = 0: 返回函数调用成功 nRet >0: 返回调用失败的错误码

3.10.2.2. 示例

//定义轴 ID

int nAxisID = 0;

//定义运动方向

int nDirection= 0;

//定义运动距离

double dDistance= 5;

//定义运动坐标类型

int nToolMotion= 0;

//空间相对运动

int nRet= HRIF_MoveRelL(0,0,nAxisID ,nDirection,dDistance,nToolMotion);

3.10.3. HRIF_WayPointRel

3.10.3.1. 描述：路点相对运动。

✓ 输入变量

参数	名称	数据类型	有效范围	内容
boxID	电箱 ID	unsigned int	0~5	电箱 ID 号, 默认值= 0
rbtID	机器人 ID	unsigned int	0~5	机器人 ID 号, 默认值= 0
nType	运动类型	int	0/1	0: 关节相对运动 1: 直线相对运动
nPointList	是否使用点位列表点位	int	0/1	0: 不使用点位列表中点位 1: 使用点位列表中点位
dPos_X- dPos_Rz	空间位置	double		nPointList= 1: 点位空间位置 dPos_X: X 坐标, 单位[mm] dPos_Y: Y 坐标, 单位[mm] dPos_Z: Z 坐标, 单位[mm] dPos_Rx: Rx 坐标, 单位[°] dPos_Ry: Ry 坐标, 单位[°] dPos_Rz: Rz 坐标, 单位[°] nPointList= 0, 均为 0
dPos_J1- dPos_J6	关节位置	double		nPointList= 1: 点位关节位置 dPos_J1: 关节 1 坐标, 单位[°] dPos_J2: 关节 2 坐标, 单位[°] dPos_J3: 关节 3 坐标, 单位[°] dPos_J4: 关节 4 坐标, 单位[°] dPos_J5: 关节 5 坐标, 单位[°] dPos_J6: 关节 6 坐标, 单位[°] nPointList= 0, 均为 0
nrelMoveType	相对运动类型	int	0/1	0: 绝对值 1: 叠加值 2: tool 模式

nAxisMask_ 1- nAxisMask_ 6	是否运动	int	0/1	各轴/各方向是否运动 0: 不运动 1: 运动
dTarget_1- dTarget_6	运动距离	double	-	nType= 0 并 nAxisMask= 1: 该方向运动 绝对距离或叠加距离 nType= 1 并 nAxisMask= 1: 该轴运动绝对距离或叠加距离 nAxisMask= 0: 无效
sTcpName	工具坐标名称	string	-	目标空间坐标所处的工具坐标系名称, 与示教器页面的名称对应, 当 nIsUseJoint= 1 时无效, 可使用默认名称 “TCP”
sUcsName	用户坐标名称	string	-	目标空间坐标所处的用户坐标系名称, 与示教器页面的名称对应, 当 nIsUseJoint= 1 时无效, 可使用默认名称 “Base”
dVelocity	速度	double	0.1<速度<加速度	运动最大速度, 关节运动时单位[°/s], 空间运动时 X, Y, Z 单位[mm/s], Rx, Ry, Rz 单位[°/s]
dAcc	加速度	double	>0.1	运动最大加速度, 关节运动时单位[°/s²], 空间运动时 X, Y, Z 单位[mm/s²], Rx, Ry, Rz 单位[°/s²]
dRadius	过渡半径	double	0-1000	过渡半径, 单位[mm]
nIsUseJoint	是否使用关节坐标	int	0/1	是否使用关节角度作为目标点, 如果 nMoveType= 0 时, 则 nIsUseJoint 有效: 0: 不使用关节角度 1: 使用关节角度
nIsSeek	是否检测 DI 停止	int	0/1	如果 nIsSeek 为 1, 则开启检测 DI 停止, 路点运动过程中如果电箱的 nIOBit 位索引的 DI 的状态= nIOState 时, 机器人停止运动, 否则运动到目标点完成运动
nIOBit	检测的 DI 索引	int	0~7	电箱对应 DI 索引, nIsSeek= 0 时无效

nIOState	检测的 DI 状态	int	0/1	检测的 DI 状态, , nIsSeek=0 时无效
strCmdID	命令 ID	string	-	当前路点 ID , 可以自定义, 也可以按顺序设置为“1”,“2”,“3”.

✓ 返回值

返回值	名称	数据类型	有效范围	内容
nRet	返回值	int	>=0 的整型值	nRet = 0: 返回函数调用成功 nRet >0: 返回调用失败的错误码

3.10.3.2. 示例

// 定义运动类型

```
int nType = 0;
```

// 定义是否使用点位列表的点位

```
int nPointList= 0;
```

// 定义空间目标位置

```
double dPos_X = 420; double dPos_Y = 0; double dPos_Z = 445;
double dPos_Rx = 180; double dPos_Ry = 0; double dPos_Rz = 180;
```

// 定义关节目标位置

```
double dPos_J1 = 0; double dPos_J2 = 0; double dPos_J3 = 90;
double dPos_J4 = 0; double dPos_J5 = 90; double dPos_J6 = 0;
```

// 定义相对运动类型

```
int nrelMoveType= 1;
```

// 定义各轴 各方向是否运动

```
int nAxisMask_1 = 1; int nAxisMask_2 = 0; int nAxisMask_3 = 0;
int nAxisMask_4 = 0; int nAxisMask_5 = 0; int nAxisMask_6 = 0;
```

// 定义运动距离

```
double dTarget_1= 5; double dTarget_2= 0; double dTarget_3= 0;
double dTarget_4= 0; double dTarget_5= 0; double dTarget_6= 0;
```

// 定义工具坐标变量

```
string sTcpName = "TCP";
```

// 定义用户坐标变量

```
string sUcsName = "Base";
```

// 定义运动速度

```
double dVelocity = 50;
// 定义运动加速度

double dAcc = 100;
// 定义过渡半径

double dRadius = 50;
// 定义是否使用关节角度

int nIsUseJoint = 1;
// 定义是否使用检测 DI 停止

int nIsSeek = 0;
// 定义检测的 DI 索引

int nIOBit = 0;
// 定义检测的 DI 状态

int nIOState = 0;
// 定义路点 ID

string strCmdID = "0";
// 路点相对运动

int nRet= HRIF_WayPointRel(0,0, nType ,nPointList,dPos_X, dPos_Y, dPos_Z, dPos_Rx, dPos_Ry, dPos_Rz,
dPos_J1,dPos_J2,dPos_J3,dPos_J4,dPos_J5,dPos_J6,nrelMoveType,nAxisMask_1,nAxisMask_2,nAxisMask_3,nAxisMask_4,nAxisMask_5,nAxisMask_6,dTarget_1,dTarget_2,dTarget_3,dTarget_4,dTarget_5,dTarget_6,sTcpName ,
sUcsName, dVelocity, dAcc, dRadius,nIsUseJoint, nIsSeek, nIOBit, nIOState, strCmdID);
```

3.10.4. HRIF_WayPointRel_nJ

3.10.4.1. 描述：路点相对运动。**注意：这是扩展轴版本所用接口**

✓ 输入变量

参数	名称	数据类型	有效范围	内容
boxID	电箱 ID	unsigned int	0~5	电箱 ID 号, 默认值= 0
rbtID	机器人 ID	unsigned int	0~5	机器人 ID 号, 默认值= 0
nType	运动类型	int	0/1	0: 关节相对运动 1: 直线相对运动
nPointList	是否使用点位列表点位	int	0/1	0: 不使用点位列表中点位 1: 使用点位列表中点位
dPos_X- dPos_Rz	空间位置	double		nPointList= 1: 点位空间位置 dPos_X: X 坐标, 单位[mm] dPos_Y: Y 坐标, 单位[mm] dPos_Z: Z 坐标, 单位[mm] dPos_Rx: Rx 坐标, 单位[°] dPos_Ry: Ry 坐标, 单位[°] dPos_Rz: Rz 坐标, 单位[°] nPointList= 0, 均为 0
JointsPos	关节位置	JointsData		nPointList= 1: 点位关节位置 JointsPos.dJoint[i]: 关节坐标, 单位[°] nPointList= 0, 均为 0
nrelMoveType	相对运动类型	int	0/1	0: 绝对值 1: 叠加值 2: tool 模式
AxisMask	是否运动	JointsData	0/1	各轴/各方向是否运动 0: 不运动 1: 运动
Target	运动距离	JointsData	-	nType= 0 并 nAxisMask= 1: 该方向运动 绝对距离或叠加距离

				nType= 1 并 nAxisMask= 1: 该轴运动绝对距离或叠加距离 nAxisMask= 0: 无效
sTcpName	工具坐标名称	string	-	目标空间坐标所处的工具坐标系名称, 与示教器页面的名称对应, 当 nIsUseJoint= 1 时无效, 可使用默认名称 “TCP”
sUcsName	用户坐标名称	string	-	目标空间坐标所处的用户坐标系名称, 与示教器页面的名称对应, 当 nIsUseJoint= 1 时无效, 可使用默认名称 “Base”
dVelocity	速度	double	$0.1 < \text{速度} < \text{加速度}$	运动最大速度, 关节运动时单位[°/s], 空间运动时 X, Y, Z 单位[mm/s], Rx, Ry, Rz 单位[°/s]
dAcc	加速度	double	> 0.1	运动最大加速度, 关节运动时单位[°/s ²], 空间运动时 X, Y, Z 单位[mm/s ²], Rx, Ry, Rz 单位[°/s ²]
dRadius	过渡半径	double	0-1000	过渡半径, 单位[mm]
nIsUseJoint	是否使用关节坐标	int	0/1	是否使用关节角度作为目标点, 如果 nMoveType= 0 时, 则 nIsUseJoint 有效: 0: 不使用关节角度 1: 使用关节角度
nIsSeek	是否检测 DI 停止	int	0/1	如果 nIsSeek 为 1, 则开启检测 DI 停止, 路点运动过程中如果电箱的 nIOBit 位索引的 DI 的状态= nIOState 时, 机器人停止运动, 否则运动到目标点完成运动
nIOBit	检测的 DI 索引	int	0~7	电箱对应 DI 索引, nIsSeek= 0 时无效
nIOState	检测的 DI 状态	int	0/1	检测的 DI 状态, , nIsSeek= 0 时无效
strCmdID	命令 ID	string	-	当前路点 ID, 可以自定义, 也可以按顺序设置为“1”, “2”, “3”.

✓ 返回值

返回值	名称	数据类型	有效范围	内容
nRet	返回值	int	≥ 0 的整型值	nRet = 0: 返回函数调用成功 nRet > 0: 返回调用失败的错误码

3.10.4.2. 示例

// 定义运动类型 (空间相对运动)

```
int nType = 0;
```

// 定义是否使用点位列表的点位

```
int nPointList= 0;
```

// 定义空间目标位置

```
double dPos_X = 420.000; double dPos_Y = -0.000; double dPos_Z = 443.500;
```

```
double dPos_Rx = -180.000; double dPos_Ry = -0.000; double dPos_Rz = 180.000;
```

// 定义关节目标位置

```
JointsData JointsPos;
```

```
JointsPos.dJoint[0]=0.000; JointsPos.dJoint[1]=0.000; JointsPos.dJoint[2]=90.000; JointsPos.dJoint[3]=0.000;
```

```
JointsPos.dJoint[4]=90.000; JointsPos.dJoint[5]=0.000; JointsPos.dJoint[6]=-0.000;
```

// 定义相对运动类型

```
int nrelMoveType= 1;
```

// 定义各轴 各方向是否运动

```
JointsData nAxisMask;
```

```
nAxisMask.dJoint[0] = 1; nAxisMask.dJoint[1] = 1; nAxisMask.dJoint[2] = 0;
```

```
nAxisMask.dJoint[3] = 0; nAxisMask.dJoint[4] = 0; nAxisMask.dJoint[5] = 0;
```

// 定义运动距离

```
JointsData dTarget;
```

```
dTarget.dJoint[0] = 10; dTarget.dJoint[1] = 10; dTarget.dJoint[2] = 0;
```

```
dTarget.dJoint[3] = 0; dTarget.dJoint[4] = 0; dTarget.dJoint[5] = 0;
```

// 定义工具坐标变量

```
string sTcpName = "TCP";
```

// 定义用户坐标变量

```
string sUcsName = "Base";
```

// 定义运动速度

```
double dVelocity = 200;
```

// 定义运动加速度

```
double dAcc = 500;
// 定义过渡半径

double dRadius = 0.000;
// 定义是否使用关节角度

int nIsUseJoint = 0;
// 定义是否使用检测 DI 停止

int nIsSeek = 0;
// 定义检测的 DI 索引

int nIOBit = 0;
// 定义检测的 DI 状态

int nIOState = 0;
// 定义路点 ID

string strCmdID = "0";
// 路点相对运动

int nRet= HRIF_WayPointRel_nJ(0,0, nType ,nPointList,dPos_X, dPos_Y, dPos_Z, dPos_Rx, dPos_Ry, dPos_Rz,
JointsPos,nrelMoveType,nAxisMask,dTarget,
sTcpName , sUcsName, dVelocity, dAcc, dRadius,nIsUseJoint, nIsSeek, nIOBit, nIOState, strCmdID);
-----
// 定义运动类型 (关节相对运动)

int nType = 0;
// 定义是否使用点位列表的点位

int nPointList = 0;
// 定义空间目标位置

double dPos_X = 0.000; double dPos_Y = 0.000; double dPos_Z = 0.000;
double dPos_Rx = 0.000; double dPos_Ry = 0.000; double dPos_Rz = 0.000;
// 定义关节目标位置

JointsData JointsPos;
JointsPos.dJoint[0]=0.000; JointsPos.dJoint[1] = 0.000; JointsPos.dJoint[2] = 0.000; JointsPos.dJoint[3] = 0.000;
JointsPos.dJoint[4] = 0.000; JointsPos.dJoint[5] = 0.000; JointsPos.dJoint[6] = 0.000;
// 定义相对运动类型

int nrelMoveType = 0;
// 定义运动距离

JointsData dTarget;
```

```
dTarget.dJoint[0] = 0;dTarget.dJoint[1] = 0;dTarget.dJoint[2] = 90;dTarget.dJoint[3] = 0;  
dTarget.dJoint[4] = 90;dTarget.dJoint[5] = 0;dTarget.dJoint[6] = 0;
```

```
// 定义运动距离
```

```
JointsData nAxisMask;  
nAxisMask.dJoint[0] = 1; nAxisMask.dJoint[1] = 1; nAxisMask.dJoint[2] = 0; nAxisMask.dJoint[3] = 0;  
nAxisMask.dJoint[4] = 0; nAxisMask.dJoint[5] = 0; nAxisMask.dJoint[6] = 0;
```

```
// 定义工具坐标变量
```

```
string sTcpName = "TCP";
```

```
// 定义用户坐标变量
```

```
string sUcsName = "Base";
```

```
// 定义运动速度
```

```
double dVelocity = 60;
```

```
// 定义运动加速度
```

```
double dAcc = 360;
```

```
// 定义过渡半径
```

```
double dRadius = 0.000;
```

```
// 定义是否使用关节角度
```

```
int nIsUseJoint = 0;
```

```
// 定义是否使用检测 DI 停止
```

```
int nIsSeek = 0;
```

```
// 定义检测的 DI 索引
```

```
int nIOBit = 0;
```

```
// 定义检测的 DI 状态
```

```
int nIOState = 0;
```

```
// 定义路点 ID
```

```
string strCmdID = "ID1";
```

```
// 路点相对运动
```

```
int nRet = HRIF_WayPointRel_nJ(0, 0, nType, nPointList, dPos_X, dPos_Y, dPos_Z, dPos_Rx, dPos_Ry, dPos_Rz,  
JointsPos, nrelMoveType, nAxisMask, dTarget,  
sTcpName, sUcsName, dVelocity, dAcc, dRadius, nIsUseJoint, nIsSeek, nIOBit, nIOState, strCmdID);
```


3.10.5. HRIF_IsMotionDone

3.10.5.1. 描述：判断机器人是否处于运动状态。

✓ 输入变量

输入变量	名称	数据类型	有效范围	内容
boxID	电箱 ID	unsigned int	0~5	电箱 ID 号，默认值= 0
rbtID	机器人 ID	unsigned int	0~5	机器人 ID 号，默认值= 0

✓ 输出变量

输出变量	名称	数据类型	有效范围	内容
bDone	返回值	bool	false/true	false: 运动未完成，处于运动状态 true: 运动完成，处于准备就绪状态

✓ 返回值

返回值	名称	数据类型	有效范围	内容
nRet	返回值	int	>=0 的整型值	nRet = 0: 返回函数调用成功 nRet > 0: 返回调用失败的错误码

3.10.5.2. 示例

// 判断机器人是否处于运动状态

```
bool bDone = false;
int nRet= HRIF_IsMotionDone(0,0,bDone);
```

3.10.6. HRIF_IsBlendingDone

3.10.6.1. 描述：判断路点是否运动完成

✓ 输入变量

输入变量	名称	数据类型	有效范围	内容
boxID	电箱 ID	unsigned int	0~5	电箱 ID 号，默认值= 0
rbtID	机器人 ID	unsigned int	0~5	机器人 ID 号，默认值= 0

* 注：此接口仅判断路点是否运动完成，返回 true 后，机器人可能还处于运动状态；如：开启跟随、力控、相对跟踪后，机器人处于运动状态，在没有调用路点时，此接口返回 true。

✓ 输出变量

输出变量	名称	数据类型	有效范围	内容
bDone	返回值	bool	false/true	false：运动未完成，处于运动状态 true：运动完成，处于准备就绪状态

✓ 返回值

返回值	名称	数据类型	有效范围	内容
nRet	返回值	int	>=0 的整型值	nRet = 0: 返回函数调用成功 nRet >0: 返回调用失败的错误码

3.10.6.2. 示例

// 判断路点是否运动完成

```
bool bDone = false;
int nRet= HRIF_IsBlendingDone(0,0,bDone);
```

3.10.7. HRIF_WayPointEx

3.10.7.1. 描述：路点运动。

✓ 输入变量

输入变量	名称	数据类型	有效范围	内容
boxID	电箱 ID	unsigned int	0~5	电箱 ID 号，默认值= 0
rbtID	机器人 ID	unsigned int	0~5	机器人 ID 号，默认值= 0
nMoveType	运动类型	int	0/1	0: 关节运动 1: 空间运动
dX -dRz	空间目标位置	double	-	nMoveType= 0 并 nIsUseJoint= 1: 无效, nMoveType= 0 并 nIsUseJoint= 0: 用此空间坐标作为目标位置，通过逆解计算得到关节坐标为目标关节坐标 nMoveType= 1: 目标空间位置 dX: X 坐标，单位[mm] dY: Y 坐标，单位[mm] dZ: Z 坐标，单位[mm] dRx: Rx 坐标，单位[°] dRy: Ry 坐标，单位[°] dRz: Rz 坐标，单位[°]
dJ1 -dJ6	关目标位置	double	-	nMoveType= 0 并 nIsUseJoint= 1: 使用此关节坐标作为目标关节坐标 nMoveType= 0 并 nIsUseJoint= 0: 此关节坐标仅作为计算逆解时选解的参考关节坐标 nMoveType= 1: 无效 dJ1: 关节 1 坐标，单位[°] dJ2: 关节 2 坐标，单位[°] dJ3: 关节 3 坐标，单位[°] dJ4: 关节 4 坐标，单位[°] dJ5: 关节 5 坐标，单位[°]

				dJ6: 关节 6 坐标, 单位[°]
dTcp_X-dTcp_Rz	工具坐标值	double	-	目标空间坐标所处的工具坐标系名称, 与示教器页面的名称对应, 当 nIsUseJoint= 1 时无效, 可设置为 0: dTcp_X: X 坐标, 单位[mm] dTcp_Y: Y 坐标, 单位[mm] dTcp_Z: Z 坐标, 单位[mm] dTcp_Rx: Rx 坐标, 单位[°] dTcp_Ry: Ry 坐标, 单位[°] dTcp_Rz: Rz 坐标, 单位[°]
dUcs_X-dUcs_Rz	用户坐标值	double	-	目标空间坐标所处的用户坐标系名称, 与示教器页面的名称对应, 当 nIsUseJoint= 1 时无效, 可设置为 0: dUcs_X: X 坐标, 单位[mm] dUcs_Y: Y 坐标, 单位[mm] dUcs_Z: Z 坐标, 单位[mm] dUcs_Rx: Rx 坐标, 单位[°] dUcs_Ry: Ry 坐标, 单位[°] dUcs_Rz: Rz 坐标, 单位[°]
dVelocity	速度	double	0.1<速度<加速度	运动最大速度, 关节运动时单位[°/s], 空间运动时 XYZ 单位[mm/s], Rx, Ry, Rz 单位[°/s]
dAcc	加速度	double	>0.1	运动最大加速度, 关节运动时单位[°/s²], 空间运动时 XYZ 单位[mm/s²], Rx, Ry, Rz 单位[°/s²]
dRadius	过渡半径	double	0-1000	过渡半径, 单位[mm]
nIsUseJoint	是否使用关节坐标	int	0/1	是否使用关节角度作为目标点, 如果 nMoveType= 0 时, 则 nIsUseJoint 有效: 0: 不使用关节角度 1: 使用关节角度
nIsSeek	是否检测 DI 停	int	0/1	如果 nIsSeek 为 1, 则开启检测 DI 停止, 路

	止			点运动过程中如果电箱的 nIOBit 索引的 DI 的状态= nIOState 时，机器人停止运动，否则运动到目标点完成运动
nIOBit	检测的 DI 索引	int	0-7	电箱对应 DI 索引， nIsSeek= 0 时无效
nIOState	检测的 DI 状态	int	0/1	检测的 DI 状态， nIsSeek= 0 时无效
strCmdID	命令 ID	string	-	当前路点 ID，可以自定义，也可以按顺序设置为"1"，"2"，"3"

* 注：HRIF_WayPointEx 需要设置工具坐标与用户坐标具体的值；HRIF_WayPoint 使用示教器示教的工具坐标与用户坐标名称。

✓ 返回值

返回值	名称	数据类型	有效范围	内容
nRet	返回值	int	>=0 的整型值	nRet = 0: 返回函数调用成功 nRet >0: 返回调用失败的错误码

3.10.7.2. 示例

// 定义运动类型

```
int nMoveType = 0;
```

// 定义空间目标位置

```
double dX = 0; double dY = 0; double dZ = 0;
double dRx = 0; double dRy = 0; double dRz = 0;
```

// 定义关节目标位置

```
double dJ1 = 0; double dJ2 = 0; double dJ3 = 0;
double dJ4 = 0; double dJ5 = 0; double dJ6 = 0;
```

// 定义工具坐标变量

```
double dTcp_X = 0; double dTcp_Y = 0; double dTcp_Z = 0;
double dTcp_Rx = 0; double dTcp_Ry = 0; double dTcp_Rz = 0;
```

// 定义用户坐标变量

```
double dUcs_X = 0; double dUcs_Y = 0; double dUcs_Z = 0;
double dUcs_Rx = 0; double dUcs_Ry = 0; double dUcs_Rz = 0;
```

// 定义运动速度

```
double dVelocity = 50;
```

// 定义运动加速度

double dAcc = 100;

// 定义过渡半径

double dRadius = 50;

// 定义是否使用关节角度

int nIsUseJoint = 1;

// 定义是否使用检测 DI 停止

int nIsSeek = 0;

// 定义检测的 DI 索引

int nIOBit = 0;

// 定义检测的 DI 状态

int nIOState = 0;

// 定义路点 ID

string strCmdID = "0";

// 执行路点运动

```
int nRet= HRIF_WayPointEx(0,0, nMoveType , dX , dY , dZ , dRx , dRy , dRz,  
dJ1 , dJ2 , dJ3 , dJ4 , dJ5 , dJ6 ,dTcp_X, dTcp_Y, dTcp_Z, dTcp_Rx, dTcp_Ry, dTcp_Rz,  
dUcs_X, dUcs_Y, dUcs_Z, dUcs_Rx, dUcs_Ry, dUcs_Rz,dVelocity, dAcc, dRadius, nIsUseJoint, nIsSeek, nIOBit,  
nIOState, strCmdID );
```

3.10.8. HRIF_WayPointEx_nJ

3.10.8.1. 描述：路点运动。**注意：这是扩展轴版本所用接口**

✓ 输入变量

输入变量	名称	数据类型	有效范围	内容
boxID	电箱 ID	unsigned int	0~5	电箱 ID 号，默认值= 0
rbtID	机器人 ID	unsigned int	0~5	机器人 ID 号，默认值= 0
nMoveType	运动类型	int	0/1	0: 关节运动 1: 空间运动
dX -dRz	空间目标位置	double	-	nMoveType= 0 并 nIsUseJoint= 1: 无效, nMoveType= 0 并 nIsUseJoint= 0: 用此空间坐标作为目标位置，通过逆解计算得到关节坐标为目标关节坐标 nMoveType= 1: 目标空间位置 dX: X 坐标，单位[mm] dY: Y 坐标，单位[mm] dZ: Z 坐标，单位[mm] dRx: Rx 坐标，单位[°] dRy: Ry 坐标，单位[°] dRz: Rz 坐标，单位[°]
Joints	关节目标位置	JointsData	-	nMoveType= 0 并 nIsUseJoint= 1: 使用此关节坐标作为目标关节坐标 nMoveType= 0 并 nIsUseJoint= 0: 此关节坐标仅作为计算逆解时选解的参考关节坐标 nMoveType= 1: 无效 Joints.dJoint[i]: 关节 i 坐标，单位[°]
dTcp_X-dTcp_Rz	工具坐标值	double	-	目标空间坐标所处的工具坐标系名称，与示教器页面的名称对应，当 nIsUseJoint= 1 时无效，可设置为 0: dTcp_X: X 坐标，单位[mm]

				dTcp_Y: Y 坐标, 单位[mm] dTcp_Z: Z 坐标, 单位[mm] dTcp_Rx: Rx 坐标, 单位[°] dTcp_Ry: Ry 坐标, 单位[°] dTcp_Rz: Rz 坐标, 单位[°]
dUcs_X-dUcs_Rz	用户坐标值	double	-	目标空间坐标所处的用户坐标系名称, 与示教器页面的名称对应, 当 nIsUseJoint= 1 时无效, 可设置为 0: dUcs_X: X 坐标, 单位[mm] dUcs_Y: Y 坐标, 单位[mm] dUcs_Z: Z 坐标, 单位[mm] dUcs_Rx: Rx 坐标, 单位[°] dUcs_Ry: Ry 坐标, 单位[°] dUcs_Rz: Rz 坐标, 单位[°]
dVelocity	速度	double	0.1<速度<加速度	运动最大速度, 关节运动时单位[°/s], 空间运动时 XYZ 单位[mm/s], Rx, Ry, Rz 单位[°/s]
dAcc	加速度	double	>0.1	运动最大加速度, 关节运动时单位[°/s²], 空间运动时 XYZ 单位[mm/s²], Rx, Ry, Rz 单位[°/s²]
dRadius	过渡半径	double	0-1000	过渡半径, 单位[mm]
nIsUseJoint	是否使用关节坐标	int	0/1	是否使用关节角度作为目标点, 如果 nMoveType= 0 时, 则 nIsUseJoint 有效: 0: 不使用关节角度 1: 使用关节角度
nIsSeek	是否检测 DI 停止	int	0/1	如果 nIsSeek 为 1, 则开启检测 DI 停止, 路点运动过程中如果电箱的 nIOBit 位索引的 DI 的状态= nIOState 时, 机器人停止运动, 否则运动到目标点完成运动
nIOBit	检测的 DI 索引	int	0-7	电箱对应 DI 索引, nIsSeek= 0 时无效
nIOState	检测的 DI 状态	int	0/1	检测的 DI 状态, , nIsSeek= 0 时无效

strCmdID	命令 ID	string	-	当前路点 ID，可以自定义，也可以按顺序设置为"1", "2", "3"
----------	-------	--------	---	--------------------------------------

* 注：HRIF_WayPointEx 需要设置工具坐标与用户坐标具体的值；HRIF_WayPoint 使用示教器示教的工具坐标与用户坐标名称。

✓ 返回值

返回值	名称	数据类型	有效范围	内容
nRet	返回值	int	>=0 的整型值	nRet = 0: 返回函数调用成功 nRet >0: 返回调用失败的错误码

3.10.8.2. 示例

// 定义运动类型

```
int nMoveType = 0;
```

// 定义空间目标位置

```
double dX = 0; double dY = 0; double dZ = 0;
```

```
double dRx = 0; double dRy = 0; double dRz = 0;
```

// 定义关节目标位置

```
JointsData Joints;
```

```
Joints.dJoint[0]=0;Joints.dJoint[1]=0;Joints.dJoint[2]=90;
```

```
Joints.dJoint[3]=0;Joints.dJoint[4]=90;Joints.dJoint[5]=0;
```

// 定义工具坐标变量

```
double dTcp_X = 0; double dTcp_Y = 0; double dTcp_Z = 0;
```

```
double dTcp_Rx = 0; double dTcp_Ry = 0; double dTcp_Rz = 0;
```

// 定义用户坐标变量

```
double dUcs_X = 0; double dUcs_Y = 0; double dUcs_Z = 0;
```

```
double dUcs_Rx = 0; double dUcs_Ry = 0; double dUcs_Rz = 0;
```

// 定义运动速度

```
double dVelocity = 50;
```

// 定义运动加速度

```
double dAcc = 100;
```

// 定义过渡半径

```
double dRadius = 50;
```

// 定义是否使用关节角度

```
int nIsUseJoint = 1;  
// 定义是否使用检测 DI 停止  
  
int nIsSeek = 0;  
// 定义检测的 DI 索引  
  
int nIOBit = 0;  
// 定义检测的 DI 状态  
  
int nIOState = 0;  
// 定义路点 ID  
  
string strCmdID = "0";  
// 执行路点运动  
  
int nRet= HRIF_WayPointEx_nJ(0,0, nMoveType , dX , dY , dZ , dRx , dRy , dRz,  
Joints ,dTcp_X, dTcp_Y, dTcp_Z, dTcp_Rx, dTcp_Ry, dTcp_Rz,  
dUcs_X, dUcs_Y, dUcs_Z, dUcs_Rx, dUcs_Ry, dUcs_Rz,dVelocity, dAcc, dRadius, nIsUseJoint, nIsSeek, nIOBit,  
nIOState, strCmdID );
```

3.10.9. HRIF_WayPoint

3.10.9.1. 描述：路点运动。

✓ 输入变量

输入变量	名称	数据类型	有效范围	内容
boxID	电箱 ID	unsigned int	0~5	电箱 ID 号，默认值= 0
rbtID	机器人 ID	unsigned int	0~5	机器人 ID 号，默认值= 0
nMoveType	运动类型	int	0/1	0: 关节运动 1: 直线运动
dX -dRz	空间目标位置	double	-	nMoveType= 0 并 nIsUseJoint= 1: 无效, nMoveType= 0 并 nIsUseJoint= 0: 用此空间坐标作为目标位置, 通过逆解计算得到关节坐标为目标关节坐标 nMoveType= 1: 目标空间位置 dX: X 坐标, 单位[mm] dY: Y 坐标, 单位[mm] dZ: Z 坐标, 单位[mm] dRx: Rx 坐标, 单位[°] dRy: Ry 坐标, 单位[°] dRz: Rz 坐标, 单位[°]
dJ1 -dJ6	关节目标位置	double	-	nMoveType= 0 并 nIsUseJoint= 1: 使用此关节坐标作为目标关节坐标 nMoveType= 0 并 nIsUseJoint= 0: 此关节坐标仅作为计算逆解时选解的参考关节坐标 nMoveType= 1: 无效 dJ1: 关节 1 坐标, 单位[°] dJ2: 关节 2 坐标, 单位[°] dJ3: 关节 3 坐标, 单位[°]

				dJ4: 关节 4 坐标, 单位[°] dJ5: 关节 5 坐标, 单位[°] dJ6: 关节 6 坐标, 单位[°]
sTcpName	工具坐标名称	string	-	目标空间坐标所处的工具坐标系名称, 与示教器页面的名称对应, 当 nIsUseJoint= 1 时无效, 可使用默认名称"TCP"
sUcsName	用户坐标名称	string	-	目标空间坐标所处的用户坐标系名称, 与示教器页面的名称对应, 当 nIsUseJoint= 1 时无效, 可使用默认名称"Base"
dVelocity	速度	double	0.1<速度<加速度	运动最大速度, 关节运动时单位[°/s], 空间运动时 XYZ 单位[mm/s], Rx, Ry, Rz 单位[°/s]
dAcc	加速度	double	>0.1	运动最大加速度, 关节运动时单位[°/s²], 空间运动时 XYZ 单位[mm/s²], Rx, Ry, Rz 单位[°/s²]
dRadius	过渡半径	double	0-1000	过渡半径, 单位[mm]
nIsUseJoint	是否使用关节坐标	int	0/1	是否使用关节角度作为目标点, 如果 nMoveType= 0 时, 则 nIsUseJoint 有效: 0: 不使用关节角度 1: 使用关节角度
nIsSeek	是否检测 DI 停止	int	0/1	如果 nIsSeek 为 1, 则开启检测 DI 停止, 路点运动过程中如果电箱的 nIOBit 位索引的 DI 的状态= nIOState 时, 机器人停止运动, 否则运动到目标点完成运动
nIOBit	检测的 DI 索引	int	0~7	电箱对应 DI 索引, nIsSeek= 0 时无效
nIOState	检测的 DI 状态	int	0/1	检测的 DI 状态, , nIsSeek= 0 时无效
strCmdID	命令 ID	string	-	当前路点 ID, 可以自定义, 也可以按顺序设置为"1", "2", "3"

* 注: HRIF_WayPointEx 需要设置工具坐标与用户坐标具体的值; HRIF_WayPoint 使用示教器示教的工具坐标与用户坐标名称。

✓ 返回值

返回值	名称	数据类型	有效范围	内容
nRet	返回值	int	≥ 0 的整型值	nRet = 0: 返回函数调用成功 nRet > 0: 返回调用失败的错误码

3.10.9.2. 示例

// 定义运动类型

```
int nMoveType = 0;
```

// 定义空间目标位置

```
double dX = 0; double dY = 0; double dZ = 0;
```

```
double dRx = 0; double dRy = 0; double dRz = 0;
```

// 定义关节目标位置

```
double dJ1 = 0; double dJ2 = 0; double dJ3 = 0;
```

```
double dJ4 = 0; double dJ5 = 0; double dJ6 = 0;
```

// 定义工具坐标变量

```
string sTcpName = "TCP";
```

// 定义用户坐标变量

```
string sUcsName = "Base";
```

// 定义运动速度

```
double dVelocity = 50;
```

// 定义运动加速度

```
double dAcc = 100;
```

// 定义过渡半径

```
double dRadius = 50;
```

// 定义是否使用关节角度

```
int nIsUseJoint = 1;
```

// 定义是否使用检测 DI 停止

```
int nIsSeek = 0;
```

// 定义检测的 DI 索引

```
int nIOBit = 0;
```

// 定义检测的 DI 状态

```
int nIOState = 0;
```

// 定义路点 ID

```
string strCmdID = "0";
```

```
// 执行路点运动
```

```
int nRet= HRIF_WayPoint(0, 0,nMoveType , dX , dY , dZ , dRx , dRy , dRz,  
dJ1 , dJ2 , dJ3 , dJ4 , dJ5 , dJ6 ,  
sTcpName , sUcsName, dVelocity, dAcc, dRadius,  
nIsUseJoint, nIsSeek, nIOBit, nIOState, strCmdID );
```

3.10.10. HRIF_WayPoint_nJ

3.10.10.1.描述：路点运动。**注意：这是扩展轴版本所用接口**

✓ 输入变量

输入变量	名称	数据类型	有效范围	内容
boxID	电箱 ID	unsigned int	0~5	电箱 ID 号，默认值= 0
rbtID	机器人 ID	unsigned int	0~5	机器人 ID 号，默认值= 0
nMoveType	运动类型	int	0/1	0：关节运动 1：直线运动
dX -dRz	空间目标位置	double	-	nMoveType= 0 并 nIsUseJoint= 1：无效， nMoveType= 0 并 nIsUseJoint= 0：用此空间坐标作为目标位置，通过逆解计算得到关节坐标为目标关节坐标 nMoveType= 1：目标空间位置 dX：X 坐标，单位[mm] dY：Y 坐标，单位[mm] dZ：Z 坐标，单位[mm] dRx：Rx 坐标，单位[°] dRy：Ry 坐标，单位[°] dRz：Rz 坐标，单位[°]
Joints	关关节目标位置	JointsData	-	nMoveType= 0 并 nIsUseJoint= 1：使用此关节坐标作为目标关节坐标 nMoveType= 0 并 nIsUseJoint= 0：此关节坐标仅作为计算逆解时选解的参考关节坐标 nMoveType= 1：无效 Joints.dJoint[i]：关节 i 坐标，单位[°]
sTcpName	工具坐标名称	string	-	目标空间坐标所处的工具坐标系名称，与示教器页面的名称对应，当 nIsUseJoint= 1 时无

				效，可使用默认名称"TCP"
sUcsName	用户坐标名称	string	-	目标空间坐标所处的用户坐标系名称，与示教器页面的名称对应，当 nIsUseJoint= 1 时无效，可使用默认名称"Base"
dVelocity	速度	double	0.1<速度<加速度	运动最大速度，关节运动时单位[°/s]，空间运动时 XYZ 单位[mm/s]，Rx, Ry, Rz 单位[°/s]
dAcc	加速度	double	>0.1	运动最大加速度，关节运动时单位[°/s²]，空间运动时 XYZ 单位[mm/s²]，Rx, Ry, Rz 单位[°/s²]
dRadius	过渡半径	double	0-1000	过渡半径，单位[mm]
nIsUseJoint	是否使用关节坐标	int	0/1	是否使用关节角度作为目标点，如果 nMoveType= 0 时，则 nIsJoint 有效： 0：不使用关节角度 1：使用关节角度
nIsSeek	是否检测 DI 停止	int	0/1	如果 nIsSeek 为 1，则开启检测 DI 停止，路点运动过程中如果电箱的 nIOBit 位索引的 DI 的状态= nIOState 时，机器人停止运动，否则运动到目标点完成运动
nIOBit	检测的 DI 索引	int	0~7	电箱对应 DI 索引，nIsSeek= 0 时无效
nIOState	检测的 DI 状态	int	0/1	检测的 DI 状态，nIsSeek= 0 时无效
strCmdID	命令 ID	string	-	当前路点 ID，可以自定义，也可以按顺序设置为"1"，"2"，"3"

* 注：HRIF_WayPointEx 需要设置工具坐标与用户坐标具体的值；HRIF_WayPoint 使用示教器示教的工具坐标与用户坐标名称。

✓ 返回值

返回值	名称	数据类型	有效范围	内容
nRet	返回值	int	>=0 的整型值	nRet = 0: 返回函数调用成功 nRet >0: 返回调用失败的错误码

3.10.10.2. 示例

// 定义运动类型

```
int nMoveType = 0;
```

// 定义空间目标位置

```
double dX = 0; double dY = 0; double dZ = 0;
```

```
double dRx = 0; double dRy = 0; double dRz = 0;
```

// 定义关节目标位置

```
JointsData Joints;
```

```
Joints.dJoint[0] = 90; Joints.dJoint[1] = 0; Joints.dJoint[2] = 90; Joints.dJoint[3] = 0;
```

```
Joints.dJoint[4] = 90; Joints.dJoint[5] = 0; Joints.dJoint[6] = 0;
```

// 定义工具坐标变量

```
string sTcpName = "TCP";
```

// 定义用户坐标变量

```
string sUcsName = "Base";
```

// 定义运动速度

```
double dVelocity = 50;
```

// 定义运动加速度

```
double dAcc = 100;
```

// 定义过渡半径

```
double dRadius = 50;
```

// 定义是否使用关节角度

```
int nIsUseJoint = 1;
```

// 定义是否使用检测 DI 停止

```
int nIsSeek = 0;
```

// 定义检测的 DI 索引

```
int nIOBit = 0;
```

// 定义检测的 DI 状态

```
int nIOState = 0;
```

// 定义路点 ID

```
string strCmdID = "0";
```

// 执行路点运动

```
int nRet= HRIF_WayPoint_nJ(0, 0,nMoveType , dX , dY , dZ , dRx , dRy , dRz,  
Joints , sTcpName , sUcsName, dVelocity, dAcc, dRadius, nIsUseJoint, nIsSeek, nIOBit, nIOState, strCmdID );
```

3.10.11. HRIF_WayPoint2

3.10.11.1.描述：路点运动。

✓ 输入变量

输入变量	名称	数据类型	有效范围	内容
boxID	电箱 ID	unsigned int	0~5	电箱 ID 号, 默认值= 0
rbtID	机器人 ID	unsigned int	0~5	机器人 ID 号, 默认值= 0
nMoveType	运动类型	int	0/1	0: 关节运动 1: 直线运动 2: 圆弧运动
dEndPos_X-dEndPos_R Z	空间目标位置	double	-	nMoveType= 0 并 nIsUseJoint= 1: 无效, nMoveType= 0 并 nIsUseJoint= 0: 用此空间坐标作为目标位置, 通过逆解计算得到关节坐标为目标关节坐标 nMoveType= 1: 目标空间位置 nMoveType= 2: 做为圆弧的目标位置 dEndPos_X: X 坐标, 单位[mm] dEndPos_Y: Y 坐标, 单位[mm] dEndPos_Z: Z 坐标, 单位[mm] dEndPos_Rx: Rx 坐标, 单位[°] dEndPos_Ry: Ry 坐标, 单位[°] dEndPos_Rz: Rz 坐标, 单位[°]
dAuxPos_X-dAuxPos_ Rz	空间目标位置	double	-	nMoveType= 0: 无效 nMoveType= 1: 无效 nMoveType= 2: 做为圆弧的经过位置 dAuxPos_X: X 坐标, 单位[mm] dAuxPos_Y: Y 坐标, 单位[mm] dAuxPos_Z: Z 坐标, 单位[mm] dAuxPos_Rx: Rx 坐标, 单位[°] dAuxPos_Ry: Ry 坐标, 单位[°]

				dAuxPosX_Rz: Rz 坐标, 单位[°]
dJ1 -dJ6	关目标位置	double	-	nMoveType= 0 并 nIsUseJoint= 1: 使用此 关节坐标作为目标关节坐标 nMoveType= 0 并 nIsUseJoint= 0: 此关节 坐标仅作为计算逆解时选解的参考关节 坐标 nMoveType= 1: 无效 nMoveType= 2: 无效 dJ1: 关节 1 坐标, 单位[°] dJ2: 关节 2 坐标, 单位[°] dJ3: 关节 3 坐标, 单位[°] dJ4: 关节 4 坐标, 单位[°] dJ5: 关节 5 坐标, 单位[°] dJ6: 关节 6 坐标, 单位[°]
sTcpName	工具坐标名称	string	-	目标空间坐标所处的工具坐标系名称, 与 示教器页面的名称对应, 当 nIsUseJoint= 1 时无效, 可使用默认名称"TCP"
sUcsName	用户坐标名称	string	-	目标空间坐标所处的用户坐标系名称, 与 示教器页面的名称对应, 当 nIsUseJoint= 1 时无效, 可使用默认名称"Base"
dVelocity	速度	double	> 0.1, 且小于系统 允许的最大速度	运动最大速度, 关节运动时单位[°/s], 空 间运动时 XYZ 单位[mm/s], Rx, Ry, Rz 单位[°/s]
dAcc	加速度	double	>0.1	运动最大加速度, 关节运动时单位[°/s²], 空间运动时 XYZ 单位[mm/s²], Rx, Ry, Rz 单位[°/s²]
dRadius	过渡半径	double	0-1000	过渡半径, 单位[mm]
nIsUseJoint	是否使用关节 坐标	int	0/1	是否使用关节角度作为目标点, 如果 nMoveType= 0 时, 则 nIsUseJoint 有效: 0: 不使用关节角度

				1: 使用关节角度
nIsSeek	是否检测 DI 停止	int	0/1	如果 nIsSeek 为 1, 则开启检测 DI 停止, 路点运动过程中如果电箱的 nIOBit 位索引的 DI 的状态= nIOState 时, 机器人停止运动, 否则运动到目标点完成运动
nIOBit	检测的 DI 索引	int	0~7	电箱对应 DI 索引, nIsSeek=0 时无效
nIOState	检测的 DI 状态	int	0/1	检测的 DI 状态, , nIsSeek=0 时无效
strCmdID	命令 ID	string	-	当前路点 ID, 可以自定义, 也可以按顺序设置为"1", "2", "3"

* 注: HRIF_WayPoint2 支持直线与圆弧过渡不减速功能; HRIF_WayPoint 仅支持直线与直线过渡不减速。

✓ 返回值

返回值	名称	数据类型	有效范围	内容
nRet	返回值	int	>=0 的整型值	nRet = 0: 返回函数调用成功 nRet >0: 返回调用失败的错误码

3.10.11.2.示例

// 定义运动类型

```
int nMoveType = 0;
```

// 定义空间目标位置

```
double dEndPos_X = 420; double dEndPos_Y = 0; double dEndPos_Z = 445;
```

```
double dEndPos_Rx = 180; double dEndPos_Ry = 0; double dEndPos_Rz = 180;
```

// 定义空间目标位置

```
double dAuxPos_X = 420; double dAuxPos_Y = 0; double dAuxPos_Z = 445;
```

```
double dAuxPos_Rx = 180; double dAuxPos_Ry = 0; double dAuxPos_Rz = 180;
```

// 定义关目标位置

```
double dJ1 = 0; double dJ2 = 0; double dJ3 = 90;
```

```
double dJ4 = 0; double dJ5 = 90; double dJ6 = 0;
```

// 定义工具坐标变量

```
string sTcpName = "TCP";
```

// 定义用户坐标变量

```
string sUcsName = "Base";
```

// 定义运动速度

double dVelocity = 50;

// 定义运动加速度

double dAcc = 100;

// 定义过渡半径

double dRadius = 50;

// 定义是否使用关节角度

int nIsUseJoint = 1;

// 定义是否使用检测 DI 停止

int nIsSeek = 0;

// 定义检测的 DI 索引

int nIOBit = 0;

// 定义检测的 DI 状态

int nIOState = 0;

// 定义路点 ID

string strCmdID = "0";

// 执行路点运动

```
int nRet= HRIF_WayPoint2(0,0,nMoveType ,dEndPos_X, dEndPos_Y, dEndPos_Z, dEndPos_Rx, dEndPos_Ry,
dEndPos_Rz,dAuxPos_X, dAuxPos_Y, dAuxPos_Z, dAuxPos_Rx, dAuxPos_Ry, dAuxPos_Rz,dJ1 , dJ2 , dJ3 , dJ4 ,
dJ5 , dJ6 ,sTcpName , sUcsName, dVelocity, dAcc, dRadius,
nIsUseJoint, nIsSeek, nIOBit, nIOState, strCmdID );
```

3.10.12. HRIF_WayPoint2_nJ

3.10.12.1.描述：路点运动。**注意：这是扩展轴版本所用接口**

✓ 输入变量

输入变量	名称	数据类型	有效范围	内容
boxID	电箱 ID	unsigned int	0~5	电箱 ID 号，默认值= 0
rbtID	机器人 ID	unsigned int	0~5	机器人 ID 号，默认值= 0
nMoveType	运动类型	int	0/1	0：关节运动 1：直线运动 2：圆弧运动
dEndPos_X-dEndPos_R Z	空间目标位置	double	-	nMoveType= 0 并 nIsUseJoint= 1：无效， nMoveType= 0 并 nIsUseJoint= 0：用此空间坐标作为目标位置，通过逆解计算得到关节坐标为目标关节坐标 nMoveType= 1：目标空间位置 nMoveType= 2：做为圆弧的目标位置 dEndPos_X：X 坐标，单位[mm] dEndPos_Y：Y 坐标，单位[mm] dEndPos_Z：Z 坐标，单位[mm] dEndPos_Rx：Rx 坐标，单位[°] dEndPos_Ry：Ry 坐标，单位[°] dEndPos_Rz：Rz 坐标，单位[°]
dAuxPos_X-dAuxPos_ Rz	空间目标位置	double	-	nMoveType= 0：无效 nMoveType= 1：无效 nMoveType= 2：做为圆弧的经过位置 dAuxPosX_X：X 坐标，单位[mm] dAuxPosX_Y：Y 坐标，单位[mm] dAuxPosX_Z：Z 坐标，单位[mm] dAuxPosX_Rx：Rx 坐标，单位[°] dAuxPosX_Ry：Ry 坐标，单位[°]

				dAuxPosX_Rz: Rz 坐标, 单位[°]
Joints	关目标位置	JointsData	-	nMoveType= 0 并 nIsUseJoint= 1: 使用此关节坐标作为目标关节坐标 nMoveType= 0 并 nIsUseJoint= 0: 此关节坐标仅作为计算逆解时选解的参考关节坐标 nMoveType= 1: 无效 nMoveType= 2: 无效 Joints.dJoint[i]: 关节坐标, 单位[°]
sTcpName	工具坐标名称	string	-	目标空间坐标所处的工具坐标系名称, 与示教器页面的名称对应, 当 nIsUseJoint= 1 时无效, 可使用默认名称"TCP"
sUcsName	用户坐标名称	string	-	目标空间坐标所处的用户坐标系名称, 与示教器页面的名称对应, 当 nIsUseJoint= 1 时无效, 可使用默认名称"Base"
dVelocity	速度	double	0.1<速度<加速度	运动最大速度, 关节运动时单位[°/s], 空间运动时 XYZ 单位[mm/s], Rx, Ry, Rz 单位[°/s]
dAcc	加速度	double	>0.1	运动最大加速度, 关节运动时单位[°/s²], 空间运动时 XYZ 单位[mm/s²], Rx, Ry, Rz 单位[°/s²]
dRadius	过渡半径	double	0-1000	过渡半径, 单位[mm]
nIsUseJoint	是否使用关节坐标	int	0/1	是否使用关节角度作为目标点, 如果 nMoveType= 0 时, 则 nIsUseJoint 有效: 0: 不使用关节角度 1: 使用关节角度
nIsSeek	是否检测 DI 停止	int	0/1	如果 nIsSeek 为 1, 则开启检测 DI 停止, 路点运动过程中如果电箱的 nIOBit 位索引的 DI 的状态= nIOState 时, 机器人停止运动, 否则运动到目标点完成运动

nIOBit	检测的 DI 索引	int	0~7	电箱对应 DI 索引, nIsSeek=0 时无效
nIOState	检测的 DI 状态	int	0/1	检测的 DI 状态, nIsSeek=0 时无效
strCmdID	命令 ID	string	-	当前路点 ID, 可以自定义, 也可以按顺序设置为"1", "2", "3"

* 注: HRIF_WayPoint2 支持直线与圆弧过渡不减速功能; HRIF_WayPoint 仅支持直线与直线过渡不减速。

✓ 返回值

返回值	名称	数据类型	有效范围	内容
nRet	返回值	int	>=0 的整型值	nRet = 0: 返回函数调用成功 nRet > 0: 返回调用失败的错误码

3.10.12.2.示例

// 定义运动类型

```
int nMoveType = 0;
```

// 定义空间目标位置

```
double dEndPos_X = 420; double dEndPos_Y = 0; double dEndPos_Z = 445;
```

```
double dEndPos_Rx = 180; double dEndPos_Ry = 0; double dEndPos_Rz = 180;
```

// 定义空间目标位置

```
double dAuxPos_X = 420; double dAuxPos_Y = 0; double dAuxPos_Z = 445;
```

```
double dAuxPos_Rx = 180; double dAuxPos_Ry = 0; double dAuxPos_Rz = 180;
```

// 定义关节目标位置

```
JointsData Joints;
```

```
Joints.dJoint[0] = 0; Joints.dJoint[1] = 0; Joints.dJoint[2] = 90;
```

```
Joints.dJoint[3] = 0; Joints.dJoint[4] = 90; Joints.dJoint[5] = 0;
```

// 定义工具坐标变量

```
string sTcpName = "TCP";
```

// 定义用户坐标变量

```
string sUcsName = "Base";
```

// 定义运动速度

```
double dVelocity = 50;
```

// 定义运动加速度

```
double dAcc = 100;
```

// 定义过渡半径

```
double dRadius = 50;
// 定义是否使用关节角度

int nIsUseJoint = 1;
// 定义是否使用检测 DI 停止

int nIsSeek = 0;
// 定义检测的 DI 索引

int nIOBit = 0;
// 定义检测的 DI 状态

int nIOState = 0;
// 定义路点 ID

string strCmdID = "0";
// 执行路点运动

int nRet = HRIF_WayPoint2_nJ(0, 0, nMoveType, dEndPos_X, dEndPos_Y, dEndPos_Z, dEndPos_Rx, dEndPos_Ry,
dEndPos_Rz, dAuxPos_X, dAuxPos_Y, dAuxPos_Z, dAuxPos_Rx, dAuxPos_Ry, dAuxPos_Rz, Joints, sTcpName,
sUcsName, dVelocity, dAcc, dRadius,
nIsUseJoint, nIsSeek, nIOBit, nIOState, strCmdID);
```

3.10.13. HRIF_WayPoint2ByLockAxis_nJ

3.10.13.1.描述：路点运动。**注意：这是扩展轴版本所用接口**

✓ 输入变量

输入变量	名称	数据类型	有效范围	内容
boxID	电箱 ID	unsigned int	0~5	电箱 ID 号，默认值= 0
rbtID	机器人 ID	unsigned int	0~5	机器人 ID 号，默认值= 0
nMoveType	运动类型	int	0/1	0：关节运动 1：直线运动 2：圆弧运动
dEndPos_X-dEndPos_R Z	空间目标位置	double	-	nMoveType= 0 并 nIsUseJoint= 1：无效， nMoveType= 0 并 nIsUseJoint= 0：用此空间坐标作为目标位置，通过逆解计算得到关节坐标为目标关节坐标 nMoveType= 1：目标空间位置 nMoveType= 2：做为圆弧的目标位置 dEndPos_X：X 坐标，单位[mm] dEndPos_Y：Y 坐标，单位[mm] dEndPos_Z：Z 坐标，单位[mm] dEndPos_Rx：Rx 坐标，单位[°] dEndPos_Ry：Ry 坐标，单位[°] dEndPos_Rz：Rz 坐标，单位[°]
dAuxPos_X-dAuxPos_ Rz	空间目标位置	double	-	nMoveType= 0：无效 nMoveType= 1：无效 nMoveType= 2：做为圆弧的经过位置 dAuxPos_X：X 坐标，单位[mm] dAuxPos_Y：Y 坐标，单位[mm] dAuxPos_Z：Z 坐标，单位[mm] dAuxPosX_Rx：Rx 坐标，单位[°] dAuxPos_Ry：Ry 坐标，单位[°]

				dAuxPos_Rz: Rz 坐标, 单位[°]
Joints	关目标位置	JointsData	-	nMoveType= 0 并 nIsUseJoint= 1: 使用此关节坐标作为目标关节坐标 nMoveType= 0 并 nIsUseJoint= 0: 此关节坐标仅作为计算逆解时选解的参考关节坐标 nMoveType= 1: 无效 nMoveType= 2: 无效 Joints.dJoint[i]: 关节坐标, 单位[°]
sTcpName	工具坐标名称	string	-	目标空间坐标所处的工具坐标系名称, 与示教器页面的名称对应, 当 nIsUseJoint= 1 时无效, 可使用默认名称"TCP"
sUcsName	用户坐标名称	string	-	目标空间坐标所处的用户坐标系名称, 与示教器页面的名称对应, 当 nIsUseJoint= 1 时无效, 可使用默认名称"Base"
dVelocity	速度	double	0.1<速度<加速度	运动最大速度, 关节运动时单位[°/s], 空间运动时 XYZ 单位[mm/s], Rx, Ry, Rz 单位[°/s]
dAcc	加速度	double	>0.1	运动最大加速度, 关节运动时单位[°/s²], 空间运动时 XYZ 单位[mm/s²], Rx, Ry, Rz 单位[°/s²]
dRadius	过渡半径	double	0-1000	过渡半径, 单位[mm]
nIsUseJoint	是否使用关节坐标	int	0/1	是否使用关节角度作为目标点, 如果 nMoveType= 0 时, 则 nIsUseJoint 有效: 0: 不使用关节角度 1: 使用关节角度
nIsSeek	是否检测 DI 停止	int	0/1	如果 nIsSeek 为 1, 则开启检测 DI 停止, 路点运动过程中如果电箱的 nIOBit 位索引的 DI 的状态= nIOState 时, 机器人停止运动, 否则运动到目标点完成运动

nIOBit	检测的 DI 索引	int	0~7	电箱对应 DI 索引, nIsSeek=0 时无效
nIOState	检测的 DI 状态	int	0/1	检测的 DI 状态, , nIsSeek=0 时无效
strCmdID	命令 ID	string	-	当前路点 ID, 可以自定义, 也可以按顺序设置为"1", "2", "3"
lockedFlags	关节锁定标志	int	位掩码 (Bitmask)	通过比特位 (Bit) 表示需要锁定的关节, 位值为 1 时锁定对应轴, 0 时不锁定。 位索引定义: 每个比特位对应一个关节 (例如: Bit0 → J1, Bit1 → J2, ... Bit6 → J7, Bit7 → J8, 以此类推)。 锁定逻辑: 锁定后, 该关节在运动过程中保持当前位置不变, 仅未锁定的关节参与运动。 版本依赖: 当前支持的锁定轴需查阅通信协议。 示例: 锁定 J7: 0x40 (二进制 01000000, Bit6=1) 锁定 J7 和 J8: 0xC0 (二进制 11000000, Bit6+Bit7=1)

* 注: HRIF_WayPoint2 支持直线与圆弧过渡不减速功能; HRIF_WayPoint 仅支持直线与直线过渡不减速。

✓ 返回值

返回值	名称	数据类型	有效范围	内容
nRet	返回值	int	≥0 的整型值	nRet = 0: 返回函数调用成功 nRet > 0: 返回调用失败的错误码

3.10.13.2. 示例

// 定义运动类型

```
int nMoveType = 0;
```

// 定义空间目标位置

```
double dEndPos_X = 420; double dEndPos_Y = 0; double dEndPos_Z = 445;
```

```
double dEndPos_Rx = 180; double dEndPos_Ry = 0; double dEndPos_Rz = 180;
```

// 定义空间目标位置

```
double dAuxPos_X = 420; double dAuxPos_Y = 0; double dAuxPos_Z = 445;  
double dAuxPos_Rx = 180; double dAuxPos_Ry = 0; double dAuxPos_Rz = 180;
```

```
// 定义关节目标位置
```

```
JointsData Joints;  
Joints.dJoint[0] = 0;Joints.dJoint[1] = 0;Joints.dJoint[2] = 90;  
Joints.dJoint[3] = 0;Joints.dJoint[4] = 90;Joints.dJoint[5] = 0;
```

```
// 定义工具坐标变量
```

```
string sTcpName = "TCP";
```

```
// 定义用户坐标变量
```

```
string sUcsName = "Base";
```

```
// 定义运动速度
```

```
double dVelocity = 50;
```

```
// 定义运动加速度
```

```
double dAcc = 100;
```

```
// 定义过渡半径
```

```
double dRadius = 50;
```

```
// 定义是否使用关节角度
```

```
int nIsUseJoint = 1;
```

```
// 定义是否使用检测 DI 停止
```

```
int nIsSeek = 0;
```

```
// 定义检测的 DI 索引
```

```
int nIOBit = 0;
```

```
// 定义检测的 DI 状态
```

```
int nIOState = 0;
```

```
// 定义路点 ID
```

```
string strCmdID = "0";
```

```
// 定义检测的关节锁定标志
```

```
int lockedFlags = 0x40;
```

```
// 执行路点运动
```

```
int nRet = HRIF_WayPoint2ByLockAxis_nJ(0, 0, nMoveType, dEndPos_X, dEndPos_Y, dEndPos_Z, dEndPos_Rx,  
dEndPos_Ry, dEndPos_Rz, dAuxPos_X, dAuxPos_Y, dAuxPos_Z, dAuxPos_Rx, dAuxPos_Ry, dAuxPos_Rz,Joints,  
sTcpName, sUcsName, dVelocity, dAcc, dRadius,  
nIsUseJoint, nIsSeek, nIOBit, nIOState, strCmdID,lockedFlags);
```


3.10.14. HRIF_MoveJ

3.10.14.1.描述：关节运动。

✓ 输入变量

输入变量	名称	数据类型	有效范围	内容
boxID	电箱 ID	unsigned int	0~5	电箱 ID 号，默认值= 0
rbtID	机器人 ID	unsigned int	0~5	机器人 ID 号，默认值= 0
dX -dRz	空间目标位置	double	-	nIsUseJoint= 1：无效 nIsUseJoint= 0：用此空间坐标作为目标位置，通过逆解计算得到关节坐标为目标关节坐标： dTcp_X：X 坐标，单位[mm] dTcp_Y：Y 坐标，单位[mm] dTcp_Z：Z 坐标，单位[mm] dTcp_Rx：Rx 坐标，单位[°] dTcp_Ry：Ry 坐标，单位[°] dTcp_Rz：Rz 坐标，单位[°]
dJ1 -dJ6	关关节目标位置	double	-	nIsUseJoint= 1：使用此关节坐标作为目标关节坐标 nIsUseJoint= 0：此关节坐标仅作为计算逆解时选解的参考关节坐标： dJ1：关节 1 坐标，单位[°] dJ2：关节 2 坐标，单位[°] dJ3：关节 3 坐标，单位[°] dJ4：关节 4 坐标，单位[°] dJ5：关节 5 坐标，单位[°] dJ6：关节 6 坐标，单位[°]
sTcpName	工具坐标名称	string	-	目标空间坐标所处的工具坐标系名称，与示教器页面的名称对应，当 nIsUseJoint= 1 时无

				效，可使用默认名称"TCP"
sUcsName	用户坐标名称	string	-	目标空间坐标所处的用户坐标系名称，与示教器页面的名称对应。当 nIsUseJoint= 1 时无效，可使用默认名称"Base"
dVelocity	速度	double	0.1<速度<加速度	运动最大速度，单位[°/s]
dAcc	加速度	double	>0.1	运动最大加速度，单位[°/s ²]
dRadius	过渡半径	double	-	过渡半径，单位[mm]
nIsUseJoint	是否使用关节坐标	int	0/1	是否使用关节角度作为目标点，如果 nMoveType= 0 时，则 nIsUseJoint 有效： 0：不使用关节角度 1：使用关节角度
nIsSeek	是否检测 DI 停止	int	0/1	如果 nIsSeek 为 1，则开启检测 DI 停止，路点运动过程中如果电箱的 nIOBit 位索引的 DI 的状态= nIOState 时，机器人停止运动，否则运动到目标点完成运动
nIOBit	检测的 DI 索引	int	0~7	电箱对应 DI 索引，nIsSeek= 0 时无效
nIOState	检测的 DI 状态	int	0/1	检测的 DI 状态，nIsSeek= 0 时无效
strCmdID	命令 ID	string	-	当前路点 ID，可以自定义，也可以按顺序设置为"1"，"2"，"3"

✓ 返回值

返回值	名称	数据类型	有效范围	内容
nRet	返回值	int	≥0 的整型值	nRet = 0: 返回函数调用成功 nRet >0: 返回调用失败的错误码

3.10.14.2.示例

// 定义空间目标位置

```
double dX = 0; double dY = 0; double dZ = 0;
double dRx = 0; double dRy = 0; double dRz = 0;
```

// 定义关节目标位置

```
double dJ1 = 0; double dJ2 = 0; double dJ3 = 0;
double dJ4 = 0; double dJ5 = 0; double dJ6 = 0;
```

// 定义工具坐标变量

```
string sTcpName = "TCP";
```

// 定义用户坐标变量

```
string sUcsName = "Base";
```

// 定义运动速度

```
double dVelocity = 50;
```

// 定义运动加速度

```
double dAcc = 100;
```

// 定义过渡半径

```
double dRadius = 50;
```

// 定义是否使用关节角度

```
int nIsUseJoint = 1;
```

// 定义是否使用检测 DI 停止

```
int nIsSeek = 0;
```

// 定义检测的 DI 索引

```
int nIOBit = 0;
```

// 定义检测的 DI 状态

```
int nIOState = 0;
```

// 定义路点 ID

```
string strCmdID = "0";
```

// 执行路点运动

```
int nRet= HRIF_MoveJ(0,0, dX , dY , dZ , dRx , dRy , dRz ,dJ1 , dJ2 , dJ3 , dJ4 , dJ5 , dJ6 ,sTcpName , sUcsName,  
dVelocity, dAcc, dRadius,nIsUseJoint, nIsSeek, nIOBit, nIOState, strCmdID );
```

3.10.15. HRIF_MoveJ_nJ

3.10.15.1.描述：关节运动。**注意：这是扩展轴版本所用接口**

✓ 输入变量

输入变量	名称	数据类型	有效范围	内容
boxID	电箱 ID	unsigned int	0~5	电箱 ID 号，默认值= 0
rbtID	机器人 ID	unsigned int	0~5	机器人 ID 号，默认值= 0
dX -dRz	空间目标位置	double	-	nIsUseJoint= 1：无效 nIsUseJoint= 0：用此空间坐标作为目标位置，通过逆解计算得到关节坐标为目标关节坐标： dTcp_X：X 坐标，单位[mm] dTcp_Y：Y 坐标，单位[mm] dTcp_Z：Z 坐标，单位[mm] dTcp_Rx：Rx 坐标，单位[°] dTcp_Ry：Ry 坐标，单位[°] dTcp_Rz：Rz 坐标，单位[°]
joints	关目标位置	JointsData	-	nIsUseJoint= 1：使用此关节坐标作为目标关节坐标 nIsUseJoint= 0：此关节坐标仅作为计算逆解时选解的参考关节坐标： joints.dJoint[i]：关节 i 坐标，单位[°]
sTcpName	工具坐标名称	string	-	目标空间坐标所处的工具坐标系名称，与示教器页面的名称对应，当 nIsUseJoint= 1 时无效，可使用默认名称"TCP"
sUcsName	用户坐标名称	string	-	目标空间坐标所处的用户坐标系名称，与示教器页面的名称对应。当 nIsUseJoint= 1 时无效，可使用默认名称"Base"
dVelocity	速度	double	0.1<速度<加速度	运动最大速度，单位[°/s]

dAcc	加速度	double	>0.1	运动最大加速度, 单位[°/s ²]
dRadius	过渡半径	double	-	过渡半径, 单位[mm]
nIsUseJoint	是否使用关节坐标	int	0/1	是否使用关节角度作为目标点, 如果 nMoveType= 0 时, 则 nIsUseJoint 有效: 0: 不使用关节角度 1: 使用关节角度
nIsSeek	是否检测 DI 停止	int	0/1	如果 nIsSeek 为 1, 则开启检测 DI 停止, 路 点运动过程中如果电箱的 nIOBit 位索引的 DI 的状态= nIOState 时, 机器人停止运动, 否则 运动到目标点完成运动
nIOBit	检测的 DI 索引	int	0~7	电箱对应 DI 索引, nIsSeek= 0 时无效
nIOState	检测的 DI 状态	int	0/1	检测的 DI 状态, , nIsSeek= 0 时无效
strCmdID	命令 ID	string	-	当前路点 ID, 可以自定义, 也可以按顺序 设置为"1", "2", "3"

✓ 返回值

返回值	名称	数据类型	有效范围	内容
nRet	返回值	int	>=0 的整型值	nRet = 0: 返回函数调用成功 nRet >0: 返回调用失败的错误码

3.10.15.2.示例

// 定义空间目标位置

```
double dX = 0; double dY = 0; double dZ = 0;
double dRx = 0; double dRy = 0; double dRz = 0;
```

// 定义关节目标位置

```
JointsData Joints;
Joints.dJoint[0]=90;Joints.dJoint[1]=0;Joints.dJoint[2]=90;Joints.dJoint[3]=0;
Joints.dJoint[4]=90;Joints.dJoint[5]=0;Joints.dJoint[6]=0;
```

// 定义工具坐标变量

```
string sTcpName = "TCP";
```

// 定义用户坐标变量

```
string sUcsName = "Base";
```

// 定义运动速度

double dVelocity = 50;

// 定义运动加速度

double dAcc = 100;

// 定义过渡半径

double dRadius = 50;

// 定义是否使用关节角度

int nIsUseJoint = 1;

// 定义是否使用检测 DI 停止

int nIsSeek = 0;

// 定义检测的 DI 索引

int nIOBit = 0;

// 定义检测的 DI 状态

int nIOState = 0;

// 定义路点 ID

string strCmdID = "0";

// 执行路点运动

int nRet= HRIF_MoveJ_nJ(0,0, dX , dY , dZ , dRx , dRy , dRz ,Joints ,sTcpName , sUcsName, dVelocity, dAcc, dRadius,nIsUseJoint, nIsSeek, nIOBit, nIOState, strCmdID);

3.10.16. HRIF_MoveL

3.10.16.1.描述：直线轨迹运动。

✓ 输入变量

输入变量	名称	数据类型	有效范围	内容
boxID	电箱 ID	unsigned int	0~5	电箱 ID 号，默认值= 0
rbtID	机器人 ID	unsigned int	0~5	机器人 ID 号，默认值= 0
dX -dRz	空间目标位置	double	-	目标空间位置： dTcp_X: X 坐标，单位[mm] dTcp_Y: Y 坐标，单位[mm] dTcp_Z: Z 坐标，单位[mm] dTcp_Rx: Rx 坐标，单位[°] dTcp_Ry: Ry 坐标，单位[°] dTcp_Rz: Rz 坐标，单位[°]
dJ1 -dJ6	关节参考位置	double	-	关节参考位置，建议使用目标迪卡尔坐标附近的关节坐标值： dJ1: 关节 1 坐标，单位[°] dJ2: 关节 2 坐标，单位[°] dJ3: 关节 3 坐标，单位[°] dJ4: 关节 4 坐标，单位[°] dJ5: 关节 5 坐标，单位[°] dJ6: 关节 6 坐标，单位[°]
sTcpName	工具坐标名称	string	-	目标空间坐标所处的工具坐标系名称，与示教器页面的名称对应，可使用默认名称"TCP"
sUcsName	用户坐标名称	string	-	目标空间坐标所处的用户坐标系名称，与示教器页面的名称对应，可使用默认名称"Base"
dVelocity	速度	double	0.1<速度<加速度	运动最大速度，X, Y, Z 单位[mm/s], Rx, Ry, Rz 单位[°/s]
dAcc	加速度	double	>0.1	运动最大加速度，X, Y, Z 单位[mm/s ²], Rx,

				Ry, Rz 单位[°/s ²]
dRadius	过渡半径	double	-	过渡半径, 单位[mm]
nIsSeek	是否检测 DI 停止	int	0/1	如果 nIsSeek 为 1, 则开启检测 DI 停止, 路点运动过程中如果电箱的 nIOBit 位索引的 DI 的状态= nIOState 时, 机器人停止运动, 否则运动到目标点完成运动
nIOBit	检测的 DI 索引	int	0~7	电箱对应 DI 索引, nIsSeek=0 时无效
nIOState	检测的 DI 状态	int	0/1	检测的 DI 状态, , nIsSeek=0 时无效
strCmdID	命令 ID	string	-	当前路点 ID, 可以自定义, 也可以按顺序设置为"1", "2", "3"

✓ 返回值

返回值	名称	数据类型	有效范围	内容
nRet	返回值	int	>=0 的整型值	nRet = 0: 返回函数调用成功 nRet > 0: 返回调用失败的错误码

3.10.16.2.示例

// 定义空间目标位置

```
double dX = 420; double dY = 0; double dZ = 445;
double dRx = 180; double dRy = 0; double dRz = 180;
```

// 定义关节目标位置

```
double dJ1 = 0; double dJ2 = 0; double dJ3 = 90;
double dJ4 = 0; double dJ5 = 90; double dJ6 = 0;
```

// 定义工具坐标变量

```
string sTcpName = "TCP";
```

// 定义用户坐标变量

```
string sUcsName = "Base";
```

// 定义运动速度

```
double dVelocity = 50;
```

// 定义运动加速度

```
double dAcc = 100;
```

// 定义过渡半径

```
double dRadius = 50;
// 定义是否使用检测 DI 停止
int nIsSeek = 0;
// 定义检测的 DI 索引
int nIOBit = 0;
// 定义检测的 DI 状态
int nIOState = 0;
// 定义路点 ID
string strCmdID = "0";
// 执行路点运动
int nRet= HRIF_MoveL(0,0, dX , dY , dZ , dRx , dRy , dRz ,dJ1 , dJ2 , dJ3 , dJ4 , dJ5 , dJ6 ,sTcpName , sUcsName,
dVelocity, dAcc, dRadius,nIsSeek, nIOBit, nIOState, strCmdID );
```


3.10.17. HRIF_MoveL_nJ

3.10.17.1.描述：直线轨迹运动。**注意：这是扩展轴版本所用接口**

✓ 输入变量

输入变量	名称	数据类型	有效范围	内容
boxID	电箱 ID	unsigned int	0~5	电箱 ID 号，默认值= 0
rbtID	机器人 ID	unsigned int	0~5	机器人 ID 号，默认值= 0
dX -dRz	空间目标位置	double	-	目标空间位置： dTcp_X: X 坐标，单位[mm] dTcp_Y: Y 坐标，单位[mm] dTcp_Z: Z 坐标，单位[mm] dTcp_Rx: Rx 坐标，单位[°] dTcp_Ry: Ry 坐标，单位[°] dTcp_Rz: Rz 坐标，单位[°]
Joints	关节参考位置	JointsData	-	关节参考位置，建议使用目标迪卡尔坐标附近的关节坐标值： joints.dJoint[i]:关节 i 坐标，单位[°]
sTcpName	工具坐标名称	string	-	目标空间坐标所处的工具坐标系名称，与示教器页面的名称对应，可使用默认名称"TCP"
sUcsName	用户坐标名称	string	-	目标空间坐标所处的用户坐标系名称，与示教器页面的名称对应，可使用默认名称"Base"
dVelocity	速度	double	0.1<速度<加速度	运动最大速度，X, Y, Z 单位[mm/s], Rx, Ry, Rz 单位[°/s]
dAcc	加速度	double	>0.1	运动最大加速度，X, Y, Z 单位[mm/s²], Rx, Ry, Rz 单位[°/s²]
dRadius	过渡半径	double	-	过渡半径，单位[mm]
nIsSeek	是否检测 DI 停止	int	0/1	如果 nIsSeek 为 1，则开启检测 DI 停止，路点运动过程中如果电箱的 nIOBit 位索引的 DI 的状态= nIOState 时，机器人停止运动，否则

				运动到目标点完成运动
nIOBit	检测的 DI 索引	int	0~7	电箱对应 DI 索引, nIsSeek=0 时无效
nIOState	检测的 DI 状态	int	0/1	检测的 DI 状态, nIsSeek=0 时无效
strCmdID	命令 ID	string	-	当前路点 ID, 可以自定义, 也可以按顺序设置为"1", "2", "3"

✓ 返回值

返回值	名称	数据类型	有效范围	内容
nRet	返回值	int	>=0 的整型值	nRet = 0: 返回函数调用成功 nRet > 0: 返回调用失败的错误码

3.10.17.2.示例

// 定义空间目标位置

```
double dX = 420; double dY = 0; double dZ = 445;
double dRx = 180; double dRy = 0; double dRz = 180;
```

// 定义关节目标位置

```
JointsData Joints;
Joints.dJoint[0]=0;Joints.dJoint[1]=0;Joints.dJoint[2]=90;
Joints.dJoint[3]=0;Joints.dJoint[4]=90;Joints.dJoint[5]=0;
```

// 定义工具坐标变量

```
string sTcpName = "TCP";
```

// 定义用户坐标变量

```
string sUcsName = "Base";
```

// 定义运动速度

```
double dVelocity = 50;
```

// 定义运动加速度

```
double dAcc = 100;
```

// 定义过渡半径

```
double dRadius = 50;
```

// 定义是否使用检测 DI 停止

```
int nIsSeek = 0;
```

// 定义检测的 DI 索引

```
int nIOBit = 0;
```

```
// 定义检测的 DI 状态
```

```
int nIOState = 0;
```

```
// 定义路点 ID
```

```
string strCmdID = "0";
```

```
// 执行路点运动
```

```
int nRet = HRIF_MoveL_nJ(0,0, dX , dY , dZ , dRx , dRy , dRz ,Joints ,sTcpName , sUcsName, dVelocity, dAcc,  
dRadius,nIsSeek, nIOBit, nIOState, strCmdID );
```

3.10.18. HRIF_MoveLByLockAxis_nJ

3.10.18.1.描述：直线轨迹运动。**注意：这是扩展轴版本所用接口**

✓ 输入变量

输入变量	名称	数据类型	有效范围	内容
boxID	电箱 ID	unsigned int	0~5	电箱 ID 号，默认值= 0
rbtID	机器人 ID	unsigned int	0~5	机器人 ID 号，默认值= 0
dX -dRz	空间目标位置	double	-	目标空间位置： dTcp_X: X 坐标，单位[mm] dTcp_Y: Y 坐标，单位[mm] dTcp_Z: Z 坐标，单位[mm] dTcp_Rx: Rx 坐标，单位[°] dTcp_Ry: Ry 坐标，单位[°] dTcp_Rz: Rz 坐标，单位[°]
Joints	关节参考位置	JointsData	-	关节参考位置，建议使用目标迪卡尔坐标附近的关节坐标值： joints.dJoint[i]:关节 i 坐标，单位[°]
sTcpName	工具坐标名称	string	-	目标空间坐标所处的工具坐标系名称，与示教器页面的名称对应，可使用默认名称"TCP"
sUcsName	用户坐标名称	string	-	目标空间坐标所处的用户坐标系名称，与示教器页面的名称对应，可使用默认名称"Base"
dVelocity	速度	double	0.1<速度<加速度	运动最大速度，X, Y, Z 单位[mm/s], Rx, Ry, Rz 单位[°/s]
dAcc	加速度	double	>0.1	运动最大加速度，X, Y, Z 单位[mm/s²], Rx, Ry, Rz 单位[°/s²]
dRadius	过渡半径	double	-	过渡半径，单位[mm]
nIsSeek	是否检测 DI 停止	int	0/1	如果 nIsSeek 为 1，则开启检测 DI 停止，路点运动过程中如果电箱的 nIOBit 位索引的 DI 的状态= nIOState 时，机器人停止运动，否则

				运动到目标点完成运动
nIOBit	检测的 DI 索引	int	0~7	电箱对应 DI 索引, nIsSeek=0 时无效
nIOState	检测的 DI 状态	int	0/1	检测的 DI 状态, nIsSeek=0 时无效
strCmdID	命令 ID	string	-	当前路点 ID, 可以自定义, 也可以按顺序设置为"1", "2", "3"
lockedFlags	关节锁定标志	int	位掩码 (Bitmask)	通过比特位 (Bit) 表示需要锁定的关节, 位值为 1 时锁定对应轴, 0 时不锁定。 位索引定义: 每个比特位对应一个关节 (例如: Bit0 → J1, Bit1 → J2, ... Bit6 → J7, Bit7 → J8, 以此类推)。 锁定逻辑: 锁定后, 该关节在运动过程中保持当前位置不变, 仅未锁定的关节参与运动。 版本依赖: 当前支持的锁定轴需查阅通讯协议。 示例: 锁定 J7: 0x40 (二进制 01000000, Bit6=1) 锁定 J7 和 J8: 0xC0 (二进制 11000000, Bit6+Bit7=1)

✓ 返回值

返回值	名称	数据类型	有效范围	内容
nRet	返回值	int	>=0 的整型值	nRet = 0: 返回函数调用成功 nRet > 0: 返回调用失败的错误码

3.10.18.2. 示例

// 定义空间目标位置

```
double dX = 420; double dY = 0; double dZ = 445;
double dRx = 180; double dRy = 0; double dRz = 180;
```

// 定义关节目标位置

```
JointsData Joints;
Joints.dJoint[0]=0;Joints.dJoint[1]=0;Joints.dJoint[2]=90;
Joints.dJoint[3]=0;Joints.dJoint[4]=90;Joints.dJoint[5]=0;
```

// 定义工具坐标变量

```
string sTcpName = "TCP";
```

// 定义用户坐标变量

```
string sUcsName = "Base";
```

// 定义运动速度

```
double dVelocity = 50;
```

// 定义运动加速度

```
double dAcc = 100;
```

// 定义过渡半径

```
double dRadius = 50;
```

// 定义是否使用检测 DI 停止

```
int nIsSeek = 0;
```

// 定义检测的 DI 索引

```
int nIOBit = 0;
```

// 定义检测的 DI 状态

```
int nIOState = 0;
```

// 定义路点 ID

```
string strCmdID = "0";
```

// 定义检测的关节锁定标志

```
int lockedFlags = 0x40;
```

// 执行路点运动

```
int nRet = HRIF_MoveLByLockAxis_nJ(0,0, dX , dY , dZ , dRx , dRy , dRz ,Joints ,sTcpName , sUcsName,  
dVelocity, dAcc, dRadius,nIsSeek, nIOBit, nIOState, strCmdID,lockedFlags );
```

3.10.19. HRIF_MoveC

3.10.19.1.描述：圆弧轨迹运动。

✓ 输入变量

输入变量	名称	数据类型	有效范围	内容
boxID	电箱 ID	unsigned int	0~5	电箱 ID 号，默认值= 0
rbtID	机器人 ID	unsigned int	0~5	机器人 ID 号，默认值= 0
dStartPos_X-dStartPos_Rz	圆弧起始点位置	double	-	圆弧起始点位置： dStartPos_X: X 坐标，单位[mm] dStartPos_Y: Y 坐标，单位[mm] dStartPos_Z: Z 坐标，单位[mm] dStartPos_Rx: Rx 坐标，单位[°] dStartPos_Ry: Ry 坐标，单位[°] dStartPos_Rz: Rz 坐标，单位[°]
dAuxPos_X-dAuxPos_Rz	圆弧经过点位置	double	-	圆弧经过点位置： dAuxPos_X: X 坐标，单位[mm] dAuxPos_Y: Y 坐标，单位[mm] dAuxPos_Z: Z 坐标，单位[mm] dAuxPos_Rx: Rx 坐标，单位[°] dAuxPos_Ry: Ry 坐标，单位[°] dAuxPos_Rz: Rz 坐标，单位[°]
dEndPos_X-dEndPos_Rz	圆弧结束点位置	double	-	圆弧结束点位置，如果是用整圆时，结束点也是圆上的一个经过点，整圆跑完后才停止： dEndPos_X: X 坐标，单位[mm] dEndPos_Y: Y 坐标，单位[mm] dEndPos_Z: Z 坐标，单位[mm] dEndPos_Rx: Rx 坐标，单位[°] dEndPos_Ry: Ry 坐标，单位[°]

				dEndPos_Rz: Rz 坐标, 单位[°]
nFixedPosture	是否固定姿态	int	0/1/2	圆弧整个运动过程中是否保持姿态不变: 0: 渐变姿态圆弧运动, 从起始点姿态渐变到结束点姿态, 姿态角变换不能超过 180°, 且运动路径选择最短的姿态运动路径。 1: 固定姿态圆弧运动, 固定姿态切向角, 在运动过程中始终保持姿态与圆弧路径切线方向的相对关系。 2: 新增的渐变姿态圆弧运动, 从起始点姿态渐变到结束点姿态, 姿态角变换可以超过 180°, 且运动路径选择示教的姿态运动路径。
nMoveCType	圆弧类型	int	0/1	0: 整圆 1: 圆弧
dRadLen	弧长	double	-	当使用圆弧运动时无效, 通过三个点位确定圆弧路径, 当使用整圆运动时表示整圆的圈数, 小数部分无效。
dVelocity	速度	double	0.1<速度<加速度	运动最大速度, X, Y, Z 单位[mm/s], Rx, Ry, Rz 单位[°/s]
dAcc	加速度	double	>0.1	运动最大加速度, X, Y, Z 单位[mm/s²], Rx, Ry, Rz 单位[°/s²]
dRadius	过渡半径	double	-	过渡半径, 单位[mm]
sTcpName	工具坐标名称	string	-	目标空间坐标所处的工具坐标系名称, 与示教器页面的名称对应, 当 nIsUseJoint=1 时无效, 可使用默认名称"TCP"
sUcsName	用户坐标名称	string	-	目标空间坐标所处的用户坐标系名称, 与示教器页面的名称对应, 当 nIsUseJoint=1 时无效, 可使用默认名称"Base"
strCmdID	命令 ID	string	-	当前路点 ID, 可以自定义, 也可以按顺序设置为"1", "2", "3"

✓ 返回值

返回值	名称	数据类型	有效范围	内容
nRet	返回值	int	≥ 0 的整型值	nRet = 0: 返回函数调用成功 nRet > 0: 返回调用失败的错误码

3.10.19.2. 示例

// 圆弧起始点位置

```
double dStartPos_X = 420; double dStartPos_Y = 0; double dStartPos_Z = 445;
double dStartPos_Rx = 180; double dStartPos_Ry = 0; double dStartPos_Rz = 180;
```

// 圆弧经过点位置

```
double dAuxPos_X = 420; double dAuxPos_Y = 50; double dAuxPos_Z = 445;
double dAuxPos_Rx = 180; double dAuxPos_Ry = 0; double dAuxPos_Rz = 180;
```

// 圆弧结束点位置

```
double dEndPos_X = 470; double dEndPos_Y = 0; double dEndPos_Z = 445;
double dEndPos_Rx = 180; double dEndPos_Ry = 0; double dEndPos_Rz = 180;
```

// 是否固定姿态

```
int nFixedPosure = 0;
```

// 圆弧类型

```
int nMoveCType = 0;
```

// 整圆圈数

```
double dRadLen = 1;
```

// 定义运动速度

```
double dVelocity = 50;
```

// 定义运动加速度

```
double dAcc = 100;
```

// 定义过渡半径

```
double dRadius = 50;
```

// 定义工具坐标变量

```
string sTcpName = "TCP";
```

// 定义用户坐标变量

```
string sUcsName = "Base";
```

// 定义路点 ID

```
string strCmdID = "0";
```

// 执行路点运动

```
int nRet= HRIF_MoveC(0,0,dStartPos_X, dStartPos_Y, dStartPos_Z, dStartPos_Rx, dStartPos_Ry, dStartPos_Rz,  
dAuxPos_X, dAuxPos_Y, dAuxPos_Z, dAuxPos_Rx, dAuxPos_Ry, dAuxPos_Rz, dEndPos_X, dEndPos_Y,  
dEndPos_Z, dEndPos_Rx, dEndPos_Ry, dEndPos_Rz,  
nFixedPosure, nMoveCType, dRadLen, dVelocity, dAcc, dRadius,sTcpName , sUcsName, strCmdID );
```

3.10.20. HRIF_MoveC_nJ

3.10.20.1.描述：圆弧轨迹运动。**注意：这是扩展轴版本所用接口**

✓ 输入变量

输入变量	名称	数据类型	有效范围	内容
boxID	电箱 ID	unsigned int	0~5	电箱 ID 号，默认值= 0
rbtID	机器人 ID	unsigned int	0~5	机器人 ID 号，默认值= 0
dStartPos_X-dStartPos_Rz	圆弧起始点位置	double	-	圆弧起始点位置： dStartPos_X: X 坐标，单位[mm] dStartPos_Y: Y 坐标，单位[mm] dStartPos_Z: Z 坐标，单位[mm] dStartPos_Rx: Rx 坐标，单位[°] dStartPos_Ry: Ry 坐标，单位[°] dStartPos_Rz: Rz 坐标，单位[°]
dAuxPos_X-dAuxPos_Rz	圆弧经过点位置	double	-	圆弧经过点位置： dAuxPos_X: X 坐标，单位[mm] dAuxPos_Y: Y 坐标，单位[mm] dAuxPos_Z: Z 坐标，单位[mm] dAuxPos_Rx: Rx 坐标，单位[°] dAuxPos_Ry: Ry 坐标，单位[°] dAuxPos_Rz: Rz 坐标，单位[°]
dEndPos_X-dEndPos_Rz	圆弧结束点位置	double	-	圆弧结束点位置，如果是用整圆时，结束点也是圆上的一个经过点，整圆跑完后才停止： dEndPos_X: X 坐标，单位[mm] dEndPos_Y: Y 坐标，单位[mm] dEndPos_Z: Z 坐标，单位[mm] dEndPos_Rx: Rx 坐标，单位[°] dEndPos_Ry: Ry 坐标，单位[°]

				dEndPos_Rz: Rz 坐标, 单位[°]
nFixedPosture	是否固定姿态	int	0/1/2	圆弧整个运动过程中是否保持姿态不变: 0: 渐变姿态圆弧运动, 从起始点姿态渐变到结束点姿态, 姿态角变换不能超过 180°, 且运动路径选择最短的姿态运动路径。 1: 固定姿态圆弧运动, 固定姿态切向角, 在运动过程中始终保持姿态与圆弧路径切线方向的相对关系。 2: 新增的渐变姿态圆弧运动, 从起始点姿态渐变到结束点姿态, 姿态角变换可以超过 180°, 且运动路径选择示教的姿态运动路径。
nMoveCType	圆弧类型	int	0/1	0: 整圆 1: 圆弧
dRadLen	弧长	double	-	当使用圆弧运动时无效, 通过三个点位确定圆弧路径, 当使用整圆运动时表示整圆的圈数, 小数部分无效。
dVelocity	速度	double	0.1<速度<加速度	运动最大速度, X, Y, Z 单位[mm/s], Rx, Ry, Rz 单位[°/s]
dAcc	加速度	double	>0.1	运动最大加速度, X, Y, Z 单位[mm/s²], Rx, Ry, Rz 单位[°/s²]
dRadius	过渡半径	double	-	过渡半径, 单位[mm]
sTcpName	工具坐标名称	string	-	目标空间坐标所处的工具坐标系名称, 与示教器页面的名称对应, 当 nIsUseJoint= 1 时无效, 可使用默认名称"TCP"
sUcsName	用户坐标名称	string	-	目标空间坐标所处的用户坐标系名称, 与示教器页面的名称对应, 当 nIsUseJoint= 1 时无效, 可使用默认名称"Base"
strCmdID	命令 ID	string	-	当前路点 ID, 可以自定义, 也可以按顺序设置为"1", "2", "3"
Joints	结束点的参考关	JointsDa		结束点的参考关节角度, 用于判断联动轴是否需

	节角度	ta		要运动 joints.dJoint[i]:第 i 轴的角度, 单位[°]
nIsUseCurRefACS	是否使用参考的 关节角度	int	0/1	0: 使用结束点的参考关节角度 1: 扩展轴不运动

✓ 返回值

返回值	名称	数据类型	有效范围	内容
nRet	返回值	int	>=0 的整型值	nRet = 0: 返回函数调用成功 nRet > 0: 返回调用失败的错误码

3.10.20.2. 示例
// 圆弧起始点位置

```
double dStartPos_X = 420; double dStartPos_Y = 0; double dStartPos_Z = 445;
double dStartPos_Rx = 180; double dStartPos_Ry = 0; double dStartPos_Rz = 180;
```

// 圆弧经过点位置

```
double dAuxPos_X = 420; double dAuxPos_Y = 50; double dAuxPos_Z = 445;
double dAuxPos_Rx = 180; double dAuxPos_Ry = 0; double dAuxPos_Rz = 180;
```

// 圆弧结束点位置

```
double dEndPos_X = 470; double dEndPos_Y = 0; double dEndPos_Z = 445;
double dEndPos_Rx = 180; double dEndPos_Ry = 0; double dEndPos_Rz = 180;
```

// 是否固定姿态

```
int nFixedPosure = 0;
```

// 圆弧类型

```
int nMoveCType = 0;
```

// 整圆圈数

```
double dRadLen = 1;
```

// 定义运动速度

```
double dVelocity = 50;
```

// 定义运动加速度

```
double dAcc = 100;
```

// 定义过渡半径

```
double dRadius = 50;
```

// 定义工具坐标变量

```
string sTcpName = "TCP";
```

// 定义用户坐标变量

```
string sUcsName = "Base";
```

// 定义路点 ID

```
string strCmdID = "0";
```

// 使用结束点的参考关节角度

```
int nIsUseCurRefACS = 0;
```

// 执行路点运动

```
int nRet= HRIF_MoveC_nJ(0,0,dStartPos_X, dStartPos_Y, dStartPos_Z, dStartPos_Rx, dStartPos_Ry, dStartPos_Rz,  
dAuxPos_X, dAuxPos_Y, dAuxPos_Z, dAuxPos_Rx, dAuxPos_Ry, dAuxPos_Rz, dEndPos_X, dEndPos_Y,  
dEndPos_Z, dEndPos_Rx, dEndPos_Ry, dEndPos_Rz,  
nFixedPosure, nMoveCType, dRadLen, dVelocity, dAcc, dRadius,sTcpName , sUcsName, strCmdID  
Joints,nIsUseCurRefACS);
```

3.10.21. HRIF_MoveCByLockAxis_nJ

3.10.21.1.描述：圆弧轨迹运动。**注意：这是扩展轴版本所用接口**

✓ 输入变量

输入变量	名称	数据类型	有效范围	内容
boxID	电箱 ID	unsigned int	0~5	电箱 ID 号，默认值= 0
rbtID	机器人 ID	unsigned int	0~5	机器人 ID 号，默认值= 0
dStartPos_X-dStartPos_Rz	圆弧起始点位置	double	-	圆弧起始点位置： dStartPos_X: X 坐标，单位[mm] dStartPos_Y: Y 坐标，单位[mm] dStartPos_Z: Z 坐标，单位[mm] dStartPos_Rx: Rx 坐标，单位[°] dStartPos_Ry: Ry 坐标，单位[°] dStartPos_Rz: Rz 坐标，单位[°]
dAuxPos_X-dAuxPos_Rz	圆弧经过点位置	double	-	圆弧经过点位置： dAuxPos_X: X 坐标，单位[mm] dAuxPos_Y: Y 坐标，单位[mm] dAuxPos_Z: Z 坐标，单位[mm] dAuxPos_Rx: Rx 坐标，单位[°] dAuxPos_Ry: Ry 坐标，单位[°] dAuxPos_Rz: Rz 坐标，单位[°]
dEndPos_X-dEndPos_Rz	圆弧结束点位置	double	-	圆弧结束点位置，如果是用整圆时，结束点也是圆上的一个经过点，整圆跑完后才停止： dEndPos_X: X 坐标，单位[mm] dEndPos_Y: Y 坐标，单位[mm] dEndPos_Z: Z 坐标，单位[mm] dEndPos_Rx: Rx 坐标，单位[°] dEndPos_Ry: Ry 坐标，单位[°]

				dEndPos_Rz: Rz 坐标, 单位[°]
nFixedPosure	是否固定姿态	int	0/1	圆弧整个运动过程中是否保持姿态不变: 0: 不固定姿态 1: 固定姿态
nMoveCType	圆弧类型	int	0/1	0: 整圆 1: 圆弧
dRadLen	弧长	double	-	当使用圆弧运动时无效, 通过三个点位确定圆弧路径, 当使用整圆运动时表示整圆的圈数, 小数部分无效。
dVelocity	速度	double	0.1<速度<加速度	运动最大速度, X, Y, Z 单位[mm/s], Rx, Ry, Rz 单位[°/s]
dAcc	加速度	double	>0.1	运动最大加速度, X, Y, Z 单位[mm/s ²], Rx, Ry, Rz 单位[°/s ²]
dRadius	过渡半径	double	-	过渡半径, 单位[mm]
sTcpName	工具坐标名称	string	-	目标空间坐标所处的工具坐标系名称, 与示教器页面的名称对应, 当 nIsUseJoint= 1 时无效, 可使用默认名称"TCP"
sUcsName	用户坐标名称	string	-	目标空间坐标所处的用户坐标系名称, 与示教器页面的名称对应, 当 nIsUseJoint= 1 时无效, 可使用默认名称"Base"
strCmdID	命令 ID	string	-	当前路点 ID, 可以自定义, 也可以按顺序设置为"1", "2", "3"
Joints	结束点的参考关节角度	JointsData		结束点的参考关节角度, 用于判断联动轴是否需要运动 joints.dJoint[i]:第 i 轴的角度, 单位[°]
lockedFlags	关节锁定标志	int	位掩码 (Bitmask)	通过比特位 (Bit) 表示需要锁定的关节, 位值为 1 时锁定对应轴, 0 时不锁定。 位索引定义: 每个比特位对应一个关节 (例如: Bit0 → J1, Bit1 → J2, ... Bit6 → J7, Bit7 → J8, 以此类推)。

				锁定逻辑：锁定后，该关节在运动过程中保持当前位置不变，仅未锁定的关节参与运动。 版本依赖：当前支持的锁定轴需查阅通讯协议。 示例： 锁定 J7：0x40（二进制 01000000，Bit6=1） 锁定 J7 和 J8：0xC0（二进制 11000000，Bit6+Bit7=1）
nIsUseCurRefACS	是否使用参考的 关节角度	int	0/1	0：使用结束点的参考关节角度 1：扩展轴不运动

✓ 返回值

返回值	名称	数据类型	有效范围	内容
nRet	返回值	int	≥ 0 的整型值	nRet = 0: 返回函数调用成功 nRet > 0: 返回调用失败的错误码

3.10.21.2.示例

// 圆弧起始点位置

```
double dStartPos_X = 420; double dStartPos_Y = 0; double dStartPos_Z = 445;
double dStartPos_Rx = 180; double dStartPos_Ry = 0; double dStartPos_Rz = 180;
```

// 圆弧经过点位置

```
double dAuxPos_X = 420; double dAuxPos_Y = 50; double dAuxPos_Z = 445;
double dAuxPos_Rx = 180; double dAuxPos_Ry = 0; double dAuxPos_Rz = 180;
```

// 圆弧结束点位置

```
double dEndPos_X = 470; double dEndPos_Y = 0; double dEndPos_Z = 445;
double dEndPos_Rx = 180; double dEndPos_Ry = 0; double dEndPos_Rz = 180;
```

// 是否固定姿态

```
int nFixedPosure = 0;
```

// 圆弧类型

```
int nMoveCType = 0;
```

// 整圆圈数

```
double dRadLen = 1;
```

// 定义运动速度

```
double dVelocity = 50;
// 定义运动加速度

double dAcc = 100;
// 定义过渡半径

double dRadius = 50;
// 定义工具坐标变量

string sTcpName = "TCP";
// 定义用户坐标变量

string sUcsName = "Base";
// 定义路点 ID

string strCmdID = "0";
// 定义检测的关节锁定标志

int lockedFlags = 0x40;
// 使用结束点的参考关节角度

int nIsUseCurRefACS = 0;
// 执行路点运动

int nRet= HRIF_MoveCByLockAxis_nJ(0,0,dStartPos_X, dStartPos_Y, dStartPos_Z, dStartPos_Rx, dStartPos_Ry,
dStartPos_Rz, dAuxPos_X, dAuxPos_Y, dAuxPos_Z, dAuxPos_Rx, dAuxPos_Ry, dAuxPos_Rz, dEndPos_X,
dEndPos_Y, dEndPos_Z, dEndPos_Rx, dEndPos_Ry, dEndPos_Rz,
nFixedPosure, nMoveCType, dRadLen, dVelocity, dAcc, dRadius,sTcpName , sUcsName, strCmdID,
Joints,lockedFlags ,nIsUseCurRefACS);
```

3.10.22. HRIF_MoveZ

3.10.22.1.描述：Z 型轨迹运动。

✓ 输入变量

输入变量	名称	数据类型	有效范围	内容
boxID	电箱 ID	unsigned int	0~5	电箱 ID 号，默认值= 0
rbtID	机器人 ID	unsigned int	0~5	机器人 ID 号，默认值= 0
dStartPos_X- dStartPos_Rz	Z 型起始点位置	double	-	Z 型起始点位置： dStartPos_X: X 坐标，单位[mm] dStartPos_Y: Y 坐标，单位[mm] dStartPos_Z: Z 坐标，单位[mm] dStartPos_Rx: Rx 坐标，单位[°] dStartPos_Ry: Ry 坐标，单位[°] dStartPos_Rz: Rz 坐标，单位[°]
dEndPos_X- dEndPos_Rz	Z 型结束点位置	double	-	Z 型结束点位置： dEndPos_X: X 坐标，单位[mm] dEndPos_Y: Y 坐标，单位[mm] dEndPos_Z: Z 坐标，单位[mm] dEndPos_Rx: Rx 坐标，单位[°] dEndPos_Ry: Ry 坐标，单位[°] dEndPos_Rz: Rz 坐标，单位[°]
dPlanePos_X- dPlanePos_Rz	轨迹确定平面点位置	double	-	Z 型轨迹平面点位置： dPlanePosPos_X: X 坐标，单位[mm] dPlanePosPos_Y: Y 坐标，单位[mm] dPlanePosPos_Z: Z 坐标，单位[mm] dPlanePosPos_Rx: Rx 坐标，单位[°] dPlanePosPos_Ry: Ry 坐标，单位[°] dPlanePosPos_Rz: Rz 坐标，单位[°]
dVelocity	速度	double	-	运动最大速度，X, Y, Z 单位[mm/s], Rx, Ry, Rz 单位[°/s]

dAcc	加速度	double	-	运动最大加速度, X, Y, Z 单位[mm/s ²], Rx, Ry, Rz 单位[°/s ²]
dWidth	宽度	double	-	轨迹宽度
dDensity	密度	double	-	轨迹密度, 当不使用密度时根据速度自动计算密度
nEnableDensity	是否使用密度	int	-	是否使用密度: 0: 不使用 1: 使用
nEnablePlane	是否使用平面点	int	-	是否使用平面点, 不使用时根据选择的用户坐标确定 XYZ 平面: 0: 不使用 1: 使用
nEnableWaitTime	是否开启转折点等待时间	int	-	是否开启转折点等待时间: 0: 不使用 1: 使用
nPosiTime	正向转折点等待时间 ms	int	-	正向转折点等待时间, 单位[ms]
nNegaTime	负向转折点等待时间 ms	int	-	负向转折点等待时间, 单位[ms]
dRadius	过渡半径	double	-	过渡半径, 单位[mm]
sTcpName	工具坐标名称	string	-	目标空间坐标所处的工具坐标系名称, 与示教器页面的名称对应, 当 nIsUseJoint= 1 时无效, 可使用默认名称"TCP"
sUcsName	用户坐标名称	string	-	目标空间坐标所处的用户坐标系名称, 与示教器页面的名称对应, 当 nIsUseJoint= 1 时无效, 可使用默认名称"Base"
strCmdID	命令 ID	string	-	当前路点 ID, 可以自定义, 也可以按顺序设置为"1", "2", "3"

✓ 返回值

返回值	名称	数据类型	有效范围	内容
nRet	返回值	int	≥ 0 的整型值	nRet = 0: 返回函数调用成功 nRet > 0: 返回调用失败的错误码

3.10.22.2. 示例

// 起始点位置

```
double dStartPos_X = 420; double dStartPos_Y = 0; double dStartPos_Z = 445;
double dStartPos_Rx = 180; double dStartPos_Ry = 0; double dStartPos_Rz = 180;
```

// 结束点位置

```
double dEndPos_X = 420; double dEndPos_Y = 100; double dEndPos_Z = 445;
double dEndPos_Rx = 180; double dEndPos_Ry = 0; double dEndPos_Rz = 180;
```

// 确定轨迹平面点位置

```
double dPlanePos_X = 470; double dPlanePos_Y = 50; double dPlanePos_Z = 445;
double dPlanePos_Rx = 180; double dPlanePos_Ry = 0; double dPlanePos_Rz = 180;
```

// 定义运动速度

```
double dVelocity = 50;
```

// 定义运动加速度

```
double dAcc = 2500;
```

// 宽度

```
double dWidth = 50;
```

// 密度

```
double dDensity = 10;
```

// 使用密度

```
int nEnableDensity = 1;
```

// 使用平面点

```
int nEnablePlane = 1;
```

// 是否在转折点等待 不等待

```
int nEnableWaiTime = 0;
```

// 正向转折点等待时间

```
int nPosiTime = 0;
```

// 负向转折点等待时间

```
int nNegaTime = 0;
```

// 定义过渡半径

```
double dRadius = 5;
```

// 定义工具坐标变量

```
string sTcpName = "TCP";
```

// 定义用户坐标变量

```
string sUcsName = "Base";
```

// 定义路点 ID

```
string strCmdID = "0";
```

// 执行 Z 型轨迹运动

```
int nRet= HRIF_MoveZ(0,0,dStartPos_X, dStartPos_Y, dStartPos_Z, dStartPos_Rx, dStartPos_Ry, dStartPos_Rz,  
dEndPos_X, dEndPos_Y, dEndPos_Z, dEndPos_Rx, dEndPos_Ry, dEndPos_Rz ,dPlanePos_X, dPlanePos_Y,  
dPlanePos_Z, dPlanePos_Rx, dPlanePos_Ry, dPlanePos_Rz,dVelocity, dAcc, dWidth, dDensity, nEnableDensity,  
nEnablePlane, nEnableWaiTime,nPosiTime, nNegaTime, dRadius, sTcpName, sUcsName, strCmdID );
```

3.10.23. HRIF_MoveE

3.10.23.1.描述：椭圆型轨迹运动。

✓ 输入变量

输入变量	名称	数据类型	有效范围	内容
boxID	电箱 ID	unsigned int	0~5	电箱 ID 号, 默认值= 0
rbtID	机器人 ID	unsigned int	0~5	机器人 ID 号, 默认值= 0
dP1_X-dP1_Rz	示教位置 1	double	-	示教位置 1: dP1_X: X 坐标, 单位[mm] dP1_Y: Y 坐标, 单位[mm] dP1_Z: Z 坐标, 单位[mm] dP1_Rx: Rx 坐标, 单位[°] dP1_Ry: Ry 坐标, 单位[°] dP1_Rz: Rz 坐标, 单位[°]
dP2_X-dP2_Rz	示教位置 2	double	-	示教位置 2: dP2_X: X 坐标, 单位[mm] dP2_Y: Y 坐标, 单位[mm] dP2_Z: Z 坐标, 单位[mm] dP2_Rx: Rx 坐标, 单位[°] dP2_Ry: Ry 坐标, 单位[°] dP2_Rz: Rz 坐标, 单位[°]
dP3_X-dP1_Rz	示教位置 3	double	-	示教位置 3: dP3_X: X 坐标, 单位[mm] dP3_Y: Y 坐标, 单位[mm] dP3_Z: Z 坐标, 单位[mm] dP3_Rx: Rx 坐标, 单位[°] dP3_Ry: Ry 坐标, 单位[°] dP3_Rz: Rz 坐标, 单位[°]
dP4_X-dP4_Rz	示教位置 4	double	-	示教位置 4: dP4_X: X 坐标, 单位[mm]

				dP4_Y: Y 坐标, 单位[mm] dP4_Z: Z 坐标, 单位[mm] dP4_Rx: Rx 坐标, 单位[°] dP4_Ry: Ry 坐标, 单位[°] dP4_Rz: Rz 坐标, 单位[°]
dP5_X-dP5_Rz	示教位置 5	double	-	示教位置 5: dP5_X: X 坐标, 单位[mm] dP5_Y: Y 坐标, 单位[mm] dP5_Z: Z 坐标, 单位[mm] dP5_Rx: Rx 坐标, 单位[°] dP5_Ry: Ry 坐标, 单位[°] dP5_Rz: Rz 坐标, 单位[°]
nOrientMode	运动模式	int	0/1	0:不使用固定姿态 1:使用固定姿态
nMoveType	运动类型	int	0/1	0: 椭圆圆弧 1: 整个椭圆
dArcLength	弧长	double	-	弧长, 单位[°]
dVelocity	速度	double	-	运动最大速度, X, Y, Z 单位[mm/s], Rx, Ry, Rz 单位[°/s]
dAcc	加速度	double	-	运动最大加速度, X, Y, Z 单位[mm/s²], Rx, Ry, Rz 单位[°/s²]
dRadius	过渡半径	double	-	过渡半径, 单位[mm]
sTcpName	工具坐标名称	string	-	目标空间坐标所处的工具坐标系名称, 与示 教器页面的名称对应, 当 nIsUseJoint= 1 时无 效, 可使用默认名称"TCP"
sUcsName	用户坐标名称	string	-	目标空间坐标所处的用户坐标系名称, 与示 教器页面的名称对应, 当 nIsUseJoint= 1 时无 效, 可使用默认名称"Base"
strCmdID	命令 ID	string	-	当前路点 ID, 可以自定义, 也可以按顺序 设置为"1", "2", "3"

* 注：HR6.5 系列及以后版本（包括 HR6.5.x, HR6.6.x, HRC8.x, ...）移除此命令，不再支持。

✓ 返回值

返回值	名称	数据类型	有效范围	内容
nRet	返回值	int	≥ 0 的整型值	nRet = 0: 返回函数调用成功 nRet > 0: 返回调用失败的错误码

3.10.23.2. 示例

// 示教点 1

```
double dP1[6] = { 420,0,445,180,0,180 };
```

// 示教点 2

```
double dP2[6] = { 460,0,445,180,0,180 };
```

// 示教点 3

```
double dP3[6] = { 480,10,445,180,0,180 };
```

// 示教点 4

```
double dP4[6] = { 460,20,445,180,0,180 };
```

// 示教点 5

```
double dP5[6] = { 420,20,445,180,0,180 };
```

// 运动模式

```
int nOrientMode = 0;
```

// 运动类型

```
int nMoveType = 1;
```

// 弧长

```
double dArcLength = 360;
```

// 定义运动速度

```
double dVelocity = 50;
```

// 定义运动加速度

```
double dAcc = 2500;
```

// 定义过渡半径

```
double dRadius = 5;
```

// 定义工具坐标变量

```
string sTcpName = "TCP";
```

// 定义用户坐标变量

```
string sUcsName = "Base";
```

// 定义路点 ID

```
string strCmdID = "0";
```

// 执行椭圆运动

```
int nRet= HRIF_MoveE(0,0,dP1[0],dP1[1],dP1[2],dP1[3],dP1[4],dP1[5],  
dP2[0],dP2[1],dP2[2],dP2[3],dP2[4],dP2[5],dP3[0],dP3[1],dP3[2],dP3[3],dP3[4],dP3[5],dP4[0],dP4[1],dP4[2],dP4[3],  
dP4[4],dP4[5],dP5[0],dP5[1],dP5[2],dP5[3],dP5[4],dP5[5],nOrientMode,nMoveType,dArcLength,dVelocity,dAcc,d  
Radius, sTcpName, sUcsName, strCmdID);
```

3.10.24. HRIF_MoveS

3.10.24.1.描述：阿基米德螺旋线运动，初始半径为固定 1mm 。

✓ 输入变量

输入变量	名称	数据类型	有效范围	内容
boxID	电箱 ID	unsigned int	0~5	电箱 ID 号，默认值= 0
rbtID	机器人 ID	unsigned int	0~5	机器人 ID 号，默认值= 0
dSpiralIncrement	增量半径	double	>0	螺旋运动每圈增量半径，单位[mm]
dSpiralDiameter	结束半径	double	>1	螺旋运动结束半径，单位[mm]
dVelocity	速度	double	-	运动最大速度，单位[°/s]
dAcc	加速度	double	-	运动最大加速度，单位[°/s ²]
dRadius	过渡半径	double	-	过渡半径，单位[mm]
sTcpName	工具坐标名称	string	-	目标空间坐标所处的工具坐标系名称，与示教器页面的名称对应，当 nIsUseJoint= 1 时无效，可使用默认名称"TCP"
sUcsName	用户坐标名称	string	-	目标空间坐标所处的用户坐标系名称，与示教器页面的名称对应，当 nIsUseJoint= 1 时无效，可使用默认名称"Base"
strCmdID	命令 ID	string	-	当前路点 ID，可以自定义，也可以按顺序设置为"1", "2", "3"

✓ 返回值

返回值	名称	数据类型	有效范围	内容
nRet	返回值	int	>=0 的整型值	nRet = 0: 返回函数调用成功 nRet >0: 返回调用失败的错误码

3.10.24.2.示例

// 定义增量半径

```
double dSpiralIncrement = 1;
```

// 定义结束半径

```
double dSpiralDiameter = 5;
```

// 定义工具坐标变量

```
string sTcpName = "TCP";
```

// 定义用户坐标变量

```
string sUcsName = "Base";
```

// 定义运动速度

```
double dVelocity = 50;
```

// 定义运动加速度

```
double dAcc = 100;
```

// 定义过渡半径

```
double dRadius = 50;
```

// 定义路点 ID

```
string strCmdID = "0";
```

// 执行螺旋轨迹运动

```
int nRet= HRIF_MoveS(0,0,dSpiralIncrement, dSpiralDiameter,dVelocity,dAcc, dRadius, sTcpName, sUcsName,  
strCmdID );
```

3.10.25. HRIF_MoveLinearWeave

3.10.25.1. 描述：直线摆焊轨迹运动。

✓ 输入变量

参数	名称	数据类型	有效范围	内容
nRbtID	机器人 id	unsigned int	0~5	机器 ID 号, 默认值=0
dStartPos_X-dStartPos_Rz	起始点位置	double	-	直线摆焊起始点位置: dStartPos_X: X 坐标, 单位[mm] dStartPos_Y: Y 坐标, 单位[mm] dStartPos_Z: Z 坐标, 单位[mm] dStartPos_Rx: Rx 坐标, 单位[°] dStartPos_Ry: Ry 坐标, 单位[°] dStartPos_Rz: Rz 坐标, 单位[°]
dEndPos_X-dEndPos_Rz	结束点位置	double	-	直线摆焊结束点位置: dEndPos_X: X 坐标, 单位[mm] dEndPos_Y: Y 坐标, 单位[mm] dEndPos_Z: Z 坐标, 单位[mm] dEndPos_Rx: Rx 坐标, 单位[°] dEndPos_Ry: Ry 坐标, 单位[°] dEndPos_Rz: Rz 坐标, 单位[°]
dVelocity	速度	double	-	运动最大速度, X, Y, Z 单位[mm/s], Rx, Ry, Rz 单位[°/s]
dAcc	加速度	double	-	运动最大加速度, X, Y, Z 单位[mm/s ²], Rx, Ry, Rz 单位[°/s ²]
dRadius	过渡半径	double	-	过渡半径, 单位[mm]
amplitude	宽度	double	-	摆焊的幅值, 单位[mm]
intervalDistance	密度	double	-	摆焊的间距, 单位[mm]
weaveFrameType	摆焊方向	int	-	用于决定摆焊平面和摆焊方向的选择方式 0: 摆焊平面的+X 方向为从 StartPos 到 EndPos, +Z 方向 (即摆焊平面的法线方向)

				为跟刀具坐标系的+Z 相同的方向, +Y 方向由右手定则确定。摆焊启动时的运动方向跟+Y 方向同侧。 1: 摆焊平面的+X 方向为从 StartPos 到 EndPos, Z 方向跟刀具坐标系的 Z 方向平行, +Y 方向跟刀具坐标系的+Y 方向同侧。摆焊启动时的运动方向跟+Y 方向同侧。
elevation	仰角	double	-	摆焊的仰角, 即焊道平面绕焊道前进方向的旋转角, 单位[°]
azimuth	方向角	double	-	摆焊的方向角, 即绕摆焊平面法向量的旋转角, 单位[°]
centreRise	中心隆起量	double	-	摆焊的中心隆起量, 即摆焊中心处焊炬的隆起量, 单位[mm]
nEnableWaiTime	是否开启转折点等待时间	int	-	是否开启转折点等待时间: 0: 不使用 1: 使用
nPosiTime	正向转折点等待时间	int	-	正向转折点等待时间, 单位[ms]
nNegaTime	负向转折点等待时间	int	-	负向转折点等待时间, 单位[ms]
sTcpName	工具坐标名称	string	-	目标空间坐标所处的工具坐标系名称, 与示教器页面的名称对应, 可使用默认名称 "TCP"
sUcsName	用户坐标名称	string	-	目标空间坐标所处的用户坐标系名称, 与示教器页面的名称对应, 可使用默认名称 "Base"
strCmdID	命令 ID	string	-	当前路点 ID, 可以自定义, 也可以按顺序设置为 "1", "2", "3" .
vecLastPoint	结束点空间位置	vector<string>	-	摆焊结束点空间位置

✓ 返回值

返回值	名称	数据类型	有效范围	内容
ErrorCode	错误码	int	>=0 的整型值	返回调用失败的错误码

3.10.25.2. 示例

//起始点位置

```
double dStartPos_X=420; double dStartPos_Y=0; double dStartPos_Z=445;
double dStartPos_Rx=180; double dStartPos_Ry=0; double dStartPos_Rz=180;
```

//结束点位置

```
double dEndPos_X = 520; double dEndPos_Y = 100; double dEndPos_Z = 445;
double dEndPos_Rx = 180; double dEndPos_Ry = 0; double dEndPos_Rz = 180;
```

//速度

```
double dVelocity = 10;
```

//加速度

```
double dAcc = 2500;
```

//过渡半径

```
double dRadius = 5;
```

//宽度

```
double dAmplitude = 30;
```

//密度

```
double dIntervalDistanc = 10;
```

//摆焊方向

```
int nWeaveFrameType = 0;
```

//仰角

```
double dElevation = 10;
```

//方向角

```
double dAzimuth = 0;
```

//中心隆起量

```
double dCentreRise = 0;
```

//是否开启转折点等待时间

```
int nEnableWaiTime = 0;
```

//正向转折点等待时间

```
int nPosiTime = 0;
```

//负向转折点等待时间

```
int nNegaTime = 0;
```

//工具坐标名称

```
string sTcpName = "TCP";
```

//坐标系

```
string sUcsName = "Base";
```

//命令ID

```
string sCmdID = "0";
```

```
vector<string> vecLastPoint = {};
```

//调用成功返回0

```
int nRet = HRIF_MoveLinearWeave(0, 0, dStartPos_X, dStartPos_Y, dStartPos_Z, dStartPos_Rx, dStartPos_Ry, dStartPos_Rz, dEndPos_X, dEndPos_Y, dEndPos_Z, dEndPos_Rx, dEndPos_Ry, dEndPos_Rz, dVelocity, dAcc, dRadius, dAmplitude, dIntervalDistanc, nWeaveFrameType, dElevation, dAzimuth, dCentreRise, nEnableWaiTime, nPosiTime, nNegaTime, sTcpName, sUcsName, sCmdID, vecLastPoint);
```


3.10.26. HRIF_MoveCircularWeave

3.10.26.1. 描述：圆弧摆焊轨迹运动。圆弧在不同姿态的插补方式和摆焊平面的坐标系选择下，相同的圆弧轨迹会产生不同的摆焊效果。

✓ 输入变量

参数	名称	数据类型	有效范围	内容
nRbtID	机器人 id	unsigned int	0~5	机器 ID 号，默认值=0
dStartPos_X-dStartPos_Rz	起始点位置	double	-	圆弧摆焊起始点位置： dStartPos_X: X 坐标，单位[mm] dStartPos_Y: Y 坐标，单位[mm] dStartPos_Z: Z 坐标，单位[mm] dStartPos_Rx: Rx 坐标，单位[°] dStartPos_Ry: Ry 坐标，单位[°] dStartPos_Rz: Rz 坐标，单位[°]
dAuxPos_X-dAuxPos_Rz	经过点位置	double	-	圆弧摆焊经过点位置： dAuxPos_X: X 坐标，单位[mm] dAuxPos_Y: Y 坐标，单位[mm] dAuxPos_Z: Z 坐标，单位[mm] dAuxPos_Rx: Rx 坐标，单位[°] dAuxPos_Ry: Ry 坐标，单位[°] dAuxPos_Rz: Rz 坐标，单位[°]
dEndPos_X-dEndPos_Rz	结束点位置	double	-	圆弧摆焊结束点位置： dEndPos_X: X 坐标，单位[mm] dEndPos_Y: Y 坐标，单位[mm] dEndPos_Z: Z 坐标，单位[mm] dEndPos_Rx: Rx 坐标，单位[°] dEndPos_Ry: Ry 坐标，单位[°] dEndPos_Rz: Rz 坐标，单位[°]
dVelocity	速度	double	-	运动最大速度, X, Y, Z 单位[mm/s], Rx, Ry, Rz 单位[°/s]

dAcc	加速度	double	-	运动最大加速度, X, Y, Z 单位[mm/s ²], Rx, Ry, Rz 单位[°/s ²]
dRadius	过渡半径	double	-	过渡半径, 单位[mm]
orientMode	姿态插补	int	-	圆弧的姿态插补方式 0: 从示教的起始姿态渐变到结束姿态; 1: 由起始姿态决定切向角插补方式;
nMoveWhole	圆弧类型	int	-	0: 整圆 1: 圆弧
arcLengthInRadian	弧度	double	-	当使用圆弧运动时该参数无效, 通过计算三个点位来确定圆弧路径及弧度; 当使用整圆运动时表示整圆的圈数, 小数部分有效。
amplitude	宽度	double	-	摆焊的幅值, 单位[mm]
intervalDistance	密度	double	-	摆焊的间距, 单位[mm]
weaveFrameType	摆焊方向	int	-	用于决定摆焊平面和摆焊方向的选择方式 0: 摆焊平面的+X 方向为从 StartPos 到 EndPos, +Z 方向 (即摆焊平面的法线方向) 为跟刀具坐标系的+Z 相同的方向, +Y 方向由右手定则确定。摆焊启动时的运动方向跟+Y 方向同侧。 1: 摆焊平面的+X 方向为从 StartPos 到 EndPos, Z 方向跟刀具坐标系的 Z 方向平行, +Y 方向跟刀具坐标系的+Y 方向同侧。摆焊启动时的运动方向跟+Y 方向同侧。
elevation	仰角	double	-	摆焊的仰角, 即焊道平面绕焊道前进方向的旋转角, 单位[°]
azimuth	方向角	double	-	摆焊的方向角, 即绕摆焊平面法向量的旋转角, 单位[°]
centreRise	中心隆起量	double	-	摆焊的中心隆起量, 即摆焊中心处焊炬的隆起量, 单位[mm]
nEnableWaiTime	是否开启转	int	-	是否开启转折点等待时间:

	折点等待时间			0: 不使用 1: 使用
nPosiTime	正向转折点等待时间	int	-	正向转折点等待时间, 单位[ms]
nNegaTime	负向转折点等待时间	int	-	负向转折点等待时间, 单位[ms]
sTcpName	工具坐标名称	string	-	目标空间坐标所处的工具坐标系名称, 与示教器页面的名称对应, 可使用默认名称“TCP”
sUcsName	用户坐标名称	string	-	目标空间坐标所处的用户坐标系名称, 与示教器页面的名称对应, 可使用默认名称“Base”
strCmdID	命令 ID	string	-	当前路点 ID, 可以自定义, 也可以按顺序设置为 “1”, “2”, “3” .
vecLastPoint	结束点空间位置	vector<string>	-	摆焊结束点空间位置

✓ 返回值

返回值	名称	数据类型	有效范围	内容
ErrorCode	错误码	int	>=0 的整型值	返回调用失败的错误码

3.10.26.2. 示例

// 圆弧起始点位置

```
double dStartPos_X=420; double dStartPos_Y=0; double dStartPos_Z=445;
double dStartPos_Rx=180; double dStartPos_Ry=0; double dStartPos_Rz=180;
```

// 圆弧经过点位置

```
double dAuxPos_X=470; double dAuxPos_Y=50; double dAuxPos_Z = 445;
double dAuxPos_RX = 180; double dAuxPos_RY = 0; double dAuxPos_RZ = 180;
```

// 圆弧结束点位置

```
double dEndPos_X=520; double dEndPos_Y=0; double dEndPos_Z=445;
double dEndPos_Rx = 180; double dEndPos_Ry = 0; double dEndPos_Rz = 180;
```

// 定义运动速度

```
double dVelocity = 100;
```

// 定义运动加速度

```
double dAcc = 2500;  
// 定义过渡半径  
double dRadius = 5;  
//姿态插补  
int nOrientMode=0;  
//圆弧类型  
int nMoveWhole=0;  
//弧度  
double dMoveWholeLen=0;  
//宽度  
double dAmplitude=1;  
//密度  
double dIntervalDistance=30;  
//焊摆方向  
int nWeaveFrameType=10;  
//仰角  
double dElevation=0;  
//方向角  
double dAzimuth=0;  
//中心隆起量  
double dCentreRise=0;  
//是否开启转折点等待时间  
int nEnableWaiTime=0;  
//正向转折点等待时间  
int nPosiTime=0;  
//负向转折点等待时间  
int nNegaTime=0;  
//工具坐标名称  
string sTcpName="TCP";  
//用户坐标名称  
string sUcsName="Base";
```

//命令 ID

```
string sCmdID="0";
```

// 运行结束点位

```
vector<string> vecLastPoint = {};
```

//示例参数错误, 返回 20006

```
int nRet= HRIF_MoveCircularWeave(0, 0, dStartPos_X, dStartPos_Y, dStartPos_Z, dStartPos_Rx, dStartPos_Ry,
dStartPos_Rz, dAuxPos_X, dAuxPos_Y, dAuxPos_Z, dAuxPos_RX, dAuxPos_RY, dAuxPos_RZ, dEndPos_X,
dEndPos_Y, dEndPos_Z, dEndPos_Rx, dEndPos_Ry, dEndPos_Rz, dVelocity, dAcc, dRadius, nOrientMode,
nMoveWhole, dMoveWholeLen, dAmplitude, dIntervalDistance, nWeaveFrameType, dElevation, dAzimuth,
dCentreRise, nEnableWaiTime, nPosiTime, nNegaTime, sTcpName, sUcsName, sCmdID, vecLastPoint);
```

3.10.27. HRIF_ShortJogJ

3.10.27.1.描述: 关节短点动, 运动距离 2°, 最大速度不超过 10°/s。

✓ 输入变量

输入变量	名称	数据类型	有效范围	内容
boxID	电箱 ID	unsigned int	0~5	电箱 ID 号, 默认值= 0
rbtID	机器人 ID	unsigned int	0~5	机器人 ID 号, 默认值= 0
nAxisID	关节轴 ID	int	0~5	运动的目标轴 ID , 对应关节 J1-J6
nDerection	运动方向	int	0/1	0: 负方向 1: 正方向

✓ 返回值

返回值	名称	数据类型	有效范围	内容
nRet	返回值	int	>=0 的整型值	nRet = 0: 返回函数调用成功 nRet >0: 返回调用失败的错误码

3.10.27.2.示例

// 关节轴 ID

```
int nAxisID = 0;
```

// 运动方向

```
int nDerection = 0;
```

// 关节短点动

```
int nRet= HRIF_ShortJogJ(0,0, nAxisID , nDerection);
```

3.10.28. HRIF_ShortJogL

3.10.28.1.描述：空间坐标短点动，运动距离 2mm，最大速度不超过 50mm/s.

✓ 输入变量

输入变量	名称	数据类型	有效范围	内容
boxID	电箱 ID	unsigned int	0~5	电箱 ID 号，默认值= 0
rbtID	机器人 ID	unsigned int	0~5	机器人 ID 号，默认值= 0
nAxisID	坐标系轴 ID	int	0~5	运动的目标轴 ID，对应坐标 X-Rz
nDerection	方向	int	0/1	0: 负方向 1: 正方向

✓ 返回值

返回值	名称	数据类型	有效范围	内容
nRet	返回值	int	>=0 的整型值	nRet = 0: 返回函数调用成功 nRet >0: 返回调用失败的错误码

3.10.28.2.示例

// 坐标系轴 ID

int nAxisID = 0;

// 运动方向

int nDerection = 0;

// 空间短点动

int nRet= HRIF_ShortJogL(0,0, nAxisID , nDerection);

3.10.29. HRIF_LongJogJ

3.10.29.1.描述: 关节长点动, 最大运动速度不超过 10°/s.

✓ 输入变量

输入变量	名称	数据类型	有效范围	内容
boxID	电箱 ID	unsigned int	0~5	电箱 ID 号, 默认值= 0
rbtID	机器人 ID	unsigned int	0~5	机器人 ID 号, 默认值= 0
nAxisID	轴 ID	int	0~5	运动的目标轴 ID , 对应关节 J1-J6
nDerection	方向	int	0/1	0: 负方向 1: 正方向
nState	状态	int	0/1	0: 关闭 1: 开启

* 注. 须与 HRIF_LongMoveEvent 配合使用。

✓ 返回值

返回值	名称	数据类型	有效范围	内容
nRet	返回值	int	>=0 的整型值	nRet = 0: 返回函数调用成功 nRet >0: 返回调用失败的错误码

3.10.29.2.示例

// 坐标系轴 ID

int nAxisID = 0;

// 运动方向

int nDerection = 0;

// 设置状态

int nState = 1;

// 关节长点动

int nRet= HRIF_LongJogJ(0,0, nAxisID , nDerection, nState);

3.10.30. HRIF_LongJogL

3.10.30.1.描述：空间长点动，最大运动速度不超过 30mm/s.

✓ 输入变量

输入变量	名称	数据类型	有效范围	内容
boxID	电箱 ID	unsigned int	0~5	电箱 ID 号，默认值= 0
rbtID	机器人 ID	unsigned int	0~5	机器人 ID 号，默认值= 0
nAxisID	轴 ID	int	0~5	运动的目标轴 ID，对应空间坐标 X-Rz
nDerection	方向	int	0/1	0: 负方向 1: 正方向
nState	状态	int	0/1	0: 关闭 1: 开启

* 注. 须与 HRIF_LongMoveEvent 配合使用。

✓ 返回值

返回值	名称	数据类型	有效范围	内容
nRet	返回值	int	>=0 的整型值	nRet = 0: 返回函数调用成功 nRet >0: 返回调用失败的错误码

3.10.30.2.示例

// 坐标系轴 ID

int nAxisID = 0;

// 运动方向

int nDerection = 0;

// 设置状态

int nState = 1;

// 空间长点动

int nRet= HRIF_LongJogL(0,0, nAxisID , nDerection, nState);

3.10.31. HRIF_LongMoveEvent

3.10.31.1.描述: 长点动继续指令, 当开始长点动之后, 要按 200 毫秒或更短时间为时间周期发送一次该指令, 否则长点动会停止。

✓ 输入变量

输入变量	名称	数据类型	有效范围	内容
boxID	电箱 ID	unsigned int	0~5	电箱 ID 号, 默认值= 0
rbtID	机器人 ID	unsigned int	0~5	机器人 ID 号, 默认值= 0

✓ 返回值

返回值	名称	数据类型	有效范围	内容
nRet	返回值	int	>=0 的整型值	nRet = 0: 返回函数调用成功 nRet >0: 返回调用失败的错误码

3.10.31.2.示例

// 长点动继续指令

```
int nRet= HRIF_LongMoveEvent(0,0);
```

3.11. 连续轨迹运动类控制指令

3.11.1. HRIF_StartPushMovePathJ

3.11.1.1. 描述：初始化关节连续轨迹运动。

✓ 输入变量

输入变量	名称	数据类型	有效范围	内容
boxID	电箱 ID	unsigned int	0~5	电箱 ID 号, 默认值= 0
rbtID	机器人 ID	unsigned int	0~5	机器人 ID 号, 默认值= 0
sPathName	轨迹名称	string	-	轨迹名称
dSpeedRatio	轨迹运动速度比	double	0~1	轨迹运动速度比
dRadius	过渡半径	double	≥ 0	过渡半径,单位[mm]

* 注:

HR6.2.17 及以前版本, 只能用该指令初始化 MovePathJ 轨迹。

HR6.3.3 及以后版本, 推荐使用 HRIF_InitPathJ 以替代本指令。

✓ 返回值

返回值	名称	数据类型	有效范围	内容
nRet	返回值	int	≥ 0 的整型值	nRet = 0: 返回函数调用成功 nRet > 0: 返回调用失败的错误码

3.11.1.2. 示例

// 轨迹名称

```
string sPathName = "drag_01";
```

// 速度比

```
double dSpeedRatio = 0.5;
```

// 过渡半径

```
double dRadius = 2;
```

// 初始化关节连续轨迹运动

```
int nRet = HRIF_StartPushMovePathJ(0,0, sPathName, dSpeedRatio, dRadius);
```

3.11.2. HRIF_PushMovePathJ

3.11.2.1. 描述：下发关节运动轨迹点位。

✓ 输入变量

输入变量	名称	数据类型	有效范围	内容
boxID	电箱 ID	unsigned int	0~5	电箱 ID 号，默认值= 0
rbtID	机器人 ID	unsigned int	0~5	机器人 ID 号，默认值= 0
sPathName	轨迹名称	string	-	轨迹名称
dJ1 -dJ6	关节点位	double	-	目标关节位置： dJ1: 关节 1 位置，单位[°] dJ2: 关节 2 位置，单位[°] dJ3: 关节 3 位置，单位[°] dJ4: 关节 4 位置，单位[°] dJ5: 关节 5 位置，单位[°] dJ6: 关节 6 位置，单位[°]

* 注：

调用 HRIF_StartPushMovePath 后，可多次调用此函数，一般情况下点位数量需要大于 4。

HR6.2.17 及以前版本，只能用该指令下发关节运动轨迹点位。

HR6.3.3 及以后版本，推荐使用 HRIF_PushPathPoints 以替代本指令。

✓ 返回值

返回值	名称	数据类型	有效范围	内容
nRet	返回值	int	>=0 的整型值	nRet = 0: 返回函数调用成功 nRet > 0: 返回调用失败的错误码

3.11.2.2. 示例

// 轨迹名称

```
string sPathName = "drag_01";
```

// 目标关节位置

```
double dJ1 = 0; double dJ2 = 0; double dJ3 = 90;
```

```
double dJ4 = 0; double dJ5 = 90; double dJ6 = 0;
```

// 下发关节点位

```
int nRet= HRIF_PushMovePathJ (0,0,sPathName, dJ1 , dJ2 , dJ3 , dJ4 , dJ5 , dJ6 );
```

3.11.3. HRIF_EndPushMovePathJ

3.11.3.1. 描述：轨迹下发完成并开始计算轨迹。

✓ 输入变量

输入变量	名称	数据类型	有效范围	内容
boxID	电箱 ID	unsigned int	0~5	电箱 ID 号，默认值= 0
rbtID	机器人 ID	unsigned int	0~5	机器人 ID 号，默认值= 0
sPathName	轨迹名称	string	-	轨迹名称

* 注:

HR6.2.17 及以前版本，只能用该指令结束 MovePathJ 轨迹下发并开始计算轨迹。

HR6.3.3 及以后版本，推荐使用 HRIF_EndPushPathPoints 以替代本指令。

✓ 返回值

返回值	名称	数据类型	有效范围	内容
nRet	返回值	int	>=0 的整型值	nRet = 0: 返回函数调用成功 nRet >0: 返回调用失败的错误码

3.11.3.2. 示例

// 轨迹名称

```
string sPathName = "drag_01";
```

// 下发完成，开始计算轨迹

```
int nRet = HRIF_EndPushMovePathJ(0,0,sPathName);
```

3.11.4. HRIF_MovePathJ

3.11.4.1. 描述：以关节运动的方式运行指定的轨迹。

✓ 输入变量

输入变量	名称	数据类型	有效范围	内容
boxID	电箱 ID	unsigned int	0~5	电箱 ID 号，默认值= 0
rbtID	机器人 ID	unsigned int	0~5	机器人 ID 号，默认值= 0
sPathName	轨迹名称	string	-	轨迹名称，仅适用 MovePathJ

✓ 返回值

返回值	名称	数据类型	有效范围	内容
nRet	返回值	int	>=0 的整型值	nRet = 0: 返回函数调用成功 nRet > 0: 返回调用失败的错误码

3.11.4.2. 示例

// 轨迹名称

```
string sPathName = "drag_01";
```

// 运动轨迹

```
int nRet = HRIF_MovePathJ(0,0, sPathName);
```

3.11.5. HRIF_ReadMovePathJState

3.11.5.1. 描述：读取当前的轨迹状态。

✓ 输入变量

输入变量	名称	数据类型	有效范围	内容
boxID	电箱 ID	unsigned int	0~5	电箱 ID 号，默认值= 0
rbtID	机器人 ID	unsigned int	0~5	机器人 ID 号，默认值= 0
sPathName	轨迹名称	string	-	轨迹名称

* 注：

- HR6.3 以前版本，本指令仅用于读取 MovePathJ 的状态。
- HR6.3 及以后版本，本指令先读取 MovePathJ 的状态，如果 MovePathJ 没有指定名称的轨迹，则再读取同名轨迹 MovePathL 的状态。
- HR6.3 及以后版本，推荐使用 ReadPathState 以替代本指令。HR6.5 及以后版本，将无法使用该指令。

✓ 输出变量

输出变量	名称	数据类型	有效范围	内容
nState	轨迹状态	int	0~5	当前轨迹状态： 0: 轨迹未开始示教，调用 HRIF_StartPushMovePathJ 后切换到此状态 1: 轨迹示教中，调用 HRIF_PushMovePathJ 后切换到此状态 2: 轨迹计算中，调用 HRIF_EndPushMovePathJ 后切换到此状态 3: 轨迹计算完成，计算完成后切换到此状态，可以开始运动。 4: 结束示教 5: 计算错误，需要检查点位是否合理，然后修正后重新示教。

✓ 返回值

返回值	名称	数据类型	有效范围	内容
nRet	返回值	int	>=0 的整型值	nRet = 0: 返回函数调用成功 nRet >0: 返回调用失败的错误码

3.11.5.2. 示例

// 轨迹名称

```
string sPathName = "drag_01";  
// 读取到的当前轨迹状态  
  
int nState = 0;  
// 读取轨迹状态  
  
int nRet = HRIF_ReadMovePathJState(0,0,sPathName, nState);
```


3.11.6. HRIF_UpdateMovePathJName

3.11.6.1. 描述：更新指定轨迹的名称。

✓ 输入变量

输入变量	名称	数据类型	有效范围	内容
boxID	电箱 ID	unsigned int	0~5	电箱 ID 号，默认值= 0
rbtID	机器人 ID	unsigned int	0~5	机器人 ID 号，默认值= 0
sPathName	轨迹原名称	string	-	轨迹原名称
sPathNewName	更新的轨迹名称	string	-	更新的轨迹名称

* 注：

HR6.2.17 及以前版本，只能用该指令更新指定轨迹名称。

HR6.3.3 及以后版本，推荐使用 HRIF_UpdatePathName 以替代本指令。HR6.5 及以后版本，将无法使用该指令。

✓ 返回值

返回值	名称	数据类型	有效范围	内容
nRet	返回值	int	≥ 0 的整型值	nRet = 0: 返回函数调用成功 nRet > 0: 返回调用失败的错误码

3.11.6.2. 示例

// 轨迹名称

```
string sPathName = "drag_01";
```

// 更新的轨迹名称

```
string sTrackNewName = "drag_02";
```

// 重命名轨迹名称

```
int nRet = HRIF_UpdateMovePathJName(0,0, sPathName,sPathNewName);
```

3.11.7. HRIF_DelMovePathJ

3.11.7.1. 描述：删除指定轨迹。

✓ 输入变量

输入变量	名称	数据类型	有效范围	内容
boxID	电箱 ID	unsigned int	0~5	电箱 ID 号，默认值= 0
rbtID	机器人 ID	unsigned int	0~5	机器人 ID 号，默认值= 0
sPathName	轨迹名称	string	-	轨迹名称

* 注:

HR6.2.17 及以前版本，只能用该指令删除指定轨迹名称。

HR6.3.3 及以后版本，推荐使用 HRIF_DelPath 以替代本指令。HR6.5 及以后版本，将无法使用该指令。

✓ 返回值

返回值	名称	数据类型	有效范围	内容
nRet	返回值	int	>=0 的整型值	nRet = 0: 返回函数调用成功 nRet > 0: 返回调用失败的错误码

3.11.7.2. 示例

// 轨迹名称

```
string sPathName = "drag_03";
```

// 删除轨迹

```
int nRet = HRIF_DelMovePathJ(0,0, sPathName);
```

3.11.8. HRIF_ReadTrackProcess

3.11.8.1. 描述：读取当前的轨迹运动进度。

✓ 输入变量

输入变量	名称	数据类型	有效范围	内容
boxID	电箱 ID	unsigned int	0~5	电箱 ID 号，默认值= 0
rbtID	机器人 ID	unsigned int	0~5	机器人 ID 号，默认值= 0

* 注：此接口仅对 HRIF_MovePathL 有效，对 HRIF_MovePathJ 无效。建议读取频率保持在 20ms 上下。

✓ 输出变量

输出变量	名称	数据类型	有效范围	内容
dProcess	轨迹运行进度	double	0~1	当前轨迹运动进度，当前运动进度 (0-1), >0.999999 表示运动完成
nIndex	点位索引	int	>0 的整型值	当前轨迹运动到哪一个点索引

✓ 返回值

返回值	名称	数据类型	有效范围	内容
nRet	返回值	int	>=0 的整型值	nRet = 0: 返回函数调用成功 nRet > 0: 返回调用失败的错误码

3.11.8.2. 示例

// 轨迹运行进度

```
double dProcess = 0;
```

// 点位索引

```
int nIndex = 0;
```

// 读取轨迹状态

```
int nRet = HRIF_ReadTrackProcess(0,0,dProcess, nIndex);
```

3.11.9. HRIF_InitMovePathL

3.11.9.1. 描述：初始化空间轨迹运动。

✓ 输入变量

输入变量	名称	数据类型	有效范围	内容
boxID	电箱 ID	unsigned int	0~5	电箱 ID 号，默认值= 0
rbtID	机器人 ID	unsigned int	0~5	机器人 ID 号，默认值= 0
sPathName	轨迹名称	string	-	轨迹名称
dVelocity	轨迹运动速度	double	-	轨迹运动速度，单位[mm/s]
dAcc	轨迹运动加速度	double	-	轨迹运动加速度，单位[mm/s ²]
dJerk	轨迹运动加加速度	double	-	轨迹运动加加速度，单位[mm/s ³]
sUcsName	用户坐标名称	string	-	目标空间坐标所处的用户坐标系名称，与示教器页面的名称对应-可使用默认名称 "Base"
sTcpName	工具坐标名称	string	-	目标空间坐标所处的工具坐标系名称，与示教器页面的名称对应，可使用默认名称 "TCP"

* 注：

HR6.2.17 及以前版本，只能用该指令初始化 MovePathL 轨迹。

HR6.3.3 及以后版本，推荐使用 HRIF_InitPathL 以替代本指令。

✓ 返回值

返回值	名称	数据类型	有效范围	内容
nRet	返回值	int	>=0 的整型值	nRet = 0: 返回函数调用成功 nRet > 0: 返回调用失败的错误码

3.11.9.2. 示例

// 轨迹名称

```
string sPathName = "Path_01";
```

// 定义运动速度

```
double dVelocity = 100;
```

// 定义运动加速度

```
double dAcc = 500;
```

// 定义运动加加速度

double dJerk = 10000;

// 定义用户坐标

string sUcsName = "Base";

// 定义工具

string sTcpName = "TCP";

// 初始化空间连续轨迹运动

int nRet= HRIF_InitMovePathL(0,0, sPathName, dVelocity, dAcc, dJerk, sUcsName, sTcpName);

3.11.10. HRIF_PushMovePathL

3.11.10.1.描述：下发直线运动轨迹点位。

✓ 输入变量

输入变量	名称	数据类型	有效范围	内容
boxID	电箱 ID	unsigned int	0~5	电箱 ID 号，默认值=0
rbtID	机器人 ID	unsigned int	0~5	机器人 ID 号，默认值=0
sPathName	轨迹名称	string	-	轨迹名称
dX -dRz	空间点位	double	-	目标空间位置： dX: X 坐标, 单位[mm] dY: Y 坐标, 单位[mm] dZ: Z 坐标, 单位 [mm] dRx: Rx 坐标, 单位[°] dRy: R y 坐标, 单位[°] dRz: Rz 坐标, 单位[°]

* 注:

调用 HRIF_StartPushMovePath 后，可多次调用此函数，一般情况下点位数量需要大于 4。

HR6.2.17 及以前版本，只能用该指令下发直线运动轨迹点位。

HR6.3.3 及以后版本，推荐使用 HRIF_PushPathPoints 以替代本指令。

✓ 返回值

返回值	名称	数据类型	有效范围	内容
nRet	返回值	int	>=0 的整型值	nRet = 0: 返回函数调用成功 nRet >0: 返回调用失败的错误码

3.11.10.2.示例

// 轨迹名称

```
string sPathName = "Path_01";
```

// 定义空间目标位置

```
double dX = 420; double dY = 0; double dZ = 445;
```

```
double dRx = 180; double dRy = 0; double dRz = 180;
```

// 下发空间目标点位

```
int nRet = HRIF_PushMovePathL(0,0, sPathName, dX , dY , dZ , dRx , dRy , dRz );
```

3.11.11. HRIF_PushMovePaths

3.11.11.1.描述：批量下发轨迹点位，调用一次可下发多个点位数据。

✓ 输入变量

输入变量	名称	数据类型	有效范围	内容
boxID	电箱 ID	unsigned int	0~5	电箱 ID 号，默认值= 0
rbtID	机器人 ID	unsigned int	0~5	机器人 ID 号，默认值= 0
sPathName	轨迹名称	string	-	轨迹名称
nMoveType	点位类型	int	0/1	0：下发关节点位，与 HRIF_MovePathJ 共用 1：下发空间点位，与 HRIF_MovePathL 共用
nPointsSize	点位数量	int	-	点位数量，必须与 sPoints 里的数量一致
sPoints	点位数据	string	-	点位数据，每个数据以逗号分隔

* 注：

HR6.2.17 及以前版本，只能用该指令批量下发轨迹点位。

HR6.3.3 及以后版本，推荐使用 HRIF_PushPathPoints 以替代本指令。

✓ 返回值

返回值	名称	数据类型	有效范围	内容
nRet	返回值	int	≥ 0 的整型值	nRet = 0: 返回函数调用成功 nRet > 0: 返回调用失败的错误码

3.11.11.2.示例

// 轨迹名称

```
string sPathName = "Path_01";
```

// ~~运动类型-MovePathL~~

```
int nMoveType = 1;
```

// 点位数量

```
int nPointsSize = 6;
```

```
string sPoints = "420,0,445,180,0,180,420,10,445,180,0,180,420,20,445,180,0,180,  
420,30,445,180,0,180,420,40,445,180,0,180,420,50,445,180,0,180";
```

// 下发空间目标点位

```
int nRet = HRIF_PushMovePaths(0,0, sPathName, nMoveType, nPointsSize, sPoints );
```


// 轨迹名称

```
string sPathName = "drag_01";
```

// 运动类型-MovePathJ

```
int nMoveType = 0;
```

// 点位数量

```
int nPointsSize = 9;
```

```
string sPoints = "0,0,90,0,90,0, 10,20,90,0,90,0, 20,0,90,0,90,0,  
40,0,90,0,90,0, 60,0,90,0,90,0, 60,20,90,0,90,0";
```

// 下发关节点位

```
int nRet = HRIF_PushMovePaths(0,0, sPathName, nMoveType, nPointsSize, sPoints);
```

3.11.12. HRIF_MovePathL

3.11.12.1.描述：以直线运动的方式运行指定的轨迹。

✓ 输入变量

输入变量	名称	数据类型	有效范围	内容
boxID	电箱 ID	unsigned int	0~5	电箱 ID 号，默认值= 0
rbtID	机器人 ID	unsigned int	0~5	机器人 ID 号，默认值= 0
sPathName	轨迹名称	string	-	轨迹名称，仅适用 MovePathL

✓ 返回值

返回值	名称	数据类型	有效范围	内容
nRet	返回值	int	≥ 0 的整型值	nRet = 0: 返回函数调用成功 nRet > 0: 返回调用失败的错误码

3.11.12.2.示例

// 轨迹名称

```
string sPathName = "Path_01";
```

// 开始空间连续轨迹运动

```
int nRet = HRIF_MovePathL(0,0, sPathName);
```

3.11.13. HRIF_SetMovePathOverride

3.11.13.1.描述：设置轨迹运动速度比，轨迹运动中设置有效。

✓ 输入变量

输入变量	名称	数据类型	有效范围	内容
boxID	电箱 ID	unsigned int	0~5	电箱 ID 号，默认值= 0
rbtID	机器人 ID	unsigned int	0~5	机器人 ID 号，默认值= 0
MovePathOverride	设置速度比	Double	0.01~1	需要设置的速度比，仅对 MovePathL 有效

✓ 返回值

返回值	名称	数据类型	有效范围	内容
nRet	返回值	int	>=0 的整型值	nRet = 0: 返回函数调用成功 nRet >0: 返回调用失败的错误码

3.11.13.2.示例

// 速度比

```
double MovePathOverride = 0.5;
```

//设置 MovePath 速度比

```
int nRet = HRIF_SetMovePathOverride(0,0,MovePathOverride);
```

3.11.14. HRIF_InitPath

3.11.14.1.描述：初始化轨迹。

✓ 输入变量

输入变量	名称	数据类型	有效范围	内容
boxID	电箱 ID	unsigned int	0~5	电箱 ID 号，默认值= 0
rbtID	机器人 ID	unsigned int	0~5	机器人 ID 号，默认值= 0
nRawDataType	原始点位类型	int	0/1	0: 关节点位 1: 空间点位

sPathName	轨迹名称	string	-	轨迹名称，由下划线，英文字母或者数字组成，最长 29 个字符。
dSpeedRatio	轨迹运动速度比	double	0.01~1.00	轨迹运动速度比。计算 PathJ 时需要用到此参数。
dRadius	过渡半径	double	>0	过渡半径,单位[mm]。计算 PathJ 时需要用到此参数。
dVelocity	轨迹运动速度	double	-	轨迹运动速度，单位[mm/s]。计算 PathL 时需要用到此参数。当 nRawDataType 取值为 1 时，此参数必给；nRawDataType 取值为 0 时，此参数不需要，系统自动使用最大值计算 PathL。
dAcc	轨迹运动加速度	double	-	轨迹运动加速度，单位[mm/s ²]。计算 PathL 时需要用到此参数。当 nRawDataType 取值为 1 时，此参数必给；nRawDataType 取值为 0 时，此参数不需要，系统自动使用最大值计算 PathL。
dJerk	轨迹运动加加速度	double	-	轨迹运动加加速度，单位[mm/s ³]。计算 PathL 时需要用到此参数。当 nRawDataType 取值为 1 时，此参数必给；nRawDataType 取值为 0 时，此参数不需要，系统自动使用最大值计算 PathL。
sUcsName	用户坐标名称	string	-	目标空间坐标所处的用户坐标系名称，与示教器页面的名称对应-可使用默认名称“Base”。计算 PathL 时需要用到此参数。当 nRawDataType 取值为 1 时，此参数必给；nRawDataType 取值为 0 时，此参数不需要，系统自动使用“Base”计算 PathL。
sTcpName	工具坐标名称	string	-	目标空间坐标所处的工具坐标系名称，与示教器页面的名称对应，可使用默认名称“TCP”。计算 PathL 时需要用到此参数。当

				nRawDataType 取值为 1 时，此参数必给； nRawDataType 取值为 0 时，此参数不需要， 系统自动使用 “TCP” 计算 PathL。
--	--	--	--	---

*** 注：**

此指令执行时，系统会先检查同名轨迹是否已经存在，如果存在，系统会先将其删除。

轨迹新建成功后，其状态为“示教中”。

本指令可以替换 HRIF_StartPushMovePathJ 和 HRIF_InitMovePathL。

✓ 返回值

返回值	名称	数据类型	有效范围	内容
nRet	返回值	int	>=0 的整型值	nRet = 0: 返回函数调用成功 nRet >0: 返回调用失败的错误码

3.11.14.2.示例

// 原始点位类型

```
int nRawDataType = 1;
```

// 轨迹名称

```
string trajectName = "Path_01";
```

// 轨迹运动速度比

```
double dSpeedRatio = 0.3;
```

// 过渡半径

```
double dRadius = 20;
```

// 轨迹运动速度

```
double dVelocity = 100;
```

// 轨迹运动加速度

```
double dAcc = 500;
```

// 轨迹运动加加速度

```
double dJerk = 10000;
```

// 用户坐标名称

```
string sUcsName = "Base";
```

// 工具坐标名称

```
string sTcpName = "TCP";
```

// 初始化直线运动轨迹

```
int nRet = HRIF_InitPath(0,0,nRawDataType,trajectName,dSpeedRatio,dRadius,dVelocity,dAcc, dJerk, sUcsName,
sTcpName);
```

3.11.15. HRIF_PushPathPoints

3.11.15.1.描述：向轨迹中批量推送原始点位（可多次调用）。

✓ 输入变量

输入变量	名称	数据类型	有效范围	内容
boxID	电箱 ID	unsigned int	0~5	电箱 ID 号，默认值= 0
rbtID	机器人 ID	unsigned int	0~5	机器人 ID 号，默认值= 0
sPathName	轨迹名称	string	-	轨迹名称，适用 MovePathJ 和 MovePathL
sPoints	点位数据	string	-	点位数据，每个数据以逗号分隔。每个点位 6 个坐标值，因此此字段值个数必须是 6 的倍数。系统自动计算出点位个数。

* 注：

本指令可反复调用。

对于状态为示教完、计算完成、计算出错、未示教的轨迹，亦可使用本指令。

原始点位最多可以有 10000 个。如果将要推送的点位数量加上已有的点位数量超过 10000，则本批推送的点位全部不加入到位点队列。

使用本指令后，轨迹状态变为“示教中”。

本指令可以替换 HRIF_PushMovePaths。

✓ 返回值

返回值	名称	数据类型	有效范围	内容
nRet	返回值	int	≥ 0 的整型值	nRet = 0: 返回函数调用成功 nRet > 0: 返回调用失败的错误码

3.11.15.2.示例

// 轨迹名称

```
string trajectName = "Path_01";
```

// 点位数据

```
string sPoints = "420,0,445,180,0,180, 430,10,445,180,0,180, 440,50,445,180,0,180,
                520,100,445,180,0,180, 450,50,445,180,0,180, 430,200,445,180,0,180";
```

// 向轨迹中批量推送原始点位

```
int nRet = HRIF_PushPathPoints(0,0, trajectName,sPoints);
```

// 轨迹名称

```
string trajectName = "drag_01";
```

// 点位数据

```
string sPoints = "0,0,90,0,90,0, 10,20,90,0,90,0, 20,0,90,0,90,0,
                40,0,90,0,90,0, 60,0,90,0,90,0, 60,20,90,0,90,0";
```

// 向轨迹中批量推送原始点位

```
int nRet = HRIF_PushPathPoints(0,0, trajectName,sPoints);
```

3.11.16. HRIF_EndPushPathPoints

3.11.16.1.描述：结束向轨迹中推送点位，并开始计算轨迹。

✓ 输入变量

输入变量	名称	数据类型	有效范围	内容
boxID	电箱 ID	unsigned int	0~5	电箱 ID 号，默认值= 0
rbtID	机器人 ID	unsigned int	0~5	机器人 ID 号，默认值= 0
sPathName	轨迹名称	string	-	轨迹名称，适用 MovePathJ 和 MovePathL

✓ 返回值

返回值	名称	数据类型	有效范围	内容
nRet	返回值	int	>=0 的整型值	nRet = 0: 返回函数调用成功 nRet >0: 返回调用失败的错误码

3.11.16.2.示例

// 轨迹名称

```
string sPathName= "drag_01";
```

// 下发完成，开始计算轨迹

```
int nRet = HRIF_EndPushPathPoints(0,0, sPathName);
```

3.11.17. HRIF_DelPath

3.11.17.1.描述：删除指定名称的轨迹

✓ 输入变量

输入变量	名称	数据类型	有效范围	内容
boxID	电箱 ID	unsigned int	0~5	电箱 ID 号，默认值= 0
rbtID	机器人 ID	unsigned int	0~5	机器人 ID 号，默认值= 0
sPathName	轨迹名称	string	-	轨迹名称，适用 MovePathJ 和 MovePathL

*** 注：**

本指令可以替换 HRIF_DelMovePathJ。

正在计算目标点位的轨迹，或者正在进行导入后处理的轨迹不能删除。

✓ 返回值

返回值	名称	数据类型	有效范围	内容
nRet	返回值	int	>=0 的整型值	nRet = 0: 返回函数调用成功 nRet > 0: 返回调用失败的错误码

3.11.17.2.示例

// 轨迹名称

```
string sPathName = "drag_03";
```

// 删除轨迹

```
int nRet = HRIF_DelPath(0,0, sPathName);
```

3.11.18. HRIF_ReadPathList

3.11.18.1.描述：读取轨迹列表

✓ 输入变量

输入变量	名称	数据类型	有效范围	内容
boxID	电箱 ID	unsigned int	0~5	电箱 ID 号，默认值= 0
rbtID	机器人 ID	unsigned int	0~5	机器人 ID 号，默认值= 0

result	轨迹列表	string	-	读取到的所有轨迹名称列表
--------	------	--------	---	--------------

✓ 返回值

返回值	名称	数据类型	有效范围	内容
nRet	返回值	int	≥ 0 的整型值	nRet = 0: 返回函数调用成功 nRet > 0: 返回调用失败的错误码

3.11.18.2. 示例

// 轨迹列表

```
string result = "";
```

// 读取轨迹列表

```
int nRet = HRIF_ReadPathList(0,0, result);
```

返回结果为 string 类型数据，系统中所有的轨迹名称逐一列出。

3.11.19. HRIF_ReadPathInfo

3.11.19.1. 描述：读取指定名称轨迹的信息

✓ 输入变量

输入变量	名称	数据类型	有效范围	内容
boxID	电箱 ID	unsigned int	0~5	电箱 ID 号，默认值= 0
rbtID	机器人 ID	unsigned int	0~5	机器人 ID 号，默认值= 0
sPathName	轨迹名称	string	-	轨迹名称，适用 MovePathJ 和 MovePathL

✓ 输出变量

输出变量	名称	数据类型	有效范围	内容
result	轨迹信息	string	-	读取到的指定名称轨迹信息。 返回结果参数依次说明，如下： nRawDataType : 原始点位类型 0: 关节点位 1: 空间点位 stateJ : MovePathJ 的状态，8 种状态：未

				示教(0)、示教中(1)、示教完(4)、计算中(2)、 计算出错(5)、计算完成(3)、已导入(9)、导入 后处理(10) ErrorCodeJ: MovePathJ 的错误码 stateL: MovePathL 的状态, 同 stateJ ErrorCodeL: MovePathL 的错误码 dOverride: 轨迹运动速度比 dRadius: 过渡半径, 单位: mm dLinearVel: 轨迹运动速度, 单位: mm/s dLinearAcc: 轨迹运动加速度, 单位: mm/s² dLinearJerk: 轨迹运动加加速度, 单位: mm/s³ dUcs_X-dUcs_Rz: 用户坐标 dTcp_X-dTcp_Rz: 工具坐标 nRawPointCount: 原始点位个数 HeadPoint[]: 第一个原始点位坐标 (J1/J2/J3/J4/J5/J6 或 X/Y/Z/RX/RY/RZ)
--	--	--	--	---

✓ 返回值

返回值	名称	数据类型	有效范围	内容
nRet	返回值	int	≥ 0 的整型值	nRet = 0: 返回函数调用成功 nRet > 0: 返回调用失败的错误码

3.11.19.2.示例

// 轨迹名称

```
string sPathName = "Path_01";
```

// 轨迹信息

```
string result = "";
```

// 读取指定名称轨迹的信息

```
int nRet = HRIF_ReadPathInfo(0,0, sPathName, result);
```

3.11.20. HRIF_UpdatePathName

3.11.20.1.描述：更新轨迹名称。

✓ 输入变量

输入变量	名称	数据类型	有效范围	内容
boxID	电箱 ID	unsigned int	0~5	电箱 ID 号，默认值= 0
rbtID	机器人 ID	unsigned int	0~5	机器人 ID 号，默认值= 0
sPathName	轨迹原名称	string	-	轨迹原名称，适用 MovePathJ 和 MovePathL
sPathNewName	新轨迹名称	string	-	新轨迹名称，适用 MovePathJ 和 MovePathL

✓ 返回值

返回值	名称	数据类型	有效范围	内容
nRet	返回值	int	≥ 0 的整型值	nRet = 0: 返回函数调用成功 nRet > 0: 返回调用失败的错误码

3.11.20.2.示例

// 轨迹原名称

```
string sPathName = "drag_01";
```

// 新轨迹名称

```
string sPathNewName = "drag_02";
```

// 更新轨迹名称

```
int nRet = HRIF_UpdatePathName(0,0, sPathName, sPathNewName);
```

3.11.21. HRIF_ReadPathState

3.11.21.1.描述：读取当前轨迹状态。

✓ 输入变量

输入变量	名称	数据类型	有效范围	内容
		型		

boxID	电箱 ID	unsigned int	0~5	电箱 ID 号，默认值= 0
rbtID	机器人 ID	unsigned int	0~5	机器人 ID 号，默认值= 0
sPathName	轨迹名称	string	-	轨迹名称，适用 MovePathJ 和 MovePathL
nStateJ	MovePathJ 的状态	int	0/1/2/3/4/5/9/10	0: 轨迹尚未示教，示教中的轨迹结束示教后，如果没有原始点位，则进入到本状态； 1: 轨迹正在示教中，InitPath、PushPathPoints 后进入到本状态； 2: 轨迹正在目标点位计算中，系统检测到状态为示教结束后，自动进入到本状态，本状态持续时长视轨迹线路长短和速度比而定，可能会长达 10 分钟以上。轨迹处于本状态时，不能删除。 3: 轨迹目标点位计算完成，系统计算目标轨迹成功完成后自动进入到本状态。 4: 轨迹结束示教，EndPushPathPoints 后，进入到本状态，轨迹文件导入后处理结束后也会进入到本状态； 5: 轨迹目标点位计算错误，系统计算目标轨迹出错后自动进入到本状态； 9: 轨迹已导入，前端导入某个轨迹后进入到本状态； 10: 轨迹导入后处理中，系统检测到状态为已导入后，自动进入到本状态。轨迹处于本状态时，不能删除。
nErrorCodeJ	MovePathJ 的错误码	int	-	参见错误码总表。当 nStateJ 为 5 时，此参数有意义。
nStateL	MovePathL 的状态	int	0/1/2/3/4/5/9/10	同 nStateJ。其中，状态 2 的持续时长视轨迹线路长短和速度比而定，可能会长达数十秒。

nErrorCodeL	MovePathL 的错误码	int	-	参见错误码总表。当 nStateL 为 5 时，此参数有意义。
-------------	----------------	-----	---	---------------------------------

✓ 返回值

返回值	名称	数据类型	有效范围	内容
nRet	返回值	int	≥0 的整型值	nRet = 0: 返回函数调用成功 nRet > 0: 返回调用失败的错误码

3.11.21.2.示例

```
// 轨迹名称
string sPathName = "drag_01";

// MovePathJ 的状态
int nStateJ = 0;

// MovePathJ 的错误码
int nErrorCodeJ = 0;

// MovePathL 的状态
int nStateL = 0;

// MovePathL 的错误码
int nErrorCodeL = 0;

// 更新轨迹名称
int nRet = HRIF_ReadPathState(0,0, sPathName,nStateJ,nErrorCodeJ,nStateL,nErrorCodeL);
```

3.11.22. HRIF_MovePathJOL

3.11.22.1. 描述：启动在线实施规划的 MovePathJ

✓ 输入变量：

参数	名称	数据类型	有效范围	内容
nRbtID	机器人 id	unsigned int	0~5	机器 ID 号，默认值=0
dVel	关节速度	double	-	单位[°/s]
dAcc	关节加速度	double	-	单位[°/s^2]
dTol	过渡参数	double	>=0.1	单位[°]
JOLPointsData	JOL 运动点位	vector<MovePathJOLPointsData>	-	MovePathJOL 运动点位数据。 MovePathJOLPointsData 是一个结构体，里面包含了 MovePathJOL 运动点位所需设置的各种数据

✓ MovePathJOLPointsData 结构体成员变量定义：

参数	名称	数据类型	有效范围	内容
dJ1	关节目标位置	double	-	关节 1 坐标，单位[°]
dJ2	关节目标位置	double	-	关节 2 坐标，单位[°]
dJ3	关节目标位置	double	-	关节 3 坐标，单位[°]
dJ4	关节目标位置	double	-	关节 4 坐标，单位[°]
dJ5	关节目标位置	double	-	关节 5 坐标，单位[°]
dJ6	关节目标位置	double	-	关节 6 坐标，单位[°]
nIsSetIO	是否设置 IO	unsigned int	0,1	1:该点位运动结束后时要设置 IO; 0:该点位运动结束后时不设置 IO;
nEndDOMask	需要更改的 EndDO 按 bit 标识	int	-	EndDO0~EndDO7 对应到 nEndDOMask 的 bit0~bit7，如果某个 bit 是 1，则其对应的 EndDO 的值需要更改，否则无需更改。
nEndDOVal	各个需要更改的 EndDO 的目标状态	int	-	EndDO0~EndDO7 对应到 nEndDOVal 的 bit0~bit7。 0: 低电平 1: 高电平
nBoxDOMask	需要更改的 BoxDO 按 bit 标识	int	-	BoxDO0~BoxDO7 对应到 nBoxDOMask 的 bit0~bit7，如果某个 bit 是 1，则其对应的 BoxDO 的值需要更改，否则无需更改。
nBoxDOVal	各个需要更改的	int	-	BoxDO0~BoxDO7 对应到 nBoxDOVal 的

	BoxDO 的目标状态			bit0~bit7。 0: 低电平 1: 高电平
nBoxCOMask	需要更改的 BoxCO 按 bit 标识	int	-	BoxCO~BoxCO 对应到 nBoxCOMask 的 bit0~bit7, 如果某个 bit 是 1, 则其对应的 BoxCO 的值需要更改, 否则无需更改。
nBoxCOVal	各个需要更改的 BoxCO 的目标状态	int	-	BoxCO~BoxCO 对应到 nBoxCOVal 的 bit0~bit7。 0: 低电平 1: 高电平
nBoxAOCH0_Mask	BoxAOCH0 是否需要更改的标识	int	0/1	0: 无需更改 1: 需要更改
nBoxAOCH0_Mode	模式	int	1/2	1: 电压模式 2: 电流模式
nBoxAOCH1_Mask	BoxAOCH1 是否需要更改的标识	int	0/1	0: 无需更改 1: 需要更改
nBoxAOCH1_Mode	模式	int	1/2	1: 电压模式 2: 电流模式
dbBoxAOCH0_Val	对应模拟量值	double	0~20	电流模式: 4-20mA 电压模式: 0-10V
dbBoxAOCH1_Val	对应模拟量值	double	0~20	电流模式: 4-20mA 电压模式: 0-10V

* 注: 以上参数中, 从 dJ1 到 dBoxAOCh1_Val 依次为一个点的完整数据, 均为必选。一次下发的指令, 最少要包含 2 个点的数据, 最多可包含 5000 个点的数据。

✓ 返回值

返回值	名称	数据类型	有效范围	内容
nRet	错误码	int	≥0 的整型值	返回调用失败的错误码

3.11.22.2. 示例

//关节速度

int dVel=20;

//关节加速度

int dAcc=20;

//过滤参数

```
int dTol=20;
```

//定义一个"vector"容器, 存储自定义结构体类型 *MovePathJOLPointsData* 的对象。

```
vector<MovePathJOLPointsData> JOLPointsDatas;
```

//定义关节坐标位移量

```
int JointMoveDistance=2;
```

```
for (int i = 0; i < 100; i++) {
```

//定义 MovePathJOLPointsData 对象, 存储 MovePathJOL 运动点位的所需设置的各种数据

```
MovePathJOLPointsData JOLPointsData;
```

//关节 1 坐标位置

```
JOLPointsData.dJ1 =0+i*JointMoveDistance;
```

//关节 2 坐标位置

```
JOLPointsData.dJ2 = 0;
```

//关节 3 坐标位置

```
JOLPointsData.dJ3 = 90;
```

//关节 4 坐标位置

```
JOLPointsData.dJ4 = 0;
```

//关节 5 坐标位置

```
JOLPointsData.dJ5 = 90;
```

//关节 6 坐标位置

```
JOLPointsData.dJ6 = 0;
```

//是否设置 IO

```
JOLPointsData.nIsSetIO = 1;
```

//需要更改的 EndDO 按 bit 标识

```
JOLPointsData.nEndDOMask = 7;
```

//各个需要更改的 EndDO 的目标状态

```
JOLPointsData.nEndDOVal = 2;
```

//需要更改的 BoxDO 按 bit 标识

```
JOLPointsData.nBoxDOMask = 86;
```

//各个需要更改的 BoxDO 的目标状态

```
JOLPointsData.nBoxDOVal = 255;
```



```
//需要更改的BoxCO 按bit 标识
JOLPointsData.nBoxCOMask = 255;
//各个需要更改的BoxCO 的目标状态
JOLPointsData.nBoxCOVal = 169;
//BoxAOCH0 是否需要更改的标识
JOLPointsData.nBoxAOCH0_Mask = 1;
//模式
JOLPointsData.nBoxAOCH0_Mode = 2;
//BoxAOCH1 是否需要更改的标识
JOLPointsData.nBoxAOCH1_Mask = 1;
//模式
JOLPointsData.nBoxAOCH1_Mode = 1;
//对应模拟量值
JOLPointsData.dbBoxAOCH0_Val = 6.66;
//对应模拟量值
JOLPointsData.dbBoxAOCH1_Val = 9.99;
//将单次点位信息存储进 JOLPointsDatas 容器
JOLPointsDatas.push_back(JOLPointsData);
}
//启动在线实施规划的 MovePathJ
int nRet=HRIF_MovePathJOL(0,0,dVel, dAcc, dTol, JOLPointsDatas);
```

3.11.23. HRIF_GetMovePathJOLIndex

3.11.23.1. 描述：获取 MovePathJOL 运动当前的点位索引号及轨迹运动所有点总数。

✓ 输入变量

参数	名称	数据类型	有效范围	内容
boxID	电箱 ID	unsigned int	0~5	电箱 ID 号，默认值= 0
rbtID	机器人 ID	unsigned int	0~5	机器人 ID 号，默认值= 0
nCurIndex	点位索引号	int	-	当前轨迹运动到的点位索引
nIndexNum	点位总数	int	-	当前轨迹运动所有点位总数

✓ 返回值

返回值	名称	数据类型	有效范围	内容
nRet	错误码	int	>=0 的整型值	返回调用失败的错误码

3.11.23.2. 示例

// 定义当前轨迹运动到的点位索引

int nCurIndex=0;

// 定义当前轨迹运动所有点位总数

int nIndexNum=0;

// 获取 MovePathJOL 运动当前的点位索引号及轨迹运动所有点总数

int nRet=HRIF_GetMovePathJOLIndex(0,0,nCurIndex,nIndexNum);

3.11.24. HRIF_SetPathRefJoints

3.11.24.1. 描述：设置空间轨迹计算参考关节坐标。

✓ 输入变量

参数	名称	数据类型	有效范围	内容
boxID	电箱 ID	unsigned int	0~5	电箱 ID 号，默认值= 0
rbtID	机器人 ID	unsigned int	0~5	机器人 ID 号，默认值= 0
sPathName	空间轨迹名	string	-	要设置参考关节坐标的空间轨迹的名字
dJ1	参考关节位置	double	-	关节 1 坐标，单位[°]
dJ2	参考关节位置	double	-	关节 2 坐标，单位[°]
dJ3	参考关节位置	double	-	关节 3 坐标，单位[°]
dJ4	参考关节位置	double	-	关节 4 坐标，单位[°]
dJ5	参考关节位置	double	-	关节 5 坐标，单位[°]
dJ6	参考关节位置	double	-	关节 6 坐标，单位[°]

✓ 返回值

返回值	名称	数据类型	有效范围	内容
nRet	错误码	int	>=0 的整型值	返回调用失败的错误码

3.11.24.2. 示例

// 轨迹名称

```
string sPathName = "drag_01";
```

// 目标关节位置

```
double dJ1 = 0; double dJ2 = 0; double dJ3 = 90;
```

```
double dJ4 = 0; double dJ5 = 90; double dJ6 = 0;
```

// 设置空间轨迹的参考关节坐标

```
int nRet=HRIF_SetPathRefJoints(0,0,sPathName,dJ1,dJ2,dJ3,dJ4,dJ5,dJ6);
```

3.12. Servo 运动类控制指令

3.12.1. HRIF_StartServo

3.12.1.1. 描述：启动机器人在线控制（ServoJ 或 ServoP）时，设定位置固定更新的周期/刚度增益系数*和前瞻时间。

✓ 输入变量

输入变量	名称	数据类型	有效范围	内容
boxID	电箱 ID	unsigned int	0~5	电箱 ID 号，默认值= 0
rbtID	机器人 ID	unsigned int	0~5	机器人 ID 号，默认值= 0
dServoTimeOrStiffness	更新周期/刚度增益系数	double	>0	固定更新的时间周期，单位[s] /刚度增益系数
dLookaheadTime	前瞻时间	double	>0	前瞻时间，单位[s]

* 注：

- HRIF_StartServo/HRIF_PushServoJ/HRIF_PushServoP 为一套接口，与其他 Servo 指令不共用。
- HR6.5.19 以前版本 dServoTimeOrStiffness 仅表示更新周期。
- HR6.5.19 包含及以后版本 dServoTimeOrStiffness 参数同时存在更新周期和刚度增益系数两种含义，其具体为哪种含义由其取值所决定；当其取值位于为 0~1 时，其含义为固定更新的时间周期；当其取值大于等于 1 时，其含义为刚度增益系数，并且推荐值为 8000；推荐以刚度增益系数为参数进行操作。

✓ 返回值

返回值	名称	数据类型	有效范围	内容
nRet	返回值	int	>=0 的整型值	nRet = 0: 返回函数调用成功 nRet > 0: 返回调用失败的错误码

3.12.1.2. 示例

// 更新周期

```
double dServoTimeOrStiffness= 0.02;
```

// 前瞻时间

```
double dLookaheadTime = 0.2;
```

// 启动机器人在线控制

```
int nRet = HRIF_StartServo(0,0,dServoTimeOrStifiness, dLookaheadTime);
```

```
// 刚度增益系数
```

```
double dServoTimeOrStifiness= 8000;
```

```
// 前瞻时间
```

```
double dLookaheadTime = 0.05;
```

```
// 启动机器人在线控制
```

```
int nRet = HRIF_StartServo(0,0,dServoTimeOrStifiness, dLookaheadTime);
```

3.12.2. HRIF_PushServoJ

3.12.2.1. 描述：在线关节位置命令控制，以 StartServo 设定的固定更新时间发送关节位置，机器人将实时的跟踪关节位置指令。

✓ 输入变量

输入变量	名称	数据类型	有效范围	内容
boxID	电箱 ID	unsigned int	0~5	电箱 ID 号，默认值= 0
rbtID	机器人 ID	unsigned int	0~5	机器人 ID 号，默认值= 0
dJ1 -dJ6	关节点位	double	-	目标关节位置： dJ1: 关节 1 位置，单位[°] dJ2: 关节 2 位置，单位[°] dJ3: 关节 3 位置，单位[°] dJ4: 关节 4 位置，单位[°] dJ5: 关节 5 位置，单位[°] dJ6: 关节 6 位置，单位[°]

* 注：HRIF_StartServo/HRIF_PushServoJ/HRIF_PushServoP 为一套接口，与其他 Servo 指令不共用。HR6.5.19 包含及以上版本，如果 dServoTimeOrStiffness 参数为刚度增益系数时，则原则上两次下发点位时间不超过 100 个控制器循环周期，否则影响运动精度，甚至导致 servo 停止，具体时间视控制器而定。

✓ 返回值

返回值	名称	数据类型	有效范围	内容
nRet	返回值	int	>=0 的整型值	nRet = 0: 返回函数调用成功 nRet > 0: 返回调用失败的错误码

3.12.2.2. 示例

// 目标关节位置

```
double dJ1 = 0; double dJ2 = 0; double dJ3 = 0;
double dJ4 = 0; double dJ5 = 0; double dJ6 = 0;
```

// 在线关节位置命令控制

```
int nRet= HRIF_PushServoJ(0,0,dJ1,dJ2,dJ3,dJ4,dJ5,dJ6);
```

3.12.3. HRIF_PushServoP

3.12.3.1. 描述：在线末端 TCP 位置命令控制，以 StartServo 设定的固定更新时间发送 TCP 位置，机器人将实时的跟踪目标 TCP 位置逆运算转换后的关节位置指令。

✓ 输入变量

输入变量	名称	数据类型	有效范围	内容
boxID	电箱 ID	unsigned int	0~5	电箱 ID 号，默认值=0
rbtID	机器人 ID	unsigned int	0~5	机器人 ID 号，默认值=0
vecCoord	空间点位	vector<double>	-	目标空间位置： vecCoord[1]: X 坐标，单位[mm] vecCoord[2]: Y 坐标，单位[mm] vecCoord[3]: Z 坐标，单位[mm] vecCoord[4]: Rx 坐标，单位[°] vecCoord[5]: Ry 坐标，单位[°] vecCoord[6]: Rz 坐标，单位[°]
vecUcs	用户坐标	vector<double>	-	目标位置对应的用户坐标： vecUcs[1] : X 坐标，单位[mm] vecUcs[2] : Y 坐标，单位[mm] vecUcs[3]: Z 坐标，单位[mm] vecUcs[4]: Rx 坐标，单位[°] vecUcs[5]: Ry 坐标，单位[°] vecUcs[6]: Rz 坐标，单位[°]
vecTcp	工具坐标	vector<double>	-	目标位置对应的工具坐标： vecTcp[1] : X 坐标，单位[mm] vecTcp[2]: Y 坐标，单位[mm] vecTcp[3]: Z 坐标，单位[mm] vecTcp[4]: Rx 坐标，单位[°] vecTcp[5]: Ry 坐标，单位[°] vecTcp[6]: Rz 坐标，单位[°]

* 注：HRIF_StartServo/HRIF_PushServoJ/HRIF_PushServoP 为一套接口，与其他 Servo 指令不共用。HR6.5.19 包含及以

上版本，如果 dServoTimeOrStiffness 参数为刚度增益系数时，则原则上两次下发点位时间不超过 100 个控制器循环周期，否则影响运动精度，甚至导致 servo 停止，具体时间视控制器而定。

✓ 返回值

返回值	名称	数据类型	有效范围	内容
nRet	返回值	int	>=0 的整型值	nRet = 0: 返回函数调用成功 nRet > 0: 返回调用失败的错误码

3.12.3.2. 示例

// 空间目标位置

```
vector<double> vecCoord={420,1,443.5,180,0,180};
```

// 定义工具坐标变量

```
vector<double> vecUcs={0,0,0,0,0,0};
```

// 定义用户坐标变量

```
vector<double> vecTcp={0,0,0,0,0,0};
```

// 在线空间位置命令控制

```
int nRet = HRIF_PushServoP(0,0,vecCoord,vecUcs,vecTcp);
```

3.12.4. HRIF_SpeedJ

3.12.4.1.描述：在线关节运动速度伺服控制，以该指令中指定的各个关节的速度和加速度运动指定的时长。

✓ 输入变量：

参数	名称	数据类型	有效范围	内容
boxID	电箱 ID	unsigned int	0~5	电箱 ID 号，默认值= 0
rbtID	机器人 ID	unsigned int	0~5	机器人 ID 号，默认值= 0
dJ1CmdVel-dJ6CmdVel	关节速度	double	-	关节 1 到关节 6 的命令速度，单位[°/s]
dAcc	加速度	double	-	关节运动的加速度，单位[°/s^2]
dRunTime	运行时长	double	0.001~86400	运动最长执行时长，四舍五入后保留三位小数，单位[s]

* 注：

1. 关节速度随不同机型取值范围有所不同
2. 本指令启动的运动可以用 GrpStop 指令提前终止，可以被 GrpInterrupt/GrpContinue 指令暂停/继续。
3. 如果当前指令的最长执行时长还未到，又接收到新的成功执行的 SpeedJ 指令，则前一个 SpeedJ 指令的运动提前终止，并按新的 SpeedJ 指令的参数继续运动。
4. 该指令触发的运动结束前，执行其它路点运动指令会返回失败，执行 SpeedL 指令机器人会进入错误状态。
5. 该指令触发的运动速度不受 SetOverride 指令影响。

✓ 返回值

返回值	名称	数据类型	有效范围	内容
nRet	错误码	int	≥ 0 的整型值	返回调用失败的错误码

3.12.4.2. 示例

// 定义关节命令速度

```
double dJ1CmdVel=1;double dJ2CmdVel=2;double dJ3CmdVel=3;
double dJ4CmdVel=4;double dJ5CmdVel=5;double dJ6CmdVel=6;
```

// 定义加速度

```
double dAcc=10;
```

//定义运行时间

```
double dRunTime=2;
```

//启动在线关节运动速度伺服控制

```
int nRet = HRIF_SpeedJ ( 0, 0, dJ1CmdVel, dJ2CmdVel, dJ3CmdVel, dJ4CmdVel, dJ5CmdVel, dJ6CmdVel,
dAcc, dRunTime);
```

3.12.5. HRIF_SpeedL

3.12.5.1.描述：在线空间运动速度伺服控制，以该指令中指定的位姿各个坐标的速度和加速度运动指定的时长。

✓ 输入变量：

参数	名称	数据类型	有效范围	内容
boxID	电箱 ID	unsigned int	0~5	电箱 ID 号，默认值= 0
rbtID	机器人 ID	unsigned int	0~5	机器人 ID 号，默认值= 0
dXCmdVel-dZCmdVel	X/Y/Z 速度	double	-	X/Y/Z 坐标的命令速度，单位[mm/s]

dRxCmdVel-dRzCmdVel	Rx/Ry/Rz 速度	double		Rx/Ry/Rz 坐标的命令速度, 单位[°/s]
dLinearAcc	X/Y/Z 加速度	double	-	单位[mm/s^2]
dAngularAcc	Rx/Ry/Rz 加速度	double	-	单位[°/s^2]
dRunTime	运行时长	double	0.001~86400	运动最长执行时长, 四舍五入后保留三位小数, 单位[s]

* 注:

1. 速度随不同机型取值范围有所不同
2. 本指令启动的运动可以用 GrpStop 指令提前终止, 可以被 GrpInterrupt/GrpContinue 指令暂停/继续。
3. 如果当前指令的最长执行时长还未到, 又接收到新的成功执行的 SpeedL 指令, 则前一个 SpeedL 指令的运动会上提前终止, 并按新的 SpeedL 指令的参数继续运动。
4. 该指令触发的运动结束前, 执行其它路点运动指令会返回失败, 执行 SpeedJ 指令机器人会进入错误状态。
5. 该指令触发的运动速度不受 SetOverride 指令影响。
6. 该指令参数基于系统当前选定的 UCS 和 TCP。
7. 该指令不支持 TOOL 模式。

✓ 返回值

返回值	名称	数据类型	有效范围	内容
nRet	错误码	int	>=0 的整型值	返回调用失败的错误码

3.12.5.2. 示例

// 定义命令速度

```
double dXCmdVel=2; double dYCmdVel=2; double dZCmdVel=2;
double dRxCmdVel=2; double dRyCmdVel=2; double dRzCmdVel=2;
```

//定义线加速度

```
double dLinearAcc=2;
```

//定义角加速度

```
double dAngularAcc=2;
```

//定义运行时间

```
double dRuntime=2;
```

//启动在线空间运动速度伺服控制

```
int nRet = HRIF_SpeedL (0, 0, dXCmdVel, dYCmdVel, dZCmdVel, dRxCmdVel, dRyCmdVel, dRzCmdVel,
dLinearAcc, dAngularAcc, dRuntime);
```

3.13. 相对跟踪运动类控制指令

3.13.1. HRIF_SetMoveTraceParams

3.13.1.1. 描述：设置相对跟踪运动控制参数。

✓ 输入变量

输入变量	名称	数据类型	有效范围	内容
boxID	电箱 ID	unsigned int	0~5	电箱 ID 号，默认值= 0
rbtID	机器人 ID	unsigned int	0~5	机器人 ID 号，默认值= 0
nState	跟踪状态	int	0/1	0：关闭相对跟踪运动 1：开启相对跟踪运动
dDistance	相对跟踪运动保持的相对距离	double	>0	相对跟踪运动保持的相对距离
dAwayVelocity	相对跟踪的运动的远离探寻速度	double	>0	相对跟踪的运动的远离探寻速度
dBackVelocity	相对跟踪的运动的靠近探寻速度	double	>0	相对跟踪的运动的靠近探寻速度

✓ 返回值

返回值	名称	数据类型	有效范围	内容
nRet	返回值	int	≥ 0 的整型值	nRet = 0: 返回函数调用成功 nRet > 0: 返回调用失败的错误码

3.13.1.2. 示例

// 设置跟踪状态和保持的相对距离

```
double nState = 1 ;
```

```
double dDistance = 100 ;
```

// 相对跟踪的运动的探寻速度

```
double dAwayVelocity = 50 ;
```

```
double dBackVelocity = 50 ;
```

// 设置相对跟踪运动控制参数并开启相对跟踪运动

```
int nRet= HRIF_SetMoveTraceParams(0,0,nState, dDistance,dAwayVelocity,dBackVelocity);
```

3.13.2. HRIF_SetMoveTraceInitParams

3.13.2.1. 描述：设置相对跟踪运动初始化参数。

✓ 输入变量

输入变量	名称	数据类型	有效范围	内容
boxID	电箱 ID	unsigned int	0~5	电箱 ID 号，默认值= 0
rbtID	机器人 ID	unsigned int	0~5	机器人 ID 号，默认值= 0
dK	传感器计算参数	double	-	计算公式 $y = dK * x + dB$
dB	传感器计算参数	double	-	计算公式 $y = dK * x + dB$
dMaxLimit	激光传感器检测 距离最大值	double	-	激光传感器检测距离最大值
dMinLimit	激光传感器检测 距离最小值	double	-	激光传感器检测距离最小值

✓ 返回值

返回值	名称	数据类型	有效范围	内容
nRet	返回值	int	≥ 0 的整型值	nRet = 0: 返回函数调用成功 nRet > 0: 返回调用失败的错误码

3.13.2.2. 示例

// 传感器计算参数

double dK= -14 ;

// 传感器计算参数

double dB= 135 ;

// 激光传感器检测距离最大值

double dMaxLimit = 130;

// 激光传感器检测距离最小值

double dMinLimit = 65;

// 设置跟踪状态初始化参数

int nRet= HRIF_SetMoveTraceInitParams(0,0, dK, dB, dMaxLimit, dMinLimit);

3.13.3. HRIF_SetMoveTraceUcs

3.13.3.1. 描述：设置相对跟踪运动的跟踪探寻方向。

✓ 输入变量

输入变量	名称	数据类型	有效范围	内容
boxID	电箱 ID	unsigned int	0~5	电箱 ID 号，默认值= 0
rbtID	机器人 ID	unsigned int	0~5	机器人 ID 号，默认值= 0
dX - dRz	跟踪探寻方向	double	>-180,<180	目标空间位置： dX：无效，可设置为 0 dY：无效，可设置为 0 dZ：无效，可设置为 0 dRx：Rx 方向，单位[°] dRy：Ry 方向，单位[°] dRz：Rz 方向，单位[°]

✓ 返回值

返回值	名称	数据类型	有效范围	内容
nRet	返回值	int	>=0 的整型值	nRet = 0: 返回函数调用成功 nRet > 0: 返回调用失败的错误码

3.13.3.2. 示例

// 设置跟踪方向

```
double dX = 420; double dY = 0; double dZ = 445;
```

```
double dRx = 180; double dRy = 0; double dRz = 180;
```

// 设置跟踪方向

```
int nRet= HRIF_SetMoveTraceUcs(0,0,dX , dY , dZ , dRx , dRy , dRz );
```

3.13.4. HRIF_SetTrackingState

3.13.4.1. 描述：设置传送带跟踪运动状态。

✓ 输入变量

输入变量	名称	数据类型	有效范围	内容
boxID	电箱 ID	unsigned int	0~5	电箱 ID 号，默认值= 0
rbtID	机器人 ID	unsigned int	0~5	机器人 ID 号，默认值= 0
nState	跟踪状态	int	0/1	0：关闭传送带跟踪运动 1：开启传送带跟踪运动

✓ 返回值

返回值	名称	数据类型	有效范围	内容
nRet	返回值	int	≥ 0 的整型值	nRet = 0: 返回函数调用成功 nRet > 0: 返回调用失败的错误码

3.13.4.2. 示例

// 设置传送带跟踪开启

```
int nState = 1;
```

// 开启传送带跟踪

```
int nRet = HRIF_SetTrackingState(0,0,nState);
```

3.14. 位置跟随运动类指令

3.14.1. HRIF_SetPoseTrackingMaxMotionLimit

3.14.1.1. 描述：设置位置跟随的最大跟随速度。

✓ 输入变量

输入变量	名称	数据类型	有效范围	内容
boxID	电箱 ID	unsigned int	0~5	电箱 ID 号，默认值= 0
rbtID	机器人 ID	unsigned int	0~5	机器人 ID 号，默认值= 0
dMaxLineVel	位置跟随直线最大跟随速度	double	0-3000	位置跟随直线最大跟随速度，单位[mm/s]
dMaxOriVel	位置跟随姿态最大跟随速度	double	>0	位置跟随姿态最大跟随速度，单位[°/s]

✓ 返回值

返回值	名称	数据类型	有效范围	内容
nRet	返回值	int	>=0 的整型值	nRet = 0: 返回函数调用成功 nRet > 0: 返回调用失败的错误码

3.14.1.2. 示例

// 设置最大跟随速度

```
double dMaxLineVel = 10;
```

```
double dMaxOriVel = 10;
```

```
int nRet = HRIF_SetPoseTrackingMaxMotionLimit (0,0, dMaxLineVel, dMaxOriVel);
```

3.14.2.HRIF_SetPoseTrackingStopTimeOut

3.14.2.1. 描述：设置位置跟踪超时停止时间。

✓ 输入变量

输入变量	名称	数据类型	有效范围	内容
boxID	电箱 ID	unsigned int	0~5	电箱 ID 号，默认值= 0
rbtID	机器人 ID	unsigned int	0~5	机器人 ID 号，默认值= 0
dTime	超时时间	double	>=0	位置跟随超时停止时间，单位：秒[s]

✓ 返回值

返回值	名称	数据类型	有效范围	内容
nRet	返回值	int	>=0 的整型值	nRet = 0: 返回函数调用成功 nRet >0: 返回调用失败的错误码

3.14.2.2. 示例

// 超时时间

```
double dTime = 10;
```

// 设置位置跟踪超时停止时间

```
int nRet = HRIF_SetPoseTrackingStopTimeOut(0,0,dTime);
```


3.14.3. HRIF_SetPoseTrackingPIDParams

3.14.3.1. 描述：设置位置跟随 PID 参数。

✓ 输入变量

输入变量	名称	数据类型	有效范围	内容
boxID	电箱 ID	unsigned int	0~5	电箱 ID 号, 默认值= 0
rbtID	机器人 ID	unsigned int	0~5	机器人 ID 号, 默认值= 0
dPosPID1-dPosPID3	位置跟随 PID	double	>0	位置跟随 PID
dOriPID1-dOriPID3	姿态跟随 PID	double	>0	姿态跟随 PID

✓ 返回值

返回值	名称	数据类型	有效范围	内容
nRet	返回值	int	>=0 的整型值	nRet = 0: 返回函数调用成功 nRet >0: 返回调用失败的错误码

3.14.3.2. 示例

// 设置 PID 参数

```
double dPosPID1 = 1;double dPosPID2 = 2;double dPosPID3 = 3;
double dOriPID1 = 1;double dOriPID2 = 2;double dOriPID3 = 3;
int nRet= HRIF_SetPoseTrackingPIDParams (0,0,double dPosPID1,double dPosPID2,double dPosPID3,
double dOriPID1,double dOriPID2,double dOriPID3);
```

3.14.4. HRIF_SetPoseTrackingTargetPos

3.14.4.1. 描述：设置位置跟随的目标位置。

✓ 输入变量

输入变量	名称	数据类型	有效范围	内容
boxID	电箱 ID	unsigned int	0~5	电箱 ID 号，默认值= 0
rbtID	机器人 ID	unsigned int	0~5	机器人 ID 号，默认值= 0
dX- dRz	位置跟随目标位置	double	-	位置跟随目标位置

✓ 返回值

返回值	名称	数据类型	有效范围	内容
nRet	返回值	int	>=0 的整型值	nRet = 0: 返回函数调用成功 nRet >0: 返回调用失败的错误码

3.14.4.2. 示例

// 设置PID 参数

```
double dX = 0;double dY = 0;double dZ = 0;
```

```
double dRx = 0;double dRy = 0;double dRz = 0;
```

```
int nRet= HRIF_SetPoseTrackingTargetPos (0, 0, dX, dY, dZ, dRx, dRy, dRz, nRet);
```

3.14.5. HRIF_SetPoseTrackingState

3.14.5.1. 描述：设置位置跟随状态

✓ 输入变量

输入变量	名称	数据类型	有效范围	内容
boxID	电箱 ID	unsigned int	0~5	电箱 ID 号，默认值= 0
rbtID	机器人 ID	unsigned int	0~5	机器人 ID 号，默认值= 0
nState	位置跟随状态	int	-	位置跟随状态

✓ 返回值

返回值	名称	数据类型	有效范围	内容
nRet	返回值	int	≥ 0 的整型值	nRet = 0: 返回函数调用成功 nRet > 0: 返回调用失败的错误码

3.14.5.2. 示例

// 设置PID 参数

```
int nState = 1;
```

```
int nRet= HRIF_SetPoseTrackingState (0,0,nState);
```

3.14.6. HRIF_SetUpdateTrackingPose

3.14.6.1. 描述：设置实时更新传感器位置信息。

✓ 输入变量

输入变量	名称	数据类型	有效范围	内容
boxID	电箱 ID	unsigned int	0~5	电箱 ID 号，默认值= 0
rbtID	机器人 ID	unsigned int	0~5	机器人 ID 号，默认值= 0
dX- dRz	传感器位置 信息	double	-	实时更新传感器位置信息

✓ 返回值

返回值	名称	数据类型	有效范围	内容
nRet	返回值	int	≥ 0 的整型值	nRet = 0: 返回函数调用成功 nRet > 0: 返回调用失败的错误码

3.14.6.2. 示例

// 设置PID 参数

```
double dX = 0;double dY = 0;double dZ = 0;
double dRx = 0;double dRy = 0;double dRz = 0;
int nRet= HRIF_SetUpdateTrackingPose (0,0, double dX,double dY,double dZ,
                                     double dRx,double dRy,double dRz);
```

3.15. 其他指令

3.15.1. HRIF_HRAppCmd

3.15.1.1. 描述：执行插件 App 命令。

✓ 输入变量

输入变量	名称	数据类型	有效范围	内容
boxID	电箱 ID	unsigned int	0~5	电箱 ID 号，默认值= 0
sAppName	插件名称	string	-	插件名称
sCmdName	命令名称	string	-	命令名称
sParams	参数列表	string	-	参数列表

✓ 返回值

返回值	名称	数据类型	有效范围	内容
nRet	返回值	int	>=0 的整型值	nRet = 0: 返回函数调用成功 nRet >0: 返回调用失败的错误码

3.15.1.2. 示例

//插件名称

```
string sAppName = "";
```

//命令名称

```
string sCmdName = "";
```

//参数列表

```
string sParams;
```

//执行插件命令

```
int nRet= HRIF_HRAppCmd(0, sAppName ,nCmdName , sParams);
```

3.15.2. HRIF_HRAppData

3.15.2.1. 描述：执行插件 App 命令。

✓ 输入变量

输入变量	名称	数据类型	有效范围	内容
boxID	电箱 ID	unsigned int	0~5	电箱 ID 号, 默认值= 0
sAppName	插件名称	string	-	插件名称
sCmdName	命令名称	string	-	命令名称
sParams	参数列表	string	-	参数列表

✓ 输出变量

输出变量	名称	数据类型	有效范围	内容
sResult	结果列表	vector<string>	-	结果列表

✓ 返回值

返回值	名称	数据类型	有效范围	内容
nRet	返回值	int	>=0 的整型值	nRet = 0: 返回函数调用成功 nRet >0: 返回调用失败的错误码

3.15.2.2. 示例

//插件名称

```
string sAppName = "";
```

//命令名称

```
string sCmdName = "";
```

//参数列表

```
string sParams;
```

// 结果列表

```
std::vector<string> sResult;
```

//执行插件命令

```
int nRet= HRIF_HRAppData(0, sAppName ,nCmdName , sParams,sResult);
```

3.15.3. HRIF_WriteEndHoldingRegisters

3.15.3.1. 描述：写末端连接的 Modbus 从站寄存器。

✓ 输入变量

输入变量	名称	数据类型	有效范围	内容
boxID	电箱 ID	unsigned int	0~5	电箱 ID 号，默认值= 0
rbtID	机器人 ID	unsigned int	0~5	机器人 ID 号，默认值= 0
nSlaveID	从站 ID	int	1~255	从站 ID
nFunction	功能码	int	>0	功能码： 0x01-读线圈寄存器 0x02-读线圈离散输入寄存器 0x03-读保持寄存器 0x04-读输入寄存器 0x05-写单个线圈寄存器 0x06-写单个保持寄存器 0x0f-写多个线圈寄存器 0x10-写多个保持寄存器
nRegAddr	寄存器地址	int	>0	寄存器起始地址
nRegCount	寄存器数量	int	>0	寄存器数量
vecData	寄存器数据	vector<int>	-	寄存器数据

* 注：末端为 EtherCAT 总线版本 IO 时有效。

✓ 返回值

返回值	名称	数据类型	有效范围	内容
nRet	返回值	int	≥0 的整型值	nRet=0: 返回函数调用成功 nRet>0: 返回调用失败的错误码

3.15.3.2. 示例

// 设置从站 ID

```
int nSlaveID = 1;
```

// 设置功能码

```
int nFunction = 16;
// 设置寄存器起始地址

int nRegAddr = 5004;
// 设置寄存器数量

int nRegCount = 2;
// 设置寄存器数据

vector<int> vecData= {196,34465};

int nRet= HRIF_WriteEndHoldingRegisters(0,0,nSlaveID , nFunction, nRegAddr, nRegCount, vecData);
```


3.15.4. HRIF_ReadEndHoldingRegisters

3.15.4.1. 描述：读末端连接的 Modbus 从站寄存器。

✓ 输入变量

输入变量	名称	数据类型	有效范围	内容
boxID	电箱 ID	unsigned int	0~5	电箱 ID 号，默认值= 0
rbtID	机器人 ID	unsigned int	0~5	机器人 ID 号，默认值= 0
nSlaveID	从站 ID	int	1~255	从站 ID
nFunction	功能码	int	>0	功能码： 0x01-读线圈寄存器 0x02-读线圈离散输入寄存器 0x03-读保持寄存器 0x04-读输入寄存器 0x05-写单个线圈寄存器 0x06-写单个保持寄存器 0x0f-写多个线圈寄存器 0x10-写多个保持寄存器
nRegAddr	寄存器地址	int	>0	寄存器起始地址
nRegCount	寄存器数量	int	>0	寄存器数量

* 注：末端为 EtherCAT 总线版本 IO 时有效。

✓ 输出变量

输出变量	名称	数据类型	有效范围	内容
vecData	寄存器数据	vector<int>	-	寄存器数据

✓ 返回值

返回值	名称	数据类型	有效范围	内容
nRet	返回值	int	>=0 的整型值	nRet=0: 返回函数调用成功 nRet>0: 返回调用失败的错误码

3.15.4.2. 示例

// 设置从站 ID

```
int nSlaveID = 1;
// 设置功能码
int nFunction = 3;
// 设置寄存器起始地址
int nRegAddr = 100;
// 设置寄存器数量
int nRegCount = 4;
// 读取寄存器数据
vector<int> vecData= {};
int nRet= HRIF_ReadEndHoldingRegisters(0,0,nSlaveID , nFunction, nRegAddr, nRegCount, nRegCount, vecData);
auto iter = vecData.begin();
auto end = vecData.end();
for (; iter != end; iter++)
cout << *iter << endl;
```

3.15.5. HRIF_SetEndModbusInfo

3.15.5.1. 描述：设置末端连接的 Modbus 配置

✓ 输入变量

输入变量	名称	数据类型	有效范围	内容
boxID	电箱 ID	unsigned int	0~5	电箱 ID 号，默认值= 0
rbtID	机器人 ID	unsigned int	0~5	机器人 ID 号，默认值= 0
nBaudrate	波特率	int	>0	波特率，默认为 115200
nDataBit	数据位	int	>0	数据位，默认为 8
nParityBit	校验位	int	>0	校验位 78：无校验 79：奇校验 69：偶校验
nStopBit	停止位	int	>0	停止位，默认为 1

* 注：末端为 EtherCAT 总线版本 IO 时有效。

✓ 返回值

返回值	名称	数据类型	有效范围	内容
nRet	返回值	int	>=0 的整型值	nRet=0: 返回函数调用成功 nRet>0: 返回调用失败的错误码

3.15.5.2. 示例

// 设置波特率

int nBaudrate= 115200;

// 设置数据位

int nDataBit= 8;

// 设置校验位

int nParityBit= 100;

// 设置停止位

int nStopBit= 1;

// 设置末端连接的 Modbus 配置

```
int nRet= HRIF_SetEndModbusInfo(0,0,nBaudrate, nDataBit, nParityBit, nStopBit);
```

3.16. 示例代码

1. 机器人初始化

机器人从下电到使能的状态机流程如下：

CPS FSM state changed from 7(Blackout48V) to 8(Electrifying48V)

CPS FSM state changed from 8(Electrifying48V) to 14(ControllerDisconnect)

CPS FSM state changed from 14(ControllerDisconnect) to 15(ControllerConnecting)

CPS FSM state changed from 15(ControllerConnecting) to 18(ControllerChecking)

CPS FSM state changed from 18(ControllerChecking) to 24(Disable)

CPS FSM state changed from 24(Disable) to 23(Enabling)

CPS FSM state changed from 23(Enabling) to 33(StandBy)

参考代码：

```
int nRet = -1;

// 连接机器人

const char* hostname = "192.168.0.10";
unsigned short nPort = 10003;
nRet = HRIF_Connect(0,hostname,nPort);

// 机器人本体上电

nRet = HRIF_Electrify(0);
while(1)
{
    int nCurFSM = -1;
    HRIF_ReadCurFSMFromCPS(0,0, nCurFSM);
    if(nCurFSM==14)
    {
        // 连接控制器(启动主站，连接从站)

        nRet = HRIF_Connect2Controller(0);
        break;
    }
    sleep(0.2);
}
while(1)
{
    // 判断控制器是否启动完成

    int nStarted = -1;
    nRet = HRIF_IsControllerStarted(0,nStarted);
    if(nStarted==1)
```

```
{
    break;
}
sleep(0.2);
}
while(1)
{
    int nCurFSM = -1;
    HRIF_ReadCurFSMFromCPS(0,0, nCurFSM);
    if(nCurFSM==24)
    {
        // 使能机器人
        nRet = HRIF_GrpEnable(0,0);
        break;
    }
    sleep(0.2);
}
cout << "finish." << endl;
```

2. 力控+MoveL 运动

// 连接机器人

```
const char* hostname = "10.20.60.144";
unsigned short nPort = 10003;
nRet = HRIF_Connect(0,hostname,nPort);
```

// 设置力控坐标系控制方向为 Tool 坐标系方向 1: Tool 0: 当前 ucs

```
int nMode = 1;
nRet = HRIF_SetForceToolCoordinateMotion(0,0,nMode);
```

// 设置力控策略为恒力模式 0:恒力模式 2: 越障模式

```
int nStrategy = 0;
nRet = HRIF_SetForceControlStrategy(0,0,nStrategy);
```

// 设置质量/惯量参数

```
double dXMass = 40; double dYMass = 40; double dZMass = 40;
double dRxBMass = 10; double dRyMass = 10; double dRzMass = 10;
nRet = HRIF_SetMassParams(0,0,dXMass,dYMass,dZMass,dRxBMass,dRyMass,dRzMass);
```

// 设置阻尼参数

```
double dXDamp = 800; double dYDamp = 800; double dZDamp = 800;  
double dRxDamp = 40; double dRyDamp = 40; double dRzDamp = 40;  
nRet = HRIF_SetDampParams(0,0,dXDamp,dYDamp,dZDamp,dRxDamp,dRyDamp,dRzDamp);
```

// 设置刚度参数

```
double dXStiff = 1000; double dYStiff = 1000; double dZStiff = 1000;  
double dRxStiff = 100; double dRyStiff = 100; double dRzStiff = 100;  
nRet = HRIF_SetStiffParams(0,0,dXStiff,dYStiff,dZStiff,dRxStiff,dRyStiff,dRzStiff);
```

// 设置力控探寻自由度状态 Z 方向

```
int nX = 0; int nY = 0; int nZ = 1;  
int nRx = 0; int nRy = 0; int nRz = 0;  
nRet = HRIF_SetControlFreedom(0,0, nX, nY, nZ, nRx, nRy, nRz);
```

// 设置力控目标力大小 Z 方向 20N

```
double dXForce = 0; double dYForce = 0; double dZForce = 20;  
double dRxForce = 0; double dRyForce = 0; double dRzForce = 0;  
nRet = HRIF_SetForceControlGoal(0,0,dXForce,dYForce,dZForce,dRxForce,dRyForce,dRzForce);
```

// 力控探寻直线速度

```
double dMaxLinearVelocity = 15;
```

// 力控探寻姿态角速度

```
double dMaxAngularVelocity = 5;
```

// 设置力控探寻最大直线速度及角速度

```
nRet = HRIF_SetMaxSearchVelocities(0,0,dMaxLinearVelocity,dMaxAngularVelocity);
```

// 设置各自由度 (X/Y/Z/RX/Ry/RZ) 力控探寻最大距离

```
double Dis_X = 300; double Dis_Y = 300; double Dis_Z = 300;  
double Dis_RX = 20; double Dis_RY = 20; double Dis_RZ = 20;  
nRet = HRIF_SetMaxSearchDistance(0,0,Dis_X,Dis_Y,Dis_Z,Dis_RX,Dis_RY,Dis_RZ);
```

// 设置恒力控稳定阶段边界

```
double Pos_X = 101; double Pos_Y = 100; double Pos_Z = 100;  
double Pos_RX = 20; double Pos_RY = 20; double Pos_RZ = 20;
```

```
double Neg_X = -100; double Neg_Y = -100; double Neg_Z = -100;
double Neg_RX = -20; double Neg_RY = -20; double Neg_RZ = -20;
HRIF_SetSteadyContactDeviationRange(0,0,Pos_X,Pos_Y,Pos_Z,Pos_RX,Pos_RY,Pos_RZ,Neg_X,Neg_Y,Neg_Z,Neg_RX,Neg_RY,Neg_RZ);
```

```
// 探寻起点
```

```
HRIF_WayPoint(0,0, 1, 420,0,445,180,0,180, 0,0,0,0,0,"TCP","Base",100,500,0, 0,0,0,0,"ID0");
```

```
while(1)
```

```
{
```

```
    bool bDone = false;
```

```
    // 判断运动是否完成
```

```
    int nRet = HRIF_IsMotionDone(0,0,bDone);
```

```
    if (bDone==true)
```

```
        break;
```

```
}
```

```
// 开启力控探寻
```

```
int nState = 1;
```

```
nRet = HRIF_SetForceControlState(0,0, nState);
```

```
while(1)
```

```
{
```

```
    cout << "Start force search." << endl;
```

```
    // 读取当前力控状态
```

```
    // 0: 关闭状态 1: 开力控探寻状态 2: 力控探寻完成状态 3: 力控自由驱动状态
```

```
    int nState1 = -1;
```

```
    nRet = HRIF_ReadForceControlState(0,0, nState1);
```

```
    if(nState1==2) break;
```

```
    sleep(0.2);
```

```
}
```

```
// MoveL 路点运动
```

```
HRIF_WayPoint(0,0, 1, 520,0,345,180,0,180, 0,0,0,0,0,"TCP","Base",60,500,30, 0,0,0,0,"ID0");
```

```
while(1)
```

```
{
```

```
    bool bDone = false;
```

```
    // 判断路点是否完成
```

```
    HRIF_IsBlendingDone(0,0,bDone);
```

```
    if (bDone==true)
```

```
        break;
```



```
}  
HRIF_WayPoint(0,0, 1, 520,200,345,180,0,180, 0,0,0,0,0,0,"TCP","Base",60,500,30, 0,0,0,0,"ID1");  
while(1)  
{  
    bool bDone = false;  
    // 判断路点是否完成  
    HRIF_IsBlendingDone(0,0,bDone);  
    if (bDone==true)  
        break;  
}  
// 关闭力控  
int nState2 = 0;  
nRet = HRIF_SetForceControlState(0,0, nState2);  
cout << "finish." << endl;
```

3. 恒力控+越障+MoveL 运动

在上述恒力控+MoveL 基础上修改添加以下代码即可。

```
// 设置力控策略为恒力模式 0:恒力模式 2: 越障模式  
int nStrategy = 2;  
nRet = HRIF_SetForceControlStrategy(0,0,nStrategy);  
  
// 定义 X/Y 方向切向力最大值  
double dMax = 25;  
// 定义 X/Y 方向切向力最小值  
double dMin = 10;  
// 定义越障上抬最大速度  
double dVel = 50;  
// 设置 X/Y 方向切向力最大值、最小值和上抬最大速度  
nRet = HRIF_SetTangentForceBounds(0,0,dMax,dMin,dVel);
```